

**Федеральное государственное образовательное бюджетное
учреждение высшего образования
«Финансовый университет при Правительстве Российской Федерации»
(Финансовый университет)**

Черпаков И.В.

**Программирование на языке Python:
теория и лабораторный практикум**

Последнее изменение
05.08.2025

Липецк–2025

СОДЕРЖАНИЕ

Введение.....	5
1. Алгоритмизация задач. Базовые понятия программирования.....	6
1.1. Алгоритмизация задач.....	6
Задание 1.1 (практико-ориентированное задание).....	10
1.2. Структура программы на языке Python.....	14
1.3. Объекты	17
2. Вычисления. Числовые и логический типы данных Python.....	21
Задание 2.1	29
Задание 2.2	31
3. Основные алгоритмические структуры	34
3.1. Условная инструкция	34
Задание 3.1 (практико-ориентированное задание).....	36
3.2. Теоретические сведения о циклических инструкциях	43
3.3. Цикл с предусловием	44
Задание 3.2	48
Задание 3.3	50
3.4. Генерирование числовых последовательностей.....	55
Задание 3.4 (практико-ориентированное задание).....	56
3.5. Цикл с параметром	61
Задание 3.5	63
Задание 3.6 (практико-ориентированное задание).....	69
4. Подпрограммы.....	74
4.1. Пользовательские функции	74
Задание 4.1	77
Задание 4.2	82
5. Строковый тип	85
5.1. Обработка строковых данных	85
Задание 5.1	94
5.2. Форматированные строки.....	100
Задание 5.2 (практико-ориентированное задание).....	101
Задание 5.3	105
6. Генерация случайных значений	110
Задание 6.1 (практико-ориентированное задание).....	112

7. Списковый тип.....	116
7.1. Одномерные списки	116
Задание 7.1	121
Задание 7.2	136
Задание 7.3	136
Задание 7.4 (практико-ориентированное задание).....	140
Задание 7.5	146
7.2. Генераторы списков.....	150
Задание 7.6	152
7.3. Двумерные списки.....	155
Задание 7.7 (практико-ориентированное задание).....	159
7.4. Сортировка списков. Применение лямбда-функций	174
8. Матричные операции	179
Задание 8.1	182
Задание 8.2	193
Задание 8.3	202
Задание 8.4	203
Задание 8.5	206
9. Кортежи	208
Задание 9.1	212
10. Множества	216
Задание 10.1	219
11. Словари	223
Задание 11.1 (практико-ориентированное задание).....	227
12. Создание интерфейса консольной программы	232
Задание 12.1 (практико-ориентированное задание).....	235
13. Исследование функций с использованием языка Python.....	238
13.1. Построение графиков функции, ее первой и второй производной. Анализ функции по графикам.....	238
Задание 13.1	249
13.2. Нахождение корней функции численными методами.....	251
Задание 13.2	254
13.3. Нахождение точек экстремума функции численными методами .	254
Задание 13.3	257
13.4. Нахождение точек перегиба функции численными методами	258

Задание 13.4	261
14. Обработка файлов	262
14.1. Базовые операции.....	262
Задание 14.1	264
14.2. Обработка csv-файлов	266
Задание 14.2	268
15. Регрессионный анализ.....	274
15.1. Линейная парная регрессия	274
15.2. Линейная множественная регрессия.....	274
Предметный указатель	275

Equation Chapter 1 Section 1

Введение

Задания, предлагаемые для студентов направления 38.03.05 «Бизнес-информатика» и 01.03.02 «Прикладная математика и информатика», охватывают широкий спектр задач, связанный с алгоритмизацией и программированием на языке Python. Ряд заданий имеет практико-ориентированную направленность и сформулированы таким образом, чтобы быть приближенными к реальным задачам, которые могут встречаться в практической деятельности.

1. Алгоритмизация задач. Базовые понятия программирования

1.1. Алгоритмизация задач

Цель работы: научиться выполнять алгоритмизацию задач по определенной схеме и получать результат, который в дальнейшем может быть использован для написания программы, реализующей решение задачи.

Алгоритм — это конечная, точная и однозначно определенная последовательность инструкций (операций), направленных на решение задачи за конечное число шагов. Получаемый результат должен быть достижим для всех допустимых входных данных.

В табл. 1.1 приведены свойства алгоритма, сформулированные в теории алгоритмов и информатике.

Таблица 1.1

Свойство	Описание
Конечность	Алгоритм завершается за конечное число шагов для каждого допустимого входа
Дискретность	Алгоритм состоит из отдельных, явно разделенных шагов (операций)
Однозначность	Каждая инструкция алгоритма имеет точное, недвусмысленное и формализованное определение
Массовость	Алгоритм применим ко всему множеству допустимых входных данных (а не только к частным случаям)
Результативность	Выполнение алгоритма приводит к достижению корректного результата или определению невозможности его получения
Формализуемость	Алгоритм может быть выражен средствами формального языка или системы, допускающей однозначную интерпретацию
Автоматизируемость	Алгоритм может быть выполнен автоматически без вмешательства человека (исполнителем является машина или вычислительный механизм)
Понятность	Алгоритм должен быть изложен доступным, логически структурированным образом для понимания и анализа
Эффективность	Алгоритм должен использовать минимально необходимые ресурсы (время, память) для решения задачи
Универсальность	Алгоритм может быть адаптирован или расширен для решения широкой категории сходных задач
Модульность	Алгоритм может быть разбит на функционально независимые части (модули)

Рассмотрим основные теоретические понятия, связанные с выполнением алгоритма.

Исполнитель алгоритма (исполнитель) — абстрактная или конкретная (вычислительная) система, способная реализовать шаги алгоритма. Примеры исполнителей: человек, вычислительное устройство, программная система.

Алфавит исполнителя — конечное множество символов, допустимых в качестве входных или выходных данных, управляющих символов или составных частей команд.

Команда исполнителя — элементарное предписание, понятное и выполнимое исполнителем.

Система команд исполнителя — конечное множество элементарных действий, каждое из которых может быть выполнено исполнителем без дополнительного уточнения. Совокупность этих команд определяет вычислительные возможности исполнителя.

Состояние исполнителя — совокупность всех параметров, определяющих текущее положение исполнителя и содержание его рабочей памяти в данный момент времени.

Среда исполнения алгоритма (среда исполнителя) — часть модели исполнителя, определяющая, к каким объектам и структурам данных обращаются команды исполнителя. Примеры: числовая прямая, двумерное пространство точек, таблица и т. д.

Начальное состояние — состояние исполнителя и среды исполнения до начала выполнения алгоритма. В начальном состоянии задаются все исходные данные и начальные конфигурации объектов среды.

Конечное состояние — состояние исполнителя и среды исполнения после завершения выполнения алгоритма, содержащее итоговый результат.

Допустимые входные данные — множество значений, при которых алгоритм корректно функционирует и приводит к завершению с определенным результатом.

Результат выполнения алгоритма — выходные данные, полученные после применения алгоритма к конкретному набору допустимых входных данных.

Алгоритмизация — это процесс формализации решения задачи путем построения алгоритма (рис. 1). Представляет собой преобразование исходной формулировки задачи в конструктивную процедуру, пригодную для последующего программного исполнения.



Рисунок 1

Основные этапы алгоритмизации:

1. Постановка задачи — формальное определение исходных данных, требований к результату и ограничений.

2. Анализ структуры решения — выделение подзадач и основных вычислительных шагов.

3. Выбор модели данных — определение форм представления входных, промежуточных и выходных объектов.

4. Построение алгоритма — разработка пошаговой инструкции, содержащей операции ввода и вывода, вычисления, ветвления, циклы.

5. Верификация алгоритма — проверка корректности, полноты и конечности.

6. Подготовка к программной реализации — перевод алгоритма в форму, совместимую с выбранным языком программирования.

Результатом алгоритмизации является *описание алгоритма* в одной из форм:

- словесная — описание на естественном языке;
- графическая — блок-схема или диаграмма потоков управления;
- псевдокод — формализованный алгоритмический язык;
- программная реализация — текст программы на языке программирования.

Пример 1.1

Выполнить алгоритмизацию следующей задачи. Требуется определить минимально необходимый объем продаж q , при котором прибыль предприятия достигает заданного значения $P_{\min}$. Имеется значение постоянных издержек F , величина переменных издержек на единицу продукции v , цена реализации одной единицы продукции p . Объем продаж определяется по формуле

$$P = (p - v) \cdot q - F.$$

В случае, если условие $p > v$ не выполняется, алгоритм должен корректно завершиться с сообщением о невозможности достижения заданной прибыли.

Решение

1. Постановка задачи

Требуется определить минимально необходимый объем продаж некоторого товара, при котором прибыль предприятия достигает или превышает заданное пользователем значение. Известны постоянные издержки F , переменные издержки на единицу продукции v , цена реализации одной единицы товара p и требуемая минимальная прибыль $P_{\min}$. Условием допустимости решения является выполнение неравенства $p > v$ (в противном случае прибыль не может быть положительной).

2. Анализ структуры решения

Решение задачи состоит в последовательном увеличении объема продаж q начиная с единицы, с вычислением соответствующей прибыли по формуле $P = (p - v) \cdot q - F$. Процесс повторяется до тех пор, пока $P < P_{\min}$. В процессе

необходимо проверить допустимость исходных параметров: если $p \leq v$, то следует завершить выполнение алгоритма с диагностическим сообщением.

3. Выбор модели данных

Описание модели данных приведено в табл. 1.2.

Таблица 1.2

Данные	Обозначение	Описание	Тип	Условия и ограничения
Входные данные	F	Постоянные издержки	Целочисленный / вещественный	$F \geq 0$
	v	Переменные издержки на единицу продукции	Целочисленный / вещественный	$v \geq 0$
	p	Цена единицы продукции	Целочисленный / вещественный	$p \geq 0$
	$P_{\min}$	Минимальная прибыль предприятия	Целочисленный / вещественный	$P_{\min} \geq 0$
Промежуточные данные	q	Объем продаж	Целочисленный	$q \geq 1$, вычисляемое значение
	P	Прибыль при текущем значении q	Вещественный	Вычисляемое значение
Выходные данные	$q_{\min}$	Минимально необходимый объем продаж	Целочисленный	Вычисляемое значение
	Диагностическое сообщение	Сообщение при невозможности решения	Строковый	—

4. Построение алгоритма

1. Ввести значения F , v , p , $P_{\min}$.
2. Если $p \leq v$, вывести сообщение: «Прибыль не может быть достигнута при данных параметрах» и завершить выполнение.
3. Присвоить $q = 1$.
4. Вычислить прибыль по формуле $P = (p - v) \cdot q - F$.
5. Пока $P < P_{\min}$, увеличить q на 1 и перейти к шагу 4).
6. После выхода из цикла вывести значение q как минимально необходимый объем продаж.

5. Верификация алгоритма

Алгоритм корректно обрабатывает случай недопустимых входных параметров ($p \leq v$), исключая бесконечный цикл. В остальных случаях алгоритм гарантированно завершится, так как при $p > v$ прибыль P монотонно возрастает с ростом q и в конечный момент достигнет уровня $P_{\min}$.

6. Подготовка к программной реализации

Для реализации на языке программирования следует использовать:

- ввод для параметров F , v , p , $P_{\min}$;
- проверку условия $p \leq v$;
- последовательное увеличение значения q ;
- вывод для отображения результата.

Алгоритм, записанный на псевдокоде, приведен ниже.

НАЧАЛО

ВВЕСТИ F

ВВЕСТИ v

ВВЕСТИ p

ВВЕСТИ $P_{\min}$

// Проверка выполнимости задачи

ЕСЛИ $p \leq v$ ТО

 ВЫВЕСТИ "Расчет невозможен: цена реализации не превышает
 переменные издержки."

 ЗАВЕРШИТЬ

КОНЕЦ ЕСЛИ

// Поиск минимального значения объема продаж q

$q \leftarrow 1$

ПОКА $(p - v) \cdot q - F < P_{\min}$ ВЫПОЛНЯТЬ

$q \leftarrow q + 1$

КОНЕЦ ПОКА

ВЫВЕСТИ "Минимально необходимый объем продаж: ", q

КОНЕЦ

Задание 1.1 (практико-ориентированное задание)

Выполнить алгоритмизацию задач из табл. 1.3.

Содержание отчета:

- полная формулировка задания;
- описание каждого этапа алгоритмизации; модель данных должна быть выполнена в виде таблицы со структурой, аналогичной примеру;
- результат алгоритмизации в виде псевдокода или графически (блок-схема).

Таблица 1.3

№	Задание
1	Определить минимальное количество единиц продукции q , которое необходимо реализовать предприятию, чтобы достичь заданного уровня прибыли $P_{\min}$, если известны постоянные издержки F , переменные издержки на единицу продукции v и цена реализации одной единицы p , при условии, что $p > v$. Необходимо найти минимальное натуральное q , при котором выполняется неравенство $(p - v) \cdot q - F \geq P_{\min}$
2	Найти необходимую цену продажи одной единицы продукции p , обеспечивающую достижение планируемой прибыли $P_{\min}$ при заданных объемах реализации q , уровне переменных издержек v и постоянных издержках F , если $q > 0$ и $p > v$. Вычисление производится по формуле $p \geq \frac{P_{\min} + F}{q} + v$
3	Вычислить объем прибыли P предприятия при известных значениях объема продаж q , цены реализации единицы продукции p , переменных издержек v и постоянных издержек F , используя формулу $P = (p - v) \cdot q - F$
4	Найти минимальное значение переменных издержек v , при котором производство заданного объема продукции q становится нерентабельным при цене реализации p и постоянных издержках F , если прибыль P не должна быть положительной. Требуется найти v , при котором $(p - v) \cdot q - F \leq 0$, откуда $v \geq p - \frac{F}{q}$
5	Определить, какое значение постоянных издержек F допустимо при заданных значениях q , p , v и планируемой прибыли $P_{\min}$, при условии, что $p > v$. Необходимо, чтобы $(p - v) \cdot q - F \geq P_{\min}$, откуда $F \leq (p - v) \cdot q - P_{\min}$
6	Найти необходимое увеличение объема продаж Δq , позволяющее увеличить прибыль на заданную величину ΔP при фиксированных значениях p , v и F . Увеличение Δq определяется из неравенства $(p - v) \cdot \Delta q \geq \Delta P$, откуда $\Delta q \geq \frac{\Delta P}{p - v}$ при $p > v$
7	Определить, при каком минимальном значении цены реализации p достигается безубыточность (нулевая прибыль) при заданных значениях q , v и F . Условие безубыточности $(p - v) \cdot q - F = 0$ дает формулу $p = \frac{F}{q} + v$
8	Найти допустимое значение объема продаж q , при котором обеспечивается покрытие всех затрат (переменных и постоянных) в условиях ограниченного спроса, если известны цена реализации p , переменные издержки v , постоянные издержки F и максимальный спрос $q_{\max}$. Требуется найти максимальное $q \leq q_{\max}$, при котором $(p - v) \cdot q \geq F$
9	Определить допустимое значение переменных издержек $v_{\max}$, при котором возможно достичь заданной прибыли $P_{\min}$ при фиксированных q , p и F , используя неравенство $(p - v) \cdot q - F \geq P_{\min}$, откуда $v \leq p - \frac{P_{\min} + F}{q}$

№	Задание
10	Определить минимальный объем продаж q , необходимый для покрытия всех издержек предприятия, если заданы постоянные издержки F , переменные издержки на единицу продукции v и цена реализации одной единицы p , при условии $p > v$. Требуется найти минимальное натуральное q , при котором выполняется неравенство $(p - v) \cdot q \geq F$
11	Оценить, при каком минимальном уровне цены p возможно достижение безубыточности при заданных значениях постоянных издержек F , переменных издержек v и ожидаемом объеме продаж q . Найти минимальное значение p , при котором выполняется неравенство $(p - v) \cdot q \geq F$ при условии $p > v$
12	Определить минимальное значение надбавки m к переменным издержкам v , обеспечивающее получение прибыли не ниже $P_{\min}$ при известных значениях v , F и q . Требуется найти минимальное значение m , при котором выполняется неравенство $(v + m - v) \cdot q - F \geq P_{\min}$, то есть $m \cdot q \geq F + P_{\min}$
13	Найти максимальное значение переменных издержек v , при котором предприятие при заданных цене реализации p , объеме продаж q и постоянных издержках F все еще получает прибыль не ниже $P_{\min}$. Необходимо, чтобы выполнялось неравенство $(p - v) \cdot q - F \geq P_{\min}$
14	Оценить, при каком минимальном объеме продаж q возможно покрытие заданного значения налога T , если с каждой реализованной единицы продукции предприятие получает доход $p - v$, а налог уплачивается только из положительной прибыли после покрытия постоянных издержек F . Найти минимальное натуральное q , при котором $(p - v) \cdot q - F \geq T$
15	Найти минимальный объем продаж q , обеспечивающий заданный уровень рентабельности r , выраженной как отношение прибыли к постоянным издержкам, при заданных F , v и p . Необходимо найти минимальное натуральное q , для которого $(p - v) \cdot q - F \geq r \cdot F$
16	Определить минимальный уровень налоговой льготы τ (в долях от прибыли), при котором достигается прибыль после налогообложения не ниже $P_{\min}$, если до налогообложения прибыль вычисляется по формуле $(p - v) \cdot q - F$. Необходимо найти минимальное значение τ , при котором $(1 - \tau) \cdot ((p - v) \cdot q - F) \geq P_{\min}$
17	Найти минимальное значение субсидии S , которое необходимо предоставить предприятию для обеспечения прибыли не ниже $P_{\min}$, если все остальные параметры (F , v , p , q) заданы. Требуется, чтобы выполнялось $(p - v) \cdot q - F + S \geq P_{\min}$
18	Определить минимальное значение скидки d (в рублях) на переменные издержки v , которую должен предоставить поставщик, чтобы предприятие достигло прибыли не ниже $P_{\min}$ при заданных F , p и q . Найти минимальное d , для которого $(p - (v - d)) \cdot q - F \geq P_{\min}$
19	Оценить, какое минимальное количество дополнительных единиц продукции q_{add} необходимо реализовать предприятию сверх базового объема q_0 , чтобы достигнуть заданной дополнительной прибыли ΔP , при прочих равных условиях. Найти минимальное натуральное q_{add} , при котором $(p - v) \cdot q_{\text{add}} \geq \Delta P$

№	Задание
20	Определить минимальный рост цены p в абсолютном выражении, необходимый для достижения заданного уровня прибыли $P_{\min}$ при фиксированных q, v и F. Найти минимальное значение приращения Δp, при котором $(p + \Delta p - v) \cdot q - F \geq P_{\min}$
21	Определить минимальное количество рабочих дней d, необходимое для достижения запланированной выручки W_{target}, если известна средняя дневная производительность труда r (в единицах продукции), цена реализации одной единицы продукции p и количество работников n. Найти минимальное натуральное значение d, при котором выполняется неравенство $p \cdot r \cdot n \cdot d \geq W_{\text{target}}$
22	Определить минимальный срок окупаемости инвестиций t в годах, при котором сумма дисконтированных доходов D становится не меньше вложенного капитала K, если доход за каждый год равен R, а ставка дисконтирования составляет i. Найти минимальное натуральное t, при котором выполняется неравенство $\sum_{k=1}^t \frac{R}{(1+i)^k} \geq K$
23	Определить минимальный объем хранимой продукции x на складе, необходимый для обеспечения бесперебойных продаж в течение d дней при ежедневном спросе s, если средний срок поставки от поставщика составляет t дней. Найти минимальное натуральное x, при котором выполняется неравенство $x \geq s \cdot t + s \cdot d$
24	Определить минимальное количество машин m, требуемое для доставки продукции объема V за T часов, если каждая машина способна перевезти c единиц продукции за один рейс, а средняя продолжительность одного рейса составляет h часов. Найти минимальное натуральное m, при котором выполняется неравенство $m \cdot \left\lfloor \frac{T}{h} \right\rfloor \cdot c \geq V$, где $\lfloor \cdot \rfloor$ — операция округления вниз до целого значения
25	Определить минимальный уровень розничной цены p, необходимый для обеспечения требуемой нормы прибыли π в процентах, если переменные издержки на единицу продукции составляют v, а доля постоянных издержек на единицу f. Найти минимальное значение p, при котором выполняется неравенство $\frac{p - v - f}{p} \cdot 100 \geq \pi$
26	Определить минимальное количество клиентов n, необходимых для выхода онлайн-сервиса на точку безубыточности при среднем доходе r с одного клиента и общих ежемесячных издержках F. Найти минимальное натуральное n, при котором выполняется неравенство $r \cdot n \geq F$
27	Определить минимальное количество часов аренды t производственного оборудования, необходимое для выпуска продукции объема q, если скорость производства составляет s единиц продукции в час, а максимальная доступная мощность составляет m единиц в час. Найти минимальное натуральное t, при котором выполняется неравенство $\min(s, m) \cdot t \geq q$

№	Задание
28	Определить минимальную долю рекламного бюджета x (в процентах от выручки), необходимую для обеспечения прироста спроса Δd за счет рекламной активности, если известно, что эластичность спроса по рекламе составляет e, а текущий спрос — d_0. Найти минимальное значение x, при котором выполняется неравенство $d_0 \cdot e \cdot \frac{x}{100} \geq \Delta d$
29	Определить минимальное количество смен k, которые необходимо организовать на предприятии, чтобы при производительности одной смены q и наличии s рабочих мест обеспечить ежесуточный выпуск продукции объема Q. Найти минимальное натуральное k, при котором выполняется неравенство $q \cdot s \cdot k \geq Q$
30	Определить минимальное количество закупаемых упаковок сырья n, необходимых для обеспечения производственного цикла объемом V, если одна упаковка содержит c единиц сырья, а на производство одной единицы продукции требуется u единиц сырья. Найти минимальное натуральное n, при котором выполняется неравенство $c \cdot n \geq V \cdot u$

1.2. Структура программы на языке Python

Запись алгоритма на любом языке программирования осуществляется в соответствии с определенным синтаксисом и заложенной семантикой.

Синтаксис (syntax) — совокупность правил, определяющих допустимую структуру и форму записи программных конструкций языка программирования.

Семантика (semantics) — совокупность правил, определяющих смысл и поведение синтаксически корректных конструкций при их выполнении.

Программа (program) — последовательность инструкций, записанных на языке программирования и предназначенных для выполнения вычислительной системой с целью реализации определенного алгоритма или обработки данных в соответствии с заданной логикой.

Программный код или просто *код* (program code, code) — синтаксически корректное текстовое представление программы.

Система программирования — это совокупность взаимосвязанных программных и технических средств, предназначенных для разработки, трансляции, отладки, сопровождения и выполнения программ на одном или нескольких языках программирования. Система программирования ориентирована преимущественно на реализацию всего жизненного цикла программного обеспечения, начиная с написания исходного кода и заканчивая получением исполняемого файла или интерпретируемого результата.

Среда программирования — это пользовательский интерфейс или оболочка, обеспечивающая доступ к возможностям системы программирования. Примерами сред программирования могут служить PyCharm, Visual Studio, Eclipse.

Рассмотрим основные понятия, связанные с написанием программного кода.

Операция (operation) — это абстрактное действие, выполняемое над данными. Операция описывает смысл вычисления, а не его конкретную реализацию в языке. Например, операция сложения означает получение суммы двух величин. Операции могут быть арифметическими (сложение, вычитание), логическими (конъюнкция, дизъюнкция), сравнительными (равенство, неравенство) и т. д. Операция как понятие существует независимо от языка программирования и соответствует логико-математическим или вычислительным действиям.

Оператор (operator) — это конкретное синтаксическое средство языка программирования, реализующее одну или несколько операций. Оператор — это знак или ключевое слово, которое интерпретатор языка Python распознает как команду. Например, оператор `+` реализует операцию сложения, оператор **and** реализует логическую конъюнкцию, а **is** — проверку идентичности объектов. Один и тот же оператор может использоваться для различных типов данных, реализуя соответствующую операцию в зависимости от контекста (например, `+` используется и для чисел, и для строк).

Инструкция (statement) — это минимальная самостоятельная единица выполнения в программе. Инструкция описывает полное действие, которое должен выполнить интерпретатор. Инструкцией может быть, например, присваивание (`x = 5`), вызов функции (**print**(`x`)), условная конструкция (**if** `x > 0:`), импорт модуля (**import** `math`) и т. д.

Операнд (operand) — это объект, к которому применяется операция, реализуемая посредством оператора. Термин используется в контексте операторного синтаксиса, то есть при записи выражений, содержащих операторы. Пример: в выражении `a + b`, `a` и `b` — операнды оператора `+`.

В зависимости от числа операндов операции могут быть:

— *унарными* (один операнд), например, операция `not`;

— *бинарными* (два операнда), например, `+`;

— *тернарными* (три операнда), например, тернарный оператор **...if...else...**

Аргумент (argument) — это объект, который передается функции или методу при вызове. Термин используется в контексте вызова функции или вызова процедуры. Пример: в вызове **max**(`a`, `b`), значения `a` и `b` являются аргументами функции **max**().

Основой программного кода на любом языке программирования, в том числе, и в Python является, является *лексема* — последовательность допустимых символов языка, имеющая смысл. Последовательность и правила расположения лексем определяются синтаксисом языка.

В Python выделяют типы лексем, представленные на рис. 2.

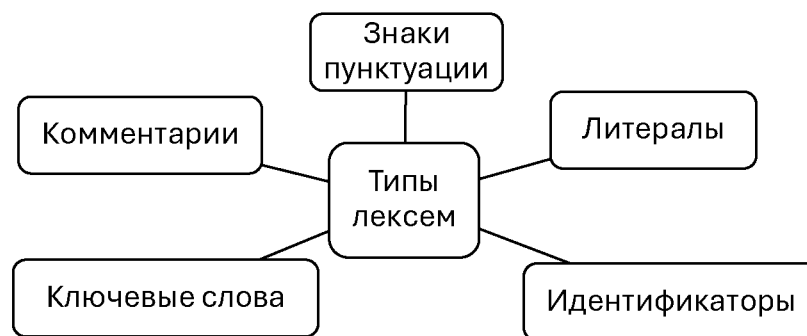


Рисунок 2

Знаки пунктуации отделяют друг от друга логические части программного кода, а так же несут различное семантическое значение в зависимости от ситуации. Например, двоеточие (:) является признаком начала вложенного блока инструкций, а символ , (запятая) выступает как в роли разделителя аргументов при вызове функций, так и в роли разделителя элементов типов-последовательностей.

Литералом считается некоторое значение или выражение, имеющее значение. Так, литералами служат целое число -3, действительное число 3.1417, строка 'primer', список ['ПН', 'ВТ', 'СР']. Говорят, что объект задается *литерально*, если его значение записано в тексте программы.

Идентификаторы (имена, *identifier*) — буквенно-числовые последовательности, которые присваиваются ссылке на некоторый объект, например, x, Price, discountedPrice. Подробнее об идентификаторах см. в п. ???.

Ключевые слова имеют специальное значение и используются только в тех случаях, для которых они определены. Их нельзя использовать в качестве идентификаторов, поэтому ключевые слова так же называют зарезервированными. Список ключевых слов можно получить при использовании модуля keyword (листинг 1.1),

Листинг 1.1

```
import keyword
print(keyword.kwlist)
```

В результате получим список ключевых слов, приведенный в листинге 1.2.

Листинг 1.2

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async',
'await', 'break', 'class', 'continue', 'def', 'del', 'elif',
'else', 'except', 'finally', 'for', 'from', 'global', 'if',
'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',
'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

Комментарии поясняют программный код, при исполнении пропускаются. Однострочный комментарий начинается с символа октоторп # («решетка», «диез», «шарп»). Все, что расположено после этого символа и до конца строки считается комментарием. Многострочные комментарии помещаются между тройными кавычками ' ' ' или " " " (листинг 1.3).

Листинг 1.3

```
# Это однострочный комментарий
a = 2 # Все, что дальше в строке, - комментарий

' ' ' Многострочный
комментарий
' ' '

" " "
Еще один
многострочный комментарий
" " "
```

1.3. Объекты

Объекты являются базовым понятием языка Python. Все, с чем имеют дело при программировании на этом языке (переменные, строки, функции и пр.), является объектами.

Под *объектом* понимается набор данных и методов (функций), взаимодействующих с этими данными.

Каждый объект является *экземпляром класса*, где *класс* понимается как общее описание данных (свойств) и операций (методов) над этими данными. Таким образом, каждый объект в Python принадлежит некоторому классу, в котором определены возможные операции над значениями объекта. Например, целые числа являются экземплярами класса `int`, строки — экземплярами класса `str` и т. д. Кроме того, сами встроенные классы, в свою очередь, относятся к классу `type`, но не являются экземплярами этого класса: в данном случае используется иной механизм объектно-ориентированного программирования.

Под *методом* понимают функцию, описанную внутри класса, которая позволяет обрабатывать объекты данного класса (выполнять операции над данными).

Под *атрибутом* в общем случае понимают характеристику, присущую всем экземплярам некоторого класса. Например, атрибутами класса `book` (книга) могут быть ISBN, автор(ы), наименование, количество страниц и т. д.

Обращаться к значениям атрибутов (получать и устанавливать значения) можно посредством специальных методов, называемых *свойствами класса*.

Замечание. Рассмотренные выше термины «метод», «атрибут» и «свойство» даны на понятийном уровне и не отражают в достаточной степени всех аспектов их применения и реализации.

Объекты в Python характеризуются как минимум следующими элементами:

- идентификатором (ID),
- типом,
- значением.

Идентификатор объекта (не путать с идентификатором-лексемой) является уникальным номером (ID), который характеризует объект. Можно считать, что идентификатор объекта является адресом объекта в памяти. Этот адрес не изменяется в течение всего времени существования объекта. Узнать ID объекта можно с помощью команды `id(name)`, где `name` — имя объекта.

Объекты в Python считаются *идентичными*, если их идентификаторы совпадают.

Тип объекта (так же говорят *тип данных объекта* или просто *тип данных*) определяет множество возможных значений объекта, набор операций над объектами и соответствующее их поведение. Тип объекта неизменен в течение всего времени существования. Узнать ID объекта можно с помощью команды `type(name)`. Результатом ее выполнения является наименование класса, к которому принадлежит объект. К слову, тип объекта так же имеет свой тип `type`, т. е. тоже является объектом.

Значение объекта определяет данные, которые хранятся по адресу ID. Возможные значения зависят от типа объекта.

Объекты называются *неизменяемыми*, если после их создания никакие операции не способны изменить их внутреннее состояние. Любая операция, предполагающая изменение, приводит к созданию нового объекта. К неизменяемым объектам относятся, например, числовые типы (`int`, `float`, `complex`), строки (`str`), кортежи (`tuple`).

Объекты называются *изменяемыми*, если их состояние может быть изменено без изменения их идентификатора в памяти. К изменяемым объектам относятся, например, списки (`list`), множества (`set`), словари (`dict`).

Замечание. В терминологии Python неизменяемые и изменяемые типы называются *немутуирующими* и *мутуирующими* соответственно (*immutable/mutable*). В русскоязычной практике обычно используют более привычные (и уже устоявшиеся) термины «неизменяемый»/«изменяемый», что не совсем корректно с точки вкладываемого в эти понятия смысла.

Пример работы с неизменяемыми и изменяемыми объектами приведен в листинге 1.4.

Листинг 1.4

```
a = 5
print(id(a)) # 140734428759080
a = a + 1
print(id(a)) # 140734428759112 (новый объект)

b = [3, 1, 2]
print(id(b)) # 2073546670336
b[1] = 9
print(b) # [3, 9, 2]
print(id(b)) # 2073546670336 (объект не изменился)
```

Замечание. Если не учитывать особенности типов объектов, и, соответственно, особенностей присвоения объектов данных типов, то можно получить некорректные и не совсем ожидаемые результаты работы программы.

Хешируемый объект — это объект, значение которого не изменяется в течение времени его существования, и который может быть использован в качестве ключа в словаре или элемента в множестве.

Хеш-значение представляет собой целое число фиксированной длины, вычисляемое по содержимому объекта и используемое в алгоритмах поиска, сопоставления и хранения данных, основанных на хеш-таблицах. Хеш-значение может быть получено с использованием функции **hash**(name), где name — имя объекта.

Формально, объект *x* является хешируемым, если:

- результат вызова **hash**(*x*) остается неизменным во времени;
- выполняется условие: если *x* == *y*, то **hash**(*x*) == **hash**(*y*).

Хешируемые объекты обладают свойством неизменяемости: внутреннее состояние объекта не должно изменяться, т. к. изменение состояния приведет к изменению хеш-значения, что нарушит корректную работу хеш-таблиц. К хешируемым объектам относят чиста, строки, логические значения, кортежи (если все элементы хешируемы).

Итерируемый объект — объект, который может последовательно предоставлять свои элементы при использовании в конструкциях, основанных на поэлементном обходе. Примеры таких конструкций: цикл **for**, функции **list()**, **tuple()**, **sum()** и др. Примеры встроенных итерируемых объектов: строки (str), списки (list), кортежи (tuple),

Сводная табл. 1.4 содержит основные типы данных с указанием их изменяемости, хешируемости и итерируемости.

Таблица 1.4

Тип данных	Наименование	Изменяемый	Хешируемый	Итерируемый
int	целое число	нет	да	нет
float	вещественное число	нет	да	нет
complex	комплексное число	нет	да	нет

Тип данных	Наименование	Изменяемый	Хешируемый	Итерируемый
bool	логический тип	нет	да	нет
str	строка	нет	да	да
tuple	кортеж	нет	да ¹	да
list	список	да	нет	да
set	множество	да	нет	да
frozenset	замороженное множество	нет	да	да
dict	словарь	да	нет	да (по ключам)
range	диапазон	нет	да	да
bytes	неизменяемая последовательность байтов	нет	да	да
bytearray	изменяемая последовательность байтов	да	нет	да

Equation Section (Next)

¹ Кортеж является хешируемым только в случае, если все его элементы являются хешируемыми

2. Вычисления. Числовые и логический типы данных Python

Цель работы: формирование практических навыков разработки программ, реализующих числовые и логические вычисления на языке программирования Python, с использованием встроенных числовых типов и логического типа, а также освоение базовых операций и выражений, связанных с обработкой числовой и булевой информации в контексте алгоритмических задач.

В Python основными типами данных для вычислений служат типы **int** (целочисленный) и **float** (вещественный или тип с плавающей точкой, плавающей запятой). Результат операций сравнения или проверки условий имеет тип **bool** (логический тип).

Стандартные математические операции приведены в табл. 2.1.

Таблица 2.1

Оператор	Описание операции	Пример выражения	Результат выполнения
+	Сложение	7 + 3	10
-	Вычитание	7 - 3	4
*	Умножение	7 * 3	21
/	Деление с плавающей точкой	7 / 3	2,333.
//	Целочисленное деление	7 // 3	2
%	Остаток от деления (деление по модулю)	7 % 3	1
**	Возведение в степень	7 ** 3	343
-x	Унарный минус	-7	-7
+x	Унарный плюс	+7	7

Все операции из табл. 2.1, кроме унарных, могут быть использованы в сокращенной записи вычислений, когда к значению переменной предполагается применить операцию и присвоить этой же переменной результат. Такая запись называется *операцией с присваиванием*. В табл. 2.2 приведены примеры подобной записи.

Таблица 2.2

Оператор	Эквивалент	Описание	Пример	Результат после исполнения
x += y	x = x + y	Сложение и присваивание	x = 5; x += 3	x == 8
x -= y	x = x - y	Вычитание и присваивание	x = 10; x -= 4	x == 6
x *= y	x = x * y	Умножение и присваивание	x = 2; x *= 4	x == 8

Оператор	Эквивалент	Описание	Пример	Результат после исполнения
$x /= y$	$x = x / y$	Деление и присваивание	$x = 9; x /= 2$	$x == 4.5$
$x /= y$	$x = x // y$	Целочисленное деление	$x = 9; x /= 2$	$x == 4$
$x \% = y$	$x = x \% y$	Остаток от деления	$x = 10; x \% = 3$	$x == 1$
$x ** = y$	$x = x ** y$	Возведение в степень	$x = 3; x ** = 2$	$x == 9$

Операторы сравнения (логические отношения) приведены в табл. 2.3.

Таблица 2.3

Оператор	Описание операции	Пример выражения	Результат выполнения
<code>==</code>	Равенство	$7 == 3$	False
<code>!=</code>	Неравенство	$7 != 3$	True
<code>></code>	Больше	$7 > 3$	True
<code><</code>	Меньше	$7 < 3$	False
<code>>=</code>	Больше или равно	$7 >= 3$	True
<code><=</code>	Меньше или равно	$7 <= 3$	False

Особенности типа **int**:

1. Тип **int** представляет знаковые целые числа произвольной длины. Ограничение на размер значения определяется доступным объемом оперативной памяти.

2. Объекты типа **int** поддерживают точные операции сложения, вычитания, умножения, целочисленного деления (`//`), остатка от деления (`%`), возведения в степень (`**`).

3. Все вычисления с объектами типа **int** являются точными. Ошибки округления отсутствуют.

4. При литеральном задании (прямой записи в тексте программы) не допускают лидирующих (начальных) нулей, т. е. запись `0034` будет ошибочной, а запись `34` будет корректной.

Особенности типа **float**:

1. Тип **float** реализует вещественные числа двойной точности и соответствует формату IEEE 754.

2. Десятичным разделителем, отделяющим целую часть от дробной, является символ `.` (точка), например, `14.186` и `0.0024`.

3. При литеральном задании значения типа **float** допускают *лидирующие* (начальные) и *закрывающие* (конечные) нули, т. е. допускается и запись `0003.400`, и запись `3.4`.

4. Вещественное число может быть задано с использованием *экспоненциальной формы записи*, при которой используется символ `e` или `E`, обозначающий основание десятичного логарифма. Например, `1.5e3` эквивалентно

$1.5 \cdot 10^3$, т. е. 1500. Разрешены положительные и отрицательные показатели степени: $2.0e-2$, $4.1E+5$. Целая и дробная часть в мантиссе (число до символа *e*) могут быть заданы аналогично обычному десятичному представлению.

5. Тип **float** допускает наличие специальных значений: положительная бесконечность `inf`, отрицательная бесконечность `-inf`, неопределенность `nan`.

6. Приблизительный диапазон абсолютных значений — от $2,2 \cdot 10^{-308}$ до $1,8 \cdot 10^{308}$. При переполнении возвращается значение `inf`, при выходе за пределы точности — `0.0`, при неопределенности — `nan`.

7. Все арифметические операции выполняются с погрешностью округления. При совместной работе с типом **int** последний неявно преобразуется в **float**.

8. Представление **float** допускает приблизительно 15–17 значащих десятичных цифр. Погрешности возникают при операциях над числами, не имеющими точного представления в двоичной системе, например, $0.1 + 0.1 + 0.1$ в Python не равно 0.3, а имеет значение 0.30000000000000004.

9. Для сравнения вещественных чисел *a* и *b* можно использовать один из следующих подходов:

- сравнение абсолютного значения разности чисел с некоторой малой величиной $\varepsilon < 10^{-14}$, например, `abs(a-b) < 10**-10`;
- применение функции `isclose(a, b)` из модуля `math` (пример приведен в листинге 2.1).

Листинг 2.1

```
from math import *

print(0.1 + 0.1 + 0.1 == 0.3) # False
print(isclose(0.1 + 0.1 + 0.1, 0.3)) # True
```

Особенности типа **bool**:

1. Тип **bool** может принимать только два значения: **True** (истина) и **False** (ложь).

2. Тип **bool** реализуется как подкласс **int**. Значение **True** соответствует числу 1, значение **False** соответствует числу 0. При использовании в арифметических выражениях **bool** ведет себя как **int**, например `True + True = 2`, `False * 10 = 0`.

3. Тип **bool** предназначен для логических операций: **and**, **or**, **not**. В логических выражениях возвращается объект **bool**.

4. В логическом контексте (**if**, **while**, **bool()** и т. п.) объекты **int**, **float** и других типов преобразуются к **bool** по следующим правилам:

— значения `0`, `0.0`, **None** и пустые последовательности интерпретируются как **False**;

— все остальные значения интерпретируются как **True**.

Логические операции Python приведены в табл. 2.4.

Таблица 2.4

Оператор	Описание	Результат (True/False)	Поведение
<code>not x</code> или <code>not(x)</code>	Логическое отрицание	True , если <code>x == False</code>	Возвращает True или False
<code>x and y</code>	Логическое И	True , если <code>x</code> и <code>y</code> — True	Возвращает <code>x</code> , если <code>x == False</code> ; иначе <code>y</code>
<code>x or y</code>	Логическое ИЛИ	True , если <code>x</code> или <code>y</code> — True	Возвращает <code>x</code> , если <code>x == True</code> ; иначе <code>y</code>

Замечание. Для вычисления выражений, содержащих другие логические операции, например, импликацию ($\rightarrow$), эквиваленцию ($\leftrightarrow$) и пр., можно воспользоваться либо формулами алгебры логики

$$x \rightarrow y \equiv \neg x \vee y, \quad (2.1)$$

$$x \leftrightarrow y \equiv (x \rightarrow y) \wedge (y \rightarrow x), \quad (2.2)$$

либо использовать операции, им эквивалентные, где в качестве аргументов выступают целочисленные значения `0` (**False**) и `1` (**True**). В табл. 2.5 приведены эквивалентные способы вычисления логических операций. В качестве операндов можно использовать как значения `0`, `1`, так и **False**, **True**.

Таблица 2.5

Логическая операция	Оригинальный оператор	Альтернативное выражение при $x, y \in \{0, 1\}$
Отрицание	<code>not x</code>	<code>1 - x</code>
Конъюнкция	<code>x and y</code>	<code>min(x, y)</code>
Дизъюнкция	<code>x or y</code>	<code>max(x, y)</code>
Импликация	$x \rightarrow y$ (в Python реализация отсутствует)	<code>x <= y</code> <code>max(1 - x, y)</code>
Эквиваленция	$x \leftrightarrow y$ (в Python реализация отсутствует)	<code>x == y</code> <code>1 - abs(x - y)</code>

При использовании операций сравнения вместо импликации и эквиваленции следует помнить, что приоритет сравнений выше (см. далее), поэтому для правильного порядка вычислений нужно использовать скобки. Например, выражение $x \rightarrow y \vee z \rightarrow x$ в Python должно быть записано как

$$(x <= (y \text{ or } z)) <= x.$$

Все три типа (**int**, **float**, **bool**) являются числовыми и совместимы в рамках системы типов Python. При выполнении операций между **int** и **float**

результатом всегда является **float**, при этом **bool** участвует как **int** с соответствующим значением 0 или 1. Неявное преобразование **int** в **float** происходит при выполнении смешанных арифметических операций с участием типов **float**. Преобразование **float** в **int** предполагает выполнение явного применения функции **int()**, при этом происходит усечение дробной части (отбрасывание без округления).

Порядок выполнения операций (*приоритет*) определяется строго регламентированной иерархией, аналогичной математическим правилам, но расширенной с учетом специфики синтаксиса и семантики языка. Порядок выполнения операций определяет, в какой последовательности выполняются операторы при отсутствии явных скобок, и влияет на результат выражения.

Операторы с высшим приоритетом выполняются раньше операторов с низшим приоритетом. При одинаковом приоритете используется правило ассоциативности, которое может быть слева направо (левоассоциативность) или справа налево (правоассоциативность). Скобки **()** изменяют стандартный порядок выполнения операций и имеют наивысший приоритет.

В табл. 2.6 представлен формализованный порядок выполнения операций в Python (от наивысшего приоритета к наименьшему).

Таблица 2.6

Приоритет	Операторы	Описание	Ассоциативность
1	(expr), [expr], {expr}, f(.), x[index], x.attr	Скалярные скобки, вызов функций, индексация, доступ к атрибутам	слева направо
2	**	Возведение в степень	справа налево
3	+x, -x	Унарный плюс, минус	справа налево
4	*, /, //, %	Умножение, деление, целочисленное деление, остаток от деления	слева направо
5	+, -	Сложение, вычитание	слева направо
6	in, not in, is, is not, <, <=, >, >=, !=, ==	Операции сравнения и принадлежности	слева направо
7	not	Логическое отрицание	справа налево
8	and	Логическое И	слева направо
9	or	Логическое ИЛИ	слева направо
10	...if... else...	Тернарный условный оператор	справа налево
11	=, +=, -=, *=, /=, //=, %=, **=	Присваивание, операторы с присваиванием	—

Особенности, связанные с приоритетом операций:

1. Скобки имеют наивысший приоритет и могут использоваться для изменения порядка выполнения любых операций.

2. Оператор `**` (возведение в степень) выполняется до унарных операций. Выражение `-2**2` интерпретируется как `-(2**2)`, результатом будет значение -4.

3. Операции присваивания имеют один из самых низких приоритетов. В выражении `x = 1 + 2 * 3` сначала выполняется умножение, затем сложение, затем присваивание.

4. Логические операторы **and**, **or**, **not** имеют более низкий приоритет по сравнению с арифметическими операциями.

5. Операции сравнения можно комбинировать в *цепочки*, например, выражение `1 < x < 10` интерпретируется как `(1 < x) and (x < 10)`.

В дополнение к встроенным операциям для расчетов используют стандартный модуль `math`. Основные функции этого модуля приведены в табл. 2.7

Таблица 2.7

Функция	Назначение	Формула/Описание
<code>math.sqrt(x)</code>	Квадратный корень	$\sqrt{x}$
<code>math.exp(x)</code>	Экспонента	e^x
<code>math.log(x)</code>	Натуральный логарифм	$\ln(x)$
<code>math.log(x, base)</code>	Логарифм по основанию base	$\log_{\text{base}}(x)$
<code>math.log10(x)</code>	Десятичный логарифм	$\log_{10}(x)$
<code>math.log2(x)</code>	Двоичный логарифм	$\log_2(x)$
<code>math.pow(x, y)</code>	Возведение в степень	x^y
<code>math.fabs(x)</code>	Абсолютное значение (модуль)	$ x $
<code>math.factorial(n)</code>	Факториал натурального числа	$n!$
<code>math.floor(x)</code>	Округление вниз до ближайшего целого	$\lfloor x \rfloor$
<code>math.ceil(x)</code>	Округление вверх до ближайшего целого	$\lceil x \rceil$
<code>math.trunc(x)</code>	Усечение дробной части	Отбрасывание после десятичной точки
<code>math.copysign(x, y)</code>	Число, равное по абсолютной величине значению x , но с тем же знаком, что и у значения y	$\begin{cases} x , y \geq 0; \\ - x , y \leq 0 \end{cases}$
<code>math.fmod(x, y)</code>	Остаток от деления по знаку делимого (в отличие от %)	$x - n \cdot y$, где $n = \text{math.trunc}(x / y)$
<code>math.isclose(a, b)</code>	Приближенное сравнение двух значений	Возвращает True или False
<code>math.isnan(x)</code>	Проверка: является ли x значением <code>nan</code>	Возвращает True или False
<code>math.isinf(x)</code>	Проверка: является ли значение x бесконечным	Возвращает True или False
<code>math.degrees(x)</code>	Преобразование радиан в градусы	$x \cdot \frac{180}{\pi}$

Функция	Назначение	Формула/Описание
<code>math.radians(x)</code>	Преобразование градусов в радианы	$x \cdot \frac{\pi}{180}$
<code>math.hypot(x, y)</code>	Евклидова норма	$\sqrt{x^2 + y^2}$
<code>math.gcd(a, b)</code>	Наибольший общий делитель (целые числа)	$НОД(a, b)$
<code>math.lcm(a, b)</code>	Наименьшее общее кратное (Python 3.9+)	$НОК(a, b)$

В табл. 2.8 приведены тригонометрические функции модуля `math`.

Таблица 2.8

Функция	Назначение	Формула
<code>math.sin(x)</code>	Синус (аргумент в радианах)	$\sin(x)$
<code>math.cos(x)</code>	Косинус (аргумент в радианах)	$\cos(x)$
<code>math.tan(x)</code>	Тангенс (аргумент в радианах)	$\tan(x)$
<code>math.asin(x)</code>	Арксинус	$\arcsin(x)$
<code>math.acos(x)</code>	Арккосинус	$\arccos(x)$
<code>math.atan(x)</code>	Арктангенс	$\arctan(x)$

Константы (постоянные значения) модуля `math` приведены в табл. 2.9.

Таблица 2.9

Имя	Значение
<code>math.pi</code>	Число $\pi \approx 3,1415926535$
<code>math.e</code>	Основание экспоненты $e \approx 2,7182818284$
<code>math.tau</code>	Полный круг $2\pi \approx 6,2831853071$
<code>math.inf</code>	Положительная бесконечность
<code>math.nan</code>	Неопределенное числовое значение

Пример 2.1

Вычислить значение выражения

$$e^{\left| \frac{\ln(x^2 + \cos^2 x)}{\arcsin\left(\frac{x}{x+3}\right)} \right|} + \sqrt{\tan\left(\frac{1}{x^2 + 1}\right)}$$

при $x = 0.2$.

Решение

Для удобства вычислений рассчитаем отдельные части выражения, после чего скомпонуем их для получения итогового значения (листинг 2.2).

Листинг 2.2

```
from math import *

x = 0.2
a = abs(log(x**2 + cos(x) ** 2) / asin(x / (x + 3)))
b = abs(tan(1 / (x**2 + 1))) ** 0.5
f = exp(a) + b
print(f) # 2.2056148191671534
```

Пример 2.2

Вычислить значение логического выражения

$$x \leftrightarrow y \rightarrow z \vee \neg x \leftrightarrow \neg(z \wedge (y \vee x)) \rightarrow z \rightarrow x \wedge y$$

для набора значений аргументов: $x = \text{True}$, $y = \text{False}$, $z = \text{False}$.

Решение

1. Проанализируем выражение. Наивысший приоритет выполнения в соответствии с табл. 2.6 имеют фрагменты $\neg(z \wedge (y \vee x))$, $x \wedge y$ и $z \vee \neg x$. Перепишем выражение, заключив эти фрагменты в скобки:

$$x \leftrightarrow y \rightarrow (z \vee \neg x) \leftrightarrow (\neg(z \wedge (y \vee x))) \rightarrow z \rightarrow (x \wedge y).$$

2. Операция импликации в математике имеет более высокий приоритет, чем эквиваленция, поэтому расставим дополнительные скобки следующим образом:

$$x \leftrightarrow (y \rightarrow (z \vee \neg x)) \leftrightarrow (((\neg(z \wedge (y \vee x))) \rightarrow z) \rightarrow (x \wedge y)).$$

3. Оставшиеся две операции эквиваленции выполняются последовательно, поэтому добавим соответствующие скобки:

$$(x \leftrightarrow (y \rightarrow (z \vee \neg x))) \leftrightarrow (((\neg(z \wedge (y \vee x))) \rightarrow z) \rightarrow (x \wedge y)).$$

4. Подставим соответствующие операторы Python, в частности, из табл. 2.5:

$$(x == (y <= (z \text{ or } \text{not}(x)))) == ((\text{not}(z \text{ and } (y \text{ or } x)) <= z) <= (x \text{ and } y))$$

5. Сформируем подпрограмму (см. п. 4) вычислений для выражения и выведем результат выполнения функции с аргументами (**True**, **False**, **False**) (листинг 2.3).

Листинг 2.3

```
def f(x, y, z):
    return (x == (y <= (z or not(x)))) == ((not(z and (y or x))
    <= z) <= (x and y))

print(f(True, False, False)) # True
```

Задание 2.1

Вычислить значение указанной функции при $x = 0.2$.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результат вычислений.

Таблица 2.10

№	$f(x)$	Контрольное значение
1	$\frac{1}{\sqrt[3]{\ln\left(\frac{x^2+1}{\sin x+2}\right)}} + \arccos\left(\frac{2}{x^2+4}\right)$	2.1542171303
2	$e^{\left(\frac{\arctan(\sqrt{x^2+3})}{\ln(x^2+1)}\right)} + \left \cos\left(\frac{1}{x+2}\right)\right $	424046954383.4161987305
3	$\sqrt{\frac{\ln(\tan(x) +1)}{\arcsin\left(\frac{x}{\sqrt{x^2+1}}\right)}} + e^{-\frac{1}{x^2+1}}$	1.3492910625
4	$\frac{1}{x+1} \cdot \arcsin\left(\frac{\sqrt{ \cos(x) }}{x+2}\right) + \ln(1+x^4)$	0.3905626774
5	$\frac{\sqrt[4]{e^{ x-\sin x }}}{\arctan\left(\frac{2x}{x^2+1}\right)} + \left \frac{1}{x} - \ln(x+1)\right $	7.5420902354
6	$\frac{1}{\sqrt{\arcsin\left(\frac{2x}{3+x^2}\right)}} + \ln(\tan^2 x + 4)$ $\sqrt{\arcsin\left(\frac{2x}{3+x^2}\right)} + e^{-x}$	2.4221194831
7	$\sqrt[3]{\frac{\ln(\sqrt{x^2+1}+2)}{\cos\left(\frac{\pi x}{x^2+2}\right)}} + \arccos\left(\frac{3}{x^2+4}\right)$	1.7845381870
8	$\left \arctan\left(\frac{\sqrt[3]{x^2+1}}{\ln(x^2+3)}\right)\right + e^{-\frac{1}{ x+1 }}$	1.1735839262
9	$\frac{\ln\left(\left \sin\left(\frac{1}{x^2+1}\right)\right + e^x\right)}{\sqrt[4]{x^2 + \cos^2 x}} + \arcsin\left(\frac{x}{x+5}\right)$	0.7520493347

№	$f(x)$	Контрольное значение
10	$e^{\left(\frac{\arcsin\left(\frac{x}{\sqrt{5+x^2}}\right)}{\sqrt{1+ \tan x }}\right)} + \ln\left(\left x^2 - \frac{1}{2}\right + 1\right)$	1.4631773477
11	$\frac{1}{\sqrt{\ln(\tan^2 x + x^2 + 1)}} + \left \arccos\left(\frac{2}{x^2 + 2}\right) - \frac{1}{x+3}\right $	3.6953845502
12	$\frac{\ln(e^{x^2} + \cos x)}{\sqrt[5]{\arctan^2 x + 1}} + \arcsin\left(\frac{1}{x^2 + 1}\right)$	1.9907232562
13	$\left \frac{\arccos\left(\frac{x^2}{x^2 + 4}\right)}{\ln(x^2 + 1)}\right + \sqrt{\frac{1}{x+1}} + \tan x $	40.8155876290
14	$e^{\left \frac{1}{x} - \ln(\sin x + 2)\right } + \sqrt[3]{\arctan\left(\frac{x}{x^2 + 2}\right)}$	67.9619700086
15	$\sqrt{\left \arcsin\left(\frac{x}{\sqrt{x^2 + 3}}\right) \cdot \ln(x^2 + 4)\right } + \frac{1}{e^{x^2 + 1}}$	0.7540961364
16	$\frac{1}{x^2 + 1} + \left \arctan\left(\sqrt{ \cos x + \frac{1}{2}}\right)\right + \ln(x^2 + 3)$	2.9561939950
17	$\frac{e^{-\left x - \frac{1}{x}\right }}{\arcsin\left(\frac{x}{\sqrt{x^2 + 2}}\right) + \ln(x^2 + 1)}$	0.0457944914
18	$\sqrt[3]{\left \frac{1}{\ln(\tan^2 x + 2)} - \arccos\left(\frac{x}{x+1}\right)\right } + \frac{1}{x^2 + 1}$	1.0826482560
19	$\left \frac{\arcsin\left(\frac{1}{x+2}\right) \cdot \ln(x^2 + 1)}{e^{ x }}\right + \frac{1}{\sqrt{x^2 + 4}}$	0.5126706468
20	$\ln\left(\left \cos\left(\frac{1}{x^2 + 2}\right)\right + \arctan(\sqrt{x^2 + 1})\right) + \frac{1}{x+3}$	0.8297707743
21	$e^{-\arctan\left(\frac{1}{x} - \cos x\right)} + \sqrt[5]{\ln(x^2 + 3)}$	1.2867087962
22	$\frac{1}{\sqrt[3]{\arccos\left(\frac{2x}{x^2 + 3}\right) + \ln(\tan x + 2)}} + e^{-x^2}$	1.7263759679

№	$f(x)$	Контрольное значение
23	$\left \frac{1}{\ln(\sqrt{1+x^2} + \cos^2 x)} + \arcsin\left(\frac{x}{5}\right) \right $	1.5035701241
24	$\sqrt{\frac{\arccos\left(\frac{1}{x+2}\right)}{\sqrt{x^2+1}} + \ln(x+3)}$	1.4969117322
25	$\frac{e^{\ln(x +1)}}{\sqrt[3]{x^2 + \arctan^2 x}} + \left \cos\left(\frac{1}{x+1}\right) \right $	3.4694802131
26	$\sqrt[4]{\arcsin\left(\frac{x^2}{x^2+4}\right) \cdot \ln(x^2+2)} + \frac{1}{e^{x+1}}$	0.5910527981
27	$\left \frac{1}{x+1} + \frac{\ln(\sqrt{x^2+3}+1)}{\arccos\left(\frac{2}{x^2+3}\right)} \right $	2.0168278256
28	$\frac{1}{\ln(\sin^2 x - \cos^2 x + 3)} + \sqrt[5]{\arctan\left(\frac{x}{2}\right)}$	1.3624089781
29	$e^{\left(\frac{\arcsin\left(\frac{x}{x+3}\right)}{\ln(x^2+1)}\right)} + \left \frac{1}{x+2} - \tan x \right $	5.1781197736
30	$\sqrt{\frac{\ln(\tan x + 2)}{e^x} + \arccos\left(\frac{x}{\sqrt{x^2+1}}\right)}$	1.4212469790

Задание 2.2

1. Вычислить значение логической функции для указанного набора значений аргументов в соответствии с табл. 2.11.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результат вычислений.

Таблица 2.11

№	Выражение	Наборы аргументов
1	$x \rightarrow \neg y \leftrightarrow z \vee \neg(x \wedge z) \rightarrow y \rightarrow \neg(x \vee y)$	$x = True, y = False, z = True$; $x = False, y = False, z = True$

№	Выражение	Наборы аргументов
2	$z \leftrightarrow x \rightarrow \neg y \vee (y \wedge z) \leftrightarrow \neg(x \vee y) \rightarrow x$	$x = \text{False}, y = \text{False}, z = \text{True};$ $x = \text{False}, y = \text{True}, z = \text{True}$
3	$y \rightarrow z \leftrightarrow \neg x \vee x \rightarrow \neg(y \wedge z) \leftrightarrow z \vee x$	$x = \text{False}, y = \text{False}, z = \text{False};$ $x = \text{True}, y = \text{True}, z = \text{False}$
4	$x \wedge y \rightarrow \neg z \leftrightarrow x \vee y \rightarrow \neg(x \wedge z) \leftrightarrow y$	$x = \text{True}, y = \text{True}, z = \text{False};$ $x = \text{False}, y = \text{True}, z = \text{False}$
5	$\neg x \vee y \leftrightarrow z \wedge (x \rightarrow y) \rightarrow \neg z \leftrightarrow y \wedge z$	$x = \text{True}, y = \text{True}, z = \text{True};$ $x = \text{True}, y = \text{False}, z = \text{False}$
6	$x \rightarrow (y \vee z) \leftrightarrow \neg(x \wedge y) \rightarrow z \leftrightarrow \neg(y \vee z)$	$x = \text{True}, y = \text{False}, z = \text{True};$ $x = \text{False}, y = \text{False}, z = \text{True}$
7	$(x \vee y) \leftrightarrow \neg z \rightarrow y \wedge \neg x \leftrightarrow z \rightarrow x \vee y$	$x = \text{False}, y = \text{False}, z = \text{True};$ $x = \text{False}, y = \text{True}, z = \text{True}$
8	$\neg(x \vee z) \rightarrow y \leftrightarrow x \wedge y \rightarrow \neg z \leftrightarrow x \wedge z$	$x = \text{False}, y = \text{False}, z = \text{False};$ $x = \text{True}, y = \text{True}, z = \text{False}$
9	$y \leftrightarrow z \rightarrow x \wedge \neg y \leftrightarrow \neg(z \vee x) \rightarrow y \vee z$	$x = \text{True}, y = \text{True}, z = \text{False};$ $x = \text{False}, y = \text{True}, z = \text{False}$
10	$(x \rightarrow z) \vee (y \leftrightarrow \neg x) \rightarrow \neg y \leftrightarrow z \wedge x$	$x = \text{True}, y = \text{True}, z = \text{True};$ $x = \text{True}, y = \text{False}, z = \text{False}$
11	$\neg(x \rightarrow y) \leftrightarrow z \vee (y \rightarrow \neg z) \rightarrow x \leftrightarrow y$	$x = \text{True}, y = \text{False}, z = \text{True};$ $x = \text{False}, y = \text{False}, z = \text{True}$
12	$(z \wedge x) \leftrightarrow \neg y \rightarrow x \vee z \leftrightarrow \neg(y \wedge x)$	$x = \text{False}, y = \text{False}, z = \text{True};$ $x = \text{False}, y = \text{True}, z = \text{True}$
13	$x \vee \neg y \leftrightarrow z \rightarrow \neg(x \vee z) \leftrightarrow y \rightarrow z$	$x = \text{False}, y = \text{False}, z = \text{False};$ $x = \text{True}, y = \text{True}, z = \text{False}$
14	$\neg(x \wedge y) \leftrightarrow z \vee x \rightarrow \neg y \leftrightarrow z \wedge x$	$x = \text{True}, y = \text{True}, z = \text{False};$ $x = \text{False}, y = \text{True}, z = \text{False}$
15	$y \wedge z \rightarrow \neg x \leftrightarrow x \vee \neg y \rightarrow z \leftrightarrow \neg x$	$x = \text{True}, y = \text{True}, z = \text{True};$ $x = \text{True}, y = \text{False}, z = \text{False}$
16	$(\neg x \vee y) \leftrightarrow z \rightarrow y \leftrightarrow \neg(z \wedge x) \rightarrow y$	$x = \text{True}, y = \text{False}, z = \text{True};$ $x = \text{False}, y = \text{False}, z = \text{True}$
17	$x \leftrightarrow y \vee z \rightarrow \neg x \wedge z \leftrightarrow y \rightarrow \neg(x \vee y)$	$x = \text{False}, y = \text{False}, z = \text{True};$ $x = \text{False}, y = \text{True}, z = \text{True}$
18	$\neg z \leftrightarrow x \wedge (y \rightarrow \neg x) \rightarrow z \leftrightarrow y \vee x$	$x = \text{False}, y = \text{False}, z = \text{False};$ $x = \text{True}, y = \text{True}, z = \text{False}$
19	$x \vee y \rightarrow z \leftrightarrow \neg(x \wedge y) \rightarrow \neg z \leftrightarrow x \rightarrow y$	$x = \text{True}, y = \text{True}, z = \text{False};$ $x = \text{False}, y = \text{True}, z = \text{False}$
20	$(\neg x \rightarrow y) \leftrightarrow z \wedge x \rightarrow y \leftrightarrow \neg(z \vee x)$	$x = \text{True}, y = \text{True}, z = \text{True};$ $x = \text{True}, y = \text{False}, z = \text{False}$
21	$z \rightarrow \neg x \leftrightarrow y \wedge x \rightarrow \neg(y \vee z) \leftrightarrow x \vee y$	$x = \text{True}, y = \text{False}, z = \text{True};$ $x = \text{False}, y = \text{False}, z = \text{True}$
22	$\neg y \rightarrow z \leftrightarrow x \vee \neg z \rightarrow y \leftrightarrow \neg(x \wedge z)$	$x = \text{False}, y = \text{False}, z = \text{True};$ $x = \text{False}, y = \text{True}, z = \text{True}$

№	Выражение	Наборы аргументов
23	$x \wedge z \rightarrow \neg y \leftrightarrow \neg(x \vee y) \rightarrow z \leftrightarrow y$	$x = False, y = False, z = False;$ $x = True, y = True, z = False$
24	$(y \rightarrow x) \vee (z \leftrightarrow \neg y) \rightarrow \neg x \leftrightarrow z \wedge x$	$x = True, y = True, z = False;$ $x = False, y = True, z = False$
25	$\neg(x \vee z) \leftrightarrow y \rightarrow \neg z \vee x \leftrightarrow y \rightarrow x$	$x = True, y = True, z = True;$ $x = True, y = False, z = False$
26	$x \leftrightarrow \neg y \rightarrow z \wedge (x \rightarrow \neg z) \leftrightarrow y \vee x$	$x = True, y = False, z = True;$ $x = False, y = False, z = True$
27	$(x \wedge y) \leftrightarrow \neg z \rightarrow \neg(x \vee y) \leftrightarrow z \rightarrow x$	$x = False, y = False, z = True;$ $x = False, y = True, z = True$
28	$z \vee x \leftrightarrow \neg y \rightarrow \neg(z \wedge x) \leftrightarrow y \rightarrow x$	$x = False, y = False, z = False;$ $x = True, y = True, z = False$
29	$\neg y \vee x \leftrightarrow z \rightarrow x \wedge \neg z \leftrightarrow y \vee z$	$x = True, y = True, z = False;$ $x = False, y = True, z = False$
30	$(x \vee \neg y) \leftrightarrow z \rightarrow \neg x \rightarrow y \leftrightarrow \neg(z \wedge y)$	$x = True, y = True, z = True;$ $x = True, y = False, z = False$

Equation Section (Next)

3. Основные алгоритмические структуры

3.1. Условная инструкция

Цель работы: формирование практических навыков разработки программ с разветвляющейся логикой на языке программирования Python, основанных на использовании условной инструкции **if**, ее формы с альтернативой (**if-else**) и множественным выбором (**if-elif-else**), тернарного оператора условия для реализации алгоритмов, предполагающих проверку условий и принятие решений в зависимости от результатов значений логических выражений.

В языке программирования Python *условная инструкция (ветвление, развилка)* реализует выполнение части программного кода в зависимости от истинности логического выражения. Синтаксис условной инструкции представлен в двух формах:

- полноценная условная инструкция (блочный синтаксис),
 - тернарный условный оператор (выражение в одной строке).
- Полноценная условная инструкция имеет вид

```
if <условие1>:  
    <блок_инструкций_1>  
elif <условие2>:  
    <блок_инструкций_2>  
...  
else:  
    <блок_инструкций_N>
```

Пояснения:

1. Каждое <условие> должно быть выражением, результат которого имеет логический тип (**True** или **False**).
 2. Выполнение конструкции начинается с проверки условия в **if**. Если оно истинно, выполняется соответствующий блок, и конструкция завершается.
 3. Если первое условие ложно, последовательно проверяются условия в **elif**.
 4. Количество блоков **elif** может быть произвольным; блок **else** не обязателен.
 5. Если ни одно из условий не выполняется, выполняется блок в **else** (если он указан).
 6. Все блоки должны быть правильно отформатированы с отступом.
- Пример использования приведен в листинге 3.1.

Листинг 3.1

```
x = 5

if x < 0:
    print («Отрицательное число»)
elif x == 0:
    print («Ноль»)
else:
    print («Положительное число»)
```

Тернарный условный оператор возвращает одно из двух значений в зависимости от условия. Синтаксис:

<выражение1> **if** <условие> **else** <выражение2>.

Используется в контексте выражений, где предполагается вычислить значение на основе логического условия, *оформляется как выражение, а не как инструкция*. Пример использования приведен в листинге 3.2.

Листинг 3.2

```
x = 10
result = «четное» if x % 2 == 0 else «нечетное»
print(result) # четное
```

В качестве условия может быть использовано любое выражение, но его результат будет приведен к логическому типу с помощью стандартных правил приведения: ноль, пустая строка, пустая последовательность, значение **None** интерпретируются как **False**, остальные значения как **True**.

Вложенные конструкции в полноценной условной конструкции допустимы, но требуют строгого соблюдения правил отступов. Вложенные конструкции в тернарном операторе так же допустимы, но это приводит к сильному ухудшению читаемости кода.

Пример 3.1

В одном вузе используется следующая шкала перевода баллов в оценки: от 0 до 50 баллов включительно — оценка «неудовлетворительно», от 51 до 70 баллов — оценка «удовлетворительно», от 71 до 85 баллов включительно — оценка «хорошо», от 86 до 100 баллов включительно — оценка «отлично». Написать программу, запрашивающую оценку в баллах и вывести на экран соответствующую оценку по качественной шкале.

Решение

Программный код решения задания приведен в листинге 3.3.

Листинг 3.3

```
n = int(input("Введите оценку в баллах: "))

if 0 <= n <= 50:
    print("Неудовлетворительно")
elif 51 <= n <= 70:
    print("Удовлетворительно")
elif 71 <= n <= 85:
    print("Хорошо")
elif 96 <= n <= 100:
    print("Отлично")
else:
    print("Невозможно перевести баллы")
```

Задание 3.1 (практико-ориентированное задание)

Выполнить задания, представленные в табл. 3.1.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результат.

Таблица 3.1

№	Задание (блочный оператор условия)	Задания (тернарный оператор условия)
1	Написать программу, запрашивающую возраст участника и время пробега дистанции 100 м в секундах, и выводящую одну из категорий: «Юниорский рекорд» (если возраст меньше 18 лет и время менее 12,0 с), «Рекордная скорость» (если возраст от 18 до 35 лет и время менее 11,0 с), «Высокая скорость для возраста» (если возраст более 35 лет и время менее 13,0 с), либо «Средний результат» во всех остальных случаях	Написать программу, запрашивающую значение температуры в градусах Цельсия и выводящую строку «ниже нуля», если значение меньше 0, либо «выше нуля» в противном случае
		Написать программу, запрашивающую возраст пользователя и выводящую строку «несовершеннолетний», если возраст меньше 18 лет, либо «совершеннолетний» в противном случае
2	Написать программу, запрашивающую массу изделия в граммах и его габарит в сантиметрах, и выводящую одну из категорий: «Мелкий стандарт» (если масса менее 100 г и размер менее 10 см), «Средний стандарт» (если масса от 100 до 500 г и размер до 30 см), «Проверка отклонений» (если масса более 500 г или размер более 30 см), либо «Несоответствие» во всех остальных случаях	Написать программу, запрашивающую имя пользователя и флаг администратора (ввод: 1 или 0) и выводящую строку «доступ разрешен», если пользователь является администратором, либо «доступ запрещен» в противном случае
		Написать программу, запрашивающую целое число и выводящую строку «нулевое», если значение равно нулю, либо «ненулевое» в противном случае

№	Задание (блочный оператор условия)	Задания (тернарный оператор условия)
3	Написать программу, запрашивающую количество совершенных заказов и средний чек в рублях, и выводящую одну из категорий: «Премиум клиент» (если заказов более 100 и средний чек более 1000), «Активный клиент» (если заказов от 30 до 100 и средний чек более 500), «Обычный клиент» (если заказов менее 30 и средний чек менее 500), либо «Анализ не завершен» в остальных случаях	Написать программу, запрашивающую оценку по 100-балльной шкале и выводящую строку «зачет», если балл не менее 60, либо «незачет» в противном случае
		Написать программу, запрашивающую число и выводящую строку «положительное», если число больше нуля, либо «отрицательное или ноль» в противном случае
4	Написать программу, запрашивающую возраст человека и наличие хронического заболевания (ввод: «да» или «нет»), и выводящую одну из категорий: «Высокий риск» (если возраст больше 60 и заболевание имеется), «Низкий риск» (если возраст до 60 и заболевания нет), «Средний риск» во всех остальных случаях	Написать программу, запрашивающую количество баллов и выводящую строку «бонус начислен», если баллов больше 90, либо «бонус не начислен» в противном случае
		Написать программу, запрашивающую количество часов и выводящую строку «рабочее время», если значение от 9 до 18 включительно, либо «внерабочее время» в противном случае
5	Написать программу, запрашивающую уровень влажности почвы в процентах и среднесуточную температуру воздуха в градусах Цельсия, и выводящую оценку условий: «Низкий урожай» (если влажность менее 30% и температура менее 10°C), «Оптимальные условия» (если влажность более 70% и температура выше 20°C), либо «Условия нестабильны» во всех остальных случаях	Написать программу, запрашивающую число и выводящую строку «кратно 3», если число делится на 3 без остатка, либо «не кратно 3» в противном случае
		Написать программу, запрашивающую цену товара и выводящую строку «доступно», если цена меньше 1000, либо «дорого» в противном случае
6	Написать программу, запрашивающую количество анкет, пройденных пользователем, и среднюю оценку его ответов, и выводящую одну из категорий: «Эксперт» (если анкет больше 50 и средняя оценка выше 4,5), «Опытный» (если анкет от 20 до 50 и средняя оценка выше 4,0), «Новичок» (если анкет меньше 20 или средняя оценка ниже 3,5), либо «Промежуточный уровень» в остальных случаях	Написать программу, запрашивающую длину отрезка и выводящую строку «короткий», если длина менее 5, либо «длинный» в противном случае
		Написать программу, запрашивающую логическое значение в виде целого числа (1 или 0) и выводящую строку «истина», если значение равно 1, либо «ложь» в противном случае
7	Написать программу, запрашивающую сумму чека в рублях и время ожидания в минутах, и выводящую одну из рекомендаций: «Выдать VIP-бонус»	Написать программу, запрашивающую возраст и выводящую строку «детский билет», если возраст менее 12, либо «взрослый билет» в противном случае

№	Задание (блочный оператор условия)	Задания (тернарный оператор условия)
	(если чек более 2000 и ожидание менее 15 минут), «Оставить негативный отзыв» (если чек менее 500 и ожидание более 30 минут), либо «Обычный клиент» в остальных случаях	Написать программу, запрашивающую число и выводящую строку «отрицательное», если оно меньше 0, либо «неотрицательное» в противном случае
8	Написать программу, запрашивающую температуру оборудования в градусах Цельсия и количество перезагрузок в сутки, и выводящую одну из категорий: «Критическое состояние» (если температура выше 90°C и перезагрузок более 5), «Рабочее состояние» (если температура в пределах от 70 до 90°C и перезагрузок не более 3), либо «Предполагается наблюдение» во всех остальных случаях	Написать программу, запрашивающую два числа и выводящую строку «равны», если числа равны, либо «не равны» в противном случае
		Написать программу, запрашивающую количество шагов за день и выводящую строку «цель достигнута», если шагов не менее 10000, либо «недостаточно шагов» в противном случае
9	Написать программу, запрашивающую текущую дату, дату окончания срока хранения и температуру хранения в градусах Цельсия, и выводящую одну из категорий: «Немедленно утилизировать» (если срок хранения истек и температура выше 5°C), «Годен» (если срок не истек и температура в норме), либо «Проверка вручную» в остальных случаях	Написать программу, запрашивающую длину строки и выводящую строку «короткая», если длина менее 10 символов, либо «длинная» в противном случае
		Написать программу, запрашивающую число и выводящую строку «делится на 5», если число кратно 5, либо «не делится на 5» в противном случае
10	Написать программу, запрашивающую баллы за теоретический и практический этап экзамена, и выводящую итоговую оценку: «отлично» (если оба результата выше 85), «хорошо» (если один результат выше 85, а другой выше 70), «удовлетворительно» (если оба выше 50), либо «неудовлетворительно» во всех остальных случаях	Написать программу, запрашивающую вещественное число и выводящую строку «в пределах нормы», если оно находится в интервале от 10 до 20 включительно, либо «вне нормы» в противном случае
		Написать программу, запрашивающую оценку по пятибалльной шкале и выводящую строку «положительная», если оценка не менее 3, либо «отрицательная» в противном случае
11	Написать программу, запрашивающую три значения: корректность пароля (ввод: «да» или «нет»), наличие двухфакторной аутентификации («да» или «нет») и количество дней с момента последнего входа, и выводящую одну из категорий: «Доступ разрешен» (если все параметры в норме), «Проверка предполагается» (если нарушен ровно один критерий), либо «Доступ запрещен» (если нарушены два и более)	Написать программу, запрашивающую текущий час в 24-часовом формате и выводящую строку «день», если час от 6 до 17 включительно, либо «ночь» в противном случае
		Написать программу, запрашивающую два целых числа и выводящую строку «первое больше», если первое число больше второго, либо «второе больше или равно» в противном случае

№	Задание (блочный оператор условия)	Задания (тернарный оператор условия)
12	Написать программу, запрашивающую номер месяца (от 1 до 12) и количество покупок, и выводящую одну из скидок: «Зимняя премиальная скидка» (если месяц декабрь и покупок более 10), «Сезонная скидка» (если месяц январь или февраль и покупок более 5), либо «Обычная скидка» во всех остальных случаях	Написать программу, запрашивающую количество литров топлива в баке и выводящую строку «топлива достаточно», если значение больше 10, либо «предполагается дозаправка» в противном случае
		Написать программу, запрашивающую количество часов сна и выводящую строку «норма соблюдена», если количество часов от 7 до 9 включительно, либо «нарушение режима» в противном случае
13	Написать программу, запрашивающую оценки по трем дисциплинам, и выводящую одну из категорий: «Отличник» (если все три оценки выше 80), «Хорошист» (если хотя бы две оценки выше 60), «С трудом сдал» (если только одна выше 50), либо «Не сдал» (если ни одна из оценок не превышает 50)	Написать программу, запрашивающую количество страниц, прочитанных за день, и выводящую строку «прогресс есть», если страниц не менее 30, либо «чтение недостаточно» в противном случае
		Написать программу, запрашивающую результат теста в процентах и выводящую строку «успешно», если процент не менее 70, либо «неудача» в противном случае
14	Написать программу, запрашивающую скорость автомобиля в км/ч и текущие погодные условия (ввод: «ясно», «дождь» или «снег»), и выводящую одно из заключений: «Повышенная опасность» (если скорость выше 100 и снег), «Контролируемое движение» (если скорость ниже 60 и ясно), либо «Оценка ситуации» в остальных случаях	Написать программу, запрашивающую значение уровня заряда аккумулятора в процентах и выводящую строку «заряд критический», если уровень ниже 15%, либо «заряд нормальный» в противном случае
		Написать программу, запрашивающую два целых числа и выводящую строку «делится нацело», если первое число делится на второе без остатка, либо «делится с остатком» в противном случае
15	Написать программу, запрашивающую остаток срока хранения в днях и температуру хранения, и выводящую одно из заключений: «Тревога» (если срок менее 3 дней и температура выше нормы), «Стабильно» (если срок более 7 дней и температура в норме), либо «Контроль» во всех остальных случаях	Написать программу, запрашивающую длину стороны квадрата в сантиметрах и выводящую строку «меньше 10 см», если длина стороны менее 10, либо «10 см и более» в противном случае
		Написать программу, запрашивающую число и выводящую строку «делится на 4», если число делится на 4 без остатка, либо «не делится на 4» в противном случае
16	Написать программу, запрашивающую количество публикаций и среднюю вовлеченность (в процентах), и выводящую одну из категорий: «Влиятельный автор» (если публикаций более 100 и вовлеченность выше 30%),	Написать программу, запрашивающую количество введенных пользователем символов и выводящую строку «короткий ввод», если длина менее 20 символов, либо «длинный ввод» в противном случае

№	Задание (блочный оператор условия)	Задания (тернарный оператор условия)
	«Активный участник» (если публикаций от 30 до 100 и вовлеченность выше 15%), либо «Обычный пользователь» в остальных случаях	Написать программу, запрашивающую количество пропущенных занятий и выводящую строку «посещение удовлетворительное», если пропущено не более 3 занятий, либо «неудовлетворительное посещение» в противном случае
17	Написать программу, запрашивающую температуру внутри помещения и наружную температуру воздуха, и выводящую необходимый режим отопления: «интенсивный режим» (если внутри менее 18°C и снаружи менее 0°C), «стабильный режим» (если внутри от 18°C до 22°C и снаружи от 0°C до 10°C), либо «отопление выключено» (если внутри выше 23°C)	Написать программу, запрашивающую массу груза в килограммах и выводящую строку «допустимый груз», если масса не превышает 500 кг, либо «превышение нормы» в противном случае Написать программу, запрашивающую текущее количество подключенных пользователей и выводящую строку «перегрузка сервера», если число больше 1000, либо «нагрузка в норме» в противном случае
18	Написать программу, запрашивающую процент выполненных заданий и среднюю оценку за курс, и выводящую результат: «Курс освоен» (если выполнено более 90% и оценка выше 80), «Пройдено с замечаниями» (если выполнено более 70% и оценка выше 60), либо «Не пройдено» в остальных случаях	Написать программу, запрашивающую текущую температуру воды и выводящую строку «вода кипит», если температура 100°C или выше, либо «вода не кипит» в противном случае Написать программу, запрашивающую скорость движения объекта и выводящую строку «превышение скорости», если значение выше 90 км/ч, либо «скорость допустима» в противном случае
19	Написать программу, запрашивающую значения активной мощности (в кВт) и напряжения (в В), и выводящую одну из категорий: «Пик нагрузки» (если мощность выше 1000 и напряжение более 230 В), «Норма» (если мощность от 500 до 1000 и напряжение стабильно), либо «Отклонение» во всех остальных случаях	Написать программу, запрашивающую объем используемой оперативной памяти в мегабайтах и выводящую строку «недостаток памяти», если значение превышает 8000 МБ, либо «памяти достаточно» в противном случае Написать программу, запрашивающую количество ошибок в отчете и выводящую строку «проверку не прошел», если ошибок более 3, либо «проверку прошел» в противном случае
20	Написать программу, запрашивающую текущий день недели, время входа (в формате HH:MM) и наличие карты доступа (ввод: «да» или «нет»), и выводящую одно из решений: «Разрешить вход» (если день рабочий, время от 08:00 до 20:00 и карта есть), «Ограничение по времени» (если рабочий день, но вход вне допустимого времени), либо «Запретить» в остальных случаях	Написать программу, запрашивающую количество дней задержки выполнения проекта и выводящую строку «штраф начислен», если задержка более 5 дней, либо «штраф не начислен» в противном случае Написать программу, запрашивающую число и выводящую строку «однозначное», если модуль числа меньше 10, либо «многозначное» в противном случае

№	Задание (блочный оператор условия)	Задания (тернарный оператор условия)
21	Написать программу, запрашивающую массу тела в килограммах и рост в сантиметрах, вычисляющую индекс массы тела по формуле $I = \frac{m}{(h/100)^2}$, где m — масса, h — рост, и выводящую одну из категорий: «дефицит массы» (если $I < 18,5$), «норма» (если $18,5 \leq I \leq 24,9$), «избыточная масса» (если $25 \leq I \leq 29,9$), либо «ожирение» (если $I \geq 30$)	Написать программу, запрашивающую дату и выводящую строку «праздничный день», если дата равна 1 января, либо «рабочий день» в противном случае
		Написать программу, запрашивающую количество часов, отработанных сотрудником за месяц, и выводящую строку «переработка», если значение превышает 160 часов, либо «в пределах нормы» в противном случае
22	Написать программу, запрашивающую сумму ежемесячного дохода и обязательные ежемесячные расходы, вычисляющую остаток и выводящую финансовую категорию: «сбережения невозможны» (если остаток меньше 0), «ноль» (если остаток равен 0), «возможны накопления» (если остаток положителен и менее 30% дохода), либо «устойчивое финансовое положение» (если остаток превышает 30% дохода)	Написать программу, запрашивающую температуру и выводящую строку «отопление включено», если значение ниже 18°C, либо «отопление отключено» в противном случае
		Написать программу, запрашивающую количество голосов, поданных «за», и общее число голосов, и выводящую строку «решение принято», если число «за» превышает половину от общего числа, либо «решение отклонено» в противном случае
23	Написать программу, запрашивающую баллы по трем модулям курса и выводящую статус: «зачет» (если все баллы не менее 50 и хотя бы один не менее 70), «условный зачет» (если два балла не менее 50), либо «незачет» в остальных случаях	Написать программу, запрашивающую массу упаковки и выводящую строку «легкая упаковка», если масса менее 1 кг, либо «тяжелая упаковка» в противном случае. Решение реализовать тернарным оператором
		Написать программу, запрашивающую текущую температуру наружного воздуха и выводящую строку «рекомендуется головной убор», если температура менее 5°C, либо «можно обойтись без головного убора» в противном случае
24	Написать программу, запрашивающую три значения: текущую температуру, уровень влажности и скорость ветра, и выводящую прогноз: «высокий риск простуды» (если температура ниже 0°C, влажность выше 80% и ветер более 10 м/с), «прохладно» (если температура от 0°C до 10°C), либо «комфортные условия» в остальных случаях	Написать программу, запрашивающую объем багажника и объем багажа, и выводящую строку «вместилось», если багаж меньше или равен объему багажника, либо «не вмещается» в противном случае
		Написать программу, запрашивающую количество символов в пароле и выводящую строку «пароль короткий», если длина менее 8, либо «пароль допустим» в противном случае

№	Задание (блочный оператор условия)	Задания (тернарный оператор условия)
25	Написать программу, запрашивающую длительность непрерывной работы за компьютером в минутах и возраст пользователя, и выводящую рекомендацию: «необходимо срочно прерваться» (если возраст менее 18 лет и длительность более 60 минут, либо возраст 18 и более лет и длительность более 120 минут), «перерыв рекомендован» (если длительность превышает 30 минут), либо «продолжайте работу» в остальных случаях	Написать программу, запрашивающую два числа и выводящую строку «оба положительные», если оба значения положительные, либо «не оба положительные» в противном случае
		Написать программу, запрашивающую ширину и высоту экрана и выводящую строку «экран широкоформатный», если отношение ширины к высоте превышает 1,6, либо «экран стандартный» в противном случае
26	Написать программу, запрашивающую дату и текущее время, определяющую, является ли текущий момент рабочим (будний день с 9:00 до 18:00), и выводящую: «рабочее время» (если условие выполняется), «вне рабочего времени» (в остальных случаях)	Написать программу, запрашивающую текущую скорость загрузки в мегабитах в секунду и выводящую строку «высокая скорость», если значение выше 50, либо «низкая скорость» в противном случае
		Написать программу, запрашивающую стоимость товара и выводящую строку «скидка применяется», если стоимость выше 5000 руб., либо «без скидки» в противном случае
27	Написать программу, запрашивающую количество завершенных проектов и среднюю оценку удовлетворенности заказчиков (по 10-балльной шкале), и выводящую одну из категорий: «профессиональный уровень» (если проектов более 15 и средняя оценка выше 8), «средний уровень» (если проектов от 5 до 15 и оценка выше 6), либо «предполагается развитие» в остальных случаях	Написать программу, запрашивающую число и выводящую строку «положительное четное», если число больше нуля и делится на 2, либо «не соответствует условию» в противном случае
		Написать программу, запрашивающую год и выводящую строку «високосный», если год кратен 4 и не кратен 100, либо кратен 400, либо «не високосный» в противном случае
28	Написать программу, запрашивающую два значения: тип поверхности (ввод: «асфальт», «грунт», «лед») и температуру воздуха, и выводящую предупреждение: «высокий риск скольжения» (если поверхность «лед» и температура выше -5°C), «умеренный риск» (если поверхность «грунт» и температура ниже 0°C), либо «риск минимален» в остальных случаях	Написать программу, запрашивающую количество дней с момента последнего обновления и выводящую строку «обновление просрочено», если значение больше 30, либо «обновление актуально» в противном случае
		Написать программу, запрашивающую время в минутах и выводящую строку «короткий сеанс», если значение менее 15, либо «долгий сеанс» в противном случае
29	Написать программу, запрашивающую суточный объем потребляемой воды в литрах и массу тела в килограммах, и выводящую рекомендацию: «недостаточное потребление» (если	Написать программу, запрашивающую длину и ширину прямоугольника и выводящую строку «квадрат», если значения равны, либо «не квадрат» в противном случае

№	Задание (блочный оператор условия)	Задания (тернарный оператор условия)
	объем менее $0,03 \cdot m$), «оптимальное потребление» (если объем от $0,03 \cdot m$ до $0,045 \cdot m$), либо «избыточное потребление» (если объем более $0,045 \cdot m$), где m — масса тела	Написать программу, запрашивающую возраст и выводящую строку «пожилой», если значение более 65 лет, либо «не пожилой» в противном случае
30	Написать программу, запрашивающую текущую температуру в помещении и наличие открытых окон (ввод: «да» или «нет»), и выводящую рекомендацию: «необходимо закрыть окна» (если температура ниже 16°C и окна открыты), «проветривание не предполагается» (если температура в пределах от 20°C до 24°C и окна закрыты), либо «проверить климат-контроль» в остальных случаях	Написать программу, запрашивающую число и выводящую строку «трехзначное», если модуль числа находится в диапазоне от 100 до 999 включительно, либо «не трехзначное» в противном случае
		Написать программу, запрашивающую код валюты и выводящую строку «доллар», если код равен «USD», либо «другая валюта» в противном случае

3.2. Теоретические сведения о циклических инструкциях

Цель работы: усвоение теоретических понятий организации циклических процессов в программировании на языке Python.

Цикл — управляющая инструкция, реализующая многократное выполнение набора некоторых однотипных инструкций.

Итерация — одно полное выполнение тела цикла.

Тело цикла — последовательность инструкций, выполняемых при каждой итерации. Тело цикла должно быть корректно структурировано по правилам синтаксиса в Python — с обязательным отступом.

Предусловие — логическое выражение, проверяемое перед каждой итерацией цикла. Если оно ложно, выполнение цикла прекращается. Используется в цикле **while**.

Цикл с предусловием — цикл, в котором условие прекращения выполнения проверяется до каждой итерации. В Python реализуется конструкцией **while**.

Цикл с параметром — цикл, в котором итерации выполняются по значениям, получаемым из итерируемого объекта. В Python реализуется конструкцией **for...in...**

Индекс итерации — переменная, принимающая очередное значение из итерируемой последовательности.

Прерывание цикла — досрочное завершение выполнения текущего цикла, инициируемое оператором **break**. После этого управление передается на первую инструкцию после тела цикла.

Пропуск итерации — механизм перехода к следующей итерации цикла без выполнения оставшихся команд текущей итерации. В Python реализуется оператором **continue**.

Бесконечный цикл — цикл, не содержащий условий выхода или таких изменений, которые могут привести к прекращению итераций. В Python задается выражением **while True:**, а выход из такого цикла осуществляется оператором **break**. (при проверке соответствующего условия).

3.3. Цикл с предусловием

Инструкция **while** используется для организации цикла с предусловием, при котором блок команд выполняется многократно до тех пор, пока заданное логическое условие остается истинным.

Особенности цикла с предусловием:

1. Требуется явный контроль над параметрами итерации (инициализация, изменение переменных, проверка условия выхода).
2. Применяется при неизвестном заранее числе итераций или при необходимости выполнения повторений до наступления определенного состояния.
3. Условие может зависеть от переменных, модифицируемых внутри тела цикла, или от внешних факторов (например, пользовательского ввода).
4. Может приводить к бесконечному циклу при отсутствии корректного механизма выхода из итерации.
5. Поддерживает инструкции **break**, **continue** (см. далее), а также необязательную конструкцию **else**, которая выполняется в случае завершения цикла без использования **break**.

Существует два синтаксических варианта оператора **while**:

- основной блочный синтаксис,
- синтаксис с необязательным блоком **else**.

Базовый синтаксис оператора **while**:

```
while <условие>:  
    <блок_инструкций>
```

Здесь <условие> — логическое выражение, результат которого приводится к логическому типу, <блок_инструкций> — одна или несколько команд, выполняемых при каждом входе в итерацию, если условие истинно. Условие проверяется до входа в тело цикла. Цикл завершает работу при первом ложном значении условия. Пример использования приведен в листинге 3.4.

Листинг 3.4

```
x = 0  
while x < 5:
```

```
print(x, end=' ')
x += 1
# 0 1 2 3 4
```

Внутри тела цикла могут быть использованы операторы **break** и **continue**:

— **break** прерывает выполнение цикла досрочно, минуя блок **else** (листинг 3.5);

— **continue** прерывает текущую итерацию и переходит к следующей проверке условия (листинг 3.6).

Листинг 3.5

```
x = 0
while True:
    if x == 3:
        break
    print(x, end=' ')
    x += 1
# 0 1 2
```

Листинг 3.6

```
x = 0
while x < 5:
    x += 1
    if x == 3:
        continue
    print(x, end=' ')
# 1 2 4 5
```

Операторы **break** и **continue** срабатывают только для того цикла, в теле которого они располагаются.

Синтаксис с необязательным блоком **else**:

```
while <условие>:
    <блок_инструкций>
else:
    <блок_else>
```

Блок **else** выполняется однократно, только в том случае, если цикл завершился естественно, то есть без прерывания через **break**. Пример использования приведен в листинге 3.7.

Листинг 3.7

```
x = 0
while x < 3:
    print(x, end=' ')
# 0 1 2
```

```
    x += 1
else:
    print()
    print("Цикл завершен")
# 0 1 2
# Цикл завершен
```

Если начальное условие ложно, тело цикла не выполнится ни разу. Если условие никогда не становится ложным и отсутствует срабатываемый оператор **break**, цикл является бесконечным.

Пример 3.2

Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Определить, существует ли среди его цифр хотя бы одна цифра, равная количеству цифр в самом числе. Вывести 1, если существует, и 0 — в противном случае.

Решение

В листинге 3.8 приведено решение данной задачи с подробными комментариями, в листинге 3.9 приведен программный код без комментариев.

Листинг 3.8

```
n = int(input("Введите число: "))

# Определяем количество цифр в числе n
k = 1 # Счетчик количества цифр
while n >= 10**k:
    k += 1 # увеличиваем счетчик
# Теперь переменная k содержит количество цифр числа n

if k > 9: # Если количество цифр в числе n больше 9...
    print(0) # ...то подходящей цифры нет
else: # ...иначе
    i = 0 # счетчик номера цифры справа налево, начинаем с 0
    while i < k: # пока не перебрали все цифры...
        t = (n // 10**i) % 10 # вычисляем цифру
        if t == k: # если текущая цифра равна количеству цифр
            в n...
                print(1) # ...то выводим 1
                break # и выходим из цикла while
            i += 1 # Увеличиваем счетчик
    else: # Если break не сработал, то подходящей цифры нет
        print(0) # и выводим 0
```

Листинг 3.9

```
n = int(input("Введите число: "))

k = 1
```

```

while n >= 10**k:
    k += 1

if k > 9:
    print(0)
else:
    i = 0
    while i < k:
        t = (n // 10**i) % 10
        if t == k:
            print(1)
            break
        i += 1
    else:
        print(0)

```

Пример 3.3

Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Требуется определить, образуют ли цифры числа строго убывающую последовательность. Проверку выполнять арифметически, извлекая цифры справа налево. Вывести 1, если условие выполняется, и 0 — в противном случае.

Решение

В листинге 3.10 приведено решение данной задачи с подробными комментариями. В листинге 3.11 приведен программный код без комментариев.

Листинг 3.10

```

n = int(input("Введите число: "))

# Подсчитываем количество цифр в n
k = 1
while n >= 10**k:
    k += 1

# Номер разряда числа: от (k - 1) до 0 включительно. Всего k цифр
i = k - 1
# Будем рассматривать i-ю и (i - 1)-ю цифру. Последняя рассматриваемая пара цифр будет иметь номера разрядов 1 и 0
while i >= 1: # Номер текущего разряда
    t1 = (n // 10**i) % 10 # Цифра на текущем разряде
    t2 = (n // 10 ** (i - 1)) % 10 # Цифра на следующем разряде
    if t2 >= t1: # Если хотя бы для одной пары цифр не выполняется условие...
        print(0) # ... то цифры числа не образуют строго убывающую последовательность, выводим 0
        break # Дальше проверять не нужно, прерываем цикл
    i -= 1 # Переходим к следующей паре цифр

```

```

else: # Если цикл завершился естественным образом, значит
      условие выполнено
      print(1) # ... то выводим 1

```

Листинг 3.11

```

n = int(input("Введите число: "))

k = 1
while n >= 10**k:
    k += 1

i = k - 1
while i >= 1:
    t1 = (n // 10**i) % 10
    t2 = (n // 10 ** (i - 1)) % 10
    if t2 >= t1:
        print(0)
        break
    i -= 1
else:
    print(1)

```

Задание 3.2

Выполнить задание в соответствии с табл. 3.2. При решении допускается использовать только конструкцию **while** и арифметические операции.

Отчет должен включать:

1. Полную формулировку задачи.
2. Программный код с обязательными пояснениями в виде комментариев.
3. Результаты работы программы с различными наборами исходных данных, демонстрирующие различные выходные значения.

Таблица 3.2

№	Задание
1	Написать программу, запрашивающую у пользователя целое положительное число $P > 0$. Требуется определить наименьшее натуральное число n , произведение десятичных цифр которого равно P . Использовать перебор значений n от 1 с проверкой произведения цифр, выполненной только через арифметику. Вывести найденное значение n
2	Написать программу, запрашивающую у пользователя два целых положительных числа a и b и реализующую упрощенную версию алгоритма Евклида для нахождения наибольшего общего делителя: из большего числа вычитается меньшее до тех пор, пока числа не станут равны. Программа должна определить количество выполненных вычитаний до получения НОД и вывести это количество

№	Задание
3	Написать программу, запрашивающую у пользователя два натуральных числа L и R , таких что $L \leq R$. Требуется определить количество чисел на интервале от L до R включительно, сумма десятичных цифр которых является четным числом. Вывести найденное количество
4	Написать программу, запрашивающую у пользователя целое число $n > 0$ и определяющую, является ли сумма его десятичных цифр квадратом целого числа. Вывести строку «да», если сумма является точным квадратом, и «нет» — в противном случае
5	Написать программу, запрашивающую у пользователя два натуральных числа n и k , и определяющую, содержит ли десятичная запись числа n цифру k . Вывести 1, если содержит, и 0 — в противном случае
6	Написать программу, запрашивающую у пользователя натуральное число n и определяющую, равна ли сумма его четных цифр сумме его нечетных цифр. Вывести 1, если суммы равны, и 0 — в противном случае
7	Написать программу, запрашивающую у пользователя натуральное число n и вычисляющую произведение всех его цифр, не равных нулю. Вывести полученное значение. Если число состоит только из нулей, вывести 0
8	Написать программу, запрашивающую у пользователя целое положительное число n и определяющую, содержит ли его десятичная запись более трех нечетных цифр. Вывести 1, если содержит, и 0 — в противном случае
9	Написать программу, запрашивающую у пользователя два натуральных числа a и b и определяющую, равна ли сумма цифр числа a сумме цифр числа b . Вывести 1, если суммы равны, и 0 — в противном случае
10	Написать программу, запрашивающую у пользователя натуральное число n и определяющую, существует ли среди его цифр хотя бы одна цифра, делящая n без остатка. Вывести 1, если существует, и 0 — в противном случае
11	Написать программу, запрашивающую у пользователя натуральное число n и вычисляющую сумму тех его цифр, которые делятся на 3. Вывести полученное значение
12	Написать программу, запрашивающую у пользователя натуральное число n и вычисляющую количество нулей в его десятичной записи. Вывести найденное количество
13	Написать программу, запрашивающую у пользователя два натуральных числа n и m и определяющую, совпадает ли максимальная цифра числа n с максимальной цифрой числа m . Вывести 1, если совпадает, и 0 — в противном случае
14	Написать программу, запрашивающую у пользователя целое положительное число n и определяющую, является ли произведение всех его цифр нечетным числом. Вывести 1, если является, и 0 — в противном случае
15	Написать программу, запрашивающую у пользователя натуральное число n и определяющую, равна ли сумма первой и последней цифры числа n четному числу. Вывести 1, если равна, и 0 — в противном случае
16	Написать программу, запрашивающую у пользователя два натуральных числа a и b и подсчитывающую количество чисел на отрезке от a до b включительно, десятичная запись которых не содержит цифры 7. Вывести найденное количество
17	Написать программу, запрашивающую у пользователя натуральное число n и определяющую, можно ли расположить его цифры в порядке невозрастания. Вывести 1, если можно, и 0 — в противном случае
18	Написать программу, запрашивающую у пользователя натуральное число n и определяющую, является ли разность между наибольшей и наименьшей его цифрами простым числом. Вывести 1, если является, и 0 — в противном случае

№	Задание
19	Написать программу, запрашивающую у пользователя натуральное число $n > 0$. На каждом шаге из числа удаляется его наименьшая цифра (без использования строк и списков), а оставшиеся цифры образуют новое число. Процесс повторяется до тех пор, пока в числе остается более одной цифры. В конце вывести полученное однозначное число
20	Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Требуется определить, может ли n быть представлено в виде суммы различных степеней двойки. Вывести 1, если может, и 0 — в противном случае
21	Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Требуется определить, сколько итераций необходимо, чтобы число n стало равным 1, если на каждом шаге оно заменяется на сумму квадратов его цифр. Если результат входит в цикл, не содержащий 1, завершить и вывести -1
22	Написать программу, запрашивающую у пользователя два натуральных числа a и b . Требуется определить, сколько чисел на отрезке от a до b имеют произведение цифр, делящееся на 4
23	Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Последовательно извлекая цифры, определить длину самой длинной подпоследовательности цифр, в которой значения убывают. Вывести эту длину
24	Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Требуется определить, можно ли из цифр числа n составить число, делящееся на 5. Вывести 1, если можно, и 0 — в противном случае
25	Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Проверить, можно ли удалить из числа n одну цифру так, чтобы сумма оставшихся цифр делилась на 9. Вывести 1, если возможно, и 0 — в противном случае
26	Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Определить минимальное число k , которое необходимо прибавить к n , чтобы сумма цифр нового числа стала кратна 7. Вывести значение k
27	Написать программу, запрашивающую у пользователя два натуральных числа a и b . Требуется определить наименьшее число в диапазоне от a до b включительно, у которого количество четных цифр равно количеству нечетных. Если такого числа нет, вывести 0
28	Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Определить, является ли произведение цифр числа палиндромом. Проверку палиндрома выполнять только арифметически. Вывести 1, если да, и 0 — в противном случае
29	Написать программу, запрашивающую у пользователя два натуральных числа a и b , таких что $a < b$. Определить, сколько чисел на отрезке от a до b являются палиндромами. Все проверки выполнять арифметически
30	Написать программу, запрашивающую у пользователя натуральное число $n > 0$. Вычислить разность между числом, полученным при зеркальном отражении цифр n , и самим числом. Вывести модуль этой разности

Задание 3.3

Выполнить задание в соответствии с табл. 3.3. При решении допускается использовать только конструкцию **while** и арифметические операции.

Отчет должен включать:

1. Полную формулировку задачи.

2. Программный код с обязательными пояснениями в виде комментариев.
3. Результаты работы программы.

Таблица 3.3

№	Задание
1	Рассмотреть всевозможные упорядоченные тройки натуральных чисел (a, b, c) , каждое из которых изменяется от 1 до 200. Для каждой такой тройки проверить, является ли число $a^2 + b^2 - 3c$ точным квадратом целого числа и одновременно сумма $a + b + c$ не превосходит заданного пользователем порога M . Перебор следует реализовать с увеличением c на 2, при $c > 200$ сбрасывать его значение и увеличивать b на 3, а при $b > 200$ увеличивать a на 4. Если найдено хотя бы одно решение, необходимо немедленно завершить все циклы и вывести найденную тройку
2	Осуществить перебор всех возможных троек (x, y, z) в диапазонах от 10 до 999, где x изменяется с шагом 7, y с шагом 11, а z с шагом 13. Для каждой тройки вычислить значение выражения $x \cdot y - z^2$ и проверить, кратно ли оно введенному пользователем числу Q . Перебор должен завершиться после нахождения первой удовлетворяющей тройки или после исчерпания всех вариантов. В случае отсутствия решения должно выводиться соответствующее сообщение
3	Рассмотреть все возможные комбинации трех положительных чисел (r, s, t) в диапазоне от 1 до 150, при этом шаг изменения r составляет 5, s — 10, t — 20. Для каждой тройки вычислить $ r - s + s - t + t - r $ и проверить, не превосходит ли сумма заданной пользователем величины L . Если такое сочетание найдено, необходимо прекратить выполнение всех вложенных циклов и вывести найденные значения. Если ни одна из троек не удовлетворяет условию, сообщить об этом
4	Перебрать всевозможные целочисленные тройки (p, q, r) , значения которых находятся в пределах от 1 до 50, при этом r изменяется с шагом 1, q — с шагом 2, p — с шагом 3. Для каждой тройки определить, является ли значение выражения $p^3 + q^2 + r$ кратным введенному числу T . В случае положительного результата прекратить перебор и вывести тройку, иначе продолжать до полного исчерпания значений или достижения $p > 50$
5	Осуществить перебор всех возможных троек чисел (u, v, w) , где u и v принимают значения от 1 до 50, а w — от 50 до 1 с шагом -1 . Для каждой тройки определить, существует ли равенство $u \cdot v = w^2 - 1$. Если равенство выполняется, необходимо завершить выполнение всех циклов и вывести найденную тройку. В противном случае продолжить перебор. Если тройка не найдена, вывести сообщение об этом
6	Провести перебор всех возможных троек (i, j, k) , значения которых ограничены диапазоном от 1 до 300, и определить, существует ли такая тройка, что $i^2 + j^2 = 2k^2$ и при этом $i + j + k$ кратно 4. Значения переменных изменяются со следующими шагами: k — на 1, j — на 3, i — на 7. Перебор следует остановить при нахождении первой подходящей тройки или при достижении $i > 300$. При нахождении первой подходящей тройки вывести ее и прекратить выполнение программы

№	Задание
7	Рассмотреть тройки положительных чисел (m, n, p) в диапазоне от 1 до 500. Для каждой тройки проверить, можно ли выразить p как целую часть от $\frac{m \cdot n}{m + n}$ и одновременно чтобы $m + n + p$ делилось на заданное пользователем число Z . Значения переменных изменяются по следующей схеме: p увеличивается на 1, при превышении границы n увеличивается на 2, а m — на 5. Если найдено хотя бы одно решение, все циклы немедленно прерываются
8	Осуществить перебор всех троек положительных чисел (a, b, c) , где каждое из чисел находится в диапазоне от 1 до 100 и изменяется с разным шагом: a — 1, b — 2, c — 3. Для каждой тройки вычислить значение $a + 2b + 3c$ и проверить, является ли оно простым числом. Если условие выполнено, вывести соответствующую тройку и завершить перебор. В противном случае продолжить до достижения максимального значения a
9	Перебрать все возможные тройки (x, y, z) , где x изменяется от 1 до 50 с шагом 1, y — от 50 до 1 с шагом -1 , z — от 1 до 25 с шагом 1. Проверить для каждой тройки, можно ли представить $x^2 + y^2 + z^2$ как сумму трех попарно различных квадратов натуральных чисел. При нахождении первой подходящей тройки вывести ее и прекратить выполнение программы
10	Рассмотреть все тройки положительных чисел (f, g, h) в диапазонах $1 \leq f \leq 200$, $1 \leq g \leq 100$, $1 \leq h \leq 50$. При каждом шаге изменять h на 2, g — на 5, f — на 10. Для каждой тройки вычислить значение $f \cdot g - h^3$ и проверить, превышает ли оно пороговое значение R , заданное пользователем. Если условие выполнено, вывести соответствующую тройку и завершить выполнение. В противном случае продолжать до завершения полного перебора
11	Рассмотреть перебор всех троек положительных целых чисел (a, b, c) , где a изменяется от 1 до 100 с шагом 2, b от 1 до 50 с шагом 1, а c от 1 до 20 с шагом 1. Требуется найти такую тройку чисел, при которой $a^2 + 2b - c^3$ кратно числу M , вводимому пользователем. Переменные a , b и c изменяются вложенным способом: при каждом увеличении c , если оно превышает 20, происходит переход к следующему значению b , при этом c устанавливается в 1. Аналогично, при $b > 50$ осуществляется переход к следующему значению a , сбрасывая b и c в начальное состояние. Поиск должен завершиться при первом найденном решении либо при превышении a значения 100
12	Реализовать перебор всех троек целых чисел (x, y, z) , где x изменяется от -20 до 20, y от -10 до 10, а z от 0 до 15. Необходимо определить, существует ли такая тройка, для которой выполняется равенство $ x - y + z^2 = P$, где P — заданное пользователем целое число. Перебор должен прекращаться при первом выполнении условия. В случае отсутствия подходящей тройки вывести соответствующее сообщение

№	Задание
13	Рассмотреть задачу поиска таких натуральных чисел (m, n, q) , удовлетворяющих условиям: $m \in [1, 50]$, $n \in [1, 30]$, $q \in [1, 10]$, при этом значения изменяются с шагом 1. Проверке подлежит условие: $m \cdot q + n^2$ имеет остаток 7 при делении на T , где T — вводимое пользователем натуральное число. Необходимо выполнить вложенный перебор, где q — самый внутренний уровень. При каждом переходе за пределы диапазона переменные сбрасываются, согласно структуре вложенных циклов. Перебор завершается при выполнении условия либо по исчерпанию всех вариантов
14	Пользователь задает целое число $S > 0$. Требуется определить, существует ли тройка натуральных чисел (u, v, w) , удовлетворяющая неравенству $u^3 + v^2 + w \leq S$ и при этом значение $u \cdot v - w^2$ делится на 4. Все значения изменяются в пределах от 1 до 30. При достижении максимального значения переменных вложенный перебор прекращается
15	Определить, существует ли тройка натуральных чисел (i, j, k) из диапазонов $i \in [1, 40]$, $j \in [1, 25]$, $k \in [1, 20]$, для которой значение выражения $(i + 2j - 3k)^2 + ijk$ делится без остатка на вводимое пользователем положительное число Q . При этом переменные увеличиваются в неравномерных шагах: i на 3, j на 2, k на 1. Перебор заканчивается при первом найденном решении
16	Рассмотреть перебор всех возможных троек положительных чисел (a, b, c) , где $a \in [1, 30]$, $b \in [1, 20]$, $c \in [1, 10]$, для которых значение выражения $ab + bc + ca$ является квадратом целого числа. Найти хотя бы одну такую тройку. Перебор осуществляется с предварительной проверкой каждого условия и немедленным выходом при первом успехе
17	Рассмотреть все тройки натуральных чисел (p, q, r) в диапазонах $p \in [1, 50]$, $q \in [1, 50]$, $r \in [1, 50]$ и определить, существует ли среди них хотя бы одна такая, при которой выполняется $p \cdot q \cdot r + p^2 + q^2 + r^2 = Z$, где Z — целое положительное число, вводимое пользователем. В случае выполнения условия требуется немедленно завершить вложенные циклы
18	Найти такую тройку натуральных чисел (x, y, z) , удовлетворяющую условиям: $x + y + z = K$, $x < y < z$, $x \cdot y \cdot z$ минимально, где K — заданное пользователем натуральное число больше 10. Перебор осуществляется с вложенными циклами, обеспечивающими строгий порядок переменных. Выход из циклов осуществляется после нахождения первого подходящего решения с минимальным произведением
19	Пользователь вводит число $A > 0$. Требуется определить, существует ли такая тройка (a, b, c) , $a, b, c \in [1, 100]$, что сумма квадратов двух наименьших чисел из тройки делится на A . Проверка осуществляется через неравенства. При первом успехе выполнить выход из всех уровней вложенности
20	Рассмотреть тройки (i, j, k) , для которых $i + 2j + 3k = N$, $i^2 + j^2 + k^2$ минимально, а $i, j, k \in [1, 30]$. Значение N вводится пользователем. Найти тройку, удовлетворяющую этим условиям, используя вложенные циклы и пошаговое обновление минимального значения. Выход из вложенности осуществляется при достижении границы по переменным

№	Задание
21	Имеется система из трех параметров a, b, c , принимающих целые положительные значения от 1 до 50. Требуется определить, существует ли тройка значений, удовлетворяющая условиям: $a^2 + b^2 = c^2$ и $a + b + c$ делится на заданное пользователем число T . При этом при каждом шаге внутреннего уровня значение c увеличивается на 1, если $c > 50$, то b увеличивается на 2, c сбрасывается в 1. Если $b > 50$, то a увеличивается на 3, b и c устанавливаются в 1. Перебор останавливается при нахождении первой допустимой тройки либо при $a > 50$
22	Пользователь задает число M . Требуется перебрать тройки натуральных чисел (x, y, z) от 1 до 80, таких что $x^3 + y^3 + z^3 = M$. Начальные значения: $x = 1, y = 1, z = 1$. При каждой итерации z увеличивается на 1. Если $z > 80$, то y увеличивается на 1, z сбрасывается. Если $y > 80$, то x увеличивается на 1, y и z сбрасываются. Перебор завершается при первом нахождении решения или при $x > 80$
23	Найти все тройки (i, j, k) таких, что $i + 2j + 3k = Q$, где Q — введенное пользователем натуральное число. Перебор осуществляется по диапазону от 1 до 60. При каждом увеличении переменной k на 2 проверяется условие. Если $k > 60$, j увеличивается на 3, k сбрасывается. Если $j > 60$, i увеличивается на 4, остальные переменные сбрасываются. Завершение — по достижении $i > 60$ или исчерпанию решений
24	Задан интервал от 1 до 100. Необходимо найти первую тройку (a, b, c) , такую что abc кратно 7, $a + b + c$ делится на 9, а $a^2 - bc$ — четное число. Перебор ведется с шагами: c увеличивается на 1, при превышении границы b увеличивается на 2 и c сбрасывается, при превышении границы a увеличивается на 3 и остальные переменные обнуляются. Завершение при нахождении решения или при $a > 100$
25	Вычислить минимальное значение выражения $ x^2 + y^2 - z^2 $, перебирая все тройки целых чисел от 1 до 70. Значения переменных изменяются с шагами: z на 3, y на 2, x на 1 при переполнении диапазонов. Найденное минимальное значение и соответствующая тройка выводятся в конце
26	Для заданного N найти тройку чисел от 1 до 50 такую, что $x^2 + y \cdot z = N$ и $x + y + z$ кратно 4. Изменение значений: z инкрементируется на 1, при $z > 50$ — y на 2, при $y > 50$ — x на 2. При каждом шаге выполняется проверка. Алгоритм завершается при нахождении решения или превышении x значения 50
27	Определить существует ли комбинация (a, b, c) в диапазоне от 10 до 90, для которой выполняется: $a + 2b + 3c = N$ и abc делится на 11. Шаги: c увеличивается на 2, b на 4, a на 5 при выходе значений переменных b, c за пределы. Проверка прекращается при $a > 90$
28	Найти все тройки (i, j, k) в диапазоне от 1 до 30, такие что $i^2 + j^2 + k^2$ делится на $i + j + k$. Выполнить циклический перебор с шагом 1 для k , при $k > 30$ — увеличение j на 2, при $j > 30$ — i на 3. Все найденные тройки выводятся на экран
29	Проверить, существует ли тройка (x, y, z) , для которой $x + y + z = 150$, $x < y < z$, $x, y, z \in [1; 100]$, и xuz кратно 17. При переполнении z увеличивается y , при переполнении y увеличивается x . Искомая тройка выводится при первом совпадении

№	Задание
30	Выбрать такую тройку (a, b, c) , для которой $a^2 + b^2 + c^2$ делится на 10, и $a \cdot b \cdot c$ максимален среди всех возможных троек, значения которых изменяются в диапазоне от 1 до 50. Перебор осуществляется с инкрементом c на 1, b на 2 и a на 3. Алгоритм завершает работу по завершении всех переборов, после чего выводит найденную тройку и максимальное значение произведения

3.4. Генерирование числовых последовательностей

Функция **range()** в языке Python реализует *генерацию последовательностей целых чисел*, которая широко применяется в циклических конструкциях, особенно с оператором **for**. Она возвращает объект специального типа **range**, который является итерируемым, но не создает полный список значений в памяти.

Синтаксис **range()** имеет три возможных формы: с одним, двумя и тремя аргументами. Все аргументы должны быть целыми числами.

Синтаксис с одним аргументом:

range(stop)

При $\text{stop} \leq 0$ генерируется пустая последовательность. При $\text{stop} \geq 1$ генерируется последовательность из stop целых чисел от 0 до $(\text{stop} - 1)$ включительно с шагом 1.

Синтаксис с двумя аргументами:

range(start, stop)

При $\text{stop} > \text{start}$ генерируется последовательность из $(\text{stop} - \text{start})$ целых чисел от start до $(\text{stop} - 1)$ включительно с шагом 1. Если $\text{stop} \leq \text{start}$, то генерируется пустая последовательность. Аргументы start и stop могут быть отрицательными числами.

Синтаксис с тремя аргументами:

range(start, stop, step)

В этом случае генерируется последовательность целых чисел, начиная со start , не превышающих $(\text{stop} - 1)$ с шагом step . Если $\text{step} > 0$, то генерируемая последовательность возрастает, значения не превышают $(\text{stop} - 1)$ — значение stop не достигается слева. Если $\text{step} < 0$, то генерируемая последовательность убывает, значения не меньше $(\text{stop} - 1)$ — значение stop не достигается справа. При $\text{step} = 0$ генерируется исключение.

Примеры генерации числовых последовательностей приведены в листинге 3.12. Чтобы просмотреть генерируемые значения, выполняется преобразование в списки.

Листинг 3.12

```
r1 = range(7)
print(list(r1))
# [0, 1, 2, 3, 4, 5, 6]

r2 = range(-3)
print(list(r2))
# []

r3 = range(3, 10)
print(list(r3))
# [3, 4, 5, 6, 7, 8, 9]

r4 = range(-5, 0)
print(list(r4))
# [-5, -4, -3, -2, -1]

r5 = range(8, 4)
print(list(r5))
# []

r6 = range(2, 20, 3)
print(list(r6))
# [2, 5, 8, 11, 14, 17]

r7 = range(20, 2, -3)
print(list(r7))
# [20, 17, 14, 11, 8, 5]

r8 = range(2, 20, -3)
print(list(r8))
# []
```

Задание 3.4 (практико-ориентированное задание)

Выполнить задания, представленные в табл. 3.4. Результатом должен быть список, получаемый в результате применения генератора списков.

Отчет должен включать:

1. Полную формулировку задачи.
2. Программный код с обязательными пояснениями в виде комментариев.
3. Результаты работы программы.

Таблица 3.4

№	Задание
1	На склад поступают ящики с продукцией, каждый из которых содержит 8 одинаковых изделий. Складской регламент допускает единовременное размещение от 90 до 280 изделий. Получить перечень допустимых объемов партий изделий при условии, что количество изделий в партии кратно 8
	Автоматическая линия отсчитывает остаточное время до завершения производственного цикла с шагом 6 минут. Таймер запускается с момента достижения значения 3 ч 35 мин и продолжает обратный отсчет до 2 ч 35 мин. Получить перечень всех фиксируемых значений таймера в минутах, при которых завершается очередной цикл
2	Упаковочный модуль формирует комплекты из 7 изделий. Объем комплекта может изменяться от 80 до 252 изделий. Определить допустимые значения количества изделий в комплекте, соответствующие возможному числу упаковочных операций
	Система мониторинга качества осуществляет регистрацию остаточного времени контроля изделия с шагом 7 минут, начиная с 2 ч 42 мин до момента достижения 1 ч 32 мин. Получить перечень всех значений таймера в минутах, на которых завершается очередной этап контроля
3	Производственный конвейер отгружает продукцию контейнерами по 9 единиц. Количество единиц продукции в одном потоке может варьироваться от 110 до 315. Получить все возможные значения объема потока продукции при условии кратности 9
	Процесс охлаждения реакционной камеры контролируется по значениям температуры, регистрируемым каждые 5 градусов. Первоначальное значение температуры составляет 240 градусов, отслеживание завершается при достижении 175 градусов. Получить перечень всех температурных значений, фиксируемых в процессе охлаждения
4	Цеховая система подготовки упаковок комплектует изделия по 12 штук. Общий объем изделий для упаковки варьируется от 125 до 390. Определить допустимые значения общего числа изделий в партии при условии кратности 12
	Цифровой таймер в системе контроля зажигания отображает значения времени с шагом 9 секунд. Обратный отсчет начинается с 100 секунд и завершается при достижении 48 секунд. Получить последовательность значений таймера в минутах, при которых срабатывает контрольная процедура
5	Механизированная линия комплектует продукцию упаковками по 11 штук. В зависимости от типа груза объем отгрузки может находиться в диапазоне от 160 до 410 изделий. Получить возможные значения объема отгружаемой продукции
	Датчики давления в гидравлической системе регистрируют уровни давления каждые 8 единиц. Начальное значение составляет 180, отслеживание прекращается при снижении до 110. Получить список значений давления, регистрируемых в процессе снижения
6	Система комплектования медицинских наборов формирует упаковки по 13 единиц. Количество единиц в заказе варьируется от 190 до 430. Определить все возможные значения общего количества единиц в заказе
	Таймер контроля стерилизации отображает значение времени с шагом 13 минут. Отсчет идет от 5 ч 30 мин до 2 ч 25 мин. Определить все значения таймера в минутах, фиксируемые в процессе стерилизации
7	На складе организована отгрузка продукции поддонами по 14 изделий. Возможное количество изделий в отгружаемой партии колеблется от 190 до 390. Получить перечень всех допустимых объемов отгрузки

	В системе контроля расхода газа регистрируются значения давления с шагом 7 единиц. Начальное значение равно 280, отслеживание заканчивается при значении 170. Получить перечень всех значений давления, зафиксированных в процессе наблюдения
8	Формирование упаковок в логистическом центре производится по 15 изделий. Количество изделий в партии может составлять от 220 до 490. Определить допустимые значения объема партии, кратные 15 В рамках проверки технического состояния оборудование фиксирует уровень сигнала с шагом 15 единиц, начиная с 460 и заканчивая при достижении 260. Получить последовательность всех значений сигнала, регистрируемых в процессе диагностики
9	Партии продукции формируются упаковками по 17 единиц. Минимальный объем партии составляет 146 единиц, максимальный объем достигает 524 единиц. Требуется сформировать полный перечень возможных объемов партий, кратных 17, которые находятся строго между указанными значениями В ходе автоматизированного тестирования каждое испытание занимает ровно 23 минуты. Временной отсчет ведется в обратном направлении: изначально таймер установлен на отметку 366 минут и непрерывно уменьшается до значения, не меньшего 156 минут. Необходимо определить все моменты времени в минутах, при которых завершается очередное испытание на протяжении всего периода работы таймера
10	Заказы комплектуются коробками вместимостью 19 единиц продукции. При этом минимальный объем заказа равен 191 единице, а максимальный не превышает 626 единиц. Следует сформировать список всех возможных значений объемов заказов, кратных 19, лежащих строго внутри данного интервала В производственном процессе каждый этап длится 25 минут. Отсчет времени в минутах ведется в сторону уменьшения, начиная с 401 минуты и прекращаясь до достижения значения, не меньшего 176 минут. Нужно определить все моменты времени, выраженные в минутах, когда происходит завершение каждого этапа в течение данного временного промежутка
11	Упаковка товара осуществляется партиями по 21 единице. Объем партии варьируется от 211 единиц до 643 единиц, при этом требуется получить перечень всех возможных значений объемов партий, кратных 21, находящихся строго между этими величинами В процессе настройки оборудования каждый этап занимает ровно 27 минут. Таймер обратного отсчета начинается с отметки 431 минуты и непрерывно уменьшается до значения, не меньшего 194 минут. Задача состоит в том, чтобы вывести список моментов времени в минутах, когда завершается очередной этап настройки
12	Партии продукции формируются упаковками по 22 единицы. Минимальный объем партии равен 221 единице, максимальный объем не превышает 637 единиц. Необходимо получить перечень всех возможных объемов партий, кратных 22, которые располагаются строго между указанными границами В процессе контроля качества каждый цикл проверки длится 29 минут. Временной счет ведется в обратном направлении, начиная с отметки 461 минута и продолжается до достижения значения, не меньшего 211 минуты. Требуется определить полный список отметок времени в минутах, при которых завершается каждая проверка
13	Транспортные партии формируются по 24 единицам продукции. Минимальный объем партии равен 231 единице, максимальный объем составляет 669 единиц. Следует сформировать перечень всех возможных значений объемов партий, кратных 24, которые находятся строго в пределах указанного диапазона

	В технологическом цикле каждый отдельный цикл длится ровно 31 минуту. Отсчет времени ведется в обратном направлении: таймер установлен на отметке 481 минута и непрерывно уменьшается до значения, не меньшего 216 минут. Задача заключается в том, чтобы получить полный список моментов времени в минутах, в которые завершается очередной цикл
14	Партии продукции комплектуются упаковками по 13 единиц. Минимальный объем партии составляет 139 единиц, максимальный — 513 единиц. Необходимо сформировать полный список возможных значений объемов партий, кратных 13, лежащих строго между этими двумя значениями В процессе настройки оборудования каждый этап занимает ровно 18 минут. Таймер обратного отсчета стартует с отметки 362 минуты и уменьшается до значения, не меньшего 152 минуты. Требуется определить все моменты времени в минутах, когда завершается очередной этап
15	Партии товаров формируются упаковками по 17 единиц. Минимальный объем партии равен 172 единицам, максимальный объем не превышает 548 единиц. Требуется сформировать перечень всех возможных значений объемов партий, кратных 17, находящихся строго между указанными значениями В процессе контроля качества каждый цикл длится 23 минуты. Таймер обратного отсчета запускается с отметки 435 минут и непрерывно уменьшается до значения, не меньшего 195 минут. Нужно определить все моменты времени в минутах, в которые завершается очередной цикл
16	Заказы комплектуются упаковками по 21 единице продукции. Минимальный объем заказа равен 204 единицам, максимальный — 632 единицам. Следует сформировать перечень всех возможных значений объемов заказов, кратных 21, расположенных строго между этими величинами В технологическом процессе продолжительность каждого цикла составляет ровно 27 минут. Отсчет времени ведется в обратном направлении, начиная с 490 минут и непрерывно уменьшается до значения, не меньшего 225 минут. Нужно определить полный перечень моментов времени в минутах, в которые завершается каждый цикл
17	Заказы формируются упаковками по 25 единиц продукции. Минимальный объем заказа составляет 245 единиц, максимальный — 685 единиц. Следует составить перечень всех возможных значений объемов заказов, кратных 25, расположенных строго внутри этого интервала В производственном процессе каждый этап длится ровно 31 минуту. Временной счет ведется в обратном направлении, начиная с 530 минут и заканчивая значением, не меньшим 260 минут. Нужно определить все моменты времени в минутах, когда завершается очередной этап
18	Партии продукции комплектуются упаковками по 27 единиц. Минимальный объем партии равен 270 единицам, максимальный — 712 единиц. Требуется сформировать список всех возможных значений объемов партий, кратных 27, находящихся строго внутри данных границ В ходе тестирования каждая операция длится ровно 33 минуты. Таймер обратного отсчета стартует с отметки 550 минут и уменьшается до значения, не меньшего 275 минут. Необходимо получить полный перечень моментов времени в минутах, в которые завершается каждая операция
19	Заказы формируются коробками по 29 единиц продукции. Минимальный объем заказа составляет 280 единиц, максимальный — 730 единиц. Следует составить перечень всех возможных значений объемов заказов, кратных 29, расположенных строго внутри заданного диапазона В производственном цикле каждый этап длится ровно 35 минут. Отсчет времени ведется в обратном направлении, начиная с 570 минут и заканчивая значением, не

	меньшим 290 минут. Требуется определить все моменты времени в минутах, когда завершается очередной этап
20	Партии товаров формируются упаковками по 31 единице. Минимальный объем партии равен 310 единицам, максимальный объем не превышает 755 единиц. Нужно получить список всех возможных значений объемов партий, кратных 31, лежащих строго в пределах указанных границ В процессе контроля качества каждый тест длится ровно 37 минут. Таймер обратного отсчета запускается с отметки 590 минут и непрерывно уменьшается до значения, не меньшего 305 минут. Следует определить все моменты времени в минутах, в которые завершается очередной тест
21	Партии продукции комплектуются упаковками по 14 единиц. Минимальный объем партии составляет 128 единиц, максимальный — 520 единиц. Требуется сформировать перечень всех возможных значений объемов партий, кратных 14, находящихся строго между указанными значениями В технологическом процессе каждый этап длится ровно 22 минуты. Временной отсчет начинается с 385 минут и уменьшается до отметки, не меньшей 170 минут. Нужно определить все моменты времени в минутах, в которых завершается очередной этап
22	Заказы формируются упаковками по 16 единиц продукции. Минимальный объем заказа равен 165 единицам, максимальный — 580 единицам. Следует составить полный список возможных значений объемов заказов, кратных 16, расположенных строго внутри этого диапазона В ходе тестирования каждая операция длится ровно 24 минуты. Таймер обратного отсчета запускается с отметки 420 минут и уменьшается до значения, не меньшего 190 минут. Необходимо определить моменты времени в минутах, когда завершается каждая операция
23	Партии товаров формируются упаковками по 18 единиц. Минимальный объем партии составляет 182 единицы, максимальный — 610 единиц. Требуется получить перечень всех возможных значений объемов партий, кратных 18, расположенных строго между этими границами В процессе контроля качества каждый тест длится ровно 26 минут. Таймер обратного отсчета начинает работу с отметки 440 минут и продолжается до значения, не меньшего 215 минут. Нужно определить все моменты времени в минутах, в которые завершается очередной тест
24	Партии продукции комплектуются упаковками по 20 единиц. Минимальный объем партии равен 208 единицам, максимальный — 640 единиц. Следует сформировать полный список всех возможных значений объемов партий, кратных 20, расположенных строго внутри указанных границ В технологическом цикле каждая операция длится ровно 28 минут. Таймер обратного отсчета стартует с отметки 470 минут и уменьшается до значения, не меньшего 230 минут. Необходимо определить моменты времени в минутах, когда завершается каждая операция
25	Заказы формируются коробками по 22 единицы продукции. Минимальный объем заказа составляет 230 единиц, максимальный — 670 единиц. Требуется получить список всех возможных значений объемов заказов, кратных 22, находящихся строго внутри данного диапазона В ходе производственного цикла каждый этап длится ровно 30 минут. Таймер обратного отсчета начинается с отметки 490 минут и продолжается до значения, не меньшего 250 минут. Нужно определить все моменты времени в минутах, в которые завершается очередной этап
26	Партии продукции формируются упаковками по 24 единицы. Минимальный объем партии составляет 256 единиц, максимальный — 700 единиц. Требуется составить

	полный перечень всех возможных значений объемов партий, кратных 24, расположенных строго внутри этого диапазона
	В технологическом процессе каждый этап длится ровно 32 минуты. Таймер обратного отсчета запускается с отметки 520 минут и продолжается до значения, не меньшего 280 минут. Нужно определить все моменты времени в минутах, в которые завершается очередной этап
27	Поставки комплектуются упаковками по 26 единиц продукции. Минимальный объем поставки равен 275 единицам, максимальный — 730 единицам. Следует получить перечень всех возможных значений объемов поставок, кратных 26, находящихся строго внутри указанных границ
	В ходе испытаний каждая операция занимает ровно 34 минуты. Таймер обратного отсчета стартует с отметки 550 минут и уменьшается до значения, не меньшего 300 минут. Необходимо определить моменты времени в минутах, в которые завершается каждая операция
28	Партии продукции формируются упаковками по 28 единиц. Минимальный объем партии составляет 294 единицы, максимальный — 760 единиц. Требуется сформировать список всех возможных значений объемов партий, кратных 28, расположенных строго между этими значениями
	В производственном цикле каждая операция длится ровно 36 минут. Таймер обратного отсчета начинается с отметки 580 минут и продолжается до значения, не меньшего 320 минут. Нужно определить моменты времени в минутах, когда завершается очередной этап
29	Заказы формируются упаковками по 30 единиц продукции. Минимальный объем заказа равен 315 единицам, максимальный — 790 единицам. Следует получить полный перечень возможных значений объемов заказов, кратных 30, находящихся строго внутри заданного диапазона
	В ходе тестирования каждая операция занимает ровно 38 минут. Таймер обратного отсчета запускается с отметки 610 минут и уменьшается до значения, не меньшего 340 минут. Необходимо определить все моменты времени в минутах, когда завершается очередной тест
30	Партии продукции комплектуются упаковками по 32 единицы. Минимальный объем партии составляет 330 единиц, максимальный — 820 единиц. Требуется составить полный список всех возможных значений объемов партий, кратных 32, расположенных строго внутри указанных границ
	В технологическом цикле каждая операция длится ровно 40 минут. Таймер обратного отсчета стартует с отметки 640 минут и продолжается до значения, не меньшего 360 минут. Нужно определить моменты времени в минутах, в которые завершается каждая операция

3.5. Цикл с параметром

Инструкция **for** реализует цикл с параметром, то есть последовательный перебор элементов итерируемого объекта.

Особенности цикла с параметром:

1. Основан на итерации по элементам итерируемого объекта. Итерационная переменная автоматически последовательно принимает значения из указанного итерируемого объекта.

2. Подходит для случаев, когда количество итераций заранее определено или определяется длиной последовательности.

3. Обеспечивает более компактную и читаемую форму записи по сравнению с эквивалентной конструкцией на основе **while**.

4. Не требует ручной инициализации, модификации и проверки счетчика итераций.

5. Поддерживает инструкции **break**, **continue**, а также необязательную конструкцию **else**, выполняемую в случае завершения цикла без прерывания.

Существуют два синтаксиса оператора **for**:

— базовый блочный синтаксис,

— синтаксис с необязательным блоком **else**.

Базовый блочный синтаксис имеет вид

```
for <переменная> in <итерируемый_объект>:  
    <блок_инструкций>
```

В этом синтаксисе <переменная> принимает на каждой итерации значение следующего элемента итерируемого объекта (является индексом итерации). <блок_инструкций> выполняется один раз для каждого индекса. Цикл завершается, когда элементы в итерируемом объекте заканчиваются.

Синтаксис с необязательным блоком **else** имеет вид

```
for <переменная> in <итерируемый_объект>:  
    <блок_инструкций>  
else:  
    <блок_else>
```

Блок **else** выполняется однократно, если цикл завершен без прерывания оператором **break**.

Пример использования цикла с параметром приведен в листинге 3.13.

Листинг 3.13

```
for i in range(2):  
    for j in range(3):  
        print(i, j)  
  
# 0 0  
# 0 1  
# 0 2  
# 1 0  
# 1 1  
# 1 2
```

Задание 3.5

Выполнить задания, представленные в табл. 3.5. Отчет должен включать:

1. Полную формулировку задачи.
2. Программный код с обязательными пояснениями в виде комментариев.
3. Результаты работы программы с различными наборами исходных данных, демонстрирующие различные выходные значения.

Таблица 3.5

№	Задания
1	Написать программу, запрашивающую у пользователя два целых числа a и b ($a < b$) и осуществляющую перебор всех целых чисел на отрезке $[a, b]$. Для каждого числа необходимо вычислить сумму квадратов его десятичных цифр. Программа должна подсчитать и вывести количество чисел, для которых эта сумма является нечетным числом
	Написать программу, запрашивающую у пользователя целое число $n > 1$ и перебирающую все пары целых чисел (i, j) , таких что $1 \leq i < j \leq n$. Для каждой пары вычисляется произведение $i \cdot j$ и сумма $i + j$. Необходимо определить количество пар, для которых произведение делится на сумму без остатка
2	Написать программу, запрашивающую у пользователя натуральное число $n > 2$ и осуществляющую перебор всех чисел от 2 до n включительно. Для каждого числа требуется определить общее количество его натуральных делителей. Необходимо вывести те числа, которые делятся на количество своих делителей без остатка
	Написать программу, запрашивающую два целых числа a и b ($a < b$) и перебирающую все числа в отрезке $[a, b]$. Для каждого числа следует вычислить сумму всех его цифр и количество четных цифр в записи. Необходимо подсчитать количество таких чисел, для которых сумма цифр равна количеству четных цифр
3	Написать программу, осуществляющую перебор всех трехзначных чисел от 100 до 999. Для каждого такого числа требуется вычислить его квадрат и определить, совпадает ли последняя цифра квадрата с последней цифрой самого числа. Программа должна вывести все такие числа
	Написать программу, перебирающую все натуральные числа от 1 до n , где n запрашивается у пользователя. Для каждого числа необходимо определить сумму всех его натуральных делителей, включая само число. Далее требуется вычислить среднее значение делителей (целочисленно) и сравнить его с самим числом. Если сумма делителей делится на их количество, это число должно быть выведено
4	Написать программу, запрашивающую натуральное число n и перебирающую все числа от 1 до n . Для каждого числа требуется определить количество его десятичных цифр, а также количество его натуральных делителей. Программа должна вывести те числа, для которых количество цифр совпадает с количеством делителей
	Написать программу, перебирающую все четырехзначные натуральные числа. Для каждого такого числа следует вычислить произведение его цифр и сравнить полученное значение с самим числом. Программа должна подсчитать количество чисел, для которых произведение цифр строго больше значения числа

№	Задания
5	Написать программу, запрашивающую у пользователя целое число $n > 2$ и осуществляющую перебор всех натуральных чисел от 2 до n . Для каждого числа необходимо определить все его собственные делители и сравнить наибольший из них с суммой остальных. Вывести те числа, для которых наибольший собственный делитель строго меньше суммы остальных
	Написать программу, запрашивающую целое число $n > 0$ и перебирающую все числа от 1 до n . Для каждого числа необходимо проверить, является ли оно палиндромом в десятичной записи, не используя при этом строк или массивов. Программа должна подсчитать и вывести количество таких палиндромов
6	Написать программу, запрашивающую целое число $n > 0$ и перебирающую все натуральные числа от 1 до n . Для каждого числа вычисляется его квадрат, после чего проверяется, дает ли квадрат в остатке 4 при делении на 5. Программа должна подсчитать и вывести количество таких чисел
	Написать программу, которая перебирает натуральные числа, начиная с 1, и для каждого такого числа проверяет, делится ли оно на хотя бы одну из своих цифр, не равных нулю. Необходимо вывести на экран первые n таких чисел, где n запрашивается у пользователя
7	Написать программу, осуществляющую перебор всех трехзначных чисел. Для каждого числа необходимо вычислить его квадрат и проверить, совпадают ли последние три цифры квадрата с самим числом. Программа должна вывести все такие числа
	Написать программу, осуществляющую перебор всех четырехзначных чисел. Для каждого числа необходимо вычислить сумму первых двух и произведение последних двух цифр. Программа должна подсчитать, сколько существует таких чисел, для которых сумма первых двух цифр равна произведению последних двух
8	Написать программу, запрашивающую число $n > 0$ и перебирающую числа от 1 до n . Для каждого числа вычисляется сумма квадратов всех предыдущих натуральных чисел (от 1 до k), и программа должна определить наименьшее k , при котором эта сумма превышает n
	Написать программу, перебирающую все числа от 10 до n ($n > 10$), запрашиваемого у пользователя. Для каждого числа необходимо проверить, содержатся ли в его десятичной записи одинаковые цифры. Вывести все числа, в которых каждая цифра встречается не более одного раза
9	Написать программу, запрашивающую число $n > 1$ и перебирающую все числа от 1 до n включительно. Для каждого числа требуется вычислить сумму всех его делителей и проверить, является ли полученная сумма степенью двойки. Подсчитать количество таких чисел
	Написать программу, запрашивающую число $n > 1$ и перебирающую все числа от 1 до n . Для каждого числа необходимо определить количество делителей. Если число имеет нечетное количество делителей и одновременно является палиндромом (в десятичной системе), его следует учесть. В конце вывести общее количество таких чисел
10	Написать программу, перебирающую все двузначные числа от 10 до 99 включительно. Для каждого числа необходимо вычислить сумму квадратов его цифр, удвоить полученное значение и сравнить с самим числом. Программа должна вывести те числа, для которых это условие выполняется

№	Задания
	Написать программу, запрашивающую число $n > 0$ и перебирающую все числа от 1 до n . Для каждого числа необходимо вычислить сумму квадратов всех его цифр и проверить, делится ли она на 3. В случае положительного результата сумма добавляется к общему счетчику. В конце вывести итоговую сумму
11	Написать программу, которая запрашивает у пользователя два целых числа a и b, таких что $a < b$, и осуществляет полный перебор всех чисел на отрезке $[a, b]$. Для каждого числа необходимо вычислить сумму его цифр, а затем проверить, делится ли исходное число на эту сумму без остатка. Программа должна вывести количество таких чисел, удовлетворяющих указанному условию Написать программу, запрашивающую у пользователя натуральное число $n > 0$ и выполняющую итеративную проверку всех чисел от 1 до n. Для каждого такого числа требуется определить количество его делителей. Если число имеет нечетное количество делителей и при этом делится на 3, оно должно быть включено в итоговую выборку. В завершение программа должна вывести количество всех таких чисел
12	Написать программу, которая по заданному натуральному числу n осуществляет последовательный перебор всех натуральных чисел от 1 до n включительно. Для каждого числа требуется определить сумму квадратов его делителей. Среди всех таких сумм необходимо найти наименьшую и вывести соответствующее число, обладающее данной суммой Написать программу, осуществляющую перебор всех двузначных натуральных чисел. Для каждого числа вычисляется произведение его цифр и проверяется, делится ли это произведение на сумму этих же цифр. Программа должна вывести все числа, удовлетворяющие указанному критерию
13	Написать программу, которая запрашивает у пользователя целое число $n > 1$ и осуществляет анализ всех чисел от 2 до n. Для каждого числа требуется определить, является ли оно простым. Если число простое, необходимо проверить, делится ли его порядковый номер в последовательности простых чисел (начиная с 2 как первого простого) на 3. Вывести все такие простые числа Написать программу, которая перебирает все трехзначные натуральные числа, для каждого из которых проверяется условие: разность между числом и числом, полученным при перестановке первой и последней цифры, равна квадрату некоторого натурального числа. Программа должна вывести такие числа
14	Написать программу, запрашивающую два натуральных числа L и R, таких что $L < R$. Требуется перебрать все числа на отрезке $[L, R]$ и для каждого из них определить, является ли сумма его цифр числом, делящимся на 4, но не делящимся на 8. Подсчитать количество таких чисел Написать программу, которая по заданному числу n перебирает все числа от 1 до n включительно. Для каждого числа проверяется, представимо ли оно в виде суммы двух различных квадратов натуральных чисел. Программа должна подсчитать и вывести общее количество таких чисел
15	Написать программу, перебирающую все четырехзначные числа. Для каждого такого числа необходимо определить сумму двух старших разрядов и сумму двух младших разрядов. Если произведение этих двух сумм делится на 9, число должно быть выведено в итоговый список

№	Задания
	Написать программу, осуществляющую полный перебор всех натуральных чисел от 1 до заданного значения n , вводимого пользователем. Для каждого числа определить все его делители и рассчитать разность между количеством четных и нечетных делителей. Если разность положительна, число включается в выборку. В завершение программа должна вывести общее количество таких чисел
16	Написать программу, которая перебирает все натуральные числа от 1 до n включительно, где n — положительное целое число, вводимое пользователем. Для каждого числа требуется определить, является ли оно одновременно произведением трех последовательных нечетных чисел. Если да, число следует отобразить на экране
	Написать программу, которая перебирает все двузначные натуральные числа и определяет, для каких из них квадрат числа заканчивается на ту же цифру, что и само число. Все такие числа должны быть выведены на экран
17	Написать программу, которая перебирает все трехзначные числа от 100 до 999. Для каждого такого числа необходимо рассчитать сумму цифр, а затем проверить, делится ли исходное число на утроенную сумму своих цифр. Все подходящие числа вывести на экран
	Написать программу, которая по заданному числу $n > 1$ осуществляет полный перебор всех натуральных чисел от 2 до n включительно. Для каждого из них необходимо вычислить сумму простых делителей. Если эта сумма делится на 3, число должно быть добавлено к общему счетчику. В конце программа должна вывести количество таких чисел
18	Написать программу, которая по заданному числу $n > 1$ перебирает все натуральные числа от 1 до n и проверяет для каждого из них, равна ли сумма квадратов его цифр количеству делителей этого числа. Подсчитать количество таких чисел
	Написать программу, которая перебирает все числа от 10 до 999, для каждого из которых определяется произведение всех цифр. Если произведение делится на сумму этих цифр и результат деления равен квадрату натурального числа, число следует отобразить на экране
19	Написать программу, которая запрашивает у пользователя два целых числа L и R , таких что $L < R$ и $L > 0$. Программа должна перебрать все целые числа на отрезке $[L, R]$. Для каждого числа необходимо определить сумму его делителей, не считая самого числа. Если полученная сумма кратна количеству цифр в десятичной записи исходного числа, такое число следует учесть в общем счетчике. По завершении перебора вывести итоговое количество таких чисел
	Написать программу, которая по заданному натуральному числу $n > 0$ перебирает все числа от 1 до n включительно. Для каждого числа необходимо определить, делится ли оно на сумму квадратов его цифр. Если условие выполняется, соответствующее число следует отобразить на экране
20	Написать программу, которая по введенному пользователем числу $n > 0$ перебирает все натуральные числа от 1 до n включительно. Для каждого числа требуется определить количество его делителей. Затем необходимо вычислить сумму всех цифр исходного числа и сравнить ее с количеством делителей. Если они совпадают, такое число следует отобразить в выходной последовательности
	Написать программу, перебирающую все двузначные натуральные числа от 10 до 99 включительно. Для каждого числа необходимо определить произведение его цифр и проверить, делится ли исходное число на это произведение. Все такие числа требуется вывести на экран в порядке возрастания

№	Задания
21	Написать программу, запрашивающую у пользователя целое положительное число $n > 0$. Для всех чисел от 1 до n включительно необходимо определить, являются ли они числами, представимыми в виде суммы двух различных квадратов натуральных чисел. Все такие числа следует отобразить в выходной последовательности
	Написать программу, осуществляющую полный перебор всех трехзначных натуральных чисел. Для каждого из них требуется вычислить разность между исходным числом и числом, полученным при обращении порядка его цифр (то есть при записи цифр в обратном порядке). Если полученная разность является квадратом целого числа, соответствующее исходное число следует вывести
22	Написать программу, которая по введенному целому числу $n > 0$ перебирает все числа от 1 до n включительно и определяет, существует ли для каждого из них пара натуральных чисел (a, b) , удовлетворяющая уравнению $a^2 + b^2 = k$, где k — исходное число. Все числа, для которых такие пары существуют, должны быть учтены в итоговом счетчике
	Написать программу, перебирающую все четырехзначные числа от 1000 до 9999 включительно. Для каждого числа необходимо выделить сумму двух старших цифр и сумму двух младших. Если разность этих двух сумм делится на 7, число должно быть выведено в результат
23	Написать программу, запрашивающую у пользователя два целых положительных числа a и b , таких что $a < b$. Для каждого числа в отрезке $[a, b]$ необходимо определить, делится ли сумма всех его делителей на сумму его цифр. Если это условие выполняется, соответствующее число должно быть учтено в итоговом счетчике
	Написать программу, осуществляющую перебор всех двузначных натуральных чисел. Для каждого числа требуется определить сумму квадратов его цифр, после чего проверить, делится ли исходное число на удвоенное значение полученной суммы. Все такие числа должны быть выведены в порядке возрастания
24	Написать программу, которая по заданному пользователем числу $n > 0$ перебирает все натуральные числа от 1 до n включительно. Для каждого числа определить сумму его делителей, не считая самого числа. Если сумма делителей совпадает с количеством нечетных цифр в записи числа, такое число должно быть включено в результат
	Написать программу, которая по заданному числу $n > 1$ перебирает все числа от 2 до n включительно. Для каждого из них необходимо определить, является ли оно простым числом. Если да, то далее требуется проверить, является ли сумма его десятичных цифр степенью двойки. Все такие простые числа следует отобразить на экране
25	Написать программу, которая по введенному пользователем числу $n > 0$ перебирает все натуральные числа от 1 до n . Для каждого числа необходимо определить произведение всех его делителей, не равных 1 и самому числу. Если это произведение делится на квадрат суммы цифр, соответствующее число следует отобразить на экране
	Написать программу, осуществляющую перебор всех трехзначных натуральных чисел. Для каждого числа необходимо определить сумму квадратов его цифр и проверить, совпадает ли она с утроенным значением суммы самих цифр. Все числа, удовлетворяющие данному критерию, следует вывести

№	Задания
26	Написать программу, которая по заданному целому числу $n > 1$ перебирает все числа от 1 до n включительно. Для каждого числа необходимо определить, делится ли квадрат суммы всех его делителей на удвоенное значение самого числа. Все числа, для которых это условие выполняется, необходимо подсчитать
	Написать программу, осуществляющую перебор всех четырехзначных чисел от 1000 до 9999. Для каждого числа необходимо определить количество четных и нечетных цифр в его записи. Если разность между количеством четных и нечетных цифр является делителем исходного числа, число следует вывести
27	Написать программу, которая по заданному числу $n > 1$ перебирает все числа от 1 до n включительно. Для каждого числа необходимо проверить, можно ли его представить в виде суммы двух чисел, произведения которых также равно этому числу. Все такие числа следует отобразить в выходной последовательности
	Написать программу, которая перебирает все числа от 10 до 99 включительно. Для каждого числа необходимо вычислить сумму квадратов цифр, произведение цифр, а затем проверить, равна ли сумма этих двух величин самому числу. Если условие выполняется, число следует вывести
28	Написать программу, которая по заданному пользователем целому числу $n > 1$ осуществляет перебор всех чисел от 1 до n включительно. Для каждого числа необходимо вычислить сумму его собственных делителей, то есть всех делителей, не включая само число. Если такая сумма в точности равна самому числу, то оно считается совершенным. Программа должна определить и вывести количество совершенных чисел в указанном диапазоне
	Написать программу, которая перебирает все трехзначные числа от 100 до 999 включительно. Для каждого числа требуется вычислить сумму квадратов его цифр. Если полученная сумма делится на количество делителей исходного числа, то данное число следует вывести на экран как удовлетворяющее условию
29	Написать программу, которая по введенному пользователем числу $n > 0$ перебирает все натуральные числа от 1 до n включительно. Для каждого числа требуется определить, делится ли произведение его делителей на сумму его цифр. Все числа, удовлетворяющие данному критерию, должны быть подсчитаны, а результат выведен в виде одного целого числа
	Написать программу, которая осуществляет перебор всех двузначных чисел от 10 до 99 включительно. Для каждого числа необходимо проверить, равна ли сумма цифр количеству всех положительных делителей этого числа. Все такие числа вывести в порядке возрастания
30	Написать программу, которая по заданному числу $n > 0$ перебирает все натуральные числа от 1 до n включительно. Для каждого числа определить сумму всех его делителей, а затем проверить, делится ли эта сумма на утроенное значение самого числа. Все числа, удовлетворяющие данному критерию, следует отобразить в выходной последовательности
	Написать программу, осуществляющую полный перебор всех четырехзначных чисел от 1000 до 9999 включительно. Для каждого числа необходимо вычислить сумму двух младших цифр и произведение двух старших цифр. Если сумма и произведение совпадают, число следует отобразить в выходной последовательности

Задание 3.6 (практико-ориентированное задание)

Выполнить задания, представленные в табл. 3.6. Отчет должен включать:

1. Полную формулировку задачи.
2. Программный код с обязательными пояснениями в виде комментариев.
3. Результаты работы программы.

Таблица 3.6

№	Задание
1	В ходе инвентаризации оборудования требуется найти первую комбинацию значений температуры, влажности и напряжения, при которых устройство переходит в аварийный режим. Диапазон температур составляет от -20 до 40 градусов с шагом 5 , влажность изменяется от 10 до 90 процентов с шагом 10 , а напряжение — от 180 до 260 вольт с шагом 5 . Устройство считается аварийным, если одновременно выполняются следующие условия: сумма температуры и влажности превышает 100 , а произведение напряжения и температуры по модулю превышает 4000 . По достижении первого совпадения перебор параметров прекращается, а найденные значения фиксируются
2	При контроле качества продукции производится анализ толщины слоя покрытия по трем независимым измерениям. Толщина первого слоя варьируется от 1 до 10 микрон, второго — от 5 до 50 микрон, третьего — от 1 до 20 микрон. Необходимо найти первую комбинацию толщин, при которой суммарная толщина превышает 60 микрон, а разность между самым толстым и самым тонким слоем превышает 40 микрон. Перебор прекращается при нахождении первой такой тройки
3	В контексте моделирования логистических маршрутов требуется определить первую тройку параметров — число складов, число машин и число часов на погрузку, — при которой общая стоимость транспортировки превышает заданное пользователем значение. Стоимость определяется как произведение числа машин на количество часов и на коэффициент, зависящий от числа складов. Диапазоны параметров: склады — от 1 до 10 , машины — от 1 до 20 , часы — от 1 до 24 . После нахождения подходящей тройки выполнение прекращается
4	В задаче подбора комбинации параметров испытаний лабораторного стенда перебираются значения давления (от 100 до 200 с шагом 10), температуры (от 20 до 100 с шагом 10) и времени выдержки (от 1 до 10 часов). Требуется найти первую тройку параметров, при которых энергия, вычисляемая как произведение давления, температуры и времени, превышает 150000 . Перебор параметров прекращается по достижении этой величины
5	В задаче проектирования химического реактора требуется подобрать первые параметры давления, объема и температуры, при которых значение уравнения состояния превышает некоторую критическую величину. Диапазоны параметров: давление — от 1 до 20 атм, объем — от 10 до 100 литров, температура — от 100 до 1000 К. Уравнение вычисляется по формуле $P \cdot V / T$. Перебор прекращается при нахождении первой удовлетворяющей конфигурации

№	Задание
6	При автоматическом тестировании аппаратного блока подбираются первые три значения уровня сигнала, числа операций и времени отклика, при которых величина $S \cdot O - T$ становится отрицательной. Уровень сигнала от 10 до 100, количество операций от 1 до 50, время отклика от 1 до 200 мс. Перебор останавливается при нахождении первой удовлетворяющей зависимости
7	В задаче инженерного моделирования требуется найти первую комбинацию высоты, массы и скорости, при которых потенциальная энергия тела в поле тяготения больше заданного значения. Высота изменяется от 10 до 100 м, масса — от 1 до 50 кг, скорость — от 0 до 20 м/с. Потенциальная энергия вычисляется как $m \cdot g \cdot h$, где $g = 9,8$, и при первом превышении порогового значения перебор прекращается
8	При автоматизированном подборе параметров освещения требуется найти первые значения яркости, расстояния и угла падения света, при которых освещенность на поверхности превышает 500 лк. Яркость источника от 100 до 1000 кд, расстояние от 1 до 10 м, угол α от 0 до 90 градусов. Освещенность вычисляется как $I \cdot \cos(\alpha) / r^2$. Перебор прекращается при первом успешном совпадении
9	В рамках моделирования системы контроля доступа на предприятии требуется определить первую тройку значений — число сотрудников, число точек входа и число часов в день, — при которой суммарная нагрузка на систему превышает критический уровень, заданный пользователем. Нагрузка вычисляется как произведение числа сотрудников, числа точек входа и количества часов. Значения параметров изменяются следующим образом: сотрудники — от 10 до 100 с шагом 10, точки входа — от 1 до 5, часы — от 1 до 12. Перебор прекращается после нахождения первой подходящей комбинации
10	В рамках расчета устойчивости строительных конструкций необходимо найти первую тройку параметров — количество вертикальных опор, высота каждой опоры и коэффициент давления ветра, — при которой результирующая нагрузка на основание превышает допустимый уровень. Нагрузка вычисляется как произведение количества опор на их высоту и на коэффициент. Диапазоны изменения: количество опор — от 1 до 20, высота — от 5 до 100 с шагом 5, коэффициент — от 1 до 10. При достижении заданного уровня расчет прерывается
11	При проектировании автоматизированной линии по сборке деталей требуется найти первую тройку параметров — количество рабочих станций, число операций на каждой станции и продолжительность одной операции — при которой общее время сборки превышает заданное значение. Общее время рассчитывается как произведение указанных трех величин. Параметры варьируются следующим образом: станции — от 2 до 10, операции — от 1 до 5, время операции — от 1 до 20 минут. Выполнение прекращается после нахождения первой удовлетворяющей тройки
12	В задаче оценки затрат на обслуживание телекоммуникационной инфраструктуры необходимо определить первую комбинацию значений: число абонентов, число обслуживаемых устройств и среднее время на диагностику одного устройства, — при которой совокупное время обслуживания превышает допустимое значение. Значения изменяются в диапазонах: абоненты — от 100 до 1000 с шагом 100, устройства — от 1 до 10, время — от 1 до 30 минут. После нахождения соответствующей комбинации вычисления прекращаются

№	Задание
13	При проектировании автоматической системы полива сельскохозяйственного участка требуется определить первую комбинацию параметров: число зон полива, количество циклов в сутки и продолжительность одного цикла, — при которой общее время работы системы превышает предельное значение. Время работы рассчитывается по формуле произведения всех трех величин. Диапазоны параметров: зоны — от 1 до 20, циклы — от 1 до 10, продолжительность — от 5 до 30 минут. После нахождения удовлетворяющей тройки выполнение прерывается
14	В задаче расчета затрат на бурение скважин необходимо определить первую тройку значений: глубина скважины, число бурильных установок и коэффициент породы, при которой стоимость работ превышает допустимую. Стоимость рассчитывается как произведение глубины, числа установок и коэффициента. Диапазоны: глубина — от 50 до 500 с шагом 50, установки — от 1 до 5, коэффициент — от 1 до 10. При достижении предельной стоимости расчет завершается
15	При планировании расписания для группы преподавателей требуется определить первую тройку параметров: число преподавателей, число занятий в день и длительность одного занятия, — при которой суммарное время преподавания превышает допустимый максимум. Параметры изменяются следующим образом: преподаватели — от 1 до 20, занятия — от 1 до 8, длительность — от 30 до 120 минут с шагом 15. Перебор прекращается при нахождении удовлетворяющей тройки
16	В рамках энергетического моделирования системы требуется определить первую комбинацию значений: число энергоблоков, число рабочих часов и коэффициент потребления топлива, — при которой суммарное потребление энергии превышает лимит. Значения изменяются в диапазонах: энергоблоки — от 1 до 6, часы — от 1 до 24, коэффициент — от 1 до 10. После нахождения тройки, удовлетворяющей условию, выполнение прекращается
17	В процессе оценки потребностей в складских площадях необходимо определить первую тройку параметров: число паллет, число ярусов хранения и объем одной паллеты, — при которой совокупный объем превышает указанный пользователем предел. Объем рассчитывается как произведение указанных трех параметров. Диапазоны: паллеты — от 50 до 500 с шагом 50, ярусы — от 1 до 5, объем — от 1 до 10 кубических метров. Перебор останавливается после нахождения первой подходящей комбинации
18	В контексте оценки производительности вычислительных систем необходимо определить первую тройку: число процессорных ядер, количество одновременно обрабатываемых потоков и время обработки одного потока, — при которой общая загрузка системы превышает заданное значение. Загрузка вычисляется как произведение всех трех параметров. Диапазоны: ядра — от 1 до 32, потоки — от 1 до 64, время — от 1 до 100 миллисекунд. Перебор останавливается после нахождения удовлетворяющей тройки
19	В контексте проверки конфигураций производственного оборудования требуется определить первую тройку параметров — число рабочих станций, число смен и число задействованных операторов, — при которой объем выпускаемой продукции за сутки превышает пороговое значение, вводимое пользователем. Объем рассчитывается как произведение числа станций, смен и операторов. Перебор осуществляется в пределах: станции — от 1 до 5, смены — от 1 до 3, операторы — от 1 до 44. По достижении искомой комбинации выполнение прекращается

№	Задание
20	В рамках задачи выбора конфигурации дежурных служб в аэропорту требуется определить первую комбинацию — количество зон контроля, число смен и число сотрудников на смену, — при которой суммарное количество персонала в течение суток превышает заданное пользователем значение. Перебор значений осуществляется в диапазоне: зоны — от 1 до 8, смены — от 1 до 3, сотрудники — от 1 до 50. После нахождения первой удовлетворяющей комбинации выполнение завершается
21	В условиях настройки параметров систем охлаждения вычислительного центра необходимо определить первую тройку: число модулей охлаждения, число вентиляторов и число часов непрерывной работы, — при которой производство этих параметров превышает заданный пользователем уровень энергопотребления. Перебор осуществляется в диапазоне: модули — от 1 до 6, вентиляторы — от 1 до 10, часы — от 1 до 24. Найденная комбинация немедленно завершает выполнение
22	В процессе моделирования укомплектованности отеля необходимо найти первую тройку: число этажей, число комнат на этаже и число человек в комнате, — при которой общее количество гостей превышает заданное пользователем значение. Диапазоны параметров: этажи — от 1 до 10, комнаты — от 1 до 15, человек — от 1 до 4. По достижении условия выполнение прерывается
23	Для задачи по определению конфигурации охраны объекта требуется определить первую тройку: число охранников, число смен и число объектов, — при которой производство этих величин достигает порогового значения, вводимого пользователем. Значения перебираются в диапазоне: охранники — от 1 до 20, смены — от 1 до 3, объекты — от 1 до 5. При нахождении подходящей комбинации выполнение прекращается
24	В задаче подбора параметров для тестирования производительности серверов необходимо найти первую тройку: число ядер, число потоков и число тестов, — при которой производство этих значений превышает заданный пользователем порог. Диапазоны: ядра — от 1 до 16, потоки — от 1 до 32, тесты — от 1 до 10. По выполнению условия выполнение прерывается
25	В рамках задач оптимизации планов уборки помещений требуется определить первую тройку: число помещений, число работников и число часов на уборку, — при которой общее человеко-часы превышают заданный пользователем порог. Диапазоны параметров: помещения — от 1 до 25, работники — от 1 до 10, часы — от 1 до 8. После нахождения подходящей тройки выполнение завершается
26	В процессе симуляции загрузки производственной линии требуется определить первую тройку параметров: число участков, число станков на участке и количество смен, — при которой общая производственная мощность, определяемая как производство этих значений, превышает порог, задаваемый пользователем. Диапазоны: участки — от 1 до 5, станки — от 1 до 10, смены — от 1 до 3. После нахождения подходящего варианта выполнение прекращается
27	В задаче моделирования подготовки мероприятий требуется найти первую комбинацию: число залов, число мероприятий в каждом зале и число участников на мероприятие, — при которой суммарное количество участников превышает заданное пользователем значение. Параметры варьируются в диапазоне: залы — от 1 до 6, мероприятия — от 1 до 10, участники — от 1 до 100. При первом выполнении условия выполнение завершается

№	Задание
28	В рамках задачи управления автопарком необходимо определить первую тройку параметров: число маршрутов, число рейсов на маршрут и число пассажиров в рейсе, — при которой общее количество перевезенных пассажиров превышает установленное пользователем значение. Перебор осуществляется по диапазонам: маршруты — от 1 до 10, рейсы — от 1 до 8, пассажиры — от 1 до 50. Выполнение завершается при нахождении первой подходящей конфигурации
29	В процессе планирования энергопотребления предприятия требуется определить первую комбинацию — число рабочих линий, число смен в день и количество часов работы каждой линии — при которой производство этих параметров превысит заданное пользователем максимальное допустимое значение энергозатрат. Параметры перебираются в диапазоне: линии — от 1 до 12, смены — от 1 до 3, часы — от 1 до 24. При нахождении удовлетворяющей комбинации алгоритм прекращает выполнение
30	В задаче расчета загрузки складского комплекса необходимо определить первую тройку параметров — количество складских помещений, число контейнеров в каждом и число рабочих дней — при которой общее количество обработанных контейнеров превысит установленный порог. Диапазоны параметров: помещения — от 1 до 15, контейнеры — от 1 до 100, дни — от 1 до 30. По достижении условия выполнение прерывается

Equation Section (Next)

4. Подпрограммы

4.1. Пользовательские функции

Подпрограмма (пользовательская функция) — именованный блок кода, предназначенный для многократного вызова и содержащий логически завершенную последовательность инструкций.

В Python подпрограмма реализуется в виде функции, определяемой при помощи ключевого слова **def**, и, возможно, возвращающей результат с использованием оператора **return**.

Объявление подпрограммы имеет следующий синтаксис:

```
def имя_функции (параметры) :  
    тело_функции  
    return результат
```

Здесь

имя_функции — идентификатор, присваиваемый объекту типа *function*;

параметры — список формальных параметров (возможно, пустой);

тело_функции — последовательность инструкций, исполняемых при вызове подпрограммы;

return результат — необязательное указание возвращаемого значения.

При вызове подпрограммы создается новое локальное пространство имен, в котором связываются значения переданных аргументов с формальными параметрами. Выполнение инструкции **return** немедленно завершает выполнение функции и возвращает указанное значение. Если **return** не указан, завершение функции происходит по достижении конца тела, и функция возвращает объект **None**.

Вызов подпрограммы осуществляется по синтаксису

```
имя_функции (параметры)
```

Подпрограмма может принимать параметры различных типов, определяющих способ передачи аргументов при вызове. С точки зрения формального описания подпрограммы, параметры служат переменными, через которые осуществляется отображение входных данных на выходные значения. Механизм задания параметров в Python поддерживает разнообразные формы передачи: позиционную, именованную, с параметрами по умолчанию, произвольную позиционную и произвольную именованную. Каждому типу параметра

соответствует строго определенный синтаксис и семантика обработки во время вызова функции.

Позиционные параметры — параметры, значения которым передаются по порядку следования при вызове. Пример использования приведен в листинге 4.1.

Листинг 4.1

```
# Подпрограмма
def f(a, b, c):
    ...

# Основная программа
f(1, 2, 3)
# Значения 1, 2, 3 связываются с параметрами a, b, c соответственно.
```

Параметры со значением по умолчанию — параметры, которым при определении функции присвоены значения, используемые при отсутствии соответствующих аргументов во время вызова. Все параметры со значением по умолчанию должны следовать после всех параметров без значения по умолчанию. Пример использования приведен в листинге 4.2.

Листинг 4.2

```
# Подпрограмма
def f(a, b=0, c=1):
    ...

# Основная программа
f(5)          # a=5, b=0, c=1
f(5, 2)       # a=5, b=2, c=1
f(5, 2, 3)    # a=5, b=2, c=3
```

Именованные параметры (ключевые аргументы) — параметры, которым значения передаются по имени (явно), независимо от порядка. Пример использования приведен в листинге 4.3.

Листинг 4.3

```
# Подпрограмма
def f(a, b, c):
    ...

# Основная программа
f(c=3, a=1, b=2)
# Внутри функции a=1, b=2, c=3
```

Произвольное количество позиционных параметров (***args**) — параметр, начинающийся с символа *****, собирает все лишние позиционные аргументы в кортеж (см. п. 9). Пример использования приведен в листинге 4.4.

Листинг 4.4

```
# Подпрограмма
def f(x, *args):
    ...

# Основная программа
f(1, 2, 3, 4)
# Внутри функции x=1, args=(2, 3, 4)
```

Произвольное количество именованных параметров (****kwargs**) — параметр, начинающийся с ******, собирает все лишние именованные аргументы в словарь (см. п. 11). Пример использования приведен в листинге 4.5.

Листинг 4.5

```
# Подпрограмма
def f(x, **kwargs):
    ...

# Основная программа
f(1, y=2, z=3)
# Внутри функции x=1, kwargs={'y': 2, 'z': 3}
```

Пример 4.1

Реализовать функцию, проверяющую, можно ли заданное натуральное число представить в виде суммы всех его цифр, умноженных на соответствующие степени числа 4, начиная с самой младшей цифры (например, $n = 123 \rightarrow 3 \cdot 4^1 + 2 \cdot 4^2 + 1 \cdot 4^3$). Использовать только арифметику и целочисленные операции. Найти все трехзначные числа, обладающие этим свойством.

Решение

Программный код решения приведен в листинге 4.6.

Листинг 4.6

```
def f(n):
    k = 0 # Счетчик количества цифр в числе
    while n >= 10**k:
        k += 1

    # Определение суммы
    s = 0
    for i in range(k):
        s += 4**(i + 1) * ((n // (10**(i))) % 10)
```

```

# Возвращаем результат сравнения суммы с числом
return s == n

flag = False # Флаг - есть ли вообще такие числа

# Перебор всех трехзначных чисел
for n in range(100, 1000):
    # Если подпрограмма вернула True, ...
    if f(n):
        print(n, end=' ') # то выводим число
        flag = True # и меняем значение флага

# Проверяем, что нужные числа есть
if not flag:
    print('Таких чисел нет')
# 128 136 144 152 160 288 296

```

Задание 4.1

Выполнить задания, представленные в табл. 4.1. Отчет должен включать:

1. Полную формулировку задачи.
2. Программный код с обязательными пояснениями в виде комментариев.
3. Результаты работы программы с различными наборами исходных данных, демонстрирующие различные выходные значения.

Таблица 4.1

№	Задания
1	Реализовать функцию, проверяющую, образует ли заданное натуральное число монотонную последовательность цифр (либо строго возрастающую, либо строго убывающую). Не использовать строки и списки. Используя функцию, определить минимальное четырехзначное число, удовлетворяющее этому условию и делящееся на 9
	Создать функцию, определяющую, можно ли представить заданное натуральное число в виде суммы квадратов двух различных натуральных чисел. Использовать ее для нахождения первого числа от 100 до 1000, для которого такое представление возможно
2	Написать функцию, вычисляющую так называемую стабильную сумму цифр: на каждом шаге вычисляется сумма цифр числа, затем сумма цифр этой суммы и так далее, пока результат не станет однозначным. Использовать ее для анализа чисел от 1 до 1000, определить количество чисел, у которых стабильная сумма равна 9
	Реализовать функцию, проверяющую, совпадают ли крайние (первая и последняя) цифры у двух заданных натуральных чисел. С ее помощью определить, сколько пар чисел от 10 до 99 имеют это свойство
3	Создать функцию, определяющую, является ли произведение цифр заданного числа точной степенью другого заданного числа k . Использовать функцию для анализа чисел от 10 до 999, проверить, сколько из них удовлетворяют условию при $k = 2$

№	Задания
	Реализовать функцию, определяющую, является ли заданное натуральное число палиндромом в двоичной системе счисления. Использовать только арифметические операции. Проверить, сколько чисел от 1 до 1000 обладают этим свойством
4	Создать функцию, которая для двух заданных чисел возвращает количество общих цифр в их десятичных записях. Не использовать списки и строки. Использовать ее для анализа всех пар чисел от 10 до 99500, удовлетворяющих этому условию
	Написать функцию, возвращающую количество уникальных цифр в заданном числе. Не использовать строковые преобразования. Использовать функцию для анализа всех чисел от 100 до 999 и найти число с наибольшим количеством различных цифр
5	Создать функцию, проверяющую, можно ли упорядочить цифры заданного числа по неубыванию без перестановки цифр. Разрешено только сравнение соседних цифр при последовательном извлечении. Использовать эту функцию для подсчета количества четырехзначных чисел с данной структурой
	Реализовать функцию, которая определяет, является ли заданное натуральное число числом Армстронга, то есть равно ли оно сумме своих цифр, возведенных в степень, равную количеству цифр. Не использовать строки, списки и другие структуры. С ее помощью найти все такие числа на интервале от 100 до 9999
6	Создать функцию, проверяющую, можно ли заданное число получить путем сложения трех его различных цифр. Не использовать структуры данных. Использовать функцию для нахождения всех четырехзначных чисел, удовлетворяющих этому условию
	Реализовать функцию, которая для заданного числа вычисляет максимальную разность между двумя его цифрами. Не использовать строки и списки. Найти число от 100 до 999, у которого эта разность наибольшая
7	Создать функцию, определяющую, чередуются ли четные и нечетные цифры в десятичной записи заданного натурального числа. Используя ее, найти наименьшее число больше 1000 с этой структурой
	Реализовать функцию, вычисляющую расстояние Хэмминга между десятичными записями двух чисел одинаковой длины. Расстояние Хэмминга — число позиций, в которых различаются соответствующие цифры двух чисел одинаковой длины. Запрещается использовать строки и массивы. Использовать эту функцию для анализа всех пар трехзначных чисел и определить максимальное найденное значение расстояния
8	Создать функцию, определяющую, можно ли перестановкой цифр заданного числа получить число, делящееся на 11. Запрещается использовать какие-либо структуры данных. Проверить это свойство для чисел от 100 до 999
	Реализовать функцию, подсчитывающую количество четных цифр, следующих за нечетной в десятичной записи числа. Использовать только арифметику и переменные. Применить ее для анализа чисел от 100 до 10000
9	Создать функцию, которая определяет, образуют ли цифры заданного числа геометрическую прогрессию. Не использовать строковые и коллекционные типы данных. Найти все трехзначные числа с этим свойством
	Реализовать функцию, определяющую, совпадают ли суммы крайних и средних цифр четырехзначного числа. Использовать ее для подсчета количества таких чисел в диапазоне от 1000 до 9999
10	Создать функцию, проверяющую, содержит ли заданное число хотя бы одну пару одинаковых подряд идущих цифр. Не использовать строки и массивы. Найти минимальное натуральное число больше 500, обладающее этим свойством

№	Задания
	Реализовать функцию, определяющую, является ли произведение цифр заданного числа палиндромом. Не использовать строки и списки. Использовать эту функцию для нахождения первого числа от 100 до 1000, удовлетворяющего этому условию
11	Создать функцию, вычисляющую количество переходов знака между соседними цифрами заданного числа (например, от четной к нечетной или от нечетной к четной). Использовать эту функцию для нахождения числа от 1000 до 9999 с максимальным количеством переходов
	Реализовать функцию, проверяющую, совпадает ли число с суммой своих цифр, каждая из которых предварительно возведена в квадрат. Запрещено использовать строки и структуры данных. Найти все такие числа от 1 до 10000
12	Создать функцию, определяющую, можно ли представить заданное натуральное число как сумму последовательных натуральных чисел. Использовать ее для нахождения наименьшего числа больше 100, удовлетворяющего этому условию
	Реализовать функцию, проверяющую, образуют ли цифры числа строго чередующуюся последовательность: больше – меньше – больше – ... или наоборот. Найти среди трехзначных чисел такие, у которых наблюдается указанная структура
13	Создать функцию, определяющую, является ли заданное натуральное число числом Харшада, то есть делится ли оно на сумму своих цифр. Использовать ее для подсчета количества таких чисел среди всех четырехзначных
	Реализовать функцию, проверяющую, можно ли разделить цифры заданного числа на две непустые группы с равной суммой. Не использовать списки, строки и рекурсию. Использовать ее для анализа чисел от 100 до 999
14	Создать функцию, которая проверяет, встречается ли хотя бы одна цифра заданного числа более одного раза. Использовать только арифметику. Определить количество четырехзначных чисел, у которых хотя бы одна цифра повторяется
	Реализовать функцию, которая определяет, состоит ли заданное число только из четных цифр. Использовать ее для анализа диапазона от 100 до 9999 и нахождения максимального числа, удовлетворяющего этому условию
15	Создать функцию, вычисляющую разность между числом, составленным из четных цифр заданного числа, и числом, составленным из нечетных цифр в порядке их появления. Использовать только арифметические операции. Найти такую разность для заданного трехзначного числа
	Реализовать функцию, определяющую, можно ли представить заданное натуральное число в виде произведения трех различных цифр, входящих в его десятичную запись. Не использовать строки, списки и множества. Применить эту функцию для анализа чисел от 100 до 999
16	Создать функцию, проверяющую, чередуются ли в заданном числе одинаковые цифры с различными. Например, в числе 2727 чередуются 2 и 7, в числе 1221 — нет. Разрешено использовать только арифметику и переменные. Найти все четырехзначные числа с этим свойством
	Реализовать функцию, определяющую, содержит ли заданное число ровно одну четную цифру, причем эта цифра должна быть строго больше всех остальных цифр. Использовать только арифметику. Определить количество таких чисел среди трехзначных
17	Создать функцию, которая определяет, образуют ли цифры числа арифметическую прогрессию. Не использовать строковые и коллекционные типы данных. Применить функцию к анализу всех трехзначных чисел
	Реализовать функцию, которая по заданному числу проверяет, равна ли сумма его цифр количеству делителей этого числа. Не использовать структуры данных. Найти все числа до 1000, удовлетворяющие данному условию

№	Задания
18	Создать функцию, определяющую, является ли сумма обратных значений цифр данного числа целым числом (например, $1/2 + 1/4 + 1/5$). Использовать только целочисленную арифметику и проверку делимости. Применить ее для анализа всех чисел от 100 до 999
	Реализовать функцию, вычисляющую число, образованное из цифр заданного числа, записанных в порядке убывания. Не использовать сортировку и списки. Найти максимальное число, полученное таким способом среди четырехзначных чисел
19	Создать функцию, проверяющую, различаются ли все цифры заданного числа на фиксированную величину d (например, 3, 5 и 7 различаются на 2). Использовать ее для поиска всех трехзначных чисел с равной разностью между любыми парами цифр
	Реализовать функцию, определяющую, можно ли число разбить на две части (последовательно по цифрам), сумма которых равна заданному значению. Использовать только арифметику. Применить к числам от 1000 до 9999
20	Создать функцию, которая определяет, содержит ли число строго более одной пары одинаковых цифр, причем пары не должны пересекаться. Использовать только арифметику. Найти такие числа среди четырехзначных
	Реализовать функцию, проверяющую, являются ли сумма четных цифр и сумма нечетных цифр заданного натурального числа взаимно простыми числами. Использовать ее для анализа чисел от 100 до 999
21	Создать функцию, которая проверяет, можно ли переставить цифры числа так, чтобы образованное число было кратно 3. Не использовать строки и списки. Найти первое четырехзначное число, для которого это возможно
	Реализовать функцию, определяющую, можно ли число представить в виде суммы его максимальной и минимальной цифры. Использовать только арифметику. Найти количество таких чисел среди чисел от 100 до 999
22	Написать функцию, которая вычисляет разность между квадратом суммы цифр и суммой квадратов цифр заданного числа. Использовать ее для нахождения числа от 100 до 999, для которого эта разность максимальна
	Создать функцию, проверяющую, являются ли все цифры заданного числа взаимно различными и образуют последовательность подряд идущих натуральных чисел. Использовать только арифметику и переменные
23	Реализовать функцию, определяющую, содержит ли заданное число хотя бы одну пару одинаковых цифр, стоящих на четных позициях справа налево. Использовать ее для подсчета количества таких чисел среди трехзначных
	Создать функцию, которая определяет, равна ли сумма крайних цифр числа разности суммы и произведения его цифр. Найти все числа от 100 до 999, удовлетворяющие этому условию
24	Реализовать функцию, проверяющую, увеличивается ли каждая следующая цифра заданного числа вдвое по сравнению с предыдущей. Использовать арифметику. Найти все такие числа среди четырехзначных
	Создать функцию, которая проверяет, является ли число числом Люка. Использовать только арифметику. Проверить, сколько таких чисел содержится в диапазоне от 1 до 1000
25	Реализовать функцию, определяющую, делится ли сумма цифр числа на разность максимальной и минимальной его цифр. Найти минимальное число от 100 до 999, удовлетворяющее этому свойству
	Создать функцию, которая проверяет, может ли заданное число быть квадратом суммы своих цифр. Использовать только арифметику. Найти такие числа среди всех натуральных до 500

№	Задания
26	Реализовать функцию, проверяющую, содержит ли десятичная запись числа хотя бы один блок из трех одинаковых подряд идущих цифр. Использовать только арифметику и переменные. Найти наименьшее число, удовлетворяющее этому условию
	Создать функцию, определяющую, образуют ли цифры числа симметричную последовательность относительно центральной цифры (например, 12321). Не использовать строки. Найти все такие числа среди пятизначных
27	Реализовать функцию, проверяющую, является ли число произведением своей первой и последней цифры, умноженным на количество цифр. Использовать ее для проверки чисел от 10 до 9999
	Создать функцию, которая определяет, существует ли в числе ровно две четные и ровно две нечетные цифры. Не использовать структуры данных. Найти все четырехзначные числа, удовлетворяющие этому условию
28	Реализовать функцию, определяющую, является ли сумма цифр числа палиндромом. Использовать только арифметику и переменные. Найти такие числа в диапазоне от 100 до 999
	Создать функцию, которая по заданному числу определяет, совпадает ли оно с суммой произведений всех пар его различных цифр. Не использовать строки и списки. Найти такие числа в пределах от 100 до 9999
29	Реализовать функцию, определяющую, можно ли при помощи только перестановки цифр числа получить простое число. Запрещено использовать структуры данных. Проверить это свойство для всех трехзначных чисел
	Создать функцию, проверяющую, состоит ли заданное число из двух одинаковых блоков, каждый из которых содержит одинаковую последовательность цифр. Например, 1212 или 4343. Использовать арифметику
30	Реализовать функцию, определяющую, делится ли разность между числом и суммой квадратов его цифр на количество цифр. Использовать ее для нахождения наименьшего четырехзначного числа, удовлетворяющего этому условию
	Создать функцию, которая определяет, образуют ли цифры заданного четырехзначного числа правильную зеркальную симметрию при разложении на две половины (например, 1221, 3443). Использовать только арифметические операции. Определить количество таких чисел на интервале от 1000 до 9999

Ряд Тейлора — это бесконечный степенной ряд, представляющий аналитическую функцию в окрестности фиксированной точки.

Если $f(x)$ — бесконечно дифференцируемая функция в точке $x = a$, то ее разложение в ряд Тейлора имеет вид

$$\sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n, \quad (4.1)$$

где $f^{(n)}(a)$ — n -я производная функции в точке a , $n!$ — факториал числа n .

Разложение в точке $a = 0$ называется *рядом Маклорена* и имеет вид

$$\sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!} x^n. \quad (4.2)$$

Значение функции в произвольной точке x_0 можно вычислить с помощью этого ряда, если x_0 лежит внутри интервала сходимости данного степенного ряда, то есть если $|x_0| < R$, где R — радиус сходимости ряда. В этом

случае бесконечная сумма членов степенного ряда сходится к точному значению функции $f(x_0)$, и вычисление значения функции в этой точке допускается. Если $|x_0| \geq R$, то ряд может либо расходиться, либо не совпадать с функцией, даже если он сходится. Следовательно, вычисление значения функции по ряду Маклорена допустимо, только если точка x_0 принадлежит области сходимости ряда и сама функция является аналитической на этом интервале.

Пример 4.2

Вычислить значение функции $f(x) = e^x$ при $x = 0.5$, используя формулу разложения в ряд Маклорена

$$\sum_{n=0}^{\infty} \frac{x^n}{n!}$$

с точностью $\varepsilon = 10^{-5}$. Вычисление членов ряда оформить в виде подпрограммы. Вывести результат, полученный из значений ряда и вычисленный с использованием математических операций.

Решение

Программный код решения приведен в листинге 4.7.

Листинг 4.7

```
from math import *

def f(x, n):
    return x**n / factorial(n)

x, eps, s, n = 0.5, 10**-6, 0, 0

while f(x, n) >= eps:
    s += f(x, n)
    n += 1
print(s)
print(exp(x))
# 1.6487211681547618
# 1.6487212707001282
```

Задание 4.2

Вычислить значение функции $f(x)$ из табл. 4.2 при $x = 0.5$, используя разложение в ряд Маклорена. Прекратить вычисления очередного члена, когда он станет меньше значения $\varepsilon = 10^{-7}$. Вычисление членов ряда оформить в виде подпрограммы. Вывести результат, полученный из значений ряда и вычисленный с использованием математических операций.

Отчет должен включать:

1. Полную формулировку задачи.

2. Программный код с обязательными пояснениями в виде комментариев.

3. Результат работы программы.

Таблица 4.2

№	Функция $f(x)$	Формула разложения в ряд Маклорена	Радиус сходимости R
1	$\sin x$	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!}$	∞
2	$\cos x$	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!}$	∞
3	$\sinh x$	$\sum_{n=0}^{\infty} \frac{x^{2n+1}}{(2n+1)!}$	∞
4	$\cosh x$	$\sum_{n=0}^{\infty} \frac{x^{2n}}{(2n)!}$	∞
5	$\ln(1+x)$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^n}{n}$	1
6	$\arctan x$	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{2n+1}$	1
7	$\arcsin x$	$\sum_{n=0}^{\infty} \frac{(2n)!}{4^n (n!)^2 (2n+1)} x^{2n+1}$	1
8	$\ln x$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{(x-1)^n}{n}$	1
9	$x \sin x$	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+2}}{(2n+1)!}$	∞
10	$x \cos x$	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n)!}$	∞
11	xe^x	$\sum_{n=1}^{\infty} \frac{nx^n}{n!}$	∞
12	e^{-x^2}	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{n!}$	∞
13	$x^2 e^x$	$\sum_{n=2}^{\infty} \frac{n(n-1)x^n}{n!}$	∞
14	$\frac{\ln(1+x)}{x}$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^{n-1}}{n}$	1
15	$\ln\left(\frac{1+x}{1-x}\right)$	$2 \sum_{n=0}^{\infty} \frac{x^{2n+1}}{2n+1}$	1
16	$\ln(1+x^2)$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^{2n}}{n}$	1
17	$\tan^{-1}(x^2)$	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{4n+2}}{2n+1}$	1

№	Функция $f(x)$	Формула разложения в ряд Маклорена	Радиус сходимости R
18	$\frac{1}{x} \ln(1+x)$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^{n-1}}{n}$	1
19	$\frac{1}{(1+x)^2}$	$\sum_{n=1}^{\infty} (-1)^n n x^{n-1}$	1
20	$\ln(1+x^2)$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^{2n}}{n}$	1
21	$\arctan(x^3)$	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{6n+3}}{2n+1}$	1
22	$\frac{1-\cos x}{x^2}$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^{2n-2}}{(2n)!}$	∞
23	$x \arctan x$	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+2}}{2n+1}$	1
24	$\ln\left(\frac{1+x}{\sqrt{1-x}}\right)$	$\sum_{n=1}^{\infty} \left(\frac{1}{n} + \frac{1}{2n}\right) x^n$	1
25	$x^3 \ln(1+x)$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^{n+3}}{n}$	1
26	$\frac{\ln(1+x)}{1+x}$	$\sum_{n=1}^{\infty} (-1)^{n+1} \left(1 - \frac{1}{n+1}\right) x^n$	1
27	$x \ln(1+x)$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^{n+1}}{n}$	1
28	$x^2 \cos x$	$\sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+2}}{(2n)!}$	∞
29	$\frac{x}{1+x^2}$	$\sum_{n=0}^{\infty} (-1)^n x^{2n+1}$	1
30	$\ln(1+x^3)$	$\sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^{3n}}{n}$	1

Equation Section (Next)

5. Строковый тип

5.1. Обработка строковых данных

Обработка строк в языке Python представляет собой совокупность операций и методов, предназначенных для создания, преобразования, анализа и сопоставления текстовых данных. Строки в Python представлены неизменяемым типом **str**, основанным на последовательности символов Unicode.

Строка литерально задается с использованием одинарных ('...'), двойных ("...") или тройных кавычек ('''...''', """...""") для создания многострочных литералов.

Пустая строка — это строка, не содержащая символов. Литерально задается как ' ' или "".

Функция **len()** возвращает длину строки (количество символов в строке).

Функции **chr()** и **ord()** реализуют взаимнообратное преобразование между целыми числами и символами в соответствии со стандартом Unicode. Эти функции являются встроенными и предназначены для работы с кодами символов.

Функция **ord(c)** возвращает целочисленный код символа *c* в таблице Unicode. Аргумент *c* должен быть строкой длины 1 (один символ). Примеры использования приведены в листинге 5.1.

Листинг 5.1

ord ('A')	# 65
ord ('a')	# 97
ord ('0')	# 48
ord ('Ф')	# 1060

Функция **chr(i)** возвращает строку из одного символа, код которого в Unicode равен целому числу *i*. Аргумент *i* должен быть целым числом в диапазоне от 0 до 1114111. Примеры использования приведены в листинге 5.2.

Листинг 5.2

chr (65)	# 'A'
chr (97)	# 'a'
chr (1060)	# 'Ф'
chr (8364)	# '€'

В листинге 5.3 приведен пример формирования строк из последовательностей букв алфавита. Обратите внимание, что буква 'ё' не располагается между букв 'е' и 'ж'.

Листинг 5.3

```
# Прописные латинские буквы
s1 = ''
for i in range(ord('A'), ord('A') + 26):
    s1 += chr(i)
print(s1)
# ABCDEFGHIJKLMNOPQRSTUVWXYZ

# Строчные русские буквы
s2 = ''
for i in range(ord('а'), ord('а') + 32):
    s2 += chr(i)
print(s2)
# абвгдежзийклмнопрстуфхцчщъыьэюя

print(ord('ё')) # 1105
```

Экранированные символы (*escape sequences, escape-последовательности*) представляют собой специальные символьные последовательности внутри строкового литерала, начинающиеся с символа обратной косой черты \, и интерпретируемые не как обычные символы, а как управляющие инструкции или символы с особым значением. Экранирование используется для представления символов, которые не могут быть напрямую записаны в строку из-за ограничений синтаксиса или печати, например символов перевода строки, табуляции, кавычек или управляющих кодов. В табл. 5.1 приведены экранированные символы, используемые в языке Python.

Таблица 5.1

Последовательность	Символ (описание)	Пример строки	Результат
\\	Обратная косая черта	'\\\\'	'\\'
\'	Апостроф (одинарная кавычка)	'It\'s fine'	"It's fine"
\"	Кавычка двойная	"She said: \"Hi\""	'She said: "Hi"'
\n	Перевод строки (LF, Line Feed)	'Line1\nLine2'	'Line1 Line2'
\r	Возврат каретки (CR, Carriage Return)	'A\rB'	'B' (при выводе на той же строке)
\t	Табуляция (горизонтальная)	'A\tB'	'A\tB'
\b	Возврат на один символ (backspace)	'ABC\bD'	'ABD'

Последовательность	Символ (описание)	Пример строки	Результат
\a	Звуковой сигнал (bell, alert)	'\a'	(сигнал в терминале, если поддерживается)
\v	Табуляция (вертикальная)	'A\vB'	'A\vB'
\ooo	Символ с восьмеричным кодом ooo (три цифры)	'\141'	'a'
\xhh	Символ с шестнадцатеричным кодом hh	'\x61'	'a'
\N{name}	Юникодный символ по имени (имя из базы Unicode)	'\N{GREEK SMALL LETTER ALPHA}'	'α'
\uXXXX	Юникодный символ с 4-значным hex-кодом	'\u03B1'	'α'
\UXXXXXXXX	Юникодный символ с 8-значным hex-кодом	'\U0001F600'	'🤪'

Сравнение символов в языке программирования Python осуществляется с использованием стандартных операторов сравнения: `==`, `!=`, `<`, `<=`, `>`, `>=`. Символы сравниваются лексикографически, то есть в соответствии с их целочисленными кодами Unicode, определяемыми функцией `ord()`.

Можно сравнивать не только отдельные символы, но и строки произвольной длины. При этом сравнение происходит лексикографически, то есть посимвольно слева направо, по правилам сравнения символов.

Расположение десятичных цифр, символов русского и английского алфавитов в таблице символов Unicode представлено в табл. 5.2.

Таблица 5.2

Класс символов	Диапазон Unicode (десятичный)
Цифры '0'–'9'	48–57
Латинские заглавные	65–90
Латинские строчные	97–122
Русские заглавные	1040–1071
Русские строчные	1072–1103
Символ 'Ё'	1025
Символ 'ё'	1105

Строка s является итерируемым объектом, индексируемым (нумеруемым по элементам-символам) следующим образом:

— неотрицательные индексы (слева направо)

$0, 1, \dots, \text{len}(s) - 2, \text{len}(s) - 1;$

— отрицательные индексы (справа налево) — от -1 до $-\text{len}(s)$

$-\text{len}(s), -(\text{len}(s) - 1), \dots, -2, -1.$

Нетрудно заметить, что разница неотрицательного индекса i_n и отрицательного индекса i_o одного и того же символа строки равна

$$i_n - i_o = \text{len}(s). \quad (5.1)$$

Это свойство можно использовать при одновременной работе с неотрицательными и отрицательными индексами.

Обращение к символу строки s по индексу i выполняется как $s[i]$.

Базовые операции над строками:

— *конкатенация* (объединение) строк $s1 + s2;$

— повторение (умножение строки на целое число) $s * k.$

Основные методы класса **str** представлены в табл. 5.5. Методы не изменяют исходную строку, а возвращают новую.

Таблица 5.3

Метод	Назначение	Пример вызова	Результат
<code>s.lower()</code>	Преобразование всех символов строки в нижний регистр	<code>'ABC'.lower()</code>	<code>'abc'</code>
<code>s.upper()</code>	Преобразование всех символов строки в верхний регистр	<code>'abc'.upper()</code>	<code>'ABC'</code>
<code>s.capitalize()</code>	Перевод первого символа в верхний регистр, остальных — в нижний	<code>'hello'.capitalize()</code>	<code>'Hello'</code>
<code>s.title()</code>	Преобразование первой буквы каждого слова в верхний регистр	<code>'hello world'.title()</code>	<code>'Hello World'</code>
<code>s.swapcase()</code>	Инверсия регистра символов	<code>'HeLlO'.swapcase()</code>	<code>'hElLo'</code>
<code>s.strip()</code>	Удаление начальных и конечных пробельных символов	<code>' text '.strip()</code>	<code>'text'</code>
<code>s.lstrip()</code>	Удаление пробелов слева	<code>' text'.lstrip()</code>	<code>'text'</code>
<code>s.rstrip()</code>	Удаление пробелов справа	<code>'text '.rstrip()</code>	<code>'text'</code>
<code>s.replace(a, b)</code>	Замена всех вхождений подстроки a на b	<code>'1-2-3'.replace('-', ',')</code>	<code>'1,2,3'</code>
<code>s.replace(a, b, k)</code>	Замена первых k вхождений подстроки a на b	<code>'1-2-3'.replace('-', ', ', k=2)</code>	<code>'1,2,3'</code>
<code>s.find(sub)</code>	Возврат индекса первого вхождения sub , либо -1	<code>'abcabc'.find('b')</code>	<code>1</code>
<code>s.rfind(sub)</code>	Возврат индекса последнего вхождения sub , либо -1	<code>'abcabc'.rfind('b')</code>	<code>4</code>

Метод	Назначение	Пример вызова	Результат
<code>s.index(sub)</code>	То же, что <code>find</code> , но при отсутствии вызывает <code>ValueError</code>	<code>'abc'.index('b')</code>	1
<code>s.count(sub)</code>	Подсчет количества вхождений <code>sub</code>	<code>'ababab'.count('ab')</code>	3
<code>s.startswith(prefix)</code>	Проверка, начинается ли строка с <code>prefix</code>	<code>'abc'.startswith('a')</code>	True
<code>s.endswith(suffix)</code>	Проверка, заканчивается ли строка на <code>suffix</code>	<code>'abc'.endswith('c')</code>	True
<code>s.isalpha()</code>	Проверка, состоит ли строка только из букв	<code>'abc'.isalpha()</code>	True
<code>s.isdigit()</code>	Проверка, состоит ли строка только из цифр	<code>'123'.isdigit()</code>	True
<code>s.isalnum()</code>	Проверка, состоит ли строка только из букв и цифр	<code>'abc123'.isalnum()</code>	True
<code>s.isspace()</code>	Проверка, состоит ли строка только из пробелов	<code>' '.isspace()</code>	True
<code>s.islower()</code>	Проверка, все ли буквы — в нижнем регистре	<code>'abc'.islower()</code>	True
<code>s.isupper()</code>	Проверка, все ли буквы — в верхнем регистре	<code>'ABC'.isupper()</code>	True
<code>s.split(sep)</code>	Разделение строки по разделителю <code>sep</code>	<code>'a,b,c'.split(',')</code>	<code>['a', 'b', 'c']</code>
<code>s.rsplit(sep)</code>	Разделение строки по <code>sep</code> справа налево	<code>'a,b,c'.rsplit(',', 1)</code>	<code>['a,b', 'c']</code>
<code>s.splitlines()</code>	Разделение строки на строки по символам перевода строки	<code>'a\nb\nc'.splitlines()</code>	<code>['a', 'b', 'c']</code>
<code>'sep'.join(seq)</code>	Объединение элементов <code>seq</code> в строку через разделитель <code>sep</code>	<code>','.join(['a', 'b', 'c'])</code>	<code>'a,b,c'</code>
<code>s.zfill(width)</code>	Дополнение строки нулями слева до длины <code>width</code>	<code>'42'.zfill(5)</code>	<code>'00042'</code>
<code>s.center(w)</code>	Центрирование строки в поле ширины <code>w</code> , заполняя пробелами	<code>'hi'.center(6)</code>	<code>' hi '</code>
<code>s.ljust(w)</code>	Выравнивание строки влево в поле ширины <code>w</code>	<code>'hi'.ljust(6)</code>	<code>'hi '</code>
<code>s.rjust(w)</code>	Выравнивание строки вправо в поле ширины <code>w</code>	<code>'hi'.rjust(6)</code>	<code>' hi '</code>
<code>s.partition(sep)</code>	Разделение строки на три части: до <code>sep</code> , <code>sep</code> , после	<code>'a=b'.partition('=')</code>	<code>('a', '=', 'b')</code>
<code>s.rpartition(sep)</code>	То же, но с конца строки	<code>'a=b=c'.rpartition('=')</code>	<code>('a=b', '=', 'c')</code>
<code>s.encode('utf-8')</code>	Преобразование строки в байтовую последовательность	<code>'abc'.encode('utf-8')</code>	<code>b'abc'</code>

Срезы строк в языке программирования Python реализуют механизм получения подстрок на основе индексации с использованием операторов `[:]`. Синтаксис срезов предоставляет мощный и компактный способ выбора диапазонов символов из строки. Срезы всегда возвращают новый объект строкового типа **str**.

Общий формат операции среза:

`s[start:stop:step]`

Здесь

- `s` — строка,
- `start` — индекс начала (включительно),
- `stop` — индекс конца (не включительно),
- `step` — шаг (интервал между индексами).

Любой из параметров может быть опущен. Если параметр опущен, он принимает значение по умолчанию.

Индексация начинается с нуля: первый символ имеет индекс 0, последний — (`len(s) - 1`). Параметры `start`, `stop` и `step` могут быть как положительными, так и отрицательными.

Если `step > 0`, то срез извлекается слева направо. Если `step < 0`, то срез извлекается справа налево.

Операция среза не вызывает ошибку при выходе границ за пределы строки — результат просто обрезается по границам.

В табл. 5.4 приведены примеры срезов над строкой `s = 'abcdef'`.

Таблица 5.4

Выражение	Результат	Описание действия
<code>s[0:3]</code>	<code>'abc'</code>	Срез с 0 по 3 (не включая 3) индексы, извлекаются первые три символа
<code>s[2:5]</code>	<code>'cde'</code>	Срез с индекса 2 до индекса 5 (не включая 5)
<code>s[:4]</code>	<code>'abcd'</code>	Срез от начала до индекса 4 (не включая 4)
<code>s[3:]</code>	<code>'def'</code>	Срез от индекса 3 до конца строки
<code>s[:]</code>	<code>'abcdef'</code>	Полная копия строки
<code>s[::1]</code>	<code>'abcdef'</code>	Копия строки с шагом 1
<code>s[::2]</code>	<code>'ace'</code>	Каждый второй символ, начиная с позиции 0
<code>s[1::2]</code>	<code>'bdf'</code>	Каждый второй символ, начиная с позиции 1
<code>s[::-1]</code>	<code>'fedcba'</code>	Инвертированная строка
<code>s[4:1:-1]</code>	<code>'edc'</code>	Срез от индекса 4 до индекса 1 (не включая 1), в обратном порядке
<code>s[-3:]</code>	<code>'def'</code>	Последние три символа строки
<code>s[:-3]</code>	<code>'abc'</code>	Все символы, кроме последних трех
<code>s[-4:-1]</code>	<code>'cde'</code>	Срез индексов от -4 до -1 (не включая -1)
<code>s[-1:-4:-1]</code>	<code>'fed'</code>	Срез от последнего символа к -4 (не включая -4), в обратном порядке
<code>s[3:3]</code>	<code>''</code>	Пустая строка, так как начало и конец совпадают
<code>s[5:0:-2]</code>	<code>'fd'</code>	Каждый второй символ с конца к индексу 1 (не включая), в обратном порядке

Проверить, что строка `s` содержит подстроку `sub` можно с использованием оператора включения (**in**) или оператора не включения (**not in**). Примеры приведены в листинге 5.4.

Листинг 5.4

```
s = 'abcabba'
print('bb' in s) # True
print('aa' in s) # False
print('abc' not in s) # False
print('cba' not in s) # True
```

Перебрать все символы строки `s` можно с использованием циклических конструкций (листинг 5.5).

Листинг 5.5

```
s = 'программа'

for i in s:
    print(i, end=' ')
# п р о г р а м м а
print()

i = 0
while i < len(s):
    print(s[i], end=' ')
    i += 1
# п р о г р а м м а
```

Пример 5.1

Написать программу, которая по введенной пользователем строке строит список всех пар соседних слов, в которых вторая буква первого слова совпадает с последней буквой второго. Слова считаются разделенными одним пробелом. Все пары, удовлетворяющие данному условию, следует вывести в порядке появления в строке. При отсутствии подходящих пар программа должна вывести «NONE».

Решение

В контексте этой задачи слово — это последовательность символов, расположенная между двумя пробелами. Чтобы для исходной строки `s` алгоритм определения пар слов работал и для последней пары слов, создадим копию строки `s` (строка `s1`) с добавленным пробелом в конце. Тогда, если в строке `s1` имеется `k` пробелов, количество пар слов будет $(k - 1)$.

Первое слово в паре — подстрока, начиная с 0-го по индексу символа до первого пробела. Его определяем через срез. После этого изменяем строку `s1`, убирая из него первое слово — применяем рез, начиная со следующего символа после первого пробела и до конца строки. Второе слово определяется

аналогично первому — через срез, начиная с начала строки `s1` и до первого пробела. При проверке пар слов учитываем, что первое слово должно иметь длину как минимум в 2 символа. Для проверки, что имеется хотя бы одна нужная пара, используется переменная `flag`, которая изначально имеет значение **False**. Если хотя бы одна пара найдена, значение `flag` меняется на **True**. После завершения перебора пар слов выполняется проверка значения `flag`. Если оно не изменилось (осталось равным **False**), значит, нужных пар слов в строке нет.

Программный код решения задания приведен в листинге 5.6.

Листинг 5.6

```
s = 'Хорошее лето настало пора неплохо потрудиться'

s1 = s + ' '
k = s.count(' ') - 1
flag = False
for i in range(k):
    slovo1 = s1[:s1.index(' ')]
    s1 = s1[s1.index(' ') + 1:]
    slovo2 = s1[:s1.index(' ')]
    if len(slovo1) >= 2 and slovo1[1] == slovo2[-1]:
        flag = True
        print(slovo1, slovo2)
if not flag:
    print('NONE')
```

Пример 5.2

Написать программу, принимающую строку, представляющую собой арифметическое выражение, в котором между каждой цифрой может отсутствовать знак арифметической операции. Программа должна определить, возможно ли расставить знаки «+» и «-» между цифрами так, чтобы итоговое выражение было равно заданному пользователем целому числу `T`. Необходимо перебрать все допустимые конфигурации вставки операций. При наличии решения вывести любое одно; если решений несколько — вывести лексикографически минимальное; если решений нет — «IMPOSSIBLE».

Решение

Программный код решения задания приведен в листинге 5.7.

Листинг 5.7

```
# Исходная строка
s = '723+612-0+34-56'
# Результирующее значение
T = 100
# Набор цифр
alph = '0123456789'
```

```

# Подсчет пар цифр, между которыми можно вставить знак операции
k = 0
for i in range(len(s) - 1):
    if s[i] in alph and s[i + 1] in alph:
        k += 1

# Генерация всевозможных наборов операций
for i in range(3**k):
    iTemp, i3 = i, ''
    while iTemp > 0:
        i3 = str(iTemp % 3) + i3
        iTemp //= 3
    i3 = i3.zfill(k) # Строка длиной k, состоящая из 0, 1 и 2

    k1 = 0
    sNew = s[0] # Новая строка со вставленными операциями
    for j in range(1, len(s)):
        if s[j] in alph and s[j - 1] in alph:
            # Сопоставление 0, 1 и 2 с операциями '' (пусто),
            # '-' и '+' соответственно
            if i3[k] == '0':
                symb = ''
            elif i3[k] == '1':
                symb = '-'
            elif i3[k] == '2':
                symb = '+'
            # К новой строке добавляем операцию,
            # соответствующую символу i3[k] и последующую цифру
            sNew += symb + s[j]
            k1 += 1
        else:
            # ...иначе просто прибавляем к новой строке цифру
            sNew += s[j]
    # Проверяем, что значение выражения равно числу T
    if eval(sNew) == T:
        print(sNew)
        break
    else:
        # Если цикл завершился без break (ни одно выражение не
        # равно T)
        print('IMPOSSIBLE')

# 7-2-3+61+2-0+34-5+6

```

Пояснения к программному коду:

1. Исходная строка *s* и значение *T* задается литерально. Так же задается и набор десятичных цифр.

2. В переменной *k* подсчитываем количество пар цифр, между которыми может потенциально стоять знак.

3. Знака операции может и не быть, в этом случае вместо знака будет стоять пустая строка '', следовательно, всего возможно 3^k вариантов размещения

операций. Для генерации всех вариантов переберем все числа от 0 до $(3^k - 1)$ включительно и переведем эти числа в троичную систему счисления (результат хранится в переменной `i3`). В таком представлении 0 будем считать пустой строкой ' ', 1 — операция '-', 2 — операция '+'. Для того, чтобы строка с операциями была ровно длины `k`, воспользуемся методом `zfill()`, добавляющей лидирующие нули до длины `k`.

4. Далее формируется новая строка `sNew`, содержащая вставляемые операции. Цифры выбираются из исходной строки `s`, а операция определяется из значения символа строки `i3`. В зависимости от того, какой символ стоит на индексе `k1` строки `i3`, определяется операция (переменная `symb`). В итоге строка `sNew` будет содержать вставленные операции.

5. Для проверки значения выражения используем функцию `eval()`, которая вычисляет значение строки `sNew`.

6. Как только найдена строка, значение которой равно `T`, программа выводит эту строку и прерывает выполнение цикла. Если такого не произошло, то получить значение `T` невозможно, и программа выводит 'IMPOSSIBLE'.

Задание 5.1

Выполнить задания, представленные в табл. 5.5.

Таблица 5.5

№	Задания
1	Написать программу, запрашивающую у пользователя строку произвольной длины. Необходимо определить, можно ли из символов этой строки составить палиндром, то есть строку, одинаково читающуюся слева направо и справа налево. Разрешается произвольная перестановка символов. Если это возможно, программа должна вывести «YES», в противном случае — «NO»
	Написать программу, которая запрашивает у пользователя строку, состоящую исключительно из латинских символов. Необходимо определить длину самой длинной подстроки, в которой символы следуют в лексикографически неубывающем порядке. В конце программа должна вывести длину найденной подстроки
2	Написать программу, принимающую на вход строку, содержащую только цифры, и определяющую, можно ли разбить ее на непустые подстроки, представляющие собой возрастающую арифметическую прогрессию. Если такая разбивка существует, необходимо вывести начальный элемент и разность прогрессии, иначе вывести сообщение об отсутствии возможности такой разбивки
	Написать программу, запрашивающую у пользователя строку, содержащую как строчные, так и прописные латинские буквы. Необходимо определить, можно ли привести все буквы строки к одному регистру (все строчные или все прописные) с минимально возможным количеством замен символов. В конце программа должна вывести минимальное количество таких преобразований

№	Задания
3	Написать программу, принимающую на вход строку, представляющую собой арифметическое выражение, записанное без пробелов (например, «12+34-5+6»). Необходимо корректно распарсить выражение, выполнить все операции сложения и вычитания в порядке следования и вывести итоговое числовое значение
	Написать программу, которая принимает строку, содержащую вложенные круглые скобки, и проверяет корректность скобочной структуры. В случае корректности необходимо дополнительно определить максимальную глубину вложенности. В случае некорректной структуры программа должна вывести сообщение об ошибке
4	Написать программу, запрашивающую у пользователя строку, содержащую слова, разделенные пробелами. Требуется определить, имеется ли среди слов палиндром, при этом палиндром должен состоять не менее чем из пяти символов. В случае наличия такого слова программа должна вывести первое из них; если ни одно слово не удовлетворяет условию — вывести сообщение об отсутствии палиндромов
	Написать программу, принимающую строку, состоящую из латинских символов и пробелов. Необходимо преобразовать строку, переставив символы в каждом слове в обратном порядке, при этом порядок следования слов в строке сохраняется. Полученная строка должна быть выведена без использования метода <code>split()</code>
5	Написать программу, которая запрашивает строку, содержащую десятичное представление целого положительного числа, и определяет, можно ли расставить в этой строке круглые скобки так, чтобы полученное выражение стало корректным арифметическим выражением, в котором сумма чисел внутри скобок делится на 7. Все такие позиции (если они существуют) необходимо вывести в виде пар индексов начала и конца скобок
	Написать программу, которая запрашивает у пользователя строку, содержащую только латинские буквы. Требуется определить, существует ли в строке такая непустая подстрока, которая одновременно начинается и заканчивается одной и той же буквой, при этом между этими буквами находится ровно столько символов, сколько составляет алфавитная позиция этой буквы. Программа должна вывести «YES», если такая подстрока существует, иначе — «NO»
6	Написать программу, принимающую на вход строку, состоящую из произвольных символов, и определяющую, является ли строка «зеркальной» по следующему правилу: строка разбивается на две равные части, и каждая позиция в первой части совпадает с соответствующей позицией второй части, считая ее в обратном порядке. Для строк нечетной длины центральный символ игнорируется. В случае зеркальности программа должна вывести «TRUE», иначе — «FALSE»
	Написать программу, которая по введенной пользователем строке, содержащей строчные латинские буквы, определяет количество различных способов удаления ровно одного символа так, чтобы оставшаяся строка стала палиндромом. Все такие позиции необходимо подсчитать и вывести количество допустимых удалений
7	Написать программу, запрашивающую у пользователя строку, содержащую произвольную комбинацию символов, включая скобки, пробелы, знаки препинания и буквы. Требуется определить, является ли строка корректной по следующему критерию: после удаления всех символов, кроме скобок, оставшаяся последовательность должна образовывать правильную скобочную конструкцию. Результат проверки вывести в виде логического значения
	Написать программу, которая по введенной строке с латинскими буквами определяет, образуют ли первые k символов строки (значение k запрашивается отдельно) повторяющийся шаблон, из которого составлена вся строка. Если строка состоит из повторений шаблона длины k , вывести «YES», иначе — «NO»

№	Задания
8	Написать программу, принимающую строку, содержащую произвольное количество слов, разделенных одним пробелом. Программа должна сформировать новую строку, в которой сохраняются только те слова, длина которых является делителем длины всей исходной строки. Полученную строку вывести в одной строке через пробел
	Написать программу, принимающую строку, содержащую смешанный текст с буквами, цифрами и служебными символами. Необходимо определить, есть ли в строке подстрока длины не менее 4, которая одновременно: начинается с согласной, заканчивается на гласную, содержит хотя бы одну цифру и не содержит подряд идущих одинаковых символов. В случае наличия такой подстроки вывести ее первую позицию, иначе — «NONE»
9	Написать программу, принимающую на вход строку, содержащую последовательность слов, разделенных пробелами. Программа должна определить, имеются ли в строке две или более пары слов, являющихся анаграммами друг друга. Если такие пары существуют, вывести количество различных пар
	Написать программу, запрашивающую строку, содержащую комбинацию цифр и символов. Требуется извлечь все числовые подстроки, интерпретировать их как целые числа и определить максимальное среди них. Если числовые подстроки отсутствуют, программа должна вывести ноль
10	Написать программу, которая по введенной пользователем строке определяет, можно ли получить строку, состоящую только из заглавных латинских букв, если последовательно удалить не более одного символа и при необходимости преобразовать оставшиеся символы к верхнему регистру. В случае возможности программа должна вывести «YES», иначе — «NO»
	Написать программу, принимающую строку, содержащую смешанный текст с буквами, цифрами и служебными символами. Необходимо определить, есть ли в строке подстрока длины не менее 4, которая одновременно: начинается с гласной, заканчивается на согласную, содержит хотя бы одну цифру и не содержит подряд идущих одинаковых символов. В случае наличия такой подстроки вывести ее первую позицию, иначе — «NONE»
11	Написать программу, которая по введенной строке составляет все возможные разбиения строки на две непустые части и определяет, сколько таких разбиений имеют свойство: первая часть состоит только из гласных, а вторая — только из согласных латинского алфавита. Все допустимые разбиения необходимо подсчитать и вывести их количество
	Написать программу, запрашивающую у пользователя строку, содержащую только строчные латинские буквы. Необходимо определить, можно ли преобразовать строку в палиндром с помощью удаления не более одного символа. В случае положительного результата следует вывести позицию удаляемого символа (нумерация с нуля), иначе — вывести сообщение об отсутствии решения
12	Написать программу, которая принимает на вход строку, представляющую собой список целых чисел, разделенных запятыми (например, «-2,3,5,10,-1»). Требуется преобразовать эту строку в список чисел и определить длину самой длинной подпоследовательности, элементы которой упорядочены по неубыванию. Использовать только строковые методы и базовые конструкции
	Написать программу, принимающую строку, содержащую одно слово на латинице. Необходимо определить, является ли это слово грамматически допустимым с точки зрения чередования гласных и согласных букв: ни одна из гласных не должна встречаться более двух раз подряд, ни одна из согласных — более трех. В случае соблюдения правила вывести «YES», иначе — «NO»

№	Задания
13	Написать программу, запрашивающую у пользователя строку, содержащую арифметическое выражение в виде последовательности цифр и знаков «+» и «-», без пробелов. Необходимо вычислить результат выражения, строго соблюдая порядок следования операций, без использования функции <code>eval()</code> или аналогов
	Написать программу, которая по введенной строке определяет, можно ли представить ее как результат многократного повторения одной и той же подстроки. В случае положительного результата следует вывести длину повторяющейся подстроки, иначе — «0»
14	Написать программу, которая по заданной строке строит статистику встречаемости всех подстрок длины 2, состоящих из строчных латинских букв. Необходимо вывести все такие подстроки, встречающиеся более одного раза, и количество их появлений, отсортированных по убыванию частоты
	Написать программу, принимающую строку, содержащую произвольную комбинацию символов. Необходимо определить, существует ли такая непустая подстрока, в которой каждый символ встречается четное число раз. В случае наличия вывести длину максимальной подстроки с этим свойством
15	Написать программу, которая по введенной строке идентифицирует все уникальные «разделители» — символы, которые не являются буквами и не являются цифрами. Программа должна вывести отсортированный список таких символов в порядке возрастания их кодов Unicode
	Написать программу, которая по заданной строке определяет, можно ли в ней найти три непересекающихся подстроки одинаковой длины, каждая из которых является палиндромом. Если такие подстроки существуют, программа должна вывести их позиции, иначе — сообщение об отсутствии
16	Написать программу, принимающую строку, содержащую слова, разделенные пробелами. Необходимо определить, являются ли все слова в строке анаграммами одного и того же базового слова. Программа должна корректно обработать как строчные, так и прописные символы
	Написать программу, принимающую строку, содержащую цифры и произвольные символы. Необходимо извлечь из строки все неперекрывающиеся последовательности цифр и определить, сколько из них представляют собой простые числа. Использовать только встроенные средства и логические конструкции
17	Написать программу, которая по заданной строке определяет, сколько различных символов встречаются в ней строго нечетное число раз. В результате вывести общее количество таких символов
	Написать программу, которая по введенной строке строит ее так называемую «сжатую форму»: последовательности одинаковых символов заменяются на пару «символ + количество повторений». Например, «aaabbc» должно быть преобразовано в «a3b2c1». Обратное преобразование реализовывать не требуется
18	Написать программу, которая по строке с произвольными символами определяет, сколько различных символов находятся одновременно в начале и в конце строки, если рассматривать только первые k символов с начала и последние k символов с конца. Значение k запрашивается у пользователя отдельно. Все такие символы должны быть выведены в алфавитном порядке
	Написать программу, принимающую строку, представляющую собой список команд в формате: «вверх», «вниз», «влево», «вправо», разделенные пробелами. Необходимо вычислить, возвращается ли объект в исходную точку после выполнения всех команд. Начальная позиция — (0,0)

№	Задания
19	Написать программу, принимающую на вход строку, содержащую арифметическое выражение с положительными целыми числами и скобками. Необходимо определить, является ли выражение корректным по скобочной структуре. Все возможные ошибки (некорректное вложение, избыточные или пропущенные скобки) должны быть обработаны
	Написать программу, которая по введенной строке находит длину самой длинной подстроки, не содержащей повторяющихся символов. В результате вывести длину найденной подстроки
20	Написать программу, принимающую строку, содержащую латинские слова, разделенные пробелами. Требуется построить новую строку, в которой порядок слов инвертирован, при этом каждое слово также записано в обратном порядке. Разделители между словами должны оставаться одинарными пробелами
	Написать программу, принимающую на вход строку, представляющую собой двоичное число. Необходимо определить, можно ли добавить не более одного символа «0» или «1» в произвольную позицию, чтобы полученное число стало палиндромом. В случае положительного ответа программа должна вывести «YES», иначе — «NO»
21	Написать программу, принимающую строку, содержащую латинские буквы. Необходимо определить, можно ли, переставив символы строки, получить строку, в которой ни один символ не повторяется дважды подряд. В случае существования такой перестановки — вывести ее в лексикографически минимальном виде
	Написать программу, которая запрашивает строку, содержащую только латинские буквы. Требуется определить, можно ли отсортировать символы строки в порядке невозрастания так, чтобы результат имел хотя бы одну непрерывную возрастающую подпоследовательность длины не менее трех. В случае наличия такой подпоследовательности вывести «YES», иначе — «NO»
22	Написать программу, которая по заданной строке определяет, содержит ли она хотя бы одно слово, в котором чередуются гласные и согласные без повторений. Все слова строки разделяются пробелами. Если такое слово найдено, вывести его, в противном случае — сообщение об отсутствии
	Написать программу, запрашивающую строку, содержащую латинские буквы, пробелы и знаки препинания. Программа должна определить, является ли строка панграммой, то есть содержит ли она все 26 строчных латинских букв. Регистр учитывать не требуется
23	Написать программу, которая по введенной строке вычисляет коэффициент сжатия, определяемый как отношение длины строки без повторяющихся подряд символов к длине исходной строки. Например, для строки «aaabbccdd» коэффициент равен $6/10$. Вывести значение с точностью до двух знаков после запятой
	Написать программу, принимающую строку, содержащую круглые, квадратные и фигурные скобки. Необходимо определить, является ли структура скобок корректной по принципу вложенности. Все три типа скобок должны правильно открываться и закрываться. При нарушении структуры вывести «INVALID»
24	Написать программу, которая по введенной пользователем строке определяет, сколько различных палиндромных подстрок содержится в ней. Подстрока считается палиндромом, если читается одинаково в обе стороны. Подсчет должен учитывать только уникальные подстроки
	Написать программу, запрашивающую строку, содержащую только латинские буквы. Требуется определить, какая подстрока максимальной длины состоит исключительно из гласных. Вывести найденную подстроку и ее длину

№	Задания
25	Написать программу, которая по строке, содержащей целые числа, разделенные пробелами, определяет наименьшее число, которое встречается более одного раза. Все числа интерпретируются как целочисленные строки и сравниваются по значению. При отсутствии дубликатов вывести «NONE»
	Написать программу, принимающую строку, состоящую из латинских букв, и проверяющую, можно ли из нее, удаляя не более двух символов, получить строку, в которой количество гласных и согласных совпадает. В случае возможности преобразования вывести «YES», иначе — «NO»
26	Написать программу, которая по введенной строке символов проверяет, является ли она корректным идентификатором в Python. Идентификатор должен начинаться с буквы или символа подчеркивания, содержать только буквы, цифры и подчеркивания и не быть ключевым словом языка. Все проверки необходимо реализовать вручную
	Написать программу, запрашивающую у пользователя строку, представляющую собой арифметическое выражение, записанное с пробелами между операндами и операциями (например, «3 + 5 * 2 - 1»). Необходимо вычислить результат выражения с учетом приоритета операций, не используя <code>eval()</code>
27	Написать программу, которая по заданной строке, содержащей текст на английском языке, подсчитывает количество слов, в которых первая и последняя буквы совпадают. Все слова в строке разделены пробелами и могут содержать знаки препинания
	Написать программу, принимающую строку, содержащую имена переменных, разделенные пробелами. Требуется определить, какие из них не удовлетворяют стилю <code>snake_case</code> (все строчные буквы, слова разделены подчеркиванием, не начинается и не заканчивается подчеркиванием, без двойных подчеркиваний). Вывести список всех некорректных имен
28	Написать программу, которая по строке с текстом определяет самую длинную последовательность слов, в которой каждое следующее слово длиннее предыдущего. Все слова считаются разделенными пробелами. В результате вывести длину найденной последовательности
	Написать программу, принимающую строку, содержащую натуральные числа, разделенные пробелами. Необходимо вычислить, сколько из этих чисел являются палиндромами при записи в десятичной системе. Все числа интерпретировать как строки и сравнивать по симметрии
29	Написать программу, принимающую строку, представляющую собой аббревиатуру (например, «NASA»), и строку текста. Программа должна определить, имеется ли в тексте такая последовательность слов, начальные буквы которых образуют заданную аббревиатуру. При наличии такой последовательности вывести соответствующие слова
	Написать программу, принимающую строку, содержащую произвольную последовательность символов. Необходимо определить длину самой длинной подстроки, содержащей хотя бы по одной цифре, одной заглавной и одной строчной букве
30	Написать программу, запрашивающую строку, содержащую дату в произвольном текстовом формате, например: «5 апреля, 2025». Программа должна выделить числовую часть, название месяца и год, и преобразовать дату в формат «2023-04-05». Все преобразования выполнять вручную, без использования библиотек даты и времени
	Написать программу, принимающую строку, содержащую имена собственные, разделенные пробелами, с произвольным использованием прописных и строчных букв. Требуется привести каждое слово в строке к виду: первая буква — прописная, остальные — строчные. Полученную строку вывести

5.2. Форматированные строки

F-строки (форматированные строковые литералы) представляют собой синтаксическую конструкцию, предназначенную для встраивания выражений непосредственно внутрь строк, с их последующим вычислением и автоматическим преобразованием в строковое представление.

F-строка объявляется с помощью префикса `f` или `F` перед строковым литералом. Выражения, заключенные в фигурные скобки `{ }`, вычисляются в момент выполнения строки и заменяются на строковое представление результата. Выражения внутри `{ }` могут быть произвольными по сложности, в том числе содержать вызовы функций и арифметические операции. F-строки интерпретируются в момент выполнения инструкции.

Пример использования F-строк приведен в листинге 5.8.

Листинг 5.8

```
name = "Иван"
age = 30
print(f"{name}у {age + 10} лет") # 'Ивану 40 лет'
```

F-строки используются так же для форматированного вывода данных. Форматная спецификация задается после выражения внутри F-строки, отделяется двоеточием `:` и имеет следующий простой общий вид:

```
f"{ выражение : формат } "
```

Замечание. Полный синтаксис форматирования при помощи F-строк имеет более сложен, но его изучение не входит в задачи данного издания.

Примеры форматов приведены в табл. 5.6.

Таблица 5.6

Формат	Описание поведения	Пример выражения	Результат строки
<code>.nf</code>	дробное число с <i>n</i> знаками после запятой	<code>f"{3.14159:.2f}"</code>	<code>'3.14'</code>
<code>nd</code>	целое число шириной <i>n</i> , выравненное по правому краю	<code>f"{42:5d}"</code>	<code>' 42 '</code>
<code><n</code>	выравнивание по левому краю	<code>f"{42:<5}"</code>	<code>'42 '</code>
<code>>n</code>	выравнивание по правому краю	<code>f"{42:>5}"</code>	<code>' 42 '</code>
<code>^n</code>	выравнивание по центру	<code>f"{42:^5}"</code>	<code>' 42 '</code>
<code>0nd</code>	целое число с ведущими нулями	<code>f"{42:05d}"</code>	<code>'00042'</code>
<code>+d</code>	всегда указывать знак числа	<code>f"{42:+d}"</code> <code>f"{-42:+d}"</code>	<code>' +42 '</code> <code>' -42 '</code>

Формат	Описание поведения	Пример выражения	Результат строки
d	пробел вместо знака +	f"{42: d}"	' 42 '
,	разделение по разрядам запятой	f"{1234567:,}"	'1,234,567'
—	разделение по разрядам подчеркиванием	f"{1234567:_}"	'1_234_567'
b	двоичное представление	f"{13:b}"	'1101'
o	восьмеричное представление	f"{10:o}"	'12'
x	шестнадцатеричное (строчные буквы)	f"{255:x}"	'ff'
X	шестнадцатеричное (прописные буквы)	f"{255:X}"	'FF'
e	экспоненциальное представление (строчные буквы)	f"{1000:e}"	'1.000000e+03'
E	экспоненциальное представление (прописные буквы)	f"{1000:E}"	'1.000000E+03'
%	процентное представление (умножает на 100 и добавляет %)	f"{0.25:%}"	'25.000000%'
#x, #b, #o	добавление префикса 0x, 0b, 0o к числу в данной системе счисления	f"{10:#b}"	'0b1010'
*<n, .*f, *>n	указание заполнителя (вместо пробела) при выравнивании или точности	f"{42:*<5}" f"{3.1:*>6.2f}"	'42***' '**3.10'

Задание 5.2 (практико-ориентированное задание)

Выполнить задания, представленные в табл. 5.7. Если в задании прямо не указан выводимый текст, сформировать текстовую часть в соответствии со смыслом задания.

Отчет должен включать:

1. Полную формулировку задачи.
2. Программный код с обязательными пояснениями в виде комментариев.
3. Результат работы программы.

Таблица 5.7

№	Задания
1	Написать программу, запрашивающую у пользователя два вещественных числа, представляющих координаты точки. Необходимо вывести строку вида: Координаты точки: (x, y), где значения x и y представлены с точностью до двух знаков после запятой, выровнены по ширине 7 символов Написать программу, запрашивающую у пользователя имя и возраст. Требуется вывести строку следующего вида: Пользователь {имя} зарегистрирован, возраст: {возраст}. Возраст необходимо вывести в поле шириной 3 символа, выровненном по правому краю

№	Задания
2	Написать программу, принимающую вещественное число r , интерпретируемое как радиус круга. Требуется вывести строку, содержащую площадь круга, округленную до четырех знаков после запятой, в формате Площадь круга радиуса r равна S
	Написать программу, запрашивающую у пользователя целое число от 0 до 255 и выводящую его в двоичном, восьмеричном и шестнадцатеричном представлениях в виде форматированной таблицы с выравниванием по правому краю
3	Написать программу, принимающую от пользователя значение температуры в градусах Цельсия. Вывести строку вида Температура: $X^{\circ}\text{C} = Y^{\circ}\text{F}$, где Y — эквивалентное значение по шкале Фаренгейта, округленное до одного знака после запятой
	Написать программу, запрашивающую у пользователя строку и число n . Вывести строку, повторенную n раз, в формате Результат: [строка * n]. Повторение выполнить внутри f-строки
4	Написать программу, запрашивающую у пользователя два вещественных числа — массу тела и рост в метрах. Необходимо вычислить индекс массы тела и вывести его в формате ИМТ: значение, округленном до двух знаков после запятой, дополнительно указав категорию (норма, дефицит, избыточный) в зависимости от диапазона значений
	Написать программу, запрашивающую дату в формате три отдельных числа: день, месяц, год. Требуется вывести дату в формате ДД.ММ.ГГГГ, обеспечив двухсимвольное отображение дня и месяца с ведущими нулями
5	Написать программу, принимающую у пользователя сумму покупки в рублях и количество товаров. Вывести строку вида Средняя стоимость товара: $X.XX$ руб., где $X.XX$ — результат деления с округлением до двух знаков
	Написать программу, запрашивающую у пользователя целое число n и выводящую таблицу квадратов чисел от 1 до n , отформатированную в два столбца: «Число» и «Квадрат», выровненных по центру в полях ширины 10 символов
6	Написать программу, запрашивающую у пользователя строку, представляющую имя файла. Вывести длину имени файла, его расширение (если есть) и выровнять вывод по правому краю с заданной шириной
	Написать программу, принимающую вещественное число и выводящую его с различными форматами: с фиксированной точкой, экспоненциальной формой, округлением до целого, каждый в отдельной строке
7	Написать программу, которая принимает три вещественных числа, представляющих стороны треугольника. Вычислить периметр и полупериметр, вывести оба значения с точностью до трех знаков после запятой, выровненных по ширине 10
	Написать программу, принимающую значение x и вычисляющую значение полинома $3x^2 - 7x + 4$. Вывести результат в формате $f(x) = \text{значение}$, округленном до трех знаков после запятой
8	Написать программу, запрашивающую у пользователя строку и выводящую сообщение вида Введенная строка состоит из N символов, где N — длина строки, вычисленная внутри f-строки
	Написать программу, которая запрашивает у пользователя целое число и выводит строку Число положительное, Число отрицательное или Ноль, используя f-строку с условным выражением (тернарным оператором)
9	Написать программу, запрашивающую три строки от пользователя и выводящую их в виде таблицы, где каждая строка выровнена по левому краю в поле шириной 20 символов

№	Задания
	Написать программу, которая запрашивает сумму и процентную ставку, а затем выводит итоговую сумму после одного года начисления процентов по формуле $S \cdot (1 + r / 100)$, округленную до двух знаков после запятой, с пометкой Итоговая сумма: X.XX руб.
10	Написать программу, запрашивающую три целых числа, представляющих оценки. Требуется вычислить их среднее арифметическое, округлить до одного знака после запятой и вывести строку Средний балл: X.X, где X.X — значение, вычисленное внутри f-строки
	Написать программу, запрашивающую у пользователя три вещественных числа, представляющих длины сторон треугольника. Необходимо вычислить его периметр и вывести строку вида: Периметр треугольника: X. Значение X должно быть округлено до двух знаков после запятой и выровнено по ширине 10 символов по правому краю
11	Написать программу, запрашивающую у пользователя целое число — количество секунд. Требуется вычислить полное количество минут и вывести строку: Прошло минут: M. Значение M должно быть выведено в поле шириной 5 символов, выровнено по центру
	Написать программу, запрашивающую два целых числа — массу тела в килограммах и рост в сантиметрах. Вывести значение индекса массы тела (ИМТ) с точностью до одного знака после запятой в виде: ИМТ: X.X
12	Написать программу, принимающую от пользователя целое число, и выводящую его квадрат и куб в виде таблицы, где все три значения (исходное число, квадрат, куб) выровнены по правому краю в полях шириной 6 символов
	Написать программу, запрашивающую у пользователя вещественное число — цену товара. Требуется вывести строку: Цена с округлением до рубля: X руб., где значение X округлено до ближайшего целого числа
13	Написать программу, принимающую от пользователя два вещественных числа — начальную и конечную температуру в Цельсиях. Требуется вывести их эквиваленты по шкале Кельвина с точностью до четырех знаков после запятой
	Написать программу, запрашивающую у пользователя два вещественных числа, представляющих координаты центра окружности, и одно — радиус. Необходимо вывести строку вида: Центр (x, y), радиус: R. Все числовые значения должны быть выведены с точностью до двух знаков после запятой
14	Написать программу, принимающую от пользователя два вещественных числа: массу вещества и его объем. Вычислить плотность и вывести ее значение, округленное до трех знаков после запятой, с выравниванием по левому краю в поле шириной 12 символов
	Написать программу, принимающую вещественное число, представляющее процент роста прибыли. Вывести это значение в строке с точностью до двух знаков после запятой и знаком процента, выровненным по ширине 6 символов по центру
15	Написать программу, запрашивающую у пользователя расстояние в километрах и время в часах. Требуется вычислить среднюю скорость и вывести ее с точностью до одного знака после запятой, дополнительно выравнивая значение в поле шириной 8 символов по правому краю
	Написать программу, принимающую от пользователя сумму депозита и процентную ставку. Вычислить итоговую сумму через один год по формуле простых процентов и вывести результат с точностью до двух знаков после запятой
16	Написать программу, которая принимает три целых числа и выводит их сумму, среднее арифметическое и произведение в виде строки с пояснением. Все значения должны быть округлены до целого и выровнены по ширине 5 символов

№	Задания
	Написать программу, принимающую вещественное число, представляющее объем куба, и вычисляющую длину ребра. Вывести ее с точностью до трех знаков после запятой
17	Написать программу, которая запрашивает два вещественных числа и выводит их арифметическое и геометрическое среднее, каждое округленное до четырех знаков после запятой
	Написать программу, принимающую натуральное число n и выводящую строку: Двоичное представление числа n : X , где X — результат применения функции $\text{bin}(n)$, и значение n — выровнено в поле шириной 5 символов
18	Написать программу, запрашивающую у пользователя целое число и выводящую его значение в шестнадцатеричной системе счисления в нижнем регистре, выровненное по центру в поле шириной 10 символов
	Написать программу, принимающую от пользователя сумму платежа и процент налога. Вывести строку: Общая сумма с налогом: X . Значение X должно быть округлено до двух знаков после запятой и выровнено по ширине 10 символов по правому краю
19	Написать программу, которая принимает от пользователя ширину и высоту прямоугольника и выводит строку: Площадь: S , где S — произведение, округленное до одного знака после запятой и выровненное в поле шириной 8 символов по центру
	Написать программу, принимающую вещественное число, представляющее температуру по шкале Фаренгейта, и выводящую ее эквивалент в Цельсиях с точностью до двух знаков после запятой
20	Написать программу, которая запрашивает у пользователя два целых числа и выводит строку: Частное: X , остаток: Y , где X и Y — результат целочисленного деления и остаток, выровненные в полях шириной 5 символов по правому краю
	Написать программу, запрашивающую у пользователя вещественное число и выводящую строку: Число в экспоненциальной форме: X , где X — представление числа с шестью знаками после запятой, записанное в формате экспоненциальной записи (e-notation), выровненное по левому краю в поле шириной 15 символов
21	Написать программу, принимающую от пользователя два вещественных числа, обозначающих основание и высоту треугольника. Вывести строку: Площадь треугольника: X , где X — результат, округленный до трех знаков после запятой, выровненный по ширине 10 символов по правому краю
	Написать программу, которая по заданному числу секунд вычисляет количество полных часов и минут и выводит в формате: Часы: H , Минуты: M , где H и M — целые значения, выровненные по центру в полях шириной 4 символа
22	Написать программу, запрашивающую три целых числа и выводящую их сумму и произведение, каждое в поле шириной 8 символов, с выравниванием по левому краю
	Написать программу, принимающую от пользователя вещественное число, представляющее коэффициент ускорения, и выводящую это значение с точностью до пяти знаков после запятой и выравниванием по правому краю в поле шириной 12
23	Написать программу, которая по заданному количеству байт выводит их объем в килобайтах и мегабайтах с точностью до двух знаков после запятой
	Написать программу, которая запрашивает у пользователя три вещественных числа и выводит строку: Среднее значение: X , где X — среднее, округленное до одного знака после запятой, с выравниванием по ширине 7 символов по центру

№	Задания
24	Написать программу, принимающую от пользователя вещественное число — длину окружности. Вычислить радиус и площадь соответствующего круга и вывести их с точностью до четырех знаков после запятой
	Написать программу, запрашивающую у пользователя вещественное число и выводящую его значение, умноженное на π , в экспоненциальной форме, выровненной по левому краю в поле шириной 20
25	Написать программу, запрашивающую у пользователя два целых числа и выводящую их целое частное и остаток, выровненные по правому краю в полях шириной 5 символов
	Написать программу, принимающую от пользователя вещественное значение скорости в м/с и выводящую ее эквивалент в км/ч, округленный до двух знаков после запятой
26	Написать программу, принимающую три целых числа и выводящую строку Минимум: X , где X — наименьшее из трех, выровненное по ширине 5 символов, округление не требуется
	Написать программу, запрашивающую у пользователя вещественное значение, представляющее температуру в Кельвинах. Вывести эквиваленты в Цельсиях и Фаренгейтах с точностью до одного знака после запятой
27	Написать программу, принимающую от пользователя три вещественных числа и выводящую строку: $A = X, B = Y, C = Z$, где X, Y и Z — соответствующие значения с округлением до трех знаков после запятой и выравниванием по ширине 9
	Написать программу, которая по введенному числу выводит строку: Число в шестнадцатеричной форме: 0x X , где X — шестнадцатеричное представление числа, записанное строчными буквами
28	Написать программу, запрашивающую два вещественных значения: начальное количество вещества и постоянную распада. Вычислить количество вещества через 10 секунд и вывести результат с точностью до четырех знаков после запятой
	Написать программу, принимающую вещественное число и выводящую его в виде: Число: X , где X — результат округления до ближайшего целого с выравниванием по центру в поле шириной 6 символов
29	Написать программу, запрашивающую у пользователя целое число и выводящую его значение в двоичной, восьмеричной и шестнадцатеричной формах, каждое выровненное по ширине 8 символов по центру
	Написать программу, запрашивающую у пользователя вещественное число, представляющее плотность, и выводящую это значение с двумя знаками после запятой и обязательным плюсом перед положительным числом
30	Написать программу, принимающую три вещественных значения, обозначающих координаты вектора. Вывести модуль вектора, округленный до двух знаков, в поле шириной 10 символов с выравниванием по правому краю
	Написать программу, принимающую два вещественных значения — начальную и конечную температуру в интервале. Требуется вывести разность с фиксированной точкой и пометкой «Температурный градиент: $X^{\circ}\text{C}$ »

Задание 5.3

На отрезке $[a, b]$ задана функция $f(x, y)$ двух переменных, $x \in [a, b]$, $y \in [c, d]$. Разбить отрезок $[a, b]$ на k интервалов одинаковой длины точками

$$\{x_1 = a, x_2, \dots, x_k, x_{k+1} = b\},$$

отрезок $[c, d]$ на p интервалов одинаковой длины точками

$$\{y_1 = c, y_2, \dots, y_p, y_{p+1} = d\}.$$

Используя инструментарий обработки строк и форматирования, вывести в консоль таблицу в форме, приведенной в листинге 5.9 ($x \in [1, 3]$, $y \in [-12; -10]$, $k = p = 9$).

Листинг 5.9

+	-----+	-----+	-----+	-----+
	x		y	f(x, y)
+	-----+	-----+	-----+	-----+
	x1 = 1.000		y1 = -12.000	-23.714
+	-----+	-----+	-----+	-----+
	x1 = 1.000		y2 = -11.800	-19.675
+	-----+	-----+	-----+	-----+
...				
+	-----+	-----+	-----+	-----+
	x10 = 3.000		y10 = -10.000	155.571
+	-----+	-----+	-----+	-----+

Все значения должны иметь три знака после десятичного разделителя. Не использовать сторонние модули (prettytable и подобные). Функции, концы отрезков и количества интервалов разбиения заданы в табл. 5.8.

Отчет должен включать:

1. Полную формулировку задачи.
2. Программный код с обязательными пояснениями в виде комментариев.
3. Результат работы программы.

Таблица 5.8

№	Функция	a	b	c	d	k	p
1	$f(x, y) = \begin{cases} x^3 - y/2 & x < 1 \wedge y < 0; \\ \sqrt{x+y^2} + x^2 & x \geq 2.5 \wedge y \geq 1; \\ \sqrt{x^2 + y^2} & \text{иначе} \end{cases}$	0	3	-1	4	10	10
2	$f(x, y) = \begin{cases} x^2 + 3y & x \leq 0 \wedge y < 1; \\ \sqrt{x^2 + y} + 2x & x > 2 \wedge y \leq -1; \\ \cos(x) + \sin(y) & \text{иначе} \end{cases}$	-1	4	-2	2	10	15
3	$f(x, y) = \begin{cases} \sin(xy) + y^2 & x < 3 \wedge y < 0; \\ \frac{x}{2} - \sqrt{y} & x \geq 4 \wedge y > 1; \\ x^2 + y^2 - 1 & \text{иначе} \end{cases}$	0	4	0	3	15	15

№	Функция	a	b	c	d	k	p
4	$f(x, y) = \begin{cases} \cos(x) + \tan(y) & x < 0.5 \wedge y < 1; \\ x^3 + y & x > 3 \wedge y \geq 0; \\ \sqrt{x^2 + y^2} & \text{иначе} \end{cases}$	-1	4	-2	2	10	10
5	$f(x, y) = \begin{cases} 2x^2 - \sin(y) & x < -2 \wedge y > 0; \\ x + y^3 & x > 1.5 \wedge y \leq 2; \\ x^2 - y^2 + 1 & \text{иначе} \end{cases}$	-4	3	-2	4	15	10
6	$f(x, y) = \begin{cases} \tan(x) - \cos(y) & x < 1.2 \wedge y < 0.5; \\ x^2 + 4y & x \geq 2.2 \wedge y \geq -1; \\ x + y - xy & \text{иначе} \end{cases}$	-2	2	-2	2	10	10
7	$f(x, y) = \begin{cases} \sqrt{x^2 + y} - y & x \leq -1 \wedge y > 1; \\ \cos(xy) + x^2 & x > 2.3 \wedge y < 1; \\ y - x^2 + 3 & \text{иначе} \end{cases}$	-3	4	-2	4	15	15
8	$f(x, y) = \begin{cases} x^2 - \tan(y) & x < 0 \wedge y < 0; \\ x^3 + 2\sqrt{y} & x > 1 \wedge y > 1; \\ \sqrt{x^2 + y^2} & \text{иначе} \end{cases}$	-2	2	-2	2	15	15
9	$f(x, y) = \begin{cases} \sin(x) + y^2 & x < 2.5 \wedge y \leq 1; \\ x^2 - \sqrt{y} + 1 & x \geq 3.5 \wedge y > 0; \\ x + y + 1 & \text{иначе} \end{cases}$	0	4	-1	5	10	15
10	$f(x, y) = \begin{cases} x^2 - 1/y & x < 1 \wedge y < 1 \wedge y \neq 0; \\ \sqrt{x + y^2} + \cos(y) & x \geq 2.1 \wedge y \geq 2; \\ y - x + \sin(xy) & \text{иначе} \end{cases}$	-1	4	-2	4	15	15
11	$f(x, y) = \begin{cases} x^2 / (1 + y^2) + \sin(x) & x < 1.5 \wedge y < 0.3; \\ \ln(x + 2) + y^3 & x \geq 3.1 \wedge y \geq 1.5; \\ x - y & \text{иначе} \end{cases}$	-1	3	-1	4	15	10
12	$f(x, y) = \begin{cases} e^x - \cos(y) & x < 0.7 \wedge y < 1.2; \\ \sqrt{x} + \frac{y}{x+1} & x > 2.5 \wedge y > 0.5; \\ xy + x + y & \text{иначе} \end{cases}$	0	5	0	4	10	10
13	$f(x, y) = \begin{cases} \tan(x) - y^2 & x < 2.3 \wedge y < 0; \\ x^3 + 4y & x \geq 3.5 \wedge y \geq 1; \\ \sqrt{x^2 + y^2} & \text{иначе} \end{cases}$	-1	3	-2	4	15	15

№	Функция	a	b	c	d	k	p
14	$f(x, y) = \begin{cases} \sin(xy) + x & x < 1 \wedge y < 0.9; \\ \frac{x}{1+y^2} + y^3 & x \geq 2.2 \wedge y \geq 1.8; \\ x^2 + y^2 & \text{иначе} \end{cases}$	-2	4	-1	5	10	10
15	$f(x, y) = \begin{cases} \ln(1+x^2) - y & x < 0.5 \wedge y < 1.5; \\ x^2 + \cos(y) & x > 3.1 \wedge y > 1; \\ y - x & \text{иначе} \end{cases}$	0	4	0	5	15	15
16	$f(x, y) = \begin{cases} 1/(x+1) + y^2 & x < 1.7 \wedge y < 0.2 \wedge x \neq -1; \\ \sqrt{x+y} + x & x \geq 3 \wedge y \geq 1; \\ \sin(x) + \cos(y) & \text{иначе} \end{cases}$	-2	4	-1	3	10	10
17	$f(x, y) = \begin{cases} e^x - y & x < 2 \wedge y < 0.1; \\ x^2 + y^3 & x \geq 2.7 \wedge y \geq 0.7; \\ \sqrt{x^2 + y^2} & \text{иначе} \end{cases}$	-1	4	-2	5	15	15
18	$f(x, y) = \begin{cases} \cos(x) + y/2 & x < 1 \wedge y < 0.8; \\ x + y^2 & x \geq 3.4 \wedge y \geq 1.2; \\ x - y + xy & \text{иначе} \end{cases}$	-1	3	-2	5	15	10
19	$f(x, y) = \begin{cases} \sqrt{x} + \sin(y) & 0 \leq x < 0.6 \wedge y < 0.4; \\ x^3 - y & x \geq 3 \wedge y \geq 1.5; \\ y^2 - x & \text{иначе} \end{cases}$	-2	4	-1	5	10	10
20	$f(x, y) = \begin{cases} \tan(x) + y & x < 1.2 \wedge y < 0.6; \\ x^2 + \sqrt{y} & x \geq 2.9 \wedge y \geq 1.1; \\ \sin(xy) & \text{иначе} \end{cases}$	0	4	0	5	15	15
21	$f(x, y) = \begin{cases} x^3 / (1+x^2) + y & x < 1.8 \wedge y < 0.7; \\ \sqrt{x} + \ln(y+2) & x \geq 3.2 \wedge y \geq 1.4; \\ x - y & \text{иначе} \end{cases}$	-2	4	-1	3	10	10
22	$f(x, y) = \begin{cases} e^{-x} + y^2 & x < 0.9 \wedge y < 0.3; \\ x^2 - \frac{1}{y+3} & x \geq 2.7 \wedge y \geq 0.9; \\ xy + \sin(x) & \text{иначе} \end{cases}$	-1	4	-2	5	15	15
23	$f(x, y) = \begin{cases} \tan(x) + \cos(y) & x < 2.1 \wedge y < 1; \\ x^3 - y & x \geq 3.6 \wedge y \geq 1.5; \\ \sqrt{x^2 + y^2} & \text{иначе} \end{cases}$	-1	5	-2	5	15	10

№	Функция	a	b	c	d	k	p
24	$f(x, y) = \begin{cases} \sin(xy) + x & x < 1.3 \wedge y < 0.5; \\ \frac{x}{1+y^2} + y^3 & x \geq 2.4 \wedge y \geq 1.7; \\ x^2 + y^2 & \text{иначе} \end{cases}$	-2	4	-1	5	10	10
25	$f(x, y) = \begin{cases} \ln(1+x^2) - y & x < 0.7 \wedge y < 1.8; \\ x^2 + \cos(y) & x > 3.3 \wedge y > 1.2; \\ y - x & \text{иначе} \end{cases}$	0	4	0	5	15	15
26	$f(x, y) = \begin{cases} \frac{1}{x+1} + y^2 & x < 1.9 \wedge y < 0.5 \wedge x \neq -1; \\ \sqrt{x+y} + x & x \geq 3.4 \wedge y \geq 1.3; \\ \sin(x) + \cos(y) & \text{иначе} \end{cases}$	-2	4	-1	3	10	10
27	$f(x, y) = \begin{cases} e^x - y & x < 2.3 \wedge y < 0.2; \\ x^2 + y^3 & x \geq 2.8 \wedge y \geq 0.8; \\ \sqrt{x^2 + y^2} & \text{иначе} \end{cases}$	-1	4	-2	5	15	15
28	$f(x, y) = \begin{cases} \cos(x) + \frac{y}{2} & x < 1.1 \wedge y < 0.9; \\ x + y^2 & x \geq 3.7 \wedge y \geq 1.4; \\ x - y + xy & \text{иначе} \end{cases}$	-1	5	-2	5	15	10
29	$f(x, y) = \begin{cases} \sqrt{x} + \sin(y) & 0 \leq x < 0.8 \wedge y < 0.6; \\ x^3 - y & x \geq 3.1 \wedge y \geq 1.7; \\ y^2 - x & \text{иначе} \end{cases}$	-2	4	-1	5	10	10
30	$f(x, y) = \begin{cases} \tan(x) + y & x < 1.4 \wedge y < 0.8; \\ x^2 + \sqrt{y} & x \geq 3.0 \wedge y \geq 1.2; \quad \dots \\ \sin(xy) & \text{иначе} \end{cases}$	0	4	0	5	15	15

Equation Section (Next)

6. Генерация случайных значений

Модуль `random` в языке Python представляет собой стандартную библиотеку, реализующую функции генерации псевдослучайных чисел. Данный модуль предоставляет обширный набор инструментов для выбора случайных элементов, генерации случайных последовательностей, перемешивания данных, получения случайных чисел различных распределений и работы с вероятностными выборками.

Все функции модуля `random` базируются на генераторе псевдослучайных чисел, который реализован как модификация алгоритма Mersenne Twister (генератор псевдослучайных чисел, разработанный М. Мацумото и Т. Нисимурой в 1997 году) и обеспечивает высокую скорость и статистически достоверное распределение.

Функция **`seed(a)`** используется для инициализации начального состояния генератора псевдослучайных чисел. Она позволяет обеспечить воспроизводимость последовательности случайных чисел.

Аргумент `a` является инициализирующим значением (*инициализатором*), и может быть любого например, ируемого типа, например, целым числом, строкой, байтовой строкой. Если аргумент `a` не передан или установлен в **`None`**, то генератор будет инициализирован текущим системным временем.

При вызове **`random.seed(a)`** генератор случайных чисел переходит в начальное состояние, соответствующее значению `a`. После этого последовательность чисел, генерируемая функциями модуля `random`, становится воспроизводимой. То есть, если дважды вызвать **`random.seed(42)`**, а затем одинаковую последовательность вызовов функций, использующих генератор, то результат в обоих случаях будет идентичен.

Основные функции и методы данного модуля приведены в табл. 6.1.

Таблица 6.1

Функция / Метод	Назначение	Пример	Результат
<code>random()</code>	Случайное число с плавающей точкой от 0.0 до 1.0 (исключая 1.0)	<code>random.seed(0); random.random()</code>	0.84442185152 50481
<code>uniform(a, b)</code>	Случайное число с плавающей точкой от <i>a</i> до <i>b</i>	<code>random.seed(0); random.uniform(10, 20)</code>	18.4442185152 50482
<code>randint(a, b)</code>	Случайное целое число из диапазона [<i>a</i> , <i>b</i>]	<code>random.seed(0); random.randint(1, 10)</code>	7

Функция / Метод	Назначение	Пример	Результат
randrange (start, stop, step)	Случайное целое из арифметической последовательности	<code>random.seed(0); random.randrange(0, 10, 2)</code>	8
choice (seq)	Случайный элемент из последовательности	<code>random.seed(0); random.choice([1, 2, 3, 4])</code>	4
choices (population, k=n)	Список из n элементов с возвращением	<code>random.seed(0); random.choices([1, 2, 3], k=4)</code>	[2, 2, 1, 3]
sample (population, k)	Список из k уникальных элементов без возвращения	<code>random.seed(0); random.sample([1, 2, 3, 4, 5], 3)</code>	[4, 5, 1]
shuffle (x)	Перемешивание элементов списка на месте	<code>random.seed(0); a=[1, 2, 3]; random.shuffle(a); a</code>	[2, 3, 1]
betavariate (alpha, beta)	Число из бета-распределения с параметрами alpha, beta	<code>random.seed(0); random.betavariate(0.5, 0.5)</code>	0.842302657126154
expovariate (lambd)	Число из экспоненциального распределения с параметром lambd	<code>random.seed(0); random.expovariate(1.5)</code>	0.17932168177616806
gammapariate (alpha, beta)	Число из гамма-распределения с параметрами alpha, beta	<code>random.seed(0); random.gammapariate(1, 2)</code>	2.876531632708509
gauss (mu, sigma)	Число из нормального распределения Гаусса	<code>random.seed(0); random.gauss(0, 1)</code>	1.2881847531554629
lognormvariate (mu, sigma)	Число из логнормального распределения	<code>random.seed(0); random.lognormvariate(0, 1)</code>	3.0848787254291545
normalvariate (mu, sigma)	Число из нормального распределения	<code>random.seed(0); random.normalvariate(0, 1)</code>	1.2881847531554629
triangular (low, high, mode)	Число из треугольного распределения	<code>random.seed(0); random.triangular(0, 10, 5)</code>	8.01357823816703
vonmisesvariate (mu, kappa)	Число из распределения фон Мизеса	<code>random.seed(0); random.vonmisesvariate(0, 4)</code>	2.7817090256340925

Функция / Метод	Назначение	Пример	Результат
<code>weibullvariate(alpha, beta)</code>	Число из распределения Вейбулла	<code>random.seed(0); random.weibullvariate(1, 1)</code>	0.8444218515250481
<code>seed(a=None)</code>	Инициализация генератора случайных чисел	<code>random.seed(42)</code>	—

Задание 6.1 (практико-ориентированное задание)

Выполнить задания в соответствии с табл. 6.2.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результаты работы программы.

Таблица 6.2

№	Задание
1	В рамках тестирования логики рекомендаций в интернет-магазине требуется имитировать поведение пользователей при просмотре товаров. Необходимо сгенерировать список, представляющий собой последовательность номеров товаров, просмотренных случайным пользователем за один визит. Всего в системе имеется N уникальных товаров, каждый из которых имеет уникальный числовой идентификатор в диапазоне от 1 до N . Количество просмотренных товаров определяется случайным образом в пределах заданного диапазона. Требуется сформировать один такой список и вывести его содержимое
2	При формировании индивидуальных заданий для студентов преподавателю необходимо создать случайную последовательность чисел, отражающую номера вопросов, включенных в конкретный вариант контрольной работы. Всего имеется M уникальных вопросов, и каждый вариант должен содержать K из них без повторений. Требуется реализовать генератор одного контрольного варианта и отобразить полученную последовательность номеров, упорядоченную по возрастанию
3	При моделировании нагрузки на службу доставки требуется сформировать массив, содержащий случайные значения объема заказов, полученных за один рабочий день. Объем заказа выражается в литрах и представлен числом с плавающей точкой в диапазоне от \$1,0 до \$50,0. Необходимо сгенерировать M таких значений и вычислить средний и максимальный объем заказа
4	В рамках проектирования обучающей игры необходимо реализовать случайный выбор заданий для пользователя из общего пула. Всего доступно T уникальных заданий, пронумерованных от 1 до T . В одной игровой сессии игроку предлагается K случайных заданий без повторений. Требуется сгенерировать одну такую выборку и отсортировать ее в порядке возрастания

№	Задание
5	При разработке системы тестирования реакции требуется сгенерировать случайные интервалы времени задержки между отображением стимулов. Каждый интервал должен быть случайным вещественным числом от \$0,5\$ до \$2,5\$ секунд. Требуется сгенерировать L таких значений и вывести их в виде списка
6	В задаче моделирования случайного движения объекта по координатной сетке необходимо на каждом шаге случайно выбирать направление движения: «вверх», «вниз», «влево», «вправо». Требуется сгенерировать последовательность из N таких направлений и вывести ее для дальнейшего анализа траектории движения
7	При генерации случайных паролей необходимо создать строку заданной длины K , содержащую случайную последовательность символов, выбранных из набора латинских букв в верхнем и нижнем регистрах, а также цифр. Требуется реализовать генератор одного пароля и отобразить результат
8	В задаче симуляции случайных исходов голосования в комитете требуется сгенерировать список из M элементов, каждый из которых случайно принимает значение «За» или «Против». По результатам моделирования необходимо подсчитать количество голосов «За» и «Против» для определения принятия решения
9	В процессе симуляции игрового автомата необходимо сгенерировать последовательность результатов выпадения трех барабанов, каждый из которых показывает случайный символ из фиксированного набора: «А», «В», «С», «D», «Е». Один запуск автомата представляет собой кортеж из трех случайных символов. Требуется сформировать и вывести результат одного запуска
10	При генерации случайных маршрутов курьеров по городу необходимо выбрать K уникальных точек доставки из общего списка из N адресов. Точки должны быть выбраны случайным образом без повторений. Полученный маршрут выводится в виде упорядоченного списка адресов
11	В рамках разработки обучающей платформы с системой случайного выбора тем для самоподготовки необходимо реализовать механизм генерации персонализированного списка тем занятий. Из общего перечня T учебных тем требуется случайным образом выбрать L различных тем, которые будут предложены студенту в качестве плана на текущий день. Сформировать и вывести такой список
12	В процессе проектирования подсистемы автоматизированного тестирования персонала необходимо реализовать механизм случайного выбора индикаторов для проверки знаний. Имеется перечень из P параметров оценки, среди которых требуется случайным образом выбрать R параметров для конкретной сессии оценки. Вывести полученный набор параметров
13	При проведении розыгрыша рекламной акции среди клиентов онлайн-сервиса требуется смоделировать случайный выбор нескольких победителей. Дано общее количество клиентов C . Требуется определить W победителей, обеспечив равновероятный выбор без повторений. Вывести список выигравших клиентов, представленных в виде их идентификаторов
14	В ходе проектирования модуля автоматического формирования команд при проведении внутренних соревнований сотрудников необходимо сгенерировать случайные составы групп. Имеется N сотрудников. Требуется случайным образом распределить их на G команд с максимально равным количеством участников. Вывести состав каждой команды

№	Задание
15	Для имитационного анализа нагрузки на сервер распределенной обработки заданий необходимо сгенерировать случайную последовательность временных меток запуска задач в пределах одного часа. Количество задач определяется заданным числом Z . Необходимо вывести отсортированный список сгенерированных времен запусков в формате минут и секунд
16	В рамках проверки качества системы автоматизированного назначения курьеров на заказы необходимо смоделировать распределение заказов между курьерами. Имеется Q заказов и K курьеров. Каждому заказу случайным образом назначается один курьер. Требуется сгенерировать распределение и подсчитать, сколько заказов получил каждый курьер
17	При реализации цифровой доски объявлений в учебном заведении необходимо протестировать механизм случайного выбора одной из активных новостей для отображения при загрузке страницы. Имеется список из N текстов новостей. Требуется выбрать одну случайную новость и вывести ее текст
18	В процессе генерации индивидуальных домашних заданий необходимо для каждого студента выбрать случайное количество задач из различных разделов курса. Имеется S студентов и R разделов, каждый из которых содержит множество задач. Для каждого студента требуется случайным образом выбрать от одного до трех разделов, а затем из каждого выбранного раздела выбрать одну задачу. Сформировать список таких заданий
19	В рамках проекта по анализу поведения покупателей в супермаркете необходимо сгенерировать случайные сценарии перемещения покупателей по отделам. Имеется D отделов магазина. За одно посещение покупатель может посетить от 2 до 5 различных отделов. Требуется сформировать и вывести одну случайную последовательность посещения отделов
20	Для симуляции механизма случайной выдачи бонусных баллов за покупки в программе лояльности необходимо сгенерировать список из N покупателей, каждый из которых получает случайное число баллов в пределах от $B_{\min}$ до $B_{\max}$. Сформировать и отобразить итоговый список в формате: идентификатор покупателя — начисленное количество баллов
21	При разработке системы тестирования для онлайн-курсов требуется обеспечить случайный порядок следования вопросов в индивидуальном тесте. Имеется банк из M вопросов. Для каждого пользователя необходимо сгенерировать случайную перестановку этих вопросов, чтобы минимизировать вероятность списывания. Вывести итоговую последовательность
22	В контексте моделирования работы службы доставки еды требуется создать последовательность случайных временных задержек между размещением заказов в течение одного дня. Задано общее количество заказов Z . Для каждого заказа необходимо сгенерировать задержку в интервале от 1 до 20 минут и вывести список задержек в хронологическом порядке
23	В рамках анализа эффективности стратегий инвестирования необходимо сгенерировать последовательность псевдослучайных процентных изменений стоимости акций. Требуется построить ряд длины N , где каждое значение представляет собой случайное изменение цены акции в пределах от -3% до $+3\%$. Вывести полученный ряд и вычислить итоговое изменение стоимости после применения всех изменений

№	Задание
24	Для симуляции работы кассовой системы супермаркета необходимо сгенерировать список покупателей с случайным временем обслуживания каждого. Количество покупателей задается числом K . Для каждого покупателя требуется случайным образом определить продолжительность обслуживания в пределах от 2 до 7 минут. Вывести список значений и рассчитать общую продолжительность обслуживания всей очереди
25	В рамках генерации искусственных профилей пользователей для тестирования веб-платформы требуется сгенерировать U профилей, каждому из которых случайным образом назначаются пол, возраст (в пределах от 18 до 65) и уровень вовлеченности (одно из трех заданных текстовых значений). Сформировать и вывести таблицу с описанием всех профилей
26	При разработке имитационной модели работы диспетчерского центра такси необходимо смоделировать поток заявок от клиентов. За один час предполагается N заявок. Для каждой заявки требуется случайным образом определить адрес подачи из заданного списка улиц, а также количество пассажиров от 1 до 4. Вывести сгенерированные данные по всем заявкам
27	В процессе разработки настольной игры с элементами случайности необходимо реализовать бросок нескольких кубиков. Требуется сгенерировать случайные значения при броске D стандартных шестигранных кубиков, а затем вычислить их сумму. Вывести все выпавшие значения и итоговую сумму очков
28	Для тестирования алгоритма сортировки необходимо сгенерировать случайный массив длины L , содержащий целые числа в диапазоне от -50 до 50 , без повторяющихся значений. После генерации требуется отсортировать массив и вывести его до и после сортировки
29	В рамках оценки модели продаж программного продукта необходимо сгенерировать случайный график покупок. Предполагается, что в течение месяца совершается N покупок, каждая из которых привязана к одной случайно выбранной дате месяца. Требуется сгенерировать и вывести таблицу с датами и количеством покупок, сгруппированных по дню
30	Для симуляции процесса набора участников конференции необходимо случайным образом отобрать из общего списка A зарегистрированных участников тех, кто получит приглашение на закрытую секцию мероприятия. Количество приглашений равно I . Необходимо обеспечить выбор без повторений и вывести итоговый список приглашенных

Equation Section (Next)

7. СПИСКОВЫЙ ТИП

7.1. Одномерные списки

Одномерный список представляет собой структуру данных, формально определяемую как линейно упорядоченную, индексируемую, изменяемую последовательность элементов, доступных по целочисленным индексам. Функция преобразования к списковому типу — **list()**.

Литерально список задается в квадратных скобках `[]`, элементы разделяются запятыми `(,)`. Элементами списка могут быть любые объекты языка Python, в том числе, и списки.

Пустой список — список, не содержащий элементов, задается как `[]`.

Перебор элементов списка можно осуществлять по значениям и по индексам. В листинге 7.1 приведен пример переборов.

Листинг 7.1

```
a = [3, -4, 'abcde', [7, 8], True]

for i in a:
    print(i, end=' ') # 3 -4 abcde [7, 8] True
print()

for i in range(len(a)):
    print(a[i], end=' ') # 3 -4 abcde [7, 8] True
```

Проход одновременно по индексам и значениям элементов списка выполняется с использованием функции **enumerate()**. Пример использования приведен в листинге 7.2.

Листинг 7.2

```
a = [3, -4, 'abcde', [7, 8], True]

# i - индекс, z - значение
for i, z in enumerate(a):
    print(i, z)
# 0 3
# 1 -4
# 2 abcde
# 3 [7, 8]
# 4 True
```

Принципы индексации и использования инструментария срезов аналогичны строковым данным.

Базовые операции над списками приведены в табл. 7.1.

Таблица 7.1

Операция	Описание поведения	Результат / Эффект	Пример	Результат примера
<code>len(L)</code>	Возвращает длину списка n	Целое число	<code>len([1, 2, 3])</code>	3
<code>x in L</code>	Проверяет наличие элемента x в списке L	Логическое значение	<code>3 in [1, 2, 3]</code>	True
<code>x not in L</code>	Проверяет отсутствие элемента x в списке L	Логическое значение	<code>4 not in [1, 2, 3]</code>	True
<code>L + M</code>	Конкатенация списков L и M	Новый список	<code>[1, 2] + [3, 4]</code>	[1, 2, 3, 4]
<code>L * n</code>	Повторяет список L n раз	Новый список	<code>[1, 2] * 3</code>	[1, 2, 1, 2, 1, 2]
<code>L[i]</code>	Обращение к элементу с индексом i	Элемент списка	<code>[10, 20, 30][1]</code>	20
<code>L[i] = x</code>	Присваивает элементу с индексом i значение x	Изменяет список	<code>a = [1, 2] a[0] = 5</code>	<code>a = [5, 2]</code>
<code>del L[i]</code>	Удаляет элемент с индексом i	Удаляет элемент	<code>a = [1, 2] del a[1]</code>	<code>a = [1]</code>
<code>L[i:j]</code>	Возвращает срез от индекса i до j (не включая j)	Новый список	<code>[0, 1, 2, 3][1:3]</code>	[1, 2]
<code>L[i:j] = M</code>	Заменяет срез <code>L[i:j]</code> на элементы списка M	Модифицирует список	<code>a = [1, 2, 3] a[1:2] = [9, 9]</code>	<code>a = [1, 9, 9, 3]</code>
<code>L.append(x)</code>	Добавляет элемент x в конец списка	Увеличивает длину списка	<code>a = [1] a.append(2)</code>	<code>a = [1, 2]</code>
<code>L.extend(M)</code>	Добавляет в конец все элементы из списка M	Увеличивает длину на длину M	<code>a = [1] a.extend([2, 3])</code>	<code>a = [1, 2, 3]</code>
<code>L.insert(i, x)</code>	Вставляет x перед элементом с индексом i	Сдвигает элементы вправо	<code>a = [1, 3] a.insert(1, 2)</code>	<code>a = [1, 2, 3]</code>
<code>L.pop()</code>	Удаляет и возвращает последний элемент списка	Уменьшает длину на 1	<code>a = [1, 2] x = a.pop()</code>	<code>x = 2 a = [1]</code>
<code>L.pop(i)</code>	Удаляет и возвращает элемент по индексу i	Уменьшает длину на 1	<code>a = [1, 2, 3] a.pop(1)</code>	<code>2 a = [1, 3]</code>
<code>L.remove(x)</code>	Удаляет первое вхождение значения x из списка	Уменьшает длину на 1	<code>a = [1, 2, 3] a.remove(2)</code>	<code>a = [1, 3]</code>
<code>L.clear()</code>	Удаляет все элементы	Превращает список в пустой	<code>a = [1, 2] a.clear()</code>	<code>a = []</code>
<code>L.index(x)</code>	Возвращает индекс первого вхождения x	Целое число	<code>[10, 20, 10].index(10)</code>	0
<code>L.count(x)</code>	Возвращает число вхождений элемента x	Целое число	<code>[1, 2, 1, 1].count(1)</code>	3
<code>L.sort()</code>	Сортирует список по возрастанию (на месте)	Упорядочивает элементы	<code>a = [3, 1, 2] a.sort()</code>	<code>a = [1, 2, 3]</code>
<code>L.reverse()</code>	Инвертирует порядок элементов (на месте)	Переставляет элементы	<code>a = [1, 2] a.reverse()</code>	<code>a = [2, 1]</code>

Операция	Описание поведения	Результат / Эффект	Пример	Результат примера
<code>L.copy()</code>	Возвращает поверхностную копию списка	Новый список с теми же элементами	<code>a = [1, 2]</code> <code>b = a.copy()</code>	<code>b = [1, 2]</code>

Помимо указанных в табл. 7.1 методов к спискам применимы встроенные функции, приведенные в табл. 7.2.

Таблица 7.2

Функция	Назначение	Требования к типам элементов списка	Пример выражения	Результат
<code>sum(L)</code>	Вычисляет сумму элементов списка	Элементы — числовые типы	<code>sum([1, 2, 3])</code>	6
<code>min(L)</code>	Находит минимальный элемент списка	Элементы — сравнимые между собой	<code>min([5, 3, 7])</code>	3
<code>max(L)</code>	Находит максимальный элемент списка	Элементы — сравнимые между собой	<code>max([5, 3, 7])</code>	7
<code>sorted(L)</code>	Возвращает новый отсортированный список	Элементы — сравнимые между собой	<code>sorted([3, 1, 2])</code>	[1, 2, 3]
<code>reversed(L)</code>	Возвращает итератор в обратном порядке	Не требуется	<code>list(reversed([1, 2, 3]))</code>	[3, 2, 1]
<code>enumerate(L)</code>	Возвращает итератор пар (i, L[i])	Не требуется	<code>list(enumerate(['a', 'b']))</code>	[(0, 'a'), (1, 'b')]
<code>zip(A, B)</code>	Возвращает итератор из кортежей попарных элементов	Списки одинаковой или согласуемой длины	<code>list(zip([1, 2], ['a', 'b']))</code>	[(1, 'a'), (2, 'b')]
<code>map(f, L)</code>	Применяет функцию f к каждому элементу списка	f определена на тип элементов	<code>list(map(str, [1, 2]))</code>	['1', '2']
<code>filter(f, L)</code>	Отбирает элементы, для которых функция f(x) возвращает True	f возвращает булев результат	<code>list(filter(lambda x: x > 0, [-1, 2]))</code>	[2]
<code>all(L)</code>	Возвращает True, если все элементы истинны	Элементы приводимы к логическому типу	<code>all([1, True, 3])</code>	True
<code>any(L)</code>	Возвращает True, если хотя бы один элемент истинен	Элементы приводимы к логическому типу	<code>any([0, False, 2])</code>	True
<code>list(iterable)</code>	Преобразует любой итерируемый объект в список	Аргумент — итерируемый объект	<code>list('abc')</code>	['a', 'b', 'c']
<code>str.join(L)</code>	Конкатенирует элементы списка строк в одну строку через разделитель	Элементы — строки	<code>','.join(['a', 'b', 'c'])</code>	'a,b,c'

Функция	Назначение	Требования к типам элементов списка	Пример выражения	Результат
<code>set(L)</code>	Преобразует список в множество (уникальные элементы)	Элементы — хешируемые	<code>set([1, 2, 2, 3])</code>	{1, 2, 3}
<code>tuple(L)</code>	Преобразует список в кортеж	Не требуется	<code>tuple([1, 2, 3])</code>	(1, 2, 3)
<code>max(L, key=f)</code>	Возвращает элемент с максимальным значением $f(x)$	$f(x)$ определена на элементы списка	<code>max(['a', 'bbb', 'cc'], key=len)</code>	'bbb'
<code>min(L, key=f)</code>	Возвращает элемент с минимальным значением $f(x)$	$f(x)$ определена на элементы списка	<code>min(['a', 'bbb', 'cc'], key=len)</code>	'a'

Пример 7.1

Пусть дан набор целочисленных значений $A = \{a_i\}$, $i = \overline{1, n}$, заданный в программе в виде списка, для которого $n = 100$, значения a_i — случайное целое число из отрезка $[-30, 30]$, сгенерированное при `seed(3)`. Вычислить

$$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i \bmod 5 = 0 \text{ и } a_i > 0.$$

$$P_1 = \prod_{i=1}^n a_i, \text{ где } i \text{ четное и } i \bmod 3 = 2.$$

$$S_2 = \sum_{i=1}^{n-2} |a_i^3 - 3a_{i+1}^3 + 2a_{i+2}^3|, \text{ где } a_i < a_{i+2}.$$

$$R = \frac{a_1^2 + a_2^2 + \dots + a_n^2 + 3a_1a_2 + 2a_{n-1}a_n}{a_1a_2 + a_2a_3 + \dots + a_{n-1}a_n + |a_1 - a_n| + 5}.$$

$$E = \max(a_1, a_2, \dots, a_n)^2 - \min(a_1^2, a_2^2, \dots, a_n^2) + |\max(a_1, \dots, a_n) - \min(a_1, \dots, a_n)|.$$

Решение

Программный код решения приведен в листинге 7.38.3.

Листинг 7.3

```

from random import *

# Параметры генерации значений
seed(3)
n = 100

# Формирование списка значений
a = []
for i in range(n):
    a.append(randint(-30, 30))
print('a =', a)

```

```

# Вычисление S1
s1 = 0
for z in a:
    if z % 5 == 0 and z > 0:
        s1 += z
print('S1 =', s1)

# Вычисление P1
p1 = 1
for i in range(n):
    if i % 2 == 0 and i % 3 == 2:
        p1 *= a[i]
print('P1 =', p1)

# Вычисление S1
s2 = 0
for i in range(n - 2):
    if a[i] < a[i + 2]:
        s2 += abs(a[i]**3 - 3 * a[i + 1]**3 + 2 * a[i + 2]**3)
print('S2 =', s2)

# Вычисление R. Отдельно вычисляется числитель R1
# и знаменатель R2
R1 = 0
for z in a:
    R1 += z**2
R1 += 3 * a[0] * a[1] + 2 * a[n - 2] * a[n - 1]
R2 = 0
for i in range(n - 1):
    R2 += a[i] * a[i + 1]
R2 += abs(a[0] - a[n - 1]) + 5
R = R1 / R2
print('R =', R)

# Вычисление E
minKv = 1000 # Переменная для хранения минимума
# из квадратов значений списка
for z in a:
    minKv = min(minKv, z**2)
E = max(a)**2 - minKv + abs(max(a) - min(a))
print('E =', E)

```

Результат выполнения приведен в листинге 7.4.

Листинг 7.4

```

a = [-15, 7, 4, -22, -7, 28, 8, 0, 10, 7, -26, 8, -30, 28, 23,
0, -14, 5, -16, -18, 15, 0, 4, 23, 5, 0, -5, 10, 25, -21, -16,
10, -21, 25, 29, 3, -6, 17, -30, 12, 19, -26, -20, 18, 7, -28,
-11, 19, -29, 22, 25, -13, 0, 8, 16, 28, 26, -6, 15, 20, 28, -
3, -5, 16, 21, 6, -2, 29, -22, 26, -7, -24, -28, -22, 1, -17, -
14, 13, -3, 19, 10, 24, -11, -4, 2, 23, -6, 6, -8, 4, 7, -4, 7,
-16, 27, -9, 13, 28, 29, -29]

```

```

S1 = 175
P1 = 2649970323000000000
S2 = 1328042
R = -23.756846473029047
E = 900

```

Задание 7.1

Пусть дан набор целочисленных значений $A = \{a_i\}$, $i = \overline{1, n}$, заданный в программе в виде списка, для которого $n = 100$, значения a_i — случайное целое число из отрезка $[-50, 50]$, сгенерированное при **seed**(k), где k — номер варианта. Найти значения выражений, приведенных в табл. 7.3. При вычислениях запрещено использовать генераторы и функции сторонних модулей.

В условии заданий используются следующие обозначения:

— $\bar{a} = \frac{1}{n} \sum_{i=1}^n a_i$ — среднее арифметическое всех значений;

— $G = \left(\prod_{i=1}^n |a_i| \right)^{1/n}$ — среднее геометрическое по модулю;

— $a \bmod b = c$ — остаток от деления числа a на число b равен c .

При вычислении значения R проверить, что знаменатель не равен 0.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результаты работы программы (исходный список значений и результаты вычислений в формате S1 = <значение>).

Таблица 7.3

№	Вычисляемые значения
1	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 - 5a_i + 4 > 0$
	$S_2 = \sum a_i$, где $i = 3q + 2$ при некотором целом $q \geq 0$ (индексы 2, 5, 8, ...)
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < 0$ и $ a_i \geq 20$
	$P_2 = \prod_{i=1}^{n-2} (a_i + 2a_{i+1} - a_{i+2})$
	$S_3 = \sum_{i=1}^n (\sqrt{ a_i } + a_i^2 - 2a_i)$
	$S_4 = \sum_{i=1}^{n-1} ((a_i + a_{i+1})^2 + a_i - a_{i+1})$

№	Вычисляемые значения
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - 3a_{i+1}a_{i+2} + 2a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(a_i - \bar{a} ^3 + \sqrt{ a_i G} \right)$
	$R = \frac{a_1^3 + a_2^3 + a_3^3 + \dots + a_n^3}{a_1a_2^2 + a_2a_3^2 + a_3a_4^2 + \dots + a_{n-1}a_n^2}$
	$E = \left(\max(a_1, a_2, \dots, a_n) - \min(a_1, a_2, \dots, a_n) \right)^2 - \min(a_1^2, a_2^2, \dots, a_n^2) + \max(a_1^2, a_2^2, \dots, a_n^2)$
2	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 + 3a_i - 10 < 0$
	$S_2 = \sum_{i=1}^n a_i$, где i делится на 4
	$P_1 = \prod_{i=1}^n a_i$, где $a_i > 2$ и a_i делится на 3
	$P_2 = \prod_{i=1}^{n-3} (2a_i - a_{i+1} + a_{i+2} + a_{i+3})$
	$S_3 = \sum_{i=1}^n \left(a_i^3 - 2a_i + \sqrt{ a_i } \right)$
	$S_4 = \sum_{i=1}^{n-1} \left(a_i^2 + a_{i+1}^2 - 2a_i a_{i+1} + 3 \right)$
	$S_5 = \sum_{i=1}^{n-2} a_i + 2a_{i+1} - a_{i+2}^2 $, где $a_i < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left((a_i - \bar{a})^2 + a_i \cdot \bar{a} \right)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1a_2 - a_2a_3 + a_3a_4 - \dots + (-1)^{n-1}a_{n-1}a_n}$
	$E = \max(a_1^2, a_2^2, \dots, a_n^2) - \left(\min(a_1, a_2, \dots, a_n) \right)^2 + 2 \cdot \max(a_1, a_2, \dots, a_n)$
3	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 + a_i > 10$
	$S_2 = \sum_{i=1}^n a_i$, где i — квадрат нечетного числа, то есть $i = 1, 9, 25, \dots$
	$P_1 = \prod_{i=1}^n a_i$, где a_i — отрицательное и нечетное число
	$P_2 = \prod_{i=1}^{n-1} (a_i + a_{i+1} + 1)$
	$S_3 = \sum_{i=1}^n \left(a_i - 3 + \sqrt{ a_i } + 1 \right)$
	$S_4 = \sum_{i=1}^{n-1} \left((a_i + 2a_{i+1})^2 - (a_i - a_{i+1}) \right)$
	$S_5 = \sum_{i=1}^{n-2} 2a_i^2 - a_{i+1}^2 + a_{i+2}^2 - 3a_i a_{i+2} $, где $a_{i+1} > a_i > 0$

№	Вычисляемые значения
	$S_6 = \sum_{i=1}^n (\bar{a} \cdot a_i^2 - G \cdot a_i + 1)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1 a_3 + a_2 a_4 + \dots + a_{n-2} a_n}$
	$E = (\max(a_1, \dots, a_n) + \min(a_1, \dots, a_n))^2 - \max(a_1^2, \dots, a_n^2) - \min(a_1^2, \dots, a_n^2)$
4	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i^2 - 4a_i + 3 \geq 0$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \text{ — нечетное и кратно } 3$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i < 0 \text{ и } a_i \bmod 4 = 0$
	$P_2 = \prod_{i=1}^{n-2} (a_i + 3a_{i+1} + a_{i+2})$
	$S_3 = \sum_{i=1}^n (a_i^2 + 2\sqrt{ a_i } + 1)$
	$S_4 = \sum_{i=1}^{n-1} (a_i^2 - 4a_i a_{i+1} + a_{i+1}^2)$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 + a_{i+1}^2 - a_{i+2}^2 + a_i a_{i+2} , \text{ где } a_i < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left((a_i - \bar{a})^2 + \frac{a_i^2}{G+1} \right)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1^2 - a_1 a_2 + a_2^2 - a_2 a_3 + \dots + a_{n-1}^2 - a_{n-1} a_n}$
	$E = (\max(a_1, \dots, a_n))^3 - 3 \cdot \min(a_1^2, \dots, a_n^2) + \max(a_1^2, \dots, a_n^2)$
5	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i^2 + 2a_i \geq 30$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \bmod 4 = 2$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i < 0 \text{ и } a_i \bmod 5 = 0$
	$P_2 = \prod_{i=1}^{n-3} (a_i + a_{i+1} - a_{i+2} + a_{i+3})$
	$S_3 = \sum_{i=1}^n ((a_i + 1)^2 - a_i)$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - a_{i+1})^2 + a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - 4a_{i+1}^2 + a_{i+2}^2 , \text{ где } a_i < a_{i+1} \text{ и } a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n (a_i - \bar{a} + a_i^2 - \bar{a}^2)$

№	Вычисляемые значения
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1 a_2 - a_2 a_3 + a_3 a_4 - \dots + (-1)^{n-1} a_{n-1} a_n}$
	$E = (\min(a_1, a_2, \dots, a_n) + \max(a_1, a_2, \dots, a_n))^2 - \min(a_1^2, a_2^2, \dots, a_n^2)$
6	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i^2 - 2a_i \geq 25$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \text{ — двузначное число}$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i > 0 \text{ и } a_i \bmod 6 = 0$
	$P_2 = \prod_{i=1}^{n-2} (3a_i - 2a_{i+1} + a_{i+2})$
	$S_3 = \sum_{i=1}^n (a_i^2 - \sqrt{ a_i } + 4)$
	$S_4 = \sum_{i=1}^{n-1} (2a_i^2 + 2a_{i+1}^2 - 3a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_{i+2}^2 - 2a_{i+1}^2 + a_i^2 + 2a_i a_{i+2} , \text{ где } a_{i+2} > a_i$
	$S_6 = \sum_{i=1}^n (\bar{a}^2 + G - a_i^2)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1 a_3 + a_2 a_4 + a_3 a_5 + \dots + a_{n-2} a_n}$
	$E = \max(a_1, \dots, a_n)^2 + \min(a_1, \dots, a_n)^2 - \max(a_1^2, \dots, a_n^2) - \min(a_1^2, \dots, a_n^2)$
7	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i^2 + 2a_i - 8 > 0$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \text{ — четное и не делится на } 4$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i > 1 \text{ и } a_i \bmod 3 = 2$
	$P_2 = \prod_{i=1}^{n-2} (a_i + a_{i+1} + a_{i+2} + 2)$
	$S_3 = \sum_{i=1}^n (a_i^2 + 3 a_i - 2a_i)$
	$S_4 = \sum_{i=1}^{n-1} ((a_i + a_{i+1})^2 - 2a_i^2)$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - 2a_{i+1} a_{i+2} + a_{i+2}^2 , \text{ где } a_{i+1} > 0 \text{ и } a_i < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i^2}{\bar{a} + 1} + \frac{ a_i }{G + 1} \right)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1 a_2 + 2a_2 a_3 + 3a_3 a_4 + \dots + (n-1)a_{n-1} a_n}$

№	Вычисляемые значения
	$E = (\min(a_1, \dots, a_n) \cdot \max(a_1, \dots, a_n))^2 - \min(a_1^2, \dots, a_n^2)$
8	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 - a_i - 6 \geq 0$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 3 = 0$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i > 0$ и $a_i \bmod 4 = 1$
	$P_2 = \prod_{i=1}^{n-2} (2a_i + a_{i+1} - a_{i+2}^2)$
	$S_3 = \sum_{i=1}^n (a_i + 2)^2 + 3a_i$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - 2a_{i+1})^2 + 1)$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 + 2a_{i+1}^2 + a_{i+2}^2 - 2a_i a_{i+2} $, где $a_{i+1} > a_i$ и $a_{i+2} > a_{i+1}$
	$S_6 = \sum_{i=1}^n \left(\frac{(a_i - \bar{a})^2}{G+1} + \bar{a} \cdot \sqrt{ a_i } \right)$
	$R = \frac{a_1^2 + a_2^2 + \dots + a_n^2}{a_1 a_2 + a_2 a_3 + \dots + a_{n-1} a_n + a_n a_1}$
	$E = (\min(a_1, \dots, a_n) + \max(a_1, \dots, a_n))^2 - (\min(a_1^2, \dots, a_n^2) + \max(a_1^2, \dots, a_n^2))$
9	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 - 3a_i > 40$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 11 = 4$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < 0$ и $a_i \bmod 9 = 6$
	$P_2 = \prod_{i=1}^{n-3} (a_{i+1} + 2a_{i+2} - a_i - a_{i+3})$
	$S_3 = \sum_{i=1}^n (a_i + \sqrt{ a_i } + 2a_i^2)$
	$S_4 = \sum_{i=1}^{n-1} (3a_i^2 - 2a_i a_{i+1} + a_{i+1}^2 + 1)$
	$S_5 = \sum_{i=1}^{n-2} 2a_i^2 - 3a_{i+1}^2 + 2a_{i+2}^2 + a_i a_{i+2} $, где $a_{i+1} > a_i$ и $a_{i+1} > a_{i+2}$
	$S_6 = \sum_{i=1}^n \left((a_i - \bar{a})^2 + \frac{a_i^2}{G+1} + \sqrt{G} \right)$
	$R = \frac{a_1^2 + a_2^2 + \dots + a_n^2}{a_1^2 + 2a_1 a_2 + a_2^2 + 2a_2 a_3 + \dots + 2a_{n-1} a_n + a_n^2}$
	$E = (\max(a_1, \dots, a_n) - \min(a_1, \dots, a_n))^2 + (\max(a_1^2, \dots, a_n^2) - \min(a_1^2, \dots, a_n^2))$

№	Вычисляемые значения
10	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 - 8a_i > 25$
	$S_2 = \sum_{i=1}^n a_i$, где i — нечетное число
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < 0$ и $a_i \bmod 6 = 4$
	$P_2 = \prod_{i=1}^{n-3} (a_i + 3a_{i+1} - 2a_{i+2} + a_{i+3})$
	$S_3 = \sum_{i=1}^n (\sqrt{ a_i } + a_i^3 - a_i)$
	$S_4 = \sum_{i=1}^{n-1} (a_i^2 + a_{i+1}^2 - 4a_i a_{i+1} + 5)$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - a_{i+1}^2 + a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n ((a_i - \bar{a})^2 + a_i \sqrt{G})$
	$R = \frac{a_1^2 + a_2^2 + \dots + a_n^2}{a_1^2 a_2 + a_2^2 a_3 + \dots + a_{n-1}^2 a_n}$
	$E = (\max(a_1, \dots, a_n))^2 - \min(a_1^2, \dots, a_n^2) + \min(a_1, \dots, a_n)$
11	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 - 10a_i + 21 > 0$
	$S_2 = \sum_{i=1}^n a_i$, где i — четное число, не делящееся на 6
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < 0$ и $a_i \bmod 8 = 5$
	$P_2 = \prod_{i=1}^{n-2} (a_i^2 - a_{i+1} a_{i+2} + 2a_{i+1}^2)$
	$S_3 = \sum_{i=1}^n (a_i + 2a_i^2 - \sqrt{ a_i })$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - a_{i+1})^2 + 3a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - 2a_{i+1} a_{i+2} + a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\bar{a} \cdot a_i + \frac{ a_i }{G+1} \right)$
	$R = \frac{a_1^3 + a_2^3 + \dots + a_n^3}{a_1 a_2 - a_2 a_3 + a_3 a_4 - \dots + (-1)^{n-1} a_{n-1} a_n}$
	$E = (\max(a_1, \dots, a_n) - \min(a_1, \dots, a_n))^3 - \max(a_1^2, \dots, a_n^2) + \min(a_1^2, \dots, a_n^2)$
12	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 + 4a_i > 50$

№	Вычисляемые значения
	$S_2 = \sum_{i=1}^n a_i$, где i — нечетное число, не делящееся на 5
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < 0$ и $a_i \bmod 7 = 4$
	$P_2 = \prod_{i=1}^{n-3} (2a_i - a_{i+1} + 3a_{i+2} - a_{i+3})$
	$S_3 = \sum_{i=1}^n (a_i^2 + a_i - \sqrt{ a_i })$
	$S_4 = \sum_{i=1}^{n-1} ((a_i + a_{i+1})^2 - 3a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - 3a_{i+1}^2 + 2a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i^2}{\bar{a} + 1} + \frac{ a_i }{G + 1} \right)$
	$R = \frac{a_1^2 + a_2^2 + \dots + a_n^2}{a_1 a_3 - a_2 a_4 + a_3 a_5 - \dots + (-1)^n a_{n-2} a_n}$
	$E = (\max(a_1, \dots, a_n) + \min(a_1, \dots, a_n))^2 - \max(a_1^2, \dots, a_n^2) - \min(a_1, \dots, a_n)$
13	$S_1 = \sum_{i=1}^n a_i$, где $a_i \bmod 7 = 4$
	$S_2 = \sum_{i=1}^n a_i$, где i — нечетное и кратно 3
	$P_1 = \prod_{i=1}^n a_i$, где $a_i > 0$ и $a_i \bmod 5 = 1$
	$P_2 = \prod_{i=1}^{n-2} (a_i - 2a_{i+1} + 3a_{i+2})$
	$S_3 = \sum_{i=1}^n (3 a_i + a_i^2 - \sqrt{ a_i })$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - a_{i+1})^2 - 4a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - a_{i+1}^2 + a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i^2}{\bar{a} + 1} + \frac{ a_i }{G + 1} \right)$
	$R = \frac{a_1^2 + a_2^2 + \dots + a_n^2}{a_1 a_2 + a_2 a_3 + \dots + a_{n-1} a_n}$
	$E = (\max(a_1, \dots, a_n) - \min(a_1, \dots, a_n))^2 - \max(a_1^2, \dots, a_n^2) \cdot \min(a_1^2, \dots, a_n^2)$
14	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 - 4a_i - 5 \geq 0$
	$S_2 = \sum_{i=1}^n a_i$, где индекс i — число, делящееся на 3, но не делящееся на 6

№	Вычисляемые значения
	$P_1 = \prod_{i=1}^n a_i$, где $a_i > 0$ и $a_i \bmod 4 = 3$
	$P_2 = \prod_{i=1}^{n-2} (2a_i - 3a_{i+1} + a_{i+2})$
	$S_3 = \sum_{i=1}^n (a_i ^{3/2} + 2a_i^2 - a_i)$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - a_{i+1})^2 + 4a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 + a_{i+1}^2 - 2a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i^2}{G+1} + \frac{ a_i }{\bar{a}+1} \right)$
	$R = \frac{a_1^2 + a_2^2 + \dots + a_n^2}{a_1 a_2 - a_2 a_3 + a_3 a_4 - \dots + (-1)^{n-1} a_{n-1} a_n}$
	$E = (\max(a_1, \dots, a_n) + \min(a_1, \dots, a_n))^2 - \min(a_1^2, \dots, a_n^2)$
15	$S_1 = \sum_{i=1}^n a_i$, где $a_i^2 < 100$ и $a_i \bmod 5 = 0$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 3 = 1$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < 0$ и $a_i \bmod 2 = 1$
	$P_2 = \prod_{i=1}^{n-2} (a_i + a_{i+2})$
	$S_3 = \sum_{i=1}^n (\sqrt{ a_i +1} + a_i^2)$
	$S_4 = \sum_{i=1}^{n-1} (2a_i^2 - 3a_{i+1}^2 + a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - a_{i+1} a_{i+2} + 1 $, где $a_i < a_{i+1}$ и $a_{i+1} > a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{(a_i - \bar{a})^2}{G+1} \right)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1 a_2 + a_3 a_4 + a_5 a_6 + \dots}$
	$E = (\max(a_1, \dots, a_n))^2 - \max(a_1^3, a_2^3, \dots, a_n^3) + (\min(a_1, \dots, a_n))^2$
16	$S_1 = \sum_{i=1}^n a_i$, где $a_i \bmod 4 = 3$ и $a_i > 0$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 3 = 0$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < -5$ и $a_i \bmod 2 = 0$

№	Вычисляемые значения
	$P_2 = \prod_{i=1}^{n-3} (a_i + a_{i+1} + a_{i+3})$
	$S_3 = \sum_{i=1}^n (a_i + 2a_i^2 - 3)$
	$S_4 = \sum_{i=1}^{n-1} (a_i a_{i+1} + a_i - a_{i+1}^2)$
	$S_5 = \sum_{i=1}^{n-2} 3a_i - 2a_{i+1} + a_{i+2} $, где $a_i < a_{i+1}$ и $a_{i+2} < a_{i+1}$
	$S_6 = \sum_{i=1}^n (a_i^2 - 2\bar{a}a_i + \bar{a}^2)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1 a_4 + a_2 a_5 + a_3 a_6 + \dots + a_{n-3} a_n}$
	$E = (\max(a_1, a_2, \dots, a_n))^2 + \min(a_1^2, a_2^2, \dots, a_n^2) - (\min(a_1, \dots, a_n))^2$
17	$S_1 = \sum_{i=1}^n a_i$, где $ a_i > 30$ и $a_i \bmod 5 = 4$
	$S_2 = \sum_{i=1}^n a_i$, где i — нечетное число, кратное 7
	$P_1 = \prod_{i=1}^n a_i$, где $a_i \bmod 3 = 1$ и $a_i < 0$
	$P_2 = \prod_{i=1}^{n-2} (a_i + a_{i+1} + 2a_{i+2})$
	$S_3 = \sum_{i=1}^n (a_i - 5 + 3a_i)$
	$S_4 = \sum_{i=1}^{n-1} (a_i^2 + 2a_i a_{i+1} + a_{i+1}^2)$
	$S_5 = \sum_{i=1}^{n-2} a_i + 2a_{i+1} - 3a_{i+2} $, где $a_i > a_{i+2}$
	Пусть $\bar{g} = \sqrt[n]{ a_1 \cdot a_2 \cdot \dots \cdot a_n }$, тогда вычислить $S_6 = \sum_{i=1}^n (a_i^2 - \bar{g})^2$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1 a_2^2 + a_2 a_3^2 + a_3 a_4^2 + \dots + a_{n-1} a_n^2}$
	$E = \max(a_1^2, a_2^2, \dots, a_n^2) - (\max(a_1, a_2, \dots, a_n))^2 + \min(a_1, a_2, \dots, a_n)$
18	$S_1 = \sum_{i=1}^n a_i$, где $ a_i > 25$ и $a_i \bmod 5 = 3$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 6 = 1$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i \leq -5$ и $a_i \bmod 2 = 1$
	$P_2 = \prod_{i=1}^{n-1} (2a_i + 3a_{i+1})$

№	Вычисляемые значения
	$S_3 = \sum_{i=1}^n \left(a_i + \frac{1}{2} a_i^2 \right)$
	$S_4 = \sum_{i=1}^{n-1} \left((a_i - a_{i+1})^2 + 4a_i \right)$
	$S_5 = \sum_{i=1}^{n-2} \left 3a_i - a_{i+1} + a_{i+2}^2 \right $, где $a_i < a_{i+1} > a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i^2}{\bar{a} + 1} \right)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2 + 3}{2a_1a_2 + 2a_2a_3 + \dots + 2a_{n-1}a_n + 1}$
	$E = \left(\max(a_1, a_2, \dots, a_n) \right)^3 - \max(a_1^3, a_2^3, \dots, a_n^3) + \min(a_1, a_2, \dots, a_n)^2$
19	$S_1 = \sum_{i=1}^n a_i$, где $a_i \bmod 3 = 1$ и $a_i < 100$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 5 = 2$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i \bmod 6 = 5$ и $a_i < 0$
	$P_2 = \prod_{i=1}^{n-1} (a_i^2 + a_{i+1}^2)$
	$S_3 = \sum_{i=1}^n \left(\sqrt{ a_i } + \frac{1}{3} a_i^2 \right)$
	$S_4 = \sum_{i=1}^{n-1} (a_i^3 - 3a_{i+1}^2 + 5)$
	$S_5 = \sum_{i=1}^{n-2} \left a_i^2 + 2a_{i+1} - 3a_{i+2} \right $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left((a_i - \bar{a})^2 + \frac{a_i}{\bar{a} + 2} \right)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2 + 4a_1}{a_1a_2 + a_2a_3 + \dots + a_{n-1}a_n + 2a_n}$
	$E = \left(\max(a_1, a_2, \dots, a_n) \right)^2 - \min(a_1^2, a_2^2, \dots, a_n^2) + \left(\min(a_1, a_2, \dots, a_n) \right)^3$
20	$S_1 = \sum_{i=1}^n a_i$, где $a_i > 0$ и $a_i \bmod 4 = 0$
	$S_2 = \sum_{i=1}^n a_i$, где i делится на 3 и $i \leq n$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < -5$ и $a_i \bmod 2 = 1$
	$P_2 = \prod_{i=1}^{n-1} (2a_i + 3a_{i+1})$
	$S_3 = \sum_{i=1}^n (a_i ^{1/3} + 5a_i)$

№	Вычисляемые значения
	$S_4 = \sum_{i=1}^{n-1} \left((a_i + 2a_{i+1})^2 - 7 \right)$
	$S_5 = \sum_{i=1}^{n-2} \left(2a_i - 4a_{i+1} + a_{i+2} \right), \text{ где } a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{(a_i - \bar{a})^2}{\bar{a}^2 + 1} \right)$
	$R = \frac{a_1^3 + a_2^3 + a_3^3 + \dots + a_n^3}{a_1 a_2 + a_2 a_3 + a_3 a_4 + \dots + a_{n-1} a_n + 1}$
	$E = \max(a_1^2, a_2^2, \dots, a_n^2) - (\min(a_1, a_2, \dots, a_n))^2 + \max(a_1, a_2, \dots, a_n)$
21	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i < 70 \text{ и } a_i \bmod 5 = 2$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \bmod 3 = 2$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i < -1 \text{ и } a_i \bmod 3 = 0$
	$P_2 = \prod_{i=1}^{n-1} (a_i - 2a_{i+1})$
	$S_3 = \sum_{i=1}^n \left(\sqrt{ a_i } + a_i^2 \right)$
	$S_4 = \sum_{i=1}^{n-1} \left(a_i^2 + a_{i+1}^2 + 4a_i a_{i+1} \right)$
	$S_5 = \sum_{i=1}^{n-2} \left(a_i + a_{i+1} - 3a_{i+2} \right), \text{ где } a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left((a_i - \bar{a})^2 + 3\bar{a} \right)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{a_1 a_2 - a_2 a_3 + a_3 a_4 - \dots + (-1)^{n+1} a_{n-1} a_n - 2}$
	$E = \max(a_1, a_2, \dots, a_n)^2 + \min(a_1^2, a_2^2, \dots, a_n^2) - 2 \cdot \min(a_1, a_2, \dots, a_n)$
22	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i \bmod 6 = 4 \text{ и } a_i > 0$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \bmod 5 = 2$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i \leq -2 \text{ и } a_i \bmod 4 = 0$
	$P_2 = \prod_{i=1}^{n-1} (a_i + 3a_{i+1})$
	$S_3 = \sum_{i=1}^n \left(a_i + \sqrt{ a_i } + a_i^2 \right)$
	$S_4 = \sum_{i=1}^{n-1} \left(2a_i^2 - 3a_{i+1}^2 + 5 \right)^2$

№	Вычисляемые значения
	$S_5 = \sum_{i=1}^{n-2} a_i^2 + a_{i+1}^2 - 2a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n ((a_i - \bar{a})^2 + 2\bar{a} \cdot a_i)$
	$R = \frac{a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2}{-a_1a_2 + a_2a_3 - a_3a_4 + \dots + (-1)^n a_{n-1}a_n + 1}$
	$E = (\max(a_1, a_2, \dots, a_n))^3 - \max(a_1^2, a_2^2, \dots, a_n^2) + 2 \cdot \min(a_1, a_2, \dots, a_n)$
23	$S_1 = \sum_{i=1}^n a_i$, где $a_i > 0$ и $a_i \bmod 3 = 2$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 4 = 1$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < 0$ и $a_i \bmod 5 \neq 0$
	$P_2 = \prod_{i=1}^{n-1} (2a_i + a_{i+1})$
	$S_3 = \sum_{i=1}^n (a_i + a_i^2 - \sqrt{ a_i })$
	$S_4 = \sum_{i=1}^{n-1} (a_i^2 + a_{i+1}^2 - 3a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - 2a_{i+1}^2 + 3a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n (a_i^2 - 2\bar{a}a_i + \bar{a}^2)$
	$R = \frac{a_1^2 + 2a_2^2 + 3a_3^2 + \dots + na_n^2}{a_1a_2 + 2a_2a_3 + 3a_3a_4 + \dots + (n-1)a_{n-1}a_n}$
	$E = (\min(a_1, a_2, \dots, a_n))^2 + \max(a_1^2, a_2^2, \dots, a_n^2) - 3 \cdot \min(a_1^2, a_2^2, \dots, a_n^2)$
24	$S_1 = \sum_{i=1}^n a_i$, где $a_i \bmod 5 = 3$ и $a_i > 10$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 3 = 0$ и i не делится на 2
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < -5$ и $a_i \bmod 2 = 1$
	$P_2 = \prod_{i=1}^{n-2} (a_i + 2a_{i+1} + a_{i+2})$
	$S_3 = \sum_{i=1}^n (a_i ^{\frac{3}{2}} - a_i^2 + 1)$
	$S_4 = \sum_{i=1}^{n-1} (a_i^2 - 2a_i a_{i+1} + a_{i+1}^2)$
	$S_5 = \sum_{i=1}^{n-2} (a_i - 2a_{i+1} + a_{i+2} + a_i a_{i+1} - a_{i+2})$, где $a_i < a_{i+2}$

№	Вычисляемые значения
	$S_6 = \sum_{i=1}^n (a_i^2 - 2\bar{a}G + \bar{a}^2)$
	$R = \frac{a_1^2 + a_2^2 + \dots + a_n^2}{a_1^2 a_2 - a_2^2 a_3 + \dots + (-1)^{n+1} a_{n-1}^2 a_n}$
	$E = (\max(a_1, \dots, a_n) - \min(a_1, \dots, a_n))^2 + (\min(a_1^2, \dots, a_n^2))^3$
25	$S_1 = \sum_{i=1}^n a_i$, где $a_i \bmod 3 = 2$ и $a_i > 0$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 5 = 1$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < -4$ и $a_i \bmod 2 = 0$
	$P_2 = \prod_{i=1}^{n-2} (a_i - 2a_{i+1} + 3a_{i+2})$
	$S_3 = \sum_{i=1}^n (\sqrt{ a_i } + a_i^2 - a_i)$
	$S_4 = \sum_{i=1}^{n-1} (a_i^2 + a_{i+1}^2 - 4a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - 3a_{i+1}^2 + 2a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i}{\bar{a} + 1} + \frac{ a_i }{G + 1} \right)$
	$R = \frac{a_1^2 + 2a_2^2 + 3a_3^2 + \dots + na_n^2}{a_1 a_2 + 2a_2 a_3 + 3a_3 a_4 + \dots + (n-1)a_{n-1} a_n}$
	$E = (\max(a_1, \dots, a_n))^2 - \min(a_1^2, \dots, a_n^2) + \min(a_1, \dots, a_n)$
26	$S_1 = \sum_{i=1}^n a_i$, где $a_i > 10$ и $a_i \bmod 7 = 3$
	$S_2 = \sum_{i=1}^n a_i$, где $i \bmod 4 = 3$
	$P_1 = \prod_{i=1}^n a_i$, где $a_i < -3$ и $a_i \bmod 2 = 0$
	$P_2 = \prod_{i=1}^{n-2} (a_i^2 - a_{i+1} a_{i+2})$
	$S_3 = \sum_{i=1}^n (\sqrt{ a_i } + 2a_i^2 - 3a_i)$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - a_{i+1})^2 + 5a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^3 - 2a_{i+1}^2 + a_{i+2} $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i^2}{\bar{a} + 1} + \frac{ a_i }{G + 1} \right)$

№	Вычисляемые значения
	$R = \frac{\sum_{i=1}^n i a_i^2 + \sum_{i=1}^n a_i }{\sum_{i=1}^{n-1} i a_i a_{i+1} + \sum_{i=1}^{n-1} a_i - a_{i+1} + 1}$
	$E = (\min(a_1, \dots, a_n))^2 + \max(a_1^2, \dots, a_n^2) - 2 \min(a_1^2, \dots, a_n^2)$
27	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i > 0 \text{ и } a_i \bmod 6 = 1$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \bmod 5 = 3$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i < -2 \text{ и } a_i \bmod 3 = 2$
	$P_2 = \prod_{i=1}^{n-1} (a_i + 2a_{i+1})$
	$S_3 = \sum_{i=1}^n \left(a_i ^{\frac{1}{2}} + a_i^2 - a_i \right)$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - a_{i+1})^2 + 3a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} 2a_i^2 - a_{i+1}^2 + a_{i+2}^2 , \text{ где } a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i}{\bar{a} + 1} + \frac{ a_i }{G + 1} \right)$
	$R = \frac{\sum_{i=1}^n i^2 a_i^2 + \sum_{i=1}^n a_i }{\sum_{i=1}^{n-1} i a_i a_{i+1} + \sum_{i=1}^{n-1} a_i - a_{i+1} + 1}$
	$E = (\max(a_1, \dots, a_n))^2 - \min(a_1^2, \dots, a_n^2) + \min(a_1, \dots, a_n)$
28	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i > 20 \text{ и } a_i \bmod 5 = 1$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \bmod 7 = 4$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i < -10 \text{ и } a_i \bmod 3 = 1$
	$P_2 = \prod_{i=1}^{n-2} (3a_i - a_{i+1} + 2a_{i+2})$
	$S_3 = \sum_{i=1}^n (\sqrt{ a_i } + 2a_i^2 - a_i)$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - 3a_{i+1})^2 + a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} a_i^2 + a_{i+1}^2 - 4a_{i+2}^2 , \text{ где } a_i < a_{i+1} < a_{i+2}$

№	Вычисляемые значения
	$S_6 = \sum_{i=1}^n \left(\frac{a_i^2}{\bar{a}+1} + \frac{ a_i }{G+2} \right)$
	$R = \frac{\sum_{i=1}^n i a_i^2 + \sum_{i=1}^n a_i }{\sum_{i=1}^{n-1} i^2 a_i a_{i+1} + \sum_{i=1}^{n-1} a_i - a_{i+1} + 1}$
	$E = (\min(a_1, \dots, a_n))^3 + \max(a_1^2, \dots, a_n^2) - \min(a_1^2, \dots, a_n^2)$
	$E = (\max(a_1, \dots, a_n))^3 - \min(a_1^3, \dots, a_n^3) + \min(a_1, \dots, a_n)^2$
29	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i \bmod 8 = 3 \text{ и } a_i > 15$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \bmod 6 = 2$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i < -7 \text{ и } a_i \bmod 5 = 4$
	$P_2 = \prod_{i=1}^{n-1} (4a_i - a_{i+1})$
	$S_3 = \sum_{i=1}^n (\sqrt{ a_i } + 3a_i^2 - 2a_i)$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - 2a_{i+1})^2 + 4a_i a_{i+1})$
	$S_5 = \sum_{i=1}^{n-2} 2a_i^2 - 3a_{i+1}^2 + a_{i+2}^2 , \text{ где } a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i^3}{\bar{a}+1} + \frac{ a_i }{G+1} \right)$
	$R = \frac{\sum_{i=1}^n i a_i^3 + \sum_{i=1}^n a_i }{\sum_{i=1}^{n-1} i^2 a_i a_{i+1} + \sum_{i=1}^{n-1} a_i - a_{i+1} + 1}$
	$E = (\max(a_1, \dots, a_n))^3 - \min(a_1^3, \dots, a_n^3) + \min(a_1, \dots, a_n)^2$
30	$S_1 = \sum_{i=1}^n a_i, \text{ где } a_i > 5 \text{ и } a_i \bmod 9 = 2$
	$S_2 = \sum_{i=1}^n a_i, \text{ где } i \bmod 7 = 1$
	$P_1 = \prod_{i=1}^n a_i, \text{ где } a_i < -3 \text{ и } a_i \bmod 4 = 3$
	$P_2 = \prod_{i=1}^{n-1} (3a_i + 2a_{i+1})$
	$S_3 = \sum_{i=1}^n (\sqrt{ a_i } + 4a_i^2 - a_i)$
	$S_4 = \sum_{i=1}^{n-1} ((a_i - 3a_{i+1})^2 + 2a_i a_{i+1})$

№	Вычисляемые значения
	$S_5 = \sum_{i=1}^{n-2} a_i^2 - 4a_{i+1}^2 + 3a_{i+2}^2 $, где $a_i < a_{i+1} < a_{i+2}$
	$S_6 = \sum_{i=1}^n \left(\frac{a_i^2}{\bar{a} + 2} + \frac{ a_i }{G + 3} \right)$
	$R = \frac{\sum_{i=1}^n i a_i^3 + \sum_{i=1}^n a_i }{\sum_{i=1}^{n-1} i^2 a_i a_{i+1} + \sum_{i=1}^{n-1} a_i + a_{i+1} + 2}$
	$E = (\min(a_1, \dots, a_n))^3 - \max(a_1^3, \dots, a_n^3) + \max(a_1, \dots, a_n)^2$

Таблица 7.4

Задание 7.2

Выполнить задания в соответствии с табл. 7.6. Списки в программном коде задать литерально.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результаты работы программы при различных исходных данных.

Таблица 7.5

Задание 7.3

Выполнить задания в соответствии с табл. 7.6. Списки в программном коде задать литерально.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результаты работы программы при различных исходных данных.

Таблица 7.6

№	Задания
1	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен A[i], если сумма цифр A[i] больше суммы цифр B[i], иначе — B[i]

№	Задания
	Заданы два одномерных списка A и B одной размерности. Получить список C, в котором каждый элемент равен 1, если $A[i]$ — квадрат, а $B[i]$ — куб натурального числа, иначе 0
2	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен длине максимальной последовательности одинаковых цифр в числе $A[i] + B[i]$
	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен $A[i]$, если разность $ A[i] - B[i] $ делится на 3, иначе $B[i]$
3	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен i , если $A[i]$ — четное число и $B[i]$ — нечетное, иначе $-i$
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий числа, обратные $A[i] + B[i]$, если результат отличен от нуля (в виде $1/(A[i] + B[i])$), иначе 0
4	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен количеству делителей числа $A[i] * B[i]$
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий True , если сумма квадратов $A[i]$ и $B[i]$ дает четное число, иначе False
5	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий минимальное значение из $A[i]$, $B[i]$ и их суммы
	Заданы два одномерных списка A и B одной размерности. Получить список C, элементы которого равны числу цифр в числе $ A[i] - B[i] $
6	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен 1, если число $A[i] * B[i]$ содержит хотя бы две одинаковые цифры, иначе 0
	Заданы два одномерных списка A и B одной размерности. Получить список C, в котором каждый элемент равен 1, если $A[i]$ является делителем $B[i] + i$, иначе 0
7	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий числа, в которых количество четных цифр суммы $A[i] + B[i]$ больше количества нечетных
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий True , если число $A[i] - B[i]$ — палиндром, иначе False
8	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен количеству совпадающих позиций, на которых в десятичной записи $A[i]$ и $B[i]$ стоят одинаковые цифры
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий сумму квадратов цифр разности $A[i] - B[i]$
9	Заданы два одномерных списка A и B одной размерности. Получить список C, в котором каждый элемент равен $A[i]$, если $B[i]$ имеет ровно два делителя (то есть простое число), иначе $A[i] + B[i]$
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий True , если количество цифр $A[i]$ и $B[i]$ одинаково, иначе False
10	Заданы два одномерных списка A и B одной размерности. Получить список C, в котором каждый элемент равен 1, если сумма цифр $A[i] + B[i]$ делится на 3 и на 4, иначе 0
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий количество позиций, на которых цифры десятичных записей $A[i]$ и $B[i]$ различаются

№	Задания
11	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен $A[i] * B[i]$, если сумма этих двух чисел четна, иначе — 0
	Заданы два одномерных списка A и B одной размерности. Получить список C, в котором каждый элемент равен наибольшей цифре в десятичной записи $A[i] + B[i]$
12	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий знак суммы $A[i] + B[i]$: 1 для положительного результата, 0 для нуля, -1 для отрицательного
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий логическое значение, равное True , если $A[i]$ делится на $B[i]$ или наоборот, иначе False
13	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий на позиции i наименьшее число, которое делится и на $A[i]$, и на $B[i]$ (наименьшее общее кратное)
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий $A[i]$, если оно больше абсолютного значения $B[i]$, иначе $B[i]$
14	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий количество нулей в десятичной записи суммы $A[i] + B[i]$
	Заданы два одномерных списка A и B одной размерности. Получить список C, где каждый элемент равен произведению всех нечетных цифр в числе $A[i] + B[i]$
15	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий значения $A[i]$, если $B[i]$ является простым числом, иначе $B[i]$
	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен количеству цифр, которые входят как в $A[i]$, так и в $B[i]$
16	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий минимальный общий делитель чисел $A[i]$ и $B[i]$, отличающийся от единицы, если таковой существует
	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен 1, если хотя бы одна цифра в $A[i]$ совпадает с любой цифрой в $B[i]$, иначе 0
17	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен количеству делений $A[i]$ на $B[i]$, пока результат остается целым числом
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий True , если $A[i] + B[i]$ — палиндром в десятичной записи, иначе False
18	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен $A[i]$, если $A[i] * B[i]$ четное число и $B[i]$ делится на 3
	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен остатку от деления суммы всех цифр $A[i] + B[i]$ на 5
19	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий на позиции i количество общих делителей чисел $A[i]$ и $B[i]$, кроме 1 и самих чисел
	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен 0, если $A[i]$ и $B[i]$ имеют одинаковый знак, иначе 1
20	Заданы два одномерных списка A и B одной размерности. Получить список C, в котором каждый элемент равен квадрату суммы квадратов цифр чисел $A[i]$ и $B[i]$

№	Задания
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий 1, если сумма цифр A[i] равна сумме цифр B[i], иначе 0
21	Заданы два одномерных списка A и B одной размерности. Получить список C, в котором каждый элемент равен A[i], если сумма A[i] + B[i] четна, и B[i] в противном случае
	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен сумме квадратов всех цифр в десятичной записи числа A[i] + B[i]
22	Заданы два одномерных списка A и B одной размерности. Получить список C, элементы которого равны количеству четных цифр в десятичной записи разности A[i] - B[i]
	Заданы два одномерных списка A и B одной размерности. Получить список C, элементы которого равны 1, если A[i] и B[i] имеют одинаковую четность, и 0 в противном случае
23	Заданы два одномерных списка A и B одной размерности. Получить список C, элементы которого равны сумме всех целых чисел от A[i] до B[i] включительно, если A[i] < B[i], иначе — 0
	Заданы два одномерных списка A и B одной размерности. Получить список C, элементы которого равны количеству общих делителей чисел A[i] и B[i]
24	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий сумму элементов A[i] и B[i], если оба числа положительны и нечетны, иначе -1
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий числа, состоящие из цифр, встречающихся одновременно в десятичных записях A[i] и B[i] (в порядке возрастания)
25	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий количество цифр в десятичной записи произведения A[i] * B[i]
	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен B[i], если A[i] является квадратом целого числа, иначе — A[i] + B[i]
26	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен сумме произведения A[i] на индекс i и остатка от деления B[i] на 3
	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен A[i], если A[i] > B[i], иначе — B[i]
27	Заданы два одномерных списка A и B одной размерности. Получить список C, каждый элемент которого равен количеству делителей суммы A[i] + B[i]
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий логическое значение True, если сумма A[i] + B[i] кратна 5, и False в противном случае
28	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий разность квадратов A[i] и B[i] только для тех позиций, на которых сумма A[i] + B[i] нечетна
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий значения B[i], если A[i] делится на B[i] без остатка, и -1 в противном случае
29	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий количество общих цифр в десятичных записях A[i] и B[i]

№	Задания
	Заданы два одномерных списка A и B одной размерности. Получить список C, в котором каждый элемент равен 1, если обе величины на данной позиции положительны, и 0 в противном случае
30	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий на позициях с четными индексами сумму $A[i] + B[i]$, а на нечетных — разность $A[i] - B[i]$
	Заданы два одномерных списка A и B одной размерности. Получить список C, содержащий значения $A[i]$, у которых значение $B[i]$ входит в диапазон от $A[i] - 5$ до $A[i] + 5$ включительно

Задание 7.4 (практико-ориентированное задание)

Выполнить задания в соответствии с табл. 7.7. Списки в программном коде задать литерально.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результат вычислений при различных исходных данных.

Таблица 7.7

№	Задания
1	В отделе маркетинга проводится анализ ценовой политики на основании собранных данных об объеме заказов по дням. Дан список, содержащий количество заказов за каждый день месяца. Требуется определить количество дней, в которых число заказов одновременно превышает среднее арифметическое значений за весь месяц и оказывается меньше максимального значения за рассматриваемый период
	В процессе верификации клиентских анкет необходимо выявить записи с потенциальной ошибкой в написании фамилий. Дан список строк, содержащих фамилии. Требуется подсчитать количество фамилий, содержащих хотя бы один символ, встречающийся в строке дважды или более раз
2	В рамках анализа пользовательской активности необходимо обработать список, содержащий количество выполненных действий за день. Требуется построить новый список, состоящий из квадратов всех нечетных значений, расположенных в обратном порядке относительно их появления в исходных данных
	В процессе анализа пользовательских идентификаторов, представленных в виде строк, необходимо определить количество строк, начинающихся и заканчивающихся одной и той же буквой
3	Для анализа периодов стабильной работы оборудования, фиксируемых в виде одномерного списка целых чисел, необходимо определить длину максимально продолжительной непрерывной подпоследовательности, состоящей исключительно из четных значений
	В задаче по управлению товарными запасами задан одномерный список строк, представляющих наименования продукции с различным регистром символов. Требуется сформировать новый список, содержащий те же наименования, приведенные к нижнему регистру, отсортированные в алфавитном порядке; при этом исходный список должен остаться без изменений

№	Задания
4	В производственном учете представлен одномерный список целых чисел, отражающий количество отработанных часов сотрудников за каждый рабочий день. Требуется определить длину максимального отрезка подряд идущих дней, в которых количество отработанных часов превышает среднее значение за весь период
	В системе анализа текстовых данных представлен одномерный список строк, содержащих клиентские отзывы с различным регистром символов и пунктуацией. Требуется сформировать новый список, отсортированный по алфавиту без учета регистра и с удалением всех знаков препинания, при этом исходный список должен остаться неизменным
5	В рамках анализа производственных данных имеется одномерный список целых чисел, каждое из которых является уникальным кодом изделия. Необходимо подсчитать сумму тех элементов, индексы которых равны сумме цифр соответствующего кода
	В системе обработки клиентских сообщений представлен одномерный список строк — отзывы пользователей. Требуется определить максимальную длину отзыва, содержащего как минимум одну заглавную и одну строчную букву латинского алфавита
6	В ходе анализа результатов автоматизированного тестирования представлен одномерный список целых чисел, отражающих оценки пользователей. Требуется определить количество последовательностей, состоящих из двух и более подряд идущих одинаковых оценок
	В системе управления контентом имеется одномерный список строк, представляющих заголовки статей. Требуется сформировать новый список, включающий только заголовки, длина которых совпадает с длиной самой короткой строки из исходного списка
7	В системе контроля качества продукции представлен одномерный список целочисленных значений, отражающих показатели последовательных партий. Требуется определить количество элементов, каждое из которых строго больше предыдущего и строго меньше следующего элемента. Первый и последний элементы исключаются из анализа ввиду отсутствия соседних значений с обеих сторон
	В системе обработки результатов экспериментов представлен одномерный список вещественных чисел. Необходимо сформировать новый список, включающий только те элементы, которые встречаются единожды в исходном наборе данных
8	В системе финансового учета предприятия представлен одномерный список целых чисел, отражающих доходы за последовательные периоды. Требуется определить максимальное значение суммы среди всех подпоследовательностей длиной три, состоящих из подряд идущих элементов, с целью выявления наиболее прибыльных трехпериодных интервалов
	В базе текстовой информации предприятия имеется одномерный список строк, представляющих описания продукции. Необходимо определить количество строк, содержащих две или более подряд идущих гласных букв латинского алфавита
9	В списке данных предприятия содержится одномерный список вещественных чисел, отражающих показатели эффективности различных подразделений. Требуется вычислить индекс элемента, значение которого минимально отличается от среднего арифметического всех элементов списка, с целью выявления подразделения, наиболее соответствующего среднему уровню. При совпадении таких минимальных расстояний выбрать элемент с наименьшим индексом

№	Задания
	В текстовом архиве организации хранится одномерный список строк, представляющих наименования товарных позиций. Необходимо сформировать новый список, состоящий из первых символов всех непустых строк исходного списка, исключая пустые элементы
10	В отчетах отдела продаж представлен одномерный список целых чисел, каждое из которых отражает количество реализованных товаров за день. Определить количество дней, в которых объем продаж совпадает с произведением суммы цифр этого значения на количество этих цифр
	В системе учета клиентов содержится одномерный список строк с именами пользователей. Требуется определить количество имен, содержащих более трех различных гласных букв латинского алфавита, для сегментации клиентов по сложности имени
11	В базе данных предприятия хранится одномерный список целых чисел, каждое из которых отражает объем производства за один рабочий день. Требуется определить количество участков подряд идущих трех дней, в которых сумма объемов превышает среднее арифметическое всех значений в списке, чтобы выявить периоды повышенной производственной активности
	В системе обработки текстовой информации хранится одномерный список строк, представляющих наименования продуктов. Требуется определить длину самой длинной строки, в которой количество латинских букв превышает количество всех остальных символов, с целью выделения наименований, содержащих преимущественно алфавитные символы для последующей текстовой аналитики
12	В системе учета производства имеется одномерный список целых чисел, каждое из которых соответствует количеству изготовленных изделий за определенный период. Требуется построить новый список, включающий только те значения, расположенные на позициях с индексами, кратными трем, для анализа производства в фиксированные интервалы времени
	В системе учета документации хранится одномерный список строк, каждая из которых представляет собой заголовок отчета за определенный период. Требуется определить индекс первого заголовка, длина которого строго превышает сумму длин всех предыдущих заголовков, для выявления отчета с аномально большим содержанием. В случае отсутствия такого заголовка вернуть значение None
13	В системе анализа данных хранится одномерный список целых чисел, представляющих показатели измерений. Требуется определить количество элементов списка, значения которых равны разности двух последующих за ними элементов, при этом последние два элемента списка исключаются из рассмотрения для предотвращения выхода за пределы индекса
	В базе данных текстовых записей хранится одномерный список строк, каждая из которых содержит наименование категории. Требуется сформировать новый список, включающий все строки, в которых число заглавных букв строго превышает количество строчных, позволяя выделить категории с особым стилевым оформлением
14	В списке вещественных чисел, отражающих измерения параметров, требуется сформировать новый список, в котором каждый элемент, начиная с третьего, заменен на среднее арифметическое двух предыдущих элементов исходного списка, сохранив при этом неизменными первые два элемента
	В базе данных с текстовыми метками представлен одномерный список строк. Требуется определить количество строк, начинающихся и завершающихся гласной буквой латинского алфавита одновременно, позволяющее выделить категории с определенным стилистическим признаком

№	Задания
15	Дан список целых чисел, представляющий суточные значения температуры воздуха в течение месяца. Требуется определить, наблюдался ли в этом периоде устойчивый рост температуры, выражающийся в наличии строго возрастающей подпоследовательности длины не менее четырех. Если такая последовательность существует, следует вернуть булево значение True , в противном случае — False
	Дан список строк, каждая из которых содержит текст заголовка электронной почты, полученной на корпоративный почтовый ящик. Необходимо построить строку, включающую в себя последние символы всех заголовков, длина которых является четным числом. Заголовки, не содержащие символов, следует исключить из рассмотрения
16	Дан список целых чисел, представляющий последовательность транзакционных изменений в бухгалтерском журнале за определенный период, где значение 0 используется для фиксации начальной и конечной точки ревизии. Требуется вычислить суммарное изменение между первой и последней точкой ревизии, исключая сами значения 0. Если в журнале отсутствует либо единственное значение 0, вернуть значение None
	Дан список строк, содержащий набор электронных сообщений, полученных по результатам анкетирования. Требуется определить количество сообщений, в которых первая буква — заглавная латинская, а последняя — точка. Пустые строки не учитывать
17	Дан список целых чисел, отражающий изменения температуры воздуха по дням в течение месяца, где положительные значения соответствуют повышению, а отрицательные — понижению температуры. Необходимо вычислить разность между количеством положительных и количеством отрицательных значений
	Дан список строк, каждая из которых представляет собой описание заказа, включающее текстовые данные и числовые идентификаторы. Требуется упорядочить описания по количеству цифр в каждой строке, при равном количестве цифр сохранить первоначальный порядок следования строк
18	Рассматривается список целых чисел, в котором каждый элемент отражает количество единиц продукции, выпущенных на соответствующем производственном участке в течение смены. Нумерация участков начинается с нуля. Необходимо определить суммарное количество участков, на которых объем выпускаемой продукции в единицах совпадает с порядковым номером участка. Иными словами, требуется вычислить сумму значений тех элементов списка, чьи числовые значения равны их индексам
	Рассматривается одномерный список строк, каждая из которых представляет собой комментарий, оставленный пользователем на сайте интернет-магазина. Необходимо определить количество таких комментариев, в которых последний символ строки является буквой и встречается в этом же комментарии не менее двух раз. Комментарии, не содержащие буквенных символов или имеющие пустое содержимое, не учитывать при анализе
19	Дан одномерный список вещественных чисел, представляющий собой набор отклонений от нормативных параметров при серийном измерении. Требуется построить новый список, содержащий только те значения, которые по модулю расположены ближе к нулю, чем к среднему арифметическому всех элементов исходного списка. В результате должен быть сформирован список, отражающий наименее критичные отклонения, требующие минимального вмешательства

№	Задания
	Рассматривается одномерный список строк, каждая из которых представляет собой пользовательское текстовое поле при вводе сведений в информационную систему. Требуется определить общее количество символов среди всех таких строк, в которых присутствует хотя бы один цифровой символ и полностью отсутствуют знаки пунктуации. Анализ подлежат только строки, содержащие хотя бы один непробельный символ
20	Определить количество элементов одномерного списка вещественных чисел, которые по абсолютному значению находятся ближе к нулю, чем к медиане списка, вычисляемой по формуле: если n — количество элементов списка, отсортированных по возрастанию как $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$, то значение медианы $\text{равно } m = x_{\frac{n+1}{2}}, \text{ если } n \text{ нечетно, или } m = \frac{x_{\frac{n}{2}} + x_{\frac{n}{2}+1}}{2}, \text{ если } n \text{ четно}$ Рассматривается одномерный список строк, каждая из которых содержит текстовые фрагменты сообщений, направленных через систему внутренней переписки организации. Необходимо определить количество строк, в которых присутствует хотя бы одно вхождение подстроки длины более двух символов, повторяющееся в пределах той же строки хотя бы дважды без учета регистра символов
21	Для анализа эффективности идентификации продукции на предприятии требуется определить количество кодов из одномерного списка целых чисел, у которых значение делится без остатка на количество цифр, входящих в состав этого числа В системе автоматизированной обработки текстовой информации необходимо определить количество строк из одномерного списка, в которых количество пробелов превышает количество всех остальных символов
22	В процессе обработки сигналов акселерометра требуется преобразовать список зафиксированных целочисленных значений ускорений в последовательность абсолютных величин, отсортированных по убыванию При автоматической генерации пользовательских идентификаторов на основе текстовых сообщений требуется из переданного списка строк извлечь первые символы только тех элементов, длина которых кратна трем. Полученные символы необходимо объединить в строку в порядке их появления в исходном списке
23	В рамках анализа данных о температурных аномалиях, зафиксированных датчиками в течение суток необходимо получить список абсолютных отклонений температур от нуля, представленных в виде целых чисел. Полученные значения требуется отсортировать по убыванию, чтобы выявить наибольшие отклонения В процессе агрегации текстовых данных из формы обратной связи необходимо сформировать строку, содержащую первые символы всех сообщений, длина которых кратна трем
24	В рамках модуля интеллектуального анализа числовых идентификаторов продукции необходимо определить количество кодов из представленного списка целых чисел, сумма цифр в которых представляет собой нечетное число При автоматическом разборе текстовых данных, поступающих в виде строк, необходимо определить количество сообщений, в которых содержатся как минимум три различные гласные буквы латинского алфавита
25	В процессе анализа временных рядов, полученных в результате измерений физической величины, необходимо трансформировать исходный список вещественных значений таким образом, чтобы каждый элемент, начиная со второго, заменялся средним геометрическим между текущим и предыдущим элементом. Первый элемент при этом должен остаться без изменений

№	Задания
	В системе обработки символьных последовательностей необходимо проверить, содержит ли поступивший список хотя бы одну строку, являющуюся палиндромом. При проверке должны учитываться регистр символов и наличие пробелов
26	В процессе калибровки сенсоров в автоматизированной измерительной системе необходимо провести трансформацию одномерного списка вещественных чисел, представляющих собой сырые показания. Требуется заменить каждый элемент, начиная со второго, на среднее арифметическое квадратов текущего и предыдущего элементов. Первый элемент следует оставить без изменений
	При автоматической генерации пользовательских идентификаторов необходимо проверить, имеются ли в списке строк такие элементы, длина которых больше восьми символов и в которых содержатся одновременно хотя бы одна заглавная латинская буква, хотя бы одна строчная и хотя бы одна цифра
27	При анализе входного потока данных, поступающего с датчиков, требуется отфильтровать значения, представляющие собой простые числа. Из исходного одномерного списка целых чисел необходимо сформировать новый список, содержащий только простые числа, отсортированные по убыванию
	В процессе проверки качества данных в системе обработки текстовых идентификаторов необходимо определить строку, в которой максимальное количество символов встречается более одного раза. В случае наличия нескольких таких строк следует выбрать первую из них по порядку следования в исходном списке
28	В ходе анализа бинарных характеристик идентификаторов оборудования необходимо определить количество числовых меток, двоичная запись которых содержит не менее трех единичных разрядов. Дан одномерный список целых чисел. Требуется вычислить количество элементов, удовлетворяющих данному условию
	При верификации корректности оформления служебной документации необходимо подсчитать общее количество заглавных букв в текстах, в которых каждое слово начинается с прописной буквы. Дан одномерный список строк. Требуется определить суммарное количество заглавных букв в тех строках, где все слова начинаются с заглавной буквы
29	В рамках оценки допустимых отклонений параметров изделий задан список измеренных значений. Требуется заменить каждое значение на ближайшее большее либо равное ему четное целое число, отражающее округленную в сторону повышения границу допуска. Результатом должен быть список округленных значений
	В процессе анализа пользовательских паролей необходимо определить количество таких паролей, в которых все символы уникальны и упорядочены в лексикографическом порядке без учета регистра
30	При обработке данных температурного мониторинга задан список вещественных значений, представляющих собой температуры, зафиксированные в разное время суток. Необходимо определить количество таких измерений, двоичное представление целой части которых содержит не менее трех единичных разрядов
	В процессе верификации электронной переписки получен список строк, каждая из которых представляет собой отдельное письмо. Требуется определить общее количество заглавных букв в тех письмах, где каждое слово начинается с прописной буквы

Задание 7.5

Выполнить задания в соответствии с табл. 7.8. Списки в программном коде задать литерально.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результаты работы программы при различных исходных данных.

Таблица 7.8

№	Задания
1	Дан одномерный список целых чисел. Необходимо определить количество всех непересекающихся непрерывных подпоследовательностей длины не менее двух, сумма элементов которых кратна 3. Подпоследовательности считаются непересекающимися, если ни один индекс не входит более чем в одну из них
	Дан одномерный список вещественных чисел. Требуется определить максимальную длину непрерывной подпоследовательности подряд идущих элементов, для которой абсолютное значение разности между каждым парой соседних элементов строго убывает по мере продвижения вдоль подпоследовательности
2	Дан одномерный список целых чисел. Для каждого элемента необходимо определить количество элементов, расположенных правее него, которые строго меньше его. Требуется построить новый список, содержащий такие количества для каждого элемента, и вывести сумму всех значений нового списка. Использование вложенных циклов не допускается
	Дан одномерный список строк. Необходимо определить длину самой длинной подпоследовательности подряд идущих строк, в которой каждая следующая строка строго больше предыдущей в лексикографическом порядке. Сравнение производится с учетом регистра
3	Дан одномерный список целых чисел. Требуется определить максимальную длину непрерывной подпоследовательности, в которой абсолютная разность между любыми двумя элементами не превышает некоторого фиксированного положительного порога, заданного литерально в теле программы
	Дан одномерный список вещественных чисел. Требуется определить, существует ли такая перестановка элементов списка, при которой образуется строго возрастающая последовательность модулей этих значений. Ответ необходимо представить в виде булевого значения
4	Дан одномерный список целых чисел. Требуется определить наибольшую сумму среди всех непрерывных подпоследовательностей, в которых сумма четных элементов равна сумме нечетных. Если таких подпоследовательностей несколько, выбрать ту, в которой сумма элементов максимальна
	Дан одномерный список строк. Требуется определить количество строк, для которых существует хотя бы одна иная строка из списка, являющаяся анаграммой (то есть содержащая те же символы в другом порядке). Строка не считается анаграммой сама себе

№	Задания
5	Дан одномерный список целых чисел. Необходимо определить количество таких элементов, которые равны среднему арифметическому двух других различных элементов из списка. При этом среднее арифметическое должно быть целым числом, и проверяемый элемент не должен совпадать с ни одним из двух составляющих среднее
	Дан одномерный список вещественных чисел. Требуется определить, существует ли в списке хотя бы одна тройка подряд идущих элементов, такая что среднее арифметическое первого и третьего элементов равно второму элементу. Ответ представить в виде булевого значения
6	Дан одномерный список целых чисел. Требуется определить количество пар элементов, стоящих на расстоянии не менее двух индексов друг от друга, сумма которых кратна 5, а разность четна
	Дан одномерный список строк. Необходимо определить длину самой длинной подпоследовательности подряд идущих строк, в которых каждый символ строки лексикографически не меньше соответствующего символа предыдущей строки на той же позиции. Строки считаются сопоставимыми, если их длины совпадают
7	Дан одномерный список вещественных чисел. Требуется определить максимальную длину непрерывной подпоследовательности, в которой каждый элемент, начиная со второго, не менее среднего арифметического всех предыдущих элементов этой подпоследовательности
	Дан одномерный список целых чисел. Необходимо определить количество таких троек подряд идущих элементов, сумма которых кратна количеству нечетных чисел среди всех элементов списка
8	Дан одномерный список строк. Требуется определить, можно ли отсортировать список в лексикографическом порядке, выполнив не более одного обмена двух элементов. Ответ представить в виде булевого значения
	Дан одномерный список целых чисел. Необходимо определить, можно ли получить строго возрастающую последовательность, удалив из исходного списка не более одного элемента. Ответ представить в виде булевого значения
9	Дан одномерный список вещественных чисел. Требуется определить количество индексов, для которых значение элемента больше, чем сумма всех элементов, находящихся левее и правее него вместе взятых
	Дан одномерный список строк. Необходимо определить количество строк, в которых есть хотя бы одна пара соседних символов, образующих палиндром длины два
10	Дан одномерный список целых чисел. Требуется построить новый список, в котором каждый элемент равен количеству различных цифр в десятичной записи соответствующего элемента исходного списка. Отрицательные числа учитывать по модулю
	Дан одномерный список вещественных чисел. Необходимо определить количество всех непрерывных подпоследовательностей длиной не менее трех элементов, в которых каждый следующий элемент больше предыдущего и общий рост превышает максимальное значение в исходном списке
11	Дан одномерный список целых чисел. Требуется определить количество различных пар элементов, произведение которых является точным квадратом целого числа и при этом оба элемента не равны нулю
	Дан одномерный список строк. Необходимо определить количество строк, каждая из которых содержит хотя бы одну такую подстроку длины не менее трех, которая совпадает с какой-либо другой подстрокой той же строки, расположенной в другой позиции

№	Задания
12	Дан одномерный список вещественных чисел. Требуется определить, можно ли переставить элементы списка так, чтобы сумма любых двух соседних элементов была положительной. Ответ представить в виде булевого значения
	Дан одномерный список целых чисел. Необходимо определить наименьшее количество элементов, которые требуется удалить, чтобы оставшаяся последовательность не содержала двух подряд идущих элементов с одинаковой четностью
13	Дан одномерный список строк. Требуется определить, существует ли такая пара строк с различными индексами, что одна является обратной по символам другой. Регистр имеет значение. Ответ представить в виде булевого значения
	Дан одномерный список целых чисел. Необходимо определить длину самой длинной подпоследовательности, в которой произведение всех элементов является четным, а сумма — нечетной
14	Дан одномерный список вещественных чисел. Требуется построить новый список, в котором каждый элемент равен среднеквадратичному отклонению текущего элемента от всех предыдущих. Первый элемент в новый список не включается
	Дан одномерный список строк. Необходимо подсчитать количество строк, в которых хотя бы один символ встречается более двух раз подряд, причем такая повторяющаяся подпоследовательность является лексикографически минимальной среди всех в данной строке
15	Дан одномерный список целых чисел. Требуется определить, содержит ли список хотя бы один элемент, значение которого совпадает с произведением всех других элементов. Наличие нулей учитывать корректно
	Дан одномерный список целых чисел. Необходимо определить количество таких элементов, сумма всех цифр которых совпадает с количеством делителей самого элемента, включая 1 и его самого
16	Дан одномерный список вещественных чисел. Требуется определить, является ли исходная последовательность локально симметричной: для каждого элемента (за исключением крайних) проверить, равен ли он среднему арифметическому соседних. Ответ представить в виде булевого значения
	Дан одномерный список строк. Необходимо определить количество строк, в которых есть хотя бы две непересекающиеся анаграммы длины не менее трех символов
17	Дан одномерный список целых чисел. Требуется найти сумму всех элементов, которые являются наибольшими в своей тройке: каждый элемент сравнивается с двумя соседями, если они существуют
	Дан одномерный список целых чисел. Необходимо построить новый список, содержащий длины максимальных строго возрастающих подпоследовательностей, начинающихся с каждого индекса, для которого такая подпоследовательность существует
18	Дан одномерный список вещественных чисел. Требуется определить минимальное количество фрагментов, на которые можно разбить последовательность, чтобы внутри каждого фрагмента максимальное и минимальное значение отличались не более чем на заданную константу (заданную литерально)
	Дан одномерный список строк. Необходимо определить количество строк, в которых ровно половина символов относится к латинскому алфавиту, а другая половина — к десятичным цифрам. Строки нечетной длины игнорировать
19	Дан одномерный список целых чисел. Требуется определить количество троек с попарно различными индексами, сумма элементов которых равна нулю. Порядок следования элементов значения не имеет

№	Задания
	Дан одномерный список вещественных чисел. Требуется определить количество таких непрерывных участков длины не менее четырех, в которых каждый последующий элемент превышает среднее всех предыдущих в пределах этого участка
20	Дан одномерный список строк. Необходимо определить количество строк, в которых встречается палиндром длины не менее пяти символов, образованный только буквами одного регистра
	Дан одномерный список целых чисел. Требуется определить, существует ли индекс, при котором сумма элементов, стоящих левее, равна сумме элементов, стоящих правее. Ответ представить в виде булевого значения
21	Дан одномерный список целых чисел. Требуется определить максимальную длину подпоследовательности подряд идущих элементов, каждый из которых равен либо сумме, либо разности двух других различных элементов этого же списка
	Дан одномерный список строк. Необходимо определить количество строк, длина которых совпадает с числом уникальных символов, встречающихся во всех остальных строках вместе взятых
22	Дан одномерный список вещественных чисел. Требуется определить количество троек подряд идущих элементов, у которых стандартное отклонение этих трех чисел не превышает фиксированного значения, заданного литерально
	Дан одномерный список целых чисел. Необходимо определить, можно ли упорядочить его элементы так, чтобы сумма любого непрерывного фрагмента длины три не делилась нацело на 3. Ответ представить в виде булевого значения
23	Дан одномерный список строк. Требуется определить количество строк, для которых найдется другая строка, представляющая собой циклический сдвиг данной. Строка не считается циклическим сдвигом самой себя
	Дан одномерный список целых чисел. Необходимо определить количество пар элементов с различными индексами, наибольший общий делитель которых равен 1
24	Дан одномерный список вещественных чисел. Требуется определить, существует ли такой непрерывный фрагмент длины не менее четырех, что каждый элемент (кроме крайних) равен среднему арифметическому соседних элементов. Ответ представить в виде булевого значения
	Дан одномерный список строк. Необходимо определить количество строк, состоящих из двух и более повторений некоторой своей собственной подстроки
25	Дан одномерный список целых чисел. Требуется определить сумму всех таких элементов, которые равны количеству собственных делителей (то есть делителей, не равных 1 и самому числу)
	Дан одномерный список вещественных чисел. Необходимо определить максимальную длину непрерывной подпоследовательности, сумма которой остается положительной при любой замене одного из ее элементов на ноль
26	Дан одномерный список целых чисел. Требуется определить количество элементов, каждое из которых можно представить в виде суммы квадратов двух различных элементов того же списка
	Дан одномерный список строк. Необходимо определить максимальную длину подпоследовательности подряд идущих строк, в которых каждый последующий символ первой строки содержится во второй строке, каждый символ второй — в третьей и так далее
27	Дан одномерный список целых чисел. Требуется определить количество индексов, для которых сумма значений в соседних позициях (если таковые имеются) строго больше текущего значения

№	Задания
	Дан одномерный список вещественных чисел. Необходимо определить, можно ли его упорядочить в строго возрастающую последовательность, удалив не более двух элементов. Ответ представить в виде булевого значения
28	Дан одномерный список строк. Требуется определить количество таких строк, в которых содержится не менее трех различных символов, встречающихся в строке строго дважды
	Дан одномерный список целых чисел. Необходимо определить количество уникальных значений, каждое из которых входит в список на позиции, номер которой кратен самому значению
29	Дан одномерный список вещественных чисел. Требуется построить новый список, каждый элемент которого равен медиане значений в текущем и двух соседних элементах (если они существуют). Для крайних элементов использовать доступные значения
	Дан одномерный список целых чисел. Необходимо определить, содержит ли он хотя бы одну последовательность трех подряд идущих элементов, образующих арифметическую прогрессию
30	Дан одномерный список строк. Требуется определить количество строк, в которых первые два символа совпадают с последними двумя в обратном порядке (если строка длины меньше четырех, не учитывать)
	Дан одномерный список целых чисел. Необходимо определить максимальную длину такой непрерывной подпоследовательности, в которой количество четных и нечетных чисел совпадает

7.2. Генераторы списков

Генератор списков (list comprehension) в языке Python представляет собой синтаксическую конструкцию, предназначенную для компактного создания списков путем итеративного применения выражения к элементам итерируемого объекта с возможностью фильтрации.

Общий синтаксис генератора списков имеет вид

[выражение **for** переменная **in** итер_объект **if** условие].

Принцип работы: переменная последовательно принимает значения из итерируемого объекта итер_объект. Для тех значений переменная, для которых выполняется условие, вычисляется выражение. Из этих значений и формируется новый список.

Условие (часть **if** условие) может быть опущено. В этом случае генератор будет возвращать значение выражение для всех элементов итерируемого объекта.

Примеры использования генераторов приведены в листинге 7.5.

Листинг 7.5

```
a = [-4, 7, 1, 0, 0, 2, -9, -3, 6, 2, 5, 8]
```

```

# Вычисление квадратов всех чисел списка
aKv = [i**2 for i in a]
print(aKv) # [16, 49, 1, 0, 0, 4, 81, 9, 36, 4, 25, 64]

# Определение количества отрицательных чисел
kOtr = len([i for i in a if i < 0])
print(kOtr) # 3

# Вычисление суммы положительных чисел списка
kPol = sum([i for i in a if i > 0])
print(kPol) # 31

# Определение максимальной длины слова в строке
s = 'Python - универсальный язык программирования'
lenMax = max([len(i) for i in s.split()])
print(lenMax) # 16

# Замена всех десятичных цифр на символ '*'
# В качестве возвращаемого значения используется тернарный
оператор условия
s = '55 лет назад 20 июля 1969 года модуль космического корабля
«Аполлон-11» опустился на поверхность Луны'
s1 = ''.join('*' if i in '0123456789' else i for i in s)
print(s1) # ** лет назад ** июля **** года модуль космического
корабля «Аполлон-**» опустился на поверхность Луны

```

Замечание. Квадратные скобки в синтаксисе генератора списков преобразуют возвращаемые значения в список. Конструкция без квадратных скобок является итератором, поэтому ее можно использовать как аргумент для функции, принимающие итерируемые объекты. К таким функциям относятся `min()`, `max()`, `sum()`, метод `''.join()` и др. В последнем примере листинга 7.5 конструкция генератора без квадратных скобок является аргументом метода `''.join()`.

В генераторах можно использовать несколько подряд идущих инструкций **for**. В листинге 7.6 приведен пример генератора списков, содержащего две циклические конструкции. Для каждого целого значения $x \in [4, 9]$ переменная y принимает значения из отрезка $[4, 7]$. Для пары значений x и y проверяется условие $x > y$. Те пары, для которых оно выполняется, являются аргументами выражения $x + y$, значения которого и формируют итоговый список.

Листинг 7.6

```

t = [x + y for x in range(4, 10) for y in range(4, 8) if x > y]
print(t)
# [9, 10, 11, 11, 12, 13, 12, 13, 14, 15, 13, 14, 15, 16]

```

Пример 7.2

Задано целое число $n > 1$. Требуется сформировать список всех простых чисел, не превышающих n , используя одну конструкцию генератора списков, допускается использование вложенных генераторов и встроенных функций.

Решение

В листинге 7.7 приведен программный код решения.

Листинг 7.7

```
n = 100

primes = [x for x in range(2, n + 1)
           if all(x % d != 0 for d in range(2, int(x**0.5) + 1))]

print(primes)
# [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53,
59, 61, 67, 71, 73, 79, 83, 89, 97]
```

Пояснения:

1. Переменная x принимает значения от минимального простого числа (2) до самого числа n .

2. Из курса арифметики известно, что все нетривиальные делители числа x (кроме 1 и самого x) не превышают значения $\sqrt{x}$. Перебор потенциальных делителей d выполнен в циклической конструкции

```
for d in range(2, int(x**0.5) + 1).
```

Этот генератор возвращает логические значения $x \% d \neq 0$ (проверка, что d не является делителем x).

3. Нам нужны только такие числа x , для которых это логическое выражение имеет значение **True** для всех потенциальных делителей d . Проверка выполняется с использованием функции `all()`, реализующей множественную конъюнкцию — если хотя бы одно логическое значение будет равно **False** (d является делителем), то функция `all()` вернет **False**. В этом случае условие для внешнего генератора будет ложным и текущее число x не войдет в генерируемый список.

Задание 7.6

Выполнить задания в соответствии с табл. 7.9 с использованием инструментария генераторов списков. Допускается вместо вложенных циклических конструкций в генераторе использовать обычную циклическую конструкцию в паре с генераторным выражением. Списки в программном коде задать литерально.

Содержание отчета:

— полная формулировка задания;

- программный код с обязательными пояснениями в виде комментариев;
- результаты работы программы при различных исходных данных.

Таблица 7.9

№	Задания
1	Задан список целых чисел. Сформировать список квадратов тех чисел, которые положительны, четны и при этом сумма их цифр больше 10
	Задан список строк, содержащих только буквы. Сформировать список тех строк, в которых ни один символ не повторяется два раза подряд
2	Задан список положительных целых чисел. Сформировать список таких чисел, которые делятся на 3 и сумма цифр которых является нечетным числом
	Задана строка. Построить список всех фрагментов длины 3, начинающихся с согласной буквы. Обработка производится посимвольно без использования сторонних библиотек
3	Задан список целых чисел. Сформировать список отрицательных чисел, у которых количество цифр в десятичной записи нечетно и первая цифра (по модулю) не равна нулю
	Задан список вещественных чисел. Построить список чисел, у которых дробная часть больше 0.7, а целая часть нечетная
4	Задан список строк. Построить список строк, в которых нет символов, совпадающих с первым символом строки, помимо самого первого вхождения
	Задан список целых чисел. Сформировать список таких чисел, для которых количество цифр в записи совпадает с квадратом некоторого числа
5	Задан список строк. Построить список длин тех строк, в которых первая и последняя буквы совпадают, а длина строки больше 4
	Задан список положительных целых чисел. Сформировать список значений, равных количеству делений на 2, которое требуется, чтобы число стало нечетным
6	Задан список положительных целых чисел. Сформировать список чисел, в двоичной записи которых содержится ровно три единицы
	Задан список строк. Сформировать список тех строк, в которых общее количество гласных букв больше количества согласных
7	Задан список целых чисел. Сформировать список квадратов чисел, у которых разность между максимальной и минимальной цифрами в десятичной записи больше трех
	Задан список строк. Построить список строк, в которых все символы имеют нечетный порядковый номер в таблице Unicode
8	Задан список положительных целых чисел. Сформировать список чисел, сумма квадратов цифр которых является точным квадратом
	Задан список строк. Сформировать список строк, в которых не встречается ни один символ, равный символу с индексом 1 в данной строке (если строка длины менее 2, не учитывать)
9	Задан список целых чисел. Сформировать список тех чисел, десятичная запись которых содержит только нечетные цифры
	Задан список строк. Построить список строк, содержащих не более одного символа, повторяющегося более одного раза
10	Задан список положительных целых чисел. Сформировать список чисел, в десятичной записи которых нет двух одинаковых цифр, расположенных рядом
	Задан список строк. Построить список длин строк, в которых сумма кодов символов делится на длину строки нацело

№	Задания
11	Задан список целых чисел. Сформировать список таких чисел, десятичная запись которых начинается и заканчивается на одну и ту же цифру
	Задан список строк. Сформировать список строк, содержащих хотя бы одну пару соседних символов, код которых отличается ровно на единицу
12	Задан список положительных целых чисел. Сформировать список чисел, количество делений которых на 3 до достижения значения меньше 3 является нечетным
	Задан список строк. Построить список строк, в которых нет двух одинаковых символов, стоящих на позициях с одинаковой четностью
13	Задан список целых чисел. Сформировать список тех чисел, которые содержат только одну цифру, повторяющуюся более одного раза
	Задан список строк. Построить список строк, в которых каждая следующая буква в лексикографическом порядке не меньше предыдущей
14	Задан список положительных целых чисел. Сформировать список чисел, у которых сумма цифр совпадает с произведением цифр
	Задан список строк. Построить список строк, в которых первые три символа различны, при этом строка состоит не менее чем из четырех символов
15	Задан список целых чисел. Сформировать список чисел, в которых в десятичной записи присутствуют ровно две четные и одна нечетная цифра
	Задан список строк. Построить список строк, длина которых равна количеству букв, код которых кратен 5
16	Задан список положительных целых чисел. Сформировать список чисел, десятичная запись которых не содержит двух последовательных цифр, различающихся более чем на единицу
	Задан список строк. Построить список строк, в которых нет символов с кодами, делящимися на 8
17	Задан список целых чисел. Сформировать список чисел, у которых количество нечетных цифр больше количества четных
	Задан список строк. Построить список строк, в которых нет ни одного символа, совпадающего с символом, находящимся в середине строки (при нечетной длине)
18	Задан список положительных целых чисел. Сформировать список чисел, в которых число цифр, больших 5, не менее двух
	Задан список строк. Построить список строк, в которых ни одна пара соседних символов не равна «ab», «cd» или «pq»
19	Задан список целых чисел. Сформировать список чисел, которые равны сумме квадратов двух других различных чисел из исходного списка
	Задан список строк. Построить список строк, в которых не менее трех символов повторяются ровно по два раза
20	Задан список положительных целых чисел. Сформировать список чисел, в которых сумма цифр четна, а произведение цифр нечетно
	Задан список строк. Построить список строк, в которых ни один символ не совпадает с символом, стоящим на той же позиции в предыдущей строке (если таковая имеется)
21	Задан список целых чисел. Сформировать список чисел, у которых абсолютная разность между первой и последней цифрами в десятичной записи не превышает двух
	Задан список строк. Сформировать список строк, в которых символ с индексом 0 встречается в строке только один раз
22	Задан список положительных целых чисел. Сформировать список чисел, в десятичной записи которых количество четных цифр в точности равно количеству нечетных

№	Задания
	Задан список строк. Сформировать список строк, в которых второй и предпоследний символы совпадают
23	Задан список целых чисел. Сформировать список чисел, сумма цифр которых кратна разности максимальной и минимальной цифры в числе
	Задан список строк. Сформировать список строк, в которых не более одной пары соседних символов совпадают
24	Задан список положительных целых чисел. Сформировать список чисел, в которых все цифры, начиная со второй, не превышают предыдущую
	Задан список строк. Сформировать список строк, содержащих не менее двух пар соседних символов, код которых различается на единицу
25	Задан список целых чисел. Сформировать список чисел, десятичная запись которых содержит ровно три различных цифры, причем старшая цифра повторяется хотя бы дважды
	Задан список строк. Сформировать список строк, в которых количество гласных строго превышает количество согласных
26	Задан список положительных целых чисел. Сформировать список чисел, десятичная запись которых состоит только из цифр 1, 2, 4, 8
	Задан список строк. Сформировать список строк, длина которых совпадает с суммой кодов двух любых различных символов, деленной на 100 с округлением вниз
27	Задан список целых чисел. Сформировать список чисел, сумма цифр которых равна количеству цифр, возведенному в квадрат
	Задан список строк. Сформировать список строк, в которых ровно один символ встречается более одного раза, а остальные — по одному разу
28	Задан список положительных целых чисел. Сформировать список чисел, в которых в десятичной записи не встречается ни одной последовательности из трех подряд идущих одинаковых цифр
	Задан список строк. Сформировать список строк, длина которых делится нацело на количество символов, имеющих четный номер в строке
29	Задан список целых чисел. Сформировать список чисел, в которых все цифры либо четные, либо нечетные
	Задан список строк. Сформировать список строк, в которых каждый символ встречается либо один, либо три раза
30	Задан список положительных целых чисел. Сформировать список чисел, в которых разность между максимальной и минимальной цифрами четна и делится на 4
	Задан список строк. Сформировать список строк, содержащих только символы с номерами, которые при делении на 7 дают остаток 3 или 5

7.3. Двумерные списки

Двумерный список представляет собой структуру данных, реализуемую как список, элементы которого сами являются списками. Пример литерального задания двумерного списка:

$a = [[1, 4, 5], [-2, 7, 3], [3, 1, 4], [5, -7, 2]]$.

Обращение к внутренним спискам осуществляется по индексу. Для приведенного списка a обращение $a[1]$ соответствует списку $[-2, 7, 3]$. Для

обращения к конкретному элементу используется двойная индексация: в рассмотренном примере `a[2][1]` соответствует значению 1.

Вывод на экран значений двумерного списка можно осуществлять по-разному (листинг 7.8). Вывод в виде двумерного списка сложно воспринимать визуально. Немного исправляет ситуацию построчный вывод. Наилучшим с визуальной точки зрения является поэлементный форматированный вывод с применением F-строк.

Листинг 7.8

```
a = [[1, 4, 5], [-2, 7, 3], [3, 1, 4], [5, -7, 2]]

# Вывод в виде двумерного списка
print(a)
# [[1, 4, 5], [-2, 7, 3], [3, 1, 4], [5, -7, 2]]
print()

# Построчный вывод списка
print(*a, sep='\n')
# [1, 4, 5]
# [-2, 7, 3]
# [3, 1, 4]
# [5, -7, 2]
print()

# Вывод с форматированием каждого элемента
for i in a:
    for j in i:
        print(f'{j:4d}', end='')
    print()
# 1    4    5
# -2    7    3
# 3    1    4
# 5   -7    2
```

Инициализировать двумерный список можно посредством генераторов списков. При попытке создать двумерный список посредством умножения списка на число получаем не совсем ожидаемое поведение (листинг 7.9).

Листинг 7.9

```
# Так создавать двумерные списки НЕЛЬЗЯ!
n, m = 4, 5
L = [[0] * m] * n
print(L)
# [[0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]

L[2][3] = 5
print(L)
# [[0, 0, 0, 5, 0], [0, 0, 0, 5, 0], [0, 0, 0, 5, 0], [0, 0, 0, 5, 0]]
```

Т. е. при изменении какого-либо элемента изменения происходят и во всех строках. Происходит это потому, что при умножении на n создаются ссылки на один и тот же объект $[0] * m$. Корректный способ задания двумерного списка приведен в листинге 7.10.

Листинг 7.10

```
n, m = 4, 5
L = [[0 for j in range(m)] for i in range(n)]
print(L)
# [[0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0,
0, 0]]

L[2][3] = 5
print(L)
# [[0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 5, 0], [0, 0, 0,
0, 0]]
```

Для генерации значений двумерных списков в учебных целях можно использовать модуль **random**. Функция **randint**(a , b) этого модуля генерирует случайное целое число из отрезка $[a, b]$. Функция **seed**(a) обеспечивает повторяемость результатов генерации случайных чисел, аргумент a — целое число. Пример генерации двумерного списка со случайными значениями приведен в листинге 7.11. Процедура вывода элементов списка на экран оформлена как подпрограмма.

Листинг 7.11

```
from random import *

# Подпрограмма вывода двумерного списка
def out(a):
    for i in a:
        for j in i:
            print(f'{j:4d}', end='')
        print()

# Генерируем двумерный список размером 4 на 5
n, m = 4, 5
seed(1)
L = [[randint(1,100) for j in range(m)] for i in range(n)]

# Вывод сгенерированного двумерного списка
out(L)
# 18  73  98   9  33
# 16  64  98  58  61
# 84  49  27  13  63
#  4  50  56  78  98
```

При работе со списками следует обратить внимание на процесс создания копий списка.

Поверхностное копирование (*shallow copy*) представляет собой создание нового списка, содержащего ссылки на те же самые объекты, что и оригинальный список. При этом сам внешний список создается заново, однако вложенные объекты (например, другие списки, словари, множества и любые изменяемые структуры) не копируются, а лишь переносятся по ссылке. Таким образом, изменение вложенных объектов через новый список приведет к изменению данных и в оригинальном списке, поскольку оба списка ссылаются на одни и те же вложенные объекты. Пример создания поверхностной копии приведен в листинге 7.12.

Листинг 7.12

```
a = [[1, 2], [3, 4]]
b = a[:] # поверхностная копия

b[0][0] = 99
print(a) # [[99, 2], [3, 4]]
```

Глубокое копирование (*deep copy*) предполагает не только создание нового внешнего списка, но и рекурсивное копирование всех вложенных объектов. В результате создается полная независимая копия оригинальной структуры данных, изменение элементов одного списка никак не влияет на другой. Глубокое копирование выполняется при помощи функции **copy.deepcopy()** из стандартного модуля **copy**. Пример использования приведен в листинге 7.13.

Листинг 7.13

```
from copy import deepcopy

a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)

b[0][0] = 99
print(a) # [[1, 2], [3, 4]]
```

В данном случае список **b** содержит полностью независимую структуру, включая все вложенные объекты.

Сравнение списков осуществляется при помощи операторов сравнения, применяемых непосредственно к объектам спискового типа.

Равенство и неравенство (**==**, **!=**) проверяют попарное равенство всех элементов на тех же позициях и одинаковую длину:

Сравнение **A == B** истинно тогда и только тогда, когда списки имеют одинаковую длину и все элементы равны: **A[i] == B[i]** для всех допустимых индексов **i**

Операторы порядка (<, >, <=, >=) реализуют лексикографическое сравнение:

- элементы сравниваются последовательно по индексам, начиная с 0.
- первые различающиеся элементы a и b сравниваются: результат выражения определяется сравнением $a < b$, $a > b$ и т. д.;
- если все сравниваемые элементы равны, но длины различны, то более короткий список считается меньшим (листинг 7.14).

Листинг 7.14

```
[1, 2, 3] == [1, 2, 3] # True
[1, 2, 3] != [1, 2, 4] # True
[1, 2] < [1, 2, 0] # True (меньшая длина)
[1, 3] > [1, 2, 99] # True (сравнение 3 > 2)
[2, 1] >= [2, 1] # True
```

Для проверки идентичности объектов (т. е. находятся ли два списка по одному и тому же адресу памяти) следует использовать оператор **is**, но не для проверки содержимого.

Задание 7.7 (практико-ориентированное задание)

Выполнить задания в соответствии с табл. 7.10. Списки в программном коде задать случайными значениями.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результаты работы программы при различных исходных данных.

Таблица 7.10

№	Задание
1	В системе учета еженедельных показателей температуры воздуха в нескольких теплицах хранится вложенный список T, состоящий из восьми подсписков, каждый из которых содержит по пять вещественных чисел с точностью до двух знаков после запятой, представляющих среднесуточные температуры (в градусах Цельсия) за пять последовательных дней. 1. Сгенерировать исходный список T с восемью подсписками по пять случайных значений в диапазоне от 15.00 до 35.00. 2. Выполнить поверхностное копирование списка T в переменную T_{shallow} и глубокое копирование в переменную T_{deep}. 3. В исходном списке T выполнить следующие изменения: — округлить каждое значение до целых чисел; — если среднее значение в подсписке превышает 30, заменить весь подсписок списком из пяти значений 15;

№	Задание
	— если минимальное значение в подписке ниже 18, заменить подписок списком из пяти значений 35. - Вывести содержимое списков T, T_{shallow} и T_{deep}. - Провести поэлементное сравнение соответствующих подписков между T и T_{shallow}, а также между T и T_{deep}, оформить результаты в комментариях. - Сформулировать выводы о влиянии изменений в исходном списке на копии при использовании поверхностного и глубокого копирования в контексте вложенных списков
2	В системе контроля качества выпускаемой продукции ведется учет параметров характеристик изделий в виде вложенного списка P, содержащего шесть подписков, каждый из которых включает восемь вещественных значений с точностью до двух знаков после запятой. Эти значения отражают измеренные параметры изделий на разных этапах технологического процесса. - Сгенерировать исходный список P с шестью подписками по восемь случайных значений в диапазоне от 0.00 до 10.00. - Выполнить поверхностное копирование списка P в переменную P_{shallow} и глубокое копирование в переменную P_{deep}. - В исходном списке P выполнить следующие изменения: - округлить все значения до одного знака после запятой; - если максимальное значение в подписке превышает 9.0, заменить весь подписок списком из восьми нулей; - если минимальное значение в подписке ниже 1.0, заменить подписок списком из восьми единиц. - Вывести содержимое списков P, P_{shallow} и P_{deep}. - Провести поэлементное сравнение соответствующих подписков между P и P_{shallow}, а также между P и P_{deep}, оформить результаты в виде комментариев. - Сделать выводы о поведении поверхностного и глубокого копирования на основании выявленных изменений в списках с вложенными структурами
3	В рамках моделирования работы склада необходимо разработать модуль хранения информации о наличии товаров в различных зонах. Информация представлена в виде вложенного списка Z, где каждая строка соответствует одной зоне хранения, а каждый элемент вложенного списка отражает количество единиц соответствующего типа товара. Все значения являются целыми неотрицательными числами. Необходимо выполнить следующие действия: - Инициализировать список Z размером $r \times c$, в котором r — количество зон, c — количество типов товаров, с использованием генератора случайных целых чисел из диапазона $[0, 50]$. - Создать поверхностную копию Z в список P и глубокую копию в список D. - Изменить значения всех товаров в одной произвольно выбранной зоне хранения в исходном списке Z на -1 (например, в случае полной утилизации или списания). - Проанализировать, какие изменения произошли в копиях P и D после модификации исходного списка. - На основе анализа сформировать логический список длины r, где каждый элемент принимает значение True, если соответствующая зона в копии P

№	Задание
	изменилась вслед за Z, и False в противном случае. Провести аналогичный анализ для копии D. 6. Сформировать словесное обоснование, иллюстрирующее различие в поведении списков P и D, с указанием причин, связанных с типом копирования
4	В системе контроля версий необходимо реализовать прототип механизма отката изменений состояния данных. Состояние представлено в виде вложенного списка S, каждая строка которого содержит текущие параметры одной из задач. Все значения являются вещественными числами, полученными из датчика. 1. Инициализировать вложенный список S размером $m \times n$, заполнив его псевдослучайными вещественными числами из диапазона $[0.0, 5.0]$, округленными до одного знака после запятой. 2. Сформировать два бэкап-снимка состояния: S_{shallow} — с использованием поверхностного копирования, S_{deep} — с использованием глубокого копирования. 3. Выполнить следующее модифицирующее преобразование над исходным состоянием S: для каждой строки определить среднее значение параметров и заменить все значения в этой строке на вычисленное среднее. 4. Провести анализ: сравнить состояния S, S_{shallow} и S_{deep}, определив, какие изменения затронули каждую из копий. Зафиксировать факт совпадения или несовпадения содержимого поэлементно. 5. Реализовать программную проверку, возвращающую список булевых значений длины m, в котором каждый элемент указывает, совпадает ли соответствующая строка в S_{shallow} с исходной S после модификации. Повторить для S_{deep}. 6. На основе полученных данных сделать вывод о применимости различных видов копирования для хранения промежуточных и контрольных снимков в системах, чувствительных к точности восстановления состояний. Указать, в каких случаях поверхностное копирование недопустимо, даже при кажущемся совпадении структуры
5	В системе анализа параметров технических устройств необходимо реализовать механизм временного прототипирования конфигураций, представленных в виде двумерного списка C, каждая строка которого содержит текущие числовые параметры одного устройства. Все значения — целые числа в диапазоне от -50 до 150. Размерность списка составляет 6×5. 1. Инициализировать двумерный список C псевдослучайными значениями из указанного диапазона. 2. Создать две копии: C_{shallow} — с использованием поверхностного копирования, C_{deep} — с использованием глубокого копирования. 3. Смоделировать изменение конфигурации в исходном списке C по следующим правилам: — каждое значение, равное 0, заменить на сумму соседних по строке элементов; — если в строке содержится хотя бы одно значение, превышающее 100, инвертировать знаки всех элементов этой строки; — если строка содержит более трех положительных значений, заменить последний элемент на произведение двух наибольших элементов строки.

№	Задание
	4. Вывести содержимое C, C_{shallow} и C_{deep} после всех преобразований. Для каждой строки зафиксировать, были ли внесены изменения относительно соответствующих строк в копиях. 5. Провести формальную проверку расхождений между C и C_{shallow}, а также между C и C_{deep}, представив результат в виде двух булевых матриц размером 6×5. 6. На основе полученных данных аргументированно объяснить, какие типы изменений затронули только исходный список C, какие повлияли на C_{shallow}, и почему структура C_{deep} осталась неизменной
6	В рамках разработки подсистемы восстановления параметров сложных инженерных объектов необходимо реализовать механизм, позволяющий фиксировать состояние набора модулей до и после изменения, при этом гарантируя возможность возврата к исходным данным. Конфигурация модулей представлена в виде списка M длины 8, каждый элемент которого является вложенным списком из четырех вещественных чисел, представляющих параметры одного модуля. 1. Сформировать список M из 8 вложенных списков, каждый из которых содержит 4 вещественных числа, равномерно распределенных в диапазоне от -10.0 до 10.0. 2. Выполнить поверхностное копирование M в список M_{shallow} и глубокое копирование в список M_{deep}. 3. Реализовать функцию корректировки конфигурации по следующим правилам: — если сумма параметров в подсписке превышает 10, заменить минимальный элемент на среднее значение всех остальных элементов этого подсписка; — если в подсписке содержатся одновременно положительные и отрицательные значения, заменить каждый элемент на его квадрат; — если все значения положительные, заменить каждый элемент на его логарифм по основанию 10, округленный до двух знаков после запятой. 4. Применить функцию корректировки к списку M. 5. Построить логическую матрицу изменений, отображающую, были ли внесены изменения в каждый из подсписков списка M относительно M_{shallow} и M_{deep}. 6. На основе анализа полученных структур описать, почему изменения, произведенные в M, повлияли на M_{shallow} и не затронули M_{deep}, с точки зрения особенностей реализации копирования вложенных объектов в Python
7	В процессе разработки алгоритма машинного обучения с ручной настройкой гиперпараметров требуется обеспечить возможность поэтапного сохранения и отката состояний конфигурации. Конфигурация параметров представлена в виде списка C, содержащего 6 вложенных списков, каждый из которых состоит из 5 целых чисел, задающих параметры одного этапа обучения. 1. Сформировать список C, где каждый из 6 вложенных списков содержит 5 случайных целых чисел из диапазона от 1 до 100. 2. Создать два набора резервных копий: — C_{shallow} — результат поверхностного копирования C; — C_{deep} — результат глубокого копирования C.

№	Задание
	3. Определить функцию модификации гиперпараметров, реализующую следующие действия: — если сумма элементов вложенного списка делится на 5, заменить все элементы на нули; — если в списке содержатся более трех четных чисел, заменить их на случайные значения от 200 до 300. 4. Применить функцию модификации к исходному списку C. 5. Проанализировать содержимое списков C_{shallow} и C_{deep} и определить, сохранили ли они исходные состояния. 6. Обосновать наблюдаемое поведение с точки зрения механизма копирования списков в Python и структуры размещения объектов в памяти при использовании поверхностного и глубокого копирования
8	При реализации подсистемы агрегации показателей по группам пользователей данные представлены в виде двумерного списка D, содержащего 10 вложенных списков, каждый из которых соответствует отдельной группе и содержит 7 элементов, представляющих собой числовые показатели за каждый день недели. 1. Сгенерировать исходный список D путем заполнения каждого из 10 вложенных списков случайными вещественными числами из диапазона от 10.0 до 100.0. 2. Выполнить поверхностное копирование D, присвоив результат переменной D_{shallow}. Выполнить глубокое копирование D, присвоив результат переменной D_{deep}. 3. Разработать функцию корректировки данных, реализующую следующие действия: — если среднее значение вложенного списка превышает 80.0, уменьшить все элементы на 10%; — если минимальное значение вложенного списка меньше 20.0, заменить его на 50.0. 4. Применить функцию корректировки к списку D. 5. Сравнить содержимое D_{shallow} и D_{deep} с модифицированным списком D и указать, какие изменения затронули резервные копии. 6. Провести объяснение полученного результата с опорой на поведение списков при поверхностном и глубоком копировании в языке Python и механизм работы с ссылочными типами данных
9	В процессе моделирования адаптивного расписания логистических операций используется исходная матрица R, представляющая собой двумерный список, в котором каждая строка описывает набор временных интервалов (в минутах) выполнения операций определенного типа на различных складах. Необходимо реализовать корректировку этой матрицы на основе изменений параметров логистической цепочки, сохранив при этом возможность анализа как исходных, так и промежуточных состояний данных. 1. Сгенерировать исходную матрицу R размером 6×5, заполнив ее случайными целыми числами из диапазона от 30 до 120. С помощью поверхностного копирования создать копию R_{shallow}. 2. С помощью глубокого копирования создать копию R_{deep}. 3. Разработать функцию корректировки расписания, реализующую следующие действия:

№	Задание
	— для каждой строки матрицы определить среднее значение и заменить в строке все значения, отклоняющиеся от среднего более чем на 25%, на удвоенное среднее значение; — если сумма значений в строке превышает 500, обнулить значения в последних двух столбцах данной строки; — если разница между максимальным и минимальным значением строки превышает 60, то заменить каждый элемент строки на среднее арифметическое данной строки, округленное до целого числа. 4. Применить функцию корректировки к исходной матрице R. 5. Сравнить содержимое списков R_{shallow} и R_{deep} с откорректированной матрицей R. Проанализировать, какие изменения затронули резервные копии. 6. Обосновать наблюдаемые различия, указав, каким образом поведение Python при копировании влияет на работу с вложенными структурами, и почему поверхностное копирование не обеспечивает полной независимости копий при изменении вложенных элементов
10	В процессе анализа структурной целостности табличных данных, поступающих от различных внешних поставщиков, используется набор таблиц, представленных в виде двумерного списка D, каждая строка которого соответствует отдельной записи, а каждый элемент строки — конкретному числовому параметру. В условиях изменяемых требований к валидации данных необходимо реализовать систему модификации структуры без потери исходного состояния. 1. Создать исходную матрицу D размером 7×6, где значения являются целыми числами, равномерно распределенными в диапазоне от -100 до 100. 2. Создать два представления исходной матрицы: D_{shallow} (с использованием поверхностного копирования) и D_{deep} (с использованием глубокого копирования). 3. Реализовать функцию очистки данных, выполняющую следующие действия: — в каждой строке удалить минимальный и максимальный элементы; — если в строке после удаления осталось менее четырех элементов, заменить ее на список из нулей той же длины; — если сумма значений в строке отрицательная, инвертировать знак всех ее элементов. 4. Применить функцию очистки к исходной матрице D. 5. Вывести содержимое всех трех списков: D, D_{shallow}, D_{deep}. 6. Проанализировать, какие изменения затронули каждую из копий. Объяснить полученные результаты с позиций поведения Python при копировании списков, учитывая вложенность структуры
11	В рамках разработки подсистемы резервного хранения данных необходимо реализовать процедуру создания изолированных копий исходных структур для их последующего восстановления после выполнения критических операций. Данные представлены в виде двумерного списка R, каждая строка которого содержит параметры, измеренные с различным интервалом времени и требующие нормализации и корректировки. 1. Сформировать двумерный список R из 10 строк по 5 элементов в каждой. Элементы — случайные вещественные числа от -10.0 до 10.0.

№	Задание
	- Создать две копии исходной структуры: R_{copy1} с использованием поверхностного копирования и R_{copy2} с использованием глубокого копирования. - Реализовать функцию трансформации данных, выполняющую следующие операции: - если максимальное значение строки превышает 8.0, то все элементы строки округляются до целых чисел; - если минимальное значение строки отрицательное, заменяется только этот минимальный элемент на его квадрат; - если сумма всех элементов строки после изменений превышает 20.0, к каждому элементу строки прибавляется 1.5. - Применить функцию трансформации к списку R. - После выполнения изменений вывести содержимое списков R, R_{copy1} и R_{copy2}. - Обосновать, какие именно копии изменились, а какие остались неизменными, с точки зрения используемого метода копирования и внутреннего представления данных в языке Python. Указать, в каком случае изменение одного вложенного списка повлияет на все представления структуры
12	В рамках тестирования модулей численного моделирования необходимо подготовить экспериментальное задание по анализу копирования многомерных структур, имитирующих матрицы данных. Имеется вложенный список A, каждая строка которого содержит параметры, рассчитанные по различным моделям. - Сформировать двумерный список A, содержащий 6 строк по 4 целых числа, каждое из которых случайным образом выбирается из диапазона от -50 до 50. - Получить две копии списка: B — с использованием поверхностного копирования, C — с использованием глубокого копирования. - Изменить значения в списке A следующим образом: - в каждой строке найти сумму четных чисел и заменить ею все элементы строки, если таких чисел не менее двух; - если в строке присутствуют хотя бы три различных отрицательных значения, заменить минимальный элемент строки на ноль; - если сумма элементов строки превышает 100, отбросить последний элемент строки. - После выполнения указанных преобразований вывести содержимое списков A, B и C. - Проанализировать, какие изменения затронули список B, какие — список C, и сделать обоснованный вывод о характере взаимосвязи между исходным и скопированными списками в контексте особенностей хранения вложенных объектов в Python. - Дополнительно провести проверку с помощью операций идентичности и сравнения (<code>is</code>, <code>==</code>) между соответствующими строками списков A, B и C, прокомментировав полученные результаты с точки зрения поверхностного и глубокого копирования
13	В процессе разработки системы резервного хранения состояния многомерных параметров требуется обеспечить корректное копирование и модификацию вложенных структур без нарушения целостности оригинальных данных. Имеется

№	Задание
	исходный двумерный список M, содержащий 5 строк по 5 целых чисел, сгенерированных случайным образом в диапазоне от 10 до 99. 1. Сформировать список M, представляющий собой таблицу показателей, полученных в ходе пяти независимых экспериментов, каждый из которых фиксирует пять различных метрик. 2. Выполнить два типа копирования: создать список P при помощи поверхностного копирования и список Q при помощи глубокого копирования. 3. Внести следующие изменения только в список M: — обнулить все диагональные элементы (где индексы строки и столбца совпадают); — для каждой строки определить медианное значение и заменить им первый элемент строки; — в строках, содержащих хотя бы одно значение больше 90, заменить последнее значение строки на сумму всех элементов строки. 4. Вывести содержимое списков M, P и Q после всех преобразований. 5. Проанализировать, какие изменения отразились на списке P, какие — на списке Q, и пояснить наблюдаемое поведение в терминах различий между поверхностным и глубоким копированием вложенных объектов. 6. Выполнить проверку идентичности объектов при помощи операций <code>is</code> и <code>==</code>, зафиксировать различия между элементами M, P и Q и интерпретировать полученные результаты с точки зрения ссылочной структуры вложенных списков
14	В системе учета показателей производительности отдельных компонентов распределенной вычислительной среды используется структура данных, представляющая собой вложенный список, где каждая вложенная подструктура содержит информацию о загрузке процессоров на различных временных интервалах. Имеется исходный вложенный список L, содержащий 4 подсписка, каждый из которых содержит по 6 вещественных чисел, представляющих уровень загрузки соответствующего процессора в течение шести последовательных минут. 1. Сгенерировать исходный список L со случайными вещественными значениями от 0.0 до 1.0 с точностью до двух знаков после запятой. 2. Выполнить поверхностное копирование списка L в переменную A и глубокое копирование в переменную B. 3. Произвести следующие изменения только в списке L: — во всех подсписках округлить значения загрузки до ближайшего кратного 0.25; — если максимальное значение в подсписке превышает 0.75, заменить весь подсписк на список из шести нулей; — если все значения подсписка меньше 0.25, заменить его на список из шести единиц. 4. Вывести содержимое всех трех списков: L, A, B. 5. Выполнить поэлементное сравнение подсписков в L и A, а также в L и B при помощи операторов <code>is</code> и <code>==</code>, и оформить результаты в виде комментариев. 6. Сформулировать выводы о том, как и почему изменения в L отразились или не отразились на списках A и B, с опорой на семантику ссылок, используемую в механизмах поверхностного и глубокого копирования
15	В системе мониторинга качества сетевых соединений используется структура данных в виде вложенного списка, где каждый подсписк отражает показатели

№	Задание
	задержки пакетов (в миллисекундах) для различных каналов связи, измеренные в течение восьми последовательных временных интервалов. Исходный список M содержит 5 подсписков, каждый из которых состоит из 8 целочисленных значений задержки в диапазоне от 10 до 100. 1. Сгенерировать исходный список M со случайными целочисленными значениями в указанном диапазоне. 2. Выполнить поверхностное копирование списка M в переменную X и глубокое копирование в переменную Y. 3. Внести изменения только в исходный список M: — увеличить на 10 % каждое значение задержки, округленное до ближайшего целого; — если среднее значение подсписка превышает 80, заменить подсписк на список из восьми нулей; — если минимальное значение подсписка меньше 20, заменить подсписк на список из восьми сотен. 4. Вывести содержимое списков M, X и Y. 5. Произвести поэлементное сравнение подсписков списков M и X, а также списков M и Y с использованием операторов <code>is</code> и <code>==</code>; оформить результаты в виде комментариев. 6. Сформулировать выводы о влиянии изменений в списке M на списки X и Y с учетом особенностей поверхностного и глубокого копирования в Python
16	В системе анализа потребления электроэнергии многоквартирного дома используется вложенный список, где каждый подсписк содержит суточные значения потребления электроэнергии (в киловатт-часах) для одного из десяти подъездов за период из семи дней. Исходный список E состоит из 10 подсписков по 7 вещественных чисел с точностью до двух знаков после запятой, значения которых варьируются от 0.00 до 50.00. 1. Сгенерировать исходный список E со случайными значениями потребления в указанном диапазоне. 2. Выполнить поверхностное копирование списка E в переменную P и глубокое копирование в переменную Q. 3. Внести изменения только в списке E: — округлить все значения до ближайшего целого числа; — если сумма потребления за неделю в подъезде превышает 200, заменить соответствующий подсписк на список из семи значений 100; — если среднее потребление в подъезде ниже 10, заменить подсписк на список из семи нулей. 4. Вывести содержимое списков E, P и Q. 5. Выполнить поэлементное сравнение подсписков списков E и P, а также E и Q с использованием операторов <code>is</code> и <code>==</code> и оформить результаты в виде комментариев. 6. Сформулировать выводы о различиях в поведении поверхностного и глубокого копирования при модификации вложенных списков в языке Python
17	В системе мониторинга температуры в серверной комнате используется вложенный список, где каждый подсписк содержит значения температуры (в градусах Цельсия) для одного из пяти серверных стоек за 24 часа, измеряемые ежечасно. Исходный список T состоит из 5 подсписков по 24 вещественных числа

№	Задание
	с точностью до двух знаков после запятой, принимающих значения в диапазоне от 15.00 до 35.00. 1. Сгенерировать исходный список T со случайными значениями температуры в указанном диапазоне. 2. Выполнить поверхностное копирование списка T в переменную S и глубокое копирование в переменную D. 3. Изменить только список T следующим образом: — округлить все значения температуры до ближайшего целого числа; — если максимальное значение температуры в подсписке превышает 30, заменить весь подсписок на список из 24 значений 30; — если минимальное значение температуры в подсписке ниже 18, заменить подсписок на список из 24 значений 18. 4. Вывести содержимое списков T, S и D. 5. Провести поэлементное сравнение подсписков в списках T и S, а также T и D, используя операторы <code>is</code> и <code>==</code>; результаты оформить в комментариях к коду. 6. Сделать выводы относительно особенностей поверхностного и глубокого копирования в Python при работе с вложенными списками на основе сделанных изменений
18	В системе анализа финансовых показателей компании используется вложенный список, где каждый из четырех подсписков содержит значения ежемесячной прибыли за один из четырех кварталов года. Каждый подсписок состоит из трех вещественных чисел с точностью до двух знаков после запятой, представляющих прибыль в миллионах условных единиц. 1. Сгенерировать исходный список P со случайными значениями прибыли в диапазоне от 0.00 до 10.00 млн. 2. Создать поверхностную копию списка P в переменную $P_{Shallow}$ и глубокую копию в переменную P_{Deep}. 3. Изменить список P следующим образом: — округлить все значения прибыли до целых чисел; — если сумма значений в подсписке меньше 10, заменить его на список из трех нулей; — если сумма значений превышает 20, заменить подсписок на список из трех значений 20. 4. Вывести содержимое списков P, $P_{Shallow}$ и P_{Deep}. 5. Выполнить поэлементное сравнение подсписков в списках P и $P_{Shallow}$, а также в P и P_{Deep}, используя операторы <code>is</code> и <code>==</code>; результаты оформить комментариями. 6. На основе сделанных операций сформулировать выводы о влиянии поверхностного и глубокого копирования на изменение вложенных структур данных в языке Python
19	В системе мониторинга показателей качества воздуха в разных районах города используется вложенный список, где каждый из пяти подсписков содержит данные о концентрации вредных веществ, измеренные в течение семи дней подряд. Каждый элемент подсписка — вещественное число с точностью до трех знаков после запятой, отражающее среднесуточное значение концентрации загрязнителей. 1. Сгенерировать исходный список Q с пятью подсписками по семь случайных значений в диапазоне от 0.000 до 0.100.

№	Задание
	2. Выполнить поверхностное копирование списка Q в переменную $CopyShallow$ и глубокое копирование в переменную $CopyDeep$. 3. В исходном списке Q выполнить следующие изменения: — округлить каждое значение до четырех знаков после запятой; — если среднее значение в подсписке превышает 0.050, заменить весь подсписок на список из семи нулей; — если максимальное значение в подсписке ниже 0.010, заменить подсписок на список из семи единиц. 4. Вывести содержимое списков Q, $CopyShallow$ и $CopyDeep$. 5. Провести поэлементное сравнение соответствующих подсписков между Q и $CopyShallow$, а также между Q и $CopyDeep$, с использованием операторов is и $==$; оформить результаты в комментариях. 6. Сформулировать заключение о разнице поведения поверхностного и глубокого копирования на основе наблюдаемых изменений в списках
20	В системе учета энергетических характеристик автономных сенсорных устройств используется вложенная структура данных, отражающая уровень потребления энергии различными модулями устройств в течение дня. Исходный список E содержит семь подсписков, соответствующих различным устройствам, каждый из которых включает по пять вещественных значений, представляющих усредненное потребление энергии (в условных единицах) в течение пяти последовательных временных интервалов. 1. Сгенерировать исходный список E, содержащий 7 подсписков по 5 случайных вещественных значений в диапазоне от 0.00 до 5.00 с точностью до двух знаков после запятой. 2. Выполнить поверхностное копирование списка E в переменную $E_{shallow}$ и глубокое копирование в переменную E_{deep}. 3. Модифицировать только список E следующим образом: — округлить все значения до одного знака после запятой; — если сумма значений подсписка превышает 20.0, заменить подсписок списком из пяти нулей; — если все значения в подсписке не превышают 0.5, заменить его списком из пяти значений 9.99. 4. Вывести на экран содержимое списков E, $E_{shallow}$ и E_{deep}. 5. Выполнить сравнение поэлементных вложенных подсписков между E и $E_{shallow}$, а также между E и E_{deep} с использованием операторов is и $==$. Добавить комментарии с интерпретацией результатов. 6. Обосновать, как механизм ссылочной семантики и особенности реализации поверхностного и глубокого копирования в языке Python приводят к наблюдаемому поведению при модификации вложенных списков
21	В системе управления запасами на складе используется вложенный список, где каждый из шести подсписков содержит по восемь целочисленных значений, отражающих ежедневное количество поступивших товаров по соответствующей товарной категории за восемь последовательных дней. 1. Сгенерировать исходный список S с шестью подсписками, каждый из которых содержит восемь случайных целых чисел в диапазоне от 0 до 1000. 2. Выполнить поверхностное копирование списка S в переменную $ShallowCopy$ и глубокое копирование в переменную $DeepCopy$. 3. В списке S выполнить следующие преобразования:

№	Задание
	— заменить значения, превышающие 800, на 800; — если сумма элементов подписка меньше 2000, заменить подписание на список из восьми нулей; — если сумма элементов подписка превышает 6000, заменить подписание на список из восьми единиц. 4. Вывести содержимое списков S, <i>ShallowCopy</i> и <i>DeepCopy</i>. 5. Провести поэлементное сравнение подписков между S и <i>ShallowCopy</i>, а также между S и <i>DeepCopy</i> с использованием операторов <code>is</code> и <code>==</code>; оформить результаты в комментариях. 6. Сформулировать выводы о том, каким образом изменения в S повлияли на копии, с учетом семантики ссылок и механизмов поверхностного и глубокого копирования
22	В системе мониторинга показателей энергопотребления здания используется вложенный список, состоящий из пяти подписков, каждый из которых содержит десять целочисленных значений, отражающих потребление электроэнергии (в ватт-часах) в течение десяти последовательных часов. 1. Сформировать исходный список E, состоящий из пяти подписков по десять случайных целых чисел в диапазоне от 50 до 500. 2. Создать поверхностную копию списка E в переменную $E_{shallow}$ и глубокую копию в переменную E_{deep}. 3. В списке E выполнить следующие преобразования: — для каждого подписка вычислить среднее значение; — если среднее значение превышает 300, заменить все элементы подписка на значение 0; — если среднее значение меньше 150, увеличить каждый элемент подписка на 100. 4. Вывести содержимое списков E, $E_{shallow}$ и E_{deep}. 5. Выполнить поэлементное сравнение подписков списков E и $E_{shallow}$, а также E и E_{deep}, используя операторы <code>is</code> и <code>==</code>, оформить результаты в виде комментариев. 6. Сформулировать выводы о влиянии изменений в списке E на копии с учетом особенностей поверхностного и глубокого копирования в языке Python
23	В системе анализа данных о температурных показателях нескольких климатических зон используется вложенный список, содержащий шесть подписков, каждый из которых включает восемь значений температуры (в градусах Цельсия), измеренных в разные часы суток. 1. Сгенерировать исходный список T, состоящий из шести подписков, каждый из которых содержит восемь случайных вещественных чисел с точностью до одного знака после запятой в диапазоне от -10.0 до 35.0. 2. Создать поверхностную копию списка T в переменную $T_{shallow}$ и глубокую копию в переменную T_{deep}. 3. В списке T выполнить корректировку: — если максимальное значение в подписке превышает 30, заменить все элементы этого подписка на среднее значение данного подписка, округленное до одного знака; — если минимальное значение в подписке меньше 0, заменить все элементы подписка на 0.0.

№	Задание
	4. Вывести содержимое списков T, T_{shallow} и T_{deep}. 5. Провести поэлементное сравнение соответствующих подсписков списков T и T_{shallow}, а также T и T_{deep} с применением операторов <code>is</code> и <code>==</code> и оформить результаты в комментариях. 6. На основании полученных данных сделать заключение о воздействии изменений в списке T на его поверхностную и глубокую копии, учитывая особенности механизмов копирования в Python
24	В системе мониторинга состояния оборудования промышленного цеха используется вложенный список S, содержащий пять подсписков, каждый из которых включает семь значений параметров вибрации, измеренных в течение недели. 1. Сгенерировать исходный список S с пятью под списками, каждый из которых содержит семь случайных вещественных чисел в диапазоне от 0.0 до 5.0 с точностью до двух знаков после запятой. 2. Выполнить поверхностное копирование списка S в переменную S_{shallow} и глубокое копирование в переменную S_{deep}. 3. В исходном списке S выполнить следующие операции: — заменить все значения под списка на 0, если среднее значение элементов под списка превышает 3.5; — заменить все значения под списка на 5, если минимальное значение под списка меньше 0.5. 4. Вывести содержимое списков S, S_{shallow} и S_{deep}. 5. Сравнить под списки между списками S и S_{shallow}, а также между S и S_{deep} с использованием операторов идентичности <code>is</code> и равенства <code>==</code>; результаты оформить в комментариях. 6. Сделать выводы о том, как изменения в исходном списке повлияли на поверхностную и глубокую копии с учетом особенностей работы ссылок в Python
25	В системе учета температурных показателей в холодильных камерах склада представлена структура данных в виде вложенного списка T, содержащего восемь подсписков, каждый из которых хранит значения температуры, измеренные с интервалом в десять минут на протяжении суток. Каждое значение — вещественное число с точностью до двух знаков после запятой, отражающее температуру в градусах Цельсия. 1. Сформировать исходный список T из восьми подсписков, содержащих по 144 случайных значений температуры в диапазоне от -25.00 до 5.00. 2. Выполнить поверхностное копирование списка T в переменную T_{shallow} и глубокое копирование в переменную T_{deep}. 3. В исходном списке T реализовать следующие преобразования: — округлить все значения до одного знака после запятой; — в каждом под списке заменить все значения на -18.0, если среднее значение под списка ниже -20.0; — в каждом под списке заменить все значения на 0.0, если максимальное значение превышает 0.0. 4. Вывести содержимое списков T, T_{shallow} и T_{deep}.

№	Задание
	5. Выполнить поэлементное сравнение соответствующих подписков между T и T_{shallow}, а также между T и T_{deep} с использованием операторов <code>is</code> и <code>==</code>; оформить полученные результаты в виде комментариев. 6. На основе анализа различий сформулировать выводы о характере и последствиях изменений для поверхностного и глубокого копирования вложенных списков в языке Python
26	В системе контроля уровня шума на промышленном объекте используется вложенный список N, состоящий из шести подписков, каждый из которых содержит по десять измерений звукового давления, выполненных с периодичностью в одну минуту. Каждое значение выражено в децибелах и представлено вещественным числом с точностью до одного знака после запятой. 1. Сгенерировать исходный список N, состоящий из шести подписков по десять случайных значений в диапазоне от 40.0 до 100.0. 2. Выполнить поверхностное копирование списка N в переменную N_{shallow} и глубокое копирование в переменную N_{deep}. 3. В исходном списке N выполнить следующие преобразования: — округлить каждое значение до целого числа; — если среднее значение в подписке превышает 75, заменить все элементы подписка на число 80; — если минимальное значение подписка ниже 45, заменить все элементы подписка на число 50. 4. Вывести содержимое списков N, N_{shallow} и N_{deep}. 5. Провести поэлементное сравнение соответствующих подписков между N и N_{shallow}, а также между N и N_{deep}, используя операторы <code>is</code> и <code>==</code>; результаты оформить в комментариях. 6. На основе наблюдений сделать выводы о различиях между поверхностным и глубоким копированием вложенных списков, отражающих изменения в исходных данных
27	В системе анализа температуры в серверных помещениях используется вложенный список T, содержащий восемь подписков, каждый из которых включает по двенадцать измерений температуры, выполненных ежечасно в течение суток. Значения представлены вещественными числами с точностью до двух знаков после запятой и выражены в градусах Цельсия. 1. Сгенерировать исходный список T с восьмью подписками по двенадцать случайных значений в диапазоне от 18.00 до 27.00. 2. Выполнить поверхностное копирование списка T в переменную T_{shallow} и глубокое копирование в переменную T_{deep}. 3. В исходном списке T выполнить следующие изменения: — округлить каждое значение до целых чисел; — если максимальное значение в подписке превышает 25, заменить весь подподсписок на список из двенадцати значений 26; — если минимальное значение подписка ниже 19, заменить подподсписок на список из двенадцати значений 18. 4. Вывести содержимое списков T, T_{shallow} и T_{deep}. 5. Провести поэлементное сравнение соответствующих подписков между T и T_{shallow}, а также между T и T_{deep} с использованием операторов <code>is</code> и <code>==</code>; результаты оформить в комментариях.

№	Задание
	6. Сделать заключение о различиях в поведении поверхностного и глубокого копирования при работе с вложенными структурами данных на примере модификаций исходного списка
28	В системе контроля качества продукции на фабрике хранится вложенный список P, состоящий из шести подсписков, каждый из которых содержит по десять значений, отражающих измерения плотности материала партии в кг/м³ с точностью до трех знаков после запятой. 1. Сгенерировать исходный список P с шестью подсписками по десять случайных вещественных чисел в диапазоне от 950.000 до 1050.000. 2. Выполнить поверхностное копирование списка P в переменную P_{shallow} и глубокое копирование в переменную P_{deep}. 3. В исходном списке P осуществить следующие изменения: — округлить каждое значение до целых чисел; — если среднее значение подсписка превышает 1000, заменить весь подсписок списком из десяти значений 1005; — если минимальное значение подсписка ниже 960, заменить подсписок списком из десяти значений 955. 4. Вывести содержимое списков P, P_{shallow} и P_{deep}. 5. Провести поэлементное сравнение соответствующих подсписков между P и P_{shallow}, а также между P и P_{deep}, используя операторы <code>is</code> и <code>==</code>; результаты оформить в комментариях. 6. Сформулировать выводы о различиях между поверхностным и глубоким копированием в контексте изменений вложенных списков и поведения ссылок на объекты
29	В системе мониторинга параметров работы нескольких серверов хранится вложенный список S, состоящий из пяти подсписков, каждый из которых содержит по восемь числовых значений, отражающих загрузку процессора сервера в процентах с точностью до двух знаков после запятой за последовательные временные интервалы. 1. Сгенерировать исходный список S с пятью подсписками по восемь случайных вещественных чисел в диапазоне от 0.00 до 100.00. 2. Выполнить поверхностное копирование списка S в переменную S_{shallow} и глубокое копирование в переменную S_{deep}. 3. В исходном списке S выполнить следующие преобразования: — округлить каждое значение вверх до целых чисел; — если максимальное значение в подсписке превышает 90, заменить весь подсписок списком из восьми значений 90; — если минимальное значение подсписка ниже 10, заменить подсписок списком из восьми значений 10. 4. Вывести содержимое списков S, S_{shallow} и S_{deep}. 5. Провести поэлементное сравнение соответствующих подсписков между S и S_{shallow}, а также между S и S_{deep}, используя операторы <code>is</code> и <code>==</code>; результаты оформить в комментариях. 6. Сформулировать заключение о поведении механизмов поверхностного и глубокого копирования в условиях изменения вложенных списков
30	В системе анализа показателей энергопотребления нескольких зданий представлен вложенный список E , состоящий из шести подсписков, каждый из которых содержит по десять вещественных числовых значений, характеризующих суточное

№	Задание
	энергопотребление (в киловатт-часах) за десять последовательных дней с точностью до двух знаков после запятой. 1. Сгенерировать исходный список E с шестью подписками по десять случайных значений в диапазоне от 15.00 до 150.00. 2. Выполнить поверхностное копирование списка E в переменную E_{shallow} и глубокое копирование в переменную E_{deep}. 3. В исходном списке E произвести следующие изменения: — округлить каждое значение до целых чисел; — если среднее значение в подписке превышает 100, заменить весь подсписок списком из десяти значений 0; — если максимальное значение в подписке ниже 20, заменить подсписок списком из десяти значений 200. 4. Вывести содержимое списков E, E_{shallow} и E_{deep}. 5. Провести поэлементное сравнение соответствующих подписок между E и E_{shallow}, а также между E и E_{deep}, с использованием операторов <code>is</code> и <code>==</code>; результаты оформить в комментариях. 6. Сделать выводы о различиях в поведении поверхностного и глубокого копирования при работе с вложенными списками после внесенных изменений

7.4. Сортировка списков. Применение лямбда-функций

Сортировка списков — это операция упорядочивания элементов в соответствии с заданным критерием сравнения. В общем случае сортировка представляет собой процесс преобразования исходной последовательности элементов в новую последовательность, в которой элементы следуют в неубывающем или невозрастающем порядке в зависимости от выбранного порядка сортировки.

В Python для сортировки списков предусмотрены два основных механизма: метод `sort()` и функция `sorted()`.

Метод `list.sort()` выполняет сортировку *на месте*, модифицируя исходный список. Синтаксис использования:

```
list.sort(key, reverse)
```

где параметр **key** задает функцию извлечения ключа сравнения для каждого элемента в формате

key=ссылка_на_функцию,

а параметр **reverse** определяет порядок сортировки (по возрастанию при значении **False**, по убыванию при значении **True**). Оба ключа являются необязательными.

Функция `sorted()` возвращает новый отсортированный список, не изменяя исходную последовательность. Синтаксис:

sorted(list, key, reverse)

При указании метода/функции без аргументов выполняется сортировка по неубыванию. Строки сортируются в лексикографическом порядке.

Аргумент **key** принимает значение ссылки на функцию, т. е. имя функции/метода без круглых скобок () в конце. Некоторые часто используемые значения аргумента **key** приведены в табл. 7.11.

Таблица 7.11

Функция/Метод	Применяется к типу	Назначение извлечения ключа
abs	числовые значения	Модуль числа
len	строка, список, кортеж, множество	Длина объекта
str.lower	строка	Приведение строки к нижнему регистру
str.upper	строка	Приведение строки к верхнему регистру
int, float	строка, число	Преобразование к числовому типу

Функция/метод, указанные в параметре **key** применяются к каждому элементу списка, получается некоторое значение. Исходный список будет отсортирован таким образом, что полученные значения будут отсортированы в указанном порядке. Пример использования функции **sorted()** с различными значениями параметров приведен в листинге 7.15.

Листинг 7.15

```
a = [-23, 46, -94, 16, 37]
b = ['Иванко', 'Иван', 'Иванюк', 'Иван01', 'Иванов', 'Иванищенко']
c = ['3', '10', '1', '2', '14']

# Сортировка по убыванию абсолютных значений
s1 = sorted(a, key=abs, reverse=True)
print(s1) # [-94, 46, 37, -23, 16]

# Сортировка по возрастанию длины строк
s2 = sorted(b, key=len)
print(s2)
# ['Иван', 'Иванко', 'Иванюк', 'Иван01', 'Иванов',
'Иванищенко']

# Сортировка строк в лексикографическом порядке по убыванию
s3 = sorted(c, reverse=True)
print(s3) # ['3', '2', '14', '10', '1']

# Сортировка строк в лексикографическом порядке после
приведения к нижнему регистру
s4 = sorted(b, key=str.lower)
print(s4)
```

```
# ['Иван', 'Иван01', 'Иванищенко', 'Иванко', 'Иванов',  
  'Иванюк']
```

Лямбда-функция — это анонимная (безымянная) функция, определяемая с помощью ключевого слова **lambda**. Такие функции позволяют создавать краткие функции без необходимости использовать конструкцию **def** и задавать имя. Основное назначение — передача простой логики как аргумента в другие функции, например, в **sorted()**, **map()**, **filter()**, **reduce()**.

Функция высшего порядка — это функция, которая либо принимает в качестве аргументов другие функции, либо возвращает другую функцию в качестве результата, либо делает и то, и другое одновременно. Указанные выше четыре функции как раз относят к подобным функциям.

Общий синтаксис задания лямбда-функции:

lambda аргументы: выражение

Лямбда-функция может принимать любое количество аргументов, но допускает только одно выражение (вычисляемое, но не инструкцию). Возвращаемое значение — результат этого выражения. Пример задания лямбда-функции:

```
lambda x, y: x + y
```

При желании можно задать имя для лямбда-функции и использовать в программе. Пример приведен в листинге 7.16.

Листинг 7.16

```
f = lambda x: x + 1  
print(f(2)) # 3  
print(f(f(4))) # 6
```

При сортировке двумерных списков имеется ряд особенностей. Так, если применить метод/функцию сортировки без параметра **key**, то вложенные списки сортируются по нулевому по индексу элементу. В случае равенства нулевых элементов они упорядочиваются по первому по индексу, а в случае и их равенства — по второму и т. д (листинг 7.17).

Листинг 7.17

```
a = [[3, 8, 2], [], [4, 2], [3, 9, 1], [4, 1], [3, 8, 4]]  
  
s1 = sorted(a)  
print(s1)  
# [[], [3, 8, 2], [3, 8, 4], [3, 9, 1], [4, 1], [4, 2]]
```

Но если параметр **key** присутствует, то сортировка выполняется по значению функции, указанной для этого параметра, но при совпадении значений порядок следования элементов в отсортированном списке будет такой, какой был в исходном списке (листинг 7.18).

Листинг 7.18

```

a = [[3, 8, 2], [], [4, 2], [3, 9, 1], [4, 1], [3, 8, 4]]

s2 = sorted(a, key=len)
print(s2)
# [[], [4, 2], [4, 1], [3, 8, 2], [3, 9, 1], [3, 8, 4]]

```

<code>lambda x: x[i]</code>	последовательность	Извлечение i-го элемента вручную
<code>lambda x: -x</code>	числовые значения	Инверсия значения для сортировки по убыванию
<code>lambda x: (x[1], x[0])</code>	кортеж	Сортировка по нескольким критериям
<code>lambda x: sum(x)</code>	последовательность чисел	Суммирование значений
<code>lambda x: max(x)</code>	последовательность	Максимальный элемент
<code>lambda x: min(x)</code>	последовательность	Минимальный элемент
<code>lambda x: x['key']</code>	словарь	Извлечение значения по ключу 'key'

Equation Section (Next)

8. Матричные операции

Двумерные (вложенные) списки удобно использовать при обработке табличных данных. Применительно к математике двумерный список соответствует матрице размерностью $n \times m$:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1(m-1)} & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2(m-1)} & a_{2m} \\ \cdots & \cdots & \ddots & \cdots & \cdots \\ a_{(n-1)1} & a_{(n-1)2} & \cdots & a_{(n-1)(m-1)} & a_{(n-1)m} \\ a_{n1} & a_{n2} & \cdots & a_{n(m-1)} & a_{nm} \end{pmatrix}$$

или в более компактной записи $A = (a_{ij})$, $i = \overline{1, n}$, $j = \overline{1, m}$.

Замечание. При обработке матриц следует помнить, что в привычной математической записи нумерация индексов начинается с 1, а в языке Python с 0. Поэтому третьей строке матрицы соответствует второй по индексу элемент двумерного списка, а первому столбцу матрицы соответствует набор элементов каждого элемента двумерного списка, имеющих в этом элементе индекс 0.

Обратиться к i -й строке матрицы A можно $A[i - 1]$. Со столбцами немного сложнее: если требуется j -й столбец, нужно отобрать из каждой строки элемент с индексом $(j - 1)$.

Пример 8.1

Сгенерировать матрицу A размерностью 6×8 , заполненную случайными числами из отрезка $[10, 50]$ при значении **seed**(15). Вывести на экран 4-ю строку и 7-й столбец матрицы A .

Решение

Программный код решения приведен в листинге 8.1. При выводе матрицы была использована ранее описанная функция `out()`.

Листинг 8.1

```
from random import *

n, m = 6, 8
seed(15)
A = [[randint(10, 50) for j in range(m)] for i in range(n)]
out(A)

# Четвертая строка матрицы A
print(A[3])
# [29, 42, 36, 24, 46, 39, 36, 41]
```

```
# Седьмой столбец матрицы A
print([i[6] for i in A])
# [11, 32, 30, 36, 30, 44]
```

При обработке квадратных матриц размерности n достаточно часто нужно проверить свойства элементов, лежащих на главной и вторичной диагоналях, а так же значения на/под ними. На рис. 3а) приведены отношения индексов для главной диагонали, на рис 3б) для вторичной. Отношения учитывают, что индексация в Python начинается с 0.

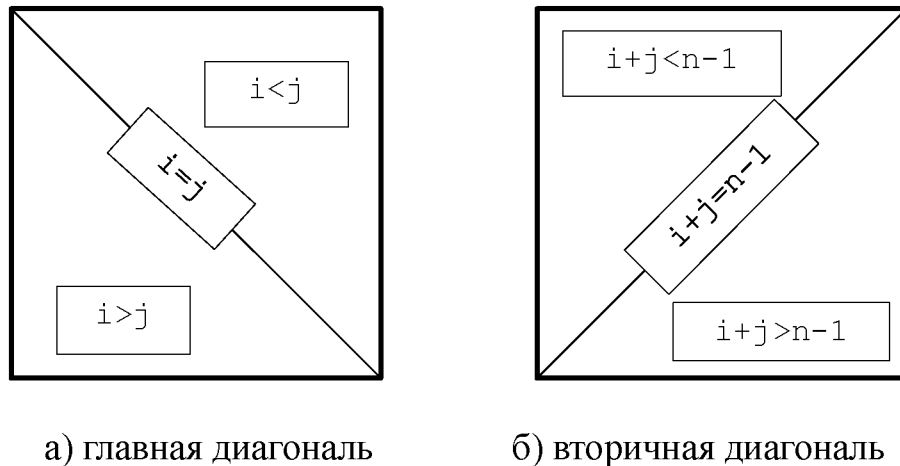


Рисунок 3. Отношение индексов для главной и вторичной диагоналей

Пример 8.2

Задана прямоугольная матрица размерности 7×9 . Определить количество столбцов и номер последнего столбца, в котором нет отрицательных элементов. Если таких столбцов нет, вывести **None**.

Решение

Программный код решения приведен в листинге 8.2.

Листинг 8.2

```
from random import *

# Подпрограмма вывода двумерного списка
def out(a):
    for i in a:
        for j in i:
            print(f'{j:4d}', end='')
        print()

# Генерируем двумерный список
n, m = 7, 9
L = [[randint(-10, 30) for j in range(m)] for i in range(n)]
# Вывод сгенерированного двумерного списка
```

```

out(L)
#  4  3 11 25 24 18 -5 17 26
# 12  0  1 29 -4 12  1 21 29
# -5 19 17  3 -6 -6 30  6 11
# 15 13 27 10 17 23 29 28 -6
#  2 16 12 23 21 12 -3 18 11
# -10  4  9 16 -2  2  7 22 29
# -8  0 27  9 -7 -3  7 30 27

c = 0
for j in range(len(L[0])):
    # j-й столбец
    st = [i[j] for i in L]
    # Если все значения неотрицательны...
    if all(k >= 0 for k in st):
        # ...увеличиваем счетчик и запоминаем номер столбца
        c += 1
        NSt = j + 1

if c == 0:
    print('None')
else:
    print(c, NSt)
# 4 8

```

Пример 8.3

Задана квадратная матрица размерности 10×10 . Определить два значения:

- сумму элементов, которые расположены выше главной и вторичной диагоналей;
- сумму элементов, которые расположены ниже главной и вторичной диагоналей.

Если первая сумма больше второй, заменить элементы главной диагонали и все элементы ниже ее на значение -1. Иначе заменить элементы вторичной диагонали значением 0.

Решение

Программный код решения приведен в листинге 8.3.

Листинг 8.3

```

from random import *

# Подпрограмма вывода двумерного списка
def out(a):
    for i in a:
        for j in i:
            print(f'{j:4d}', end='')
        print()

```

```

n = 10
L = [[randint(-10, 30) for j in range(n)] for i in range(n)]
out(L)
print()

# Вычисляем требуемые суммы
s1 = sum([L[i][j] for i in range(n) for j in range(n) if i < j
and i + j < n - 1])
s2 = sum([L[i][j] for i in range(n) for j in range(n) if i > j
and i + j > n - 1])

# Если первая сумма меньше второй...
if s1 < s2:
    for i in range(n):
        for j in range(n):
            # Заменяем все, что на главной диагонали и ниже на
-1
                if i >= j:
                    L[i][j] = -1
else: # иначе...
    for i in range(n):
        for j in range(n):
            # Заменяем элементы вторичной диагонали и ниже на 0
            if i + j == n - 1:
                L[i][j] = 0

out(L)

```

Задание 8.1

Выполнить задания в соответствии с табл. 8.1. Списки в программном коде задать случайными значениями.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- результаты работы программы при различных исходных данных.

Таблица 8.1

№	Задания
1	Задана квадратная матрица размерности 12×12 . Определить минимальный элемент, расположенный выше главной диагонали, и максимальный элемент, расположенный ниже побочной диагонали. Если разность между ними четная, заменить элементы главной диагонали на их квадраты. Иначе заменить все элементы побочной диагонали значением -1
	Задана прямоугольная матрица размерности 6×8 . Определить номер первого столбца, в котором сумма положительных элементов превышает сумму по всей строке, содержащей минимальный элемент матрицы. Если таких столбцов несколько, выбрать первый. Если ни один не удовлетворяет условию, вывести None

№	Задания
2	Задана квадратная матрица размерности 9×9 . Вычислить сумму элементов, находящихся одновременно ниже главной диагонали и выше побочной диагонали. Если полученная сумма больше нуля, заменить элементы главной диагонали на удвоенное значение номера строки. Иначе заменить нулями все элементы побочной диагонали
	Задана прямоугольная матрица размерности 5×11 . Определить номера всех строк, в которых сумма абсолютных значений элементов больше средней суммы элементов по всем строкам. Если таких строк нет, вывести None
3	Задана квадратная матрица размерности 10×10 . Для каждой строки, сумма которой кратна 3, заменить элементы с четными индексами на 0, а элементы с нечетными индексами оставить без изменения. Если ни одна строка не удовлетворяет условию, оставить матрицу без изменения
	Задана прямоугольная матрица размерности 7×10 . Вычислить сумму элементов, находящихся в нечетных по номеру строках и столбцах одновременно. Если эта сумма больше суммы всех отрицательных элементов в четных строках, заменить весь первый столбец значением 1
4	Задана квадратная матрица размерности 8×8 . Определить количество положительных элементов, расположенных строго между главной и побочной диагоналями. Если это число кратно 4, заменить соответствующие элементы на значение 99, иначе оставить без изменения
	Задана прямоугольная матрица размерности 6×9 . Для каждой строки, в которой максимальный элемент больше 50, заменить минимальный элемент на сумму всех элементов строки. Если таких строк нет, ничего не изменять
5	Задана квадратная матрица размерности 11×11 . Построить список номеров строк, в которых сумма элементов, расположенных правее главной диагонали, превышает сумму элементов, расположенных левее побочной диагонали. Если таких строк нет, вывести None
	Задана прямоугольная матрица размерности 4×12 . Найти столбец с минимальным числом положительных элементов. Если таких несколько, выбрать тот, в котором сумма этих элементов минимальна. Все элементы найденного столбца заменить значением -2
6	Задана квадратная матрица размерности 9×9 . Определить сумму элементов, расположенных на границе матрицы. Если эта сумма четна, заменить все элементы первого и последнего столбца значением 100, иначе заменить все элементы первого и последнего столбца значением -100
	Задана прямоугольная матрица размерности 6×10 . Найти номер столбца с наименьшим средним арифметическим элементов. Если таких несколько, выбрать последний из них. Элементы этого столбца заменить значением, равным номеру строки, в которой они находятся
7	Задана квадратная матрица размерности 8×8 . Определить количество элементов, лежащих одновременно ниже главной и ниже побочной диагонали. Если их количество четное, заменить все элементы выше обеих диагоналей значением 0
	Задана прямоугольная матрица размерности 5×7 . Для каждой строки вычислить разность между максимальным и минимальным элементами. Найти строку с наибольшей такой разностью. Элементы этой строки заменить их средним арифметическим, округленным вниз

№	Задания
8	Задана квадратная матрица размерности 10×10 . Определить, существует ли хотя бы одна строка, все элементы которой четны. Если да, то заменить главную диагональ значением 1. Иначе заменить побочную диагональ значением -1
	Задана прямоугольная матрица размерности 7×8 . Вычислить общее количество столбцов, в которых хотя бы один элемент совпадает с номером своей строки. Если таких столбцов больше трех, удалить первый и последний столбцы (логически — не обрабатывать их при последующих действиях)
9	Задана квадратная матрица размерности 9×9 . Определить минимальный элемент в каждом столбце. Если хотя бы в одном столбце минимальный элемент кратен 5, заменить весь соответствующий столбец на произведение номера строки и номера столбца
	Задана прямоугольная матрица размерности 6×10 . Найти номер строки, в которой сумма элементов, кратных 7, максимальна. Если таких строк несколько, выбрать последнюю. Элементы этой строки заменить значением 77
10	Задана квадратная матрица размерности 11×11 . Определить сумму элементов, находящихся строго между главной и побочной диагоналями. Если сумма кратна 3, заменить все элементы побочной диагонали значением 3
	Задана прямоугольная матрица размерности 5×9 . Определить, содержится ли хотя бы один нулевой элемент одновременно в строке с нечетным номером и в столбце с четным номером. Если условие выполняется, заменить все элементы соответствующих строк значением -9
11	Задана квадратная матрица размерности 10×10 . Найти строку с максимальным количеством отрицательных элементов. Заменить все элементы этой строки на сумму ее положительных элементов. Если таких строк несколько, выбрать первую
	Задана прямоугольная матрица размерности 6×9 . Для каждого столбца определить количество четных чисел. Если хотя бы в половине столбцов количество четных чисел превышает 3, заменить соответствующие столбцы значением 0
12	Задана квадратная матрица размерности 8×8 . Найти произведение всех элементов, расположенных на главной диагонали. Если оно нечетное, заменить все элементы побочной диагонали значением -2
	Задана прямоугольная матрица размерности 5×12 . Найти сумму элементов в каждом столбце. Определить столбец с минимальной суммой и заменить его элементы значением индекса этого столбца
13	Задана квадратная матрица размерности 9×9 . Для всех элементов, расположенных выше главной диагонали и ниже побочной диагонали, определить количество элементов, делящихся на 3. Если их больше 10, заменить все такие элементы на 3
	Задана прямоугольная матрица размерности 7×10 . Определить количество строк, в которых хотя бы три последних элемента равны между собой. Если такие строки существуют, обнулить их первые три элемента
14	Задана квадратная матрица размерности 11×11 . Найти сумму всех элементов, расположенных на внешней границе матрицы. Если сумма делится на 11, заменить все внутренние элементы (не границу) значением 11
	Задана прямоугольная матрица размерности 6×8 . Найти столбцы, в которых минимальный элемент отрицателен. Для всех таких столбцов заменить отрицательные элементы на модуль их значения

№	Задания
15	Задана квадратная матрица размерности 10×10 . Определить, существуют ли строки, содержащие элементы с повторяющимися значениями. Если да, заменить первую из них значением -5 во всех позициях
	Задана прямоугольная матрица размерности 4×9 . Для каждой строки, сумма элементов которой меньше суммы в первой строке, заменить последний элемент на среднее арифметическое всех элементов этой строки
16	Задана квадратная матрица размерности 12×12 . Определить наименьшее нечетное число, расположенное строго выше главной диагонали. Если такое число существует, заменить все элементы побочной диагонали на это значение
	Задана прямоугольная матрица размерности 5×10 . Определить количество столбцов, в которых хотя бы одно значение является квадратом нечетного числа. Если таких столбцов больше трех, заменить их элементы значением -7
17	Задана квадратная матрица размерности 9×9 . Для всех строк, где максимальный элемент равен минимальному, заменить всю строку на значения 0
	Задана прямоугольная матрица размерности 6×10 . Найти индекс строки, в которой произведение положительных чисел максимально. Если таких несколько, выбрать последнюю и заменить все ее элементы на 1
18	Задана квадратная матрица размерности 10×10 . Для всех элементов, расположенных на главной и побочной диагоналях, определить, сколько из них делится на 5 . Если их больше 5 , заменить элементы обеих диагоналей на 5
	Задана прямоугольная матрица размерности 7×9 . Для каждого столбца определить максимальный элемент и его строковый индекс. Если сумма всех таких индексов четна, заменить все максимальные элементы значением 100
19	Задана квадратная матрица размерности 11×11 . Найти количество положительных элементов, расположенных одновременно выше главной и ниже побочной диагонали. Если их больше 20 , заменить все отрицательные элементы матрицы на 0
	Задана прямоугольная матрица размерности 6×10 . Для строк, в которых есть хотя бы два нуля, заменить все элементы между ними на значение 9 (если они находятся в одном ряду)
20	Задана квадратная матрица размерности 10×10 . Определить, содержит ли каждая строка хотя бы один положительный элемент. Если это условие выполняется, заменить главную диагональ значением, равным индексу строки
	Задана прямоугольная матрица размерности 5×11 . Вычислить среднее арифметическое по каждому столбцу. Если максимальное из этих значений делится на 2 , заменить все элементы соответствующего столбца значением 2
21	Задана квадратная матрица размерности 10×10 . Определить, имеются ли среди строк такие, в которых произведение всех положительных элементов превышает 1000 . Если да, заменить соответствующие строки значением -10
	Задана прямоугольная матрица размерности 6×12 . Найти количество столбцов, в которых первый элемент равен последнему. Все такие столбцы заменить значением 1
22	Задана квадратная матрица размерности 9×9 . Для каждого столбца определить минимальный элемент. Если он меньше нуля, заменить все элементы этого столбца значением -99

№	Задания
	Задана прямоугольная матрица размерности 7×11 . Определить номера строк, в которых максимальный элемент встречается более одного раза. Все такие строки заменить значением 7
23	Задана квадратная матрица размерности 8×8 . Определить сумму элементов, расположенных на границах центрального квадрата 4×4 . Если сумма больше 500, заменить соответствующие элементы значением 500
	Задана прямоугольная матрица размерности 6×9 . Для каждого столбца вычислить разность между количеством четных и нечетных чисел. Если абсолютное значение этой разности больше 3, заменить соответствующий столбец нулями
24	Задана квадратная матрица размерности 10×10 . Определить, сколько элементов главной и побочной диагоналей являются простыми числами. Если таких больше 5, заменить обе диагонали значением 1
	Задана прямоугольная матрица размерности 5×10 . Определить номера всех строк, в которых сумма элементов, стоящих на четных позициях, больше суммы элементов на нечетных. Заменить такие строки значением -5
25	Задана квадратная матрица размерности 11×11 . Для каждого элемента, стоящего на позиции с четными индексами, определить, превышает ли он среднее арифметическое всех элементов матрицы. Если да, заменить его значением -3
	Задана прямоугольная матрица размерности 6×8 . Найти столбец с наибольшим количеством элементов, равных нулю. Все элементы этого столбца заменить значением 100
26	Задана квадратная матрица размерности 9×9 . Определить, содержит ли каждая строка элементы, делящиеся на 7. Если условие выполнено, заменить всю побочную диагональ на 7
	Задана прямоугольная матрица размерности 4×12 . Определить, есть ли строки, в которых хотя бы три элемента равны между собой. Если такие строки найдены, заменить каждый повторяющийся элемент в них значением -1
27	Задана квадратная матрица размерности 10×10 . Для каждой строки вычислить разность между максимальным и минимальным значениями. Если эта разность превышает 50, заменить строку на список из значений 50
	Задана прямоугольная матрица размерности 7×10 . Определить, есть ли столбцы, в которых ни один элемент не делится на 2. Если найдены такие столбцы, заменить их элементы значением -2
28	Задана квадратная матрица размерности 8×8 . Определить количество элементов, равных 0, находящихся на позициях, симметричных относительно главной диагонали. Если их больше 3, заменить элементы главной диагонали на 0
	Задана прямоугольная матрица размерности 6×9 . Для каждой строки определить, содержит ли она подряд идущие возрастающие значения. Если найдены такие строки, заменить в них все элементы на 9
29	Задана квадратная матрица размерности 10×10 . Определить количество элементов, значение которых больше суммы индексов строки и столбца. Если таких элементов больше 30, заменить первый столбец на -1
	Задана прямоугольная матрица размерности 5×8 . Для каждого столбца определить, является ли последовательность его элементов строго убывающей. Если таких столбцов больше 2, заменить их элементы значением -5

№	Задания
30	Задана квадратная матрица размерности 9×9 . Для каждого столбца определить, является ли он симметричным относительно центра. Если таких столбцов нет, заменить побочную диагональ на -9
	Задана прямоугольная матрица размерности 6×10 . Определить количество строк, в которых все элементы различны. Если таких строк больше половины, заменить все элементы главной диагонали (если матрица квадратная) на 1, иначе ничего не изменять

Нулевая матрица — это матрица любого размера $m \times n$, все элементы которой равны нулю. Обозначается как $0_{m \times n}$ или просто 0, если размерность ясна из контекста. Формально, если $A = (a_{ij})$ — нулевая матрица, то $a_{ij} = 0$ при всех $1 \leq i \leq m$, $1 \leq j \leq n$. Нулевая матрица является нейтральным элементом по операции сложения в пространстве матриц: для любой матрицы B размерности $m \times n$ выполнено $B + 0_{m \times n} = B$.

Единичная матрица — это квадратная матрица порядка n , элементы которой равны единице на главной диагонали и нулю во всех остальных позициях. Обозначается как I_n или I . Формально, если $I = (i_{kl})$ — единичная матрица, то $i_{kl} = 1$, если $k = l$, и $i_{kl} = 0$, если $k \neq l$, при $1 \leq k, l \leq n$. Единичная матрица является нейтральным элементом по умножению в множестве квадратных матриц: для любой квадратной матрицы A порядка n выполнено $AI_n = I_n A = A$.

Пусть $A = (a_{ij})$ и $B = (b_{ij})$ — матрицы одинаковой размерности $n \times m$, где n — количество строк, а m — количество столбцов. Основные операции над матрицами определяются следующим образом.

1. *Сложение матриц* определяется покомпонентно. Результатом сложения двух матриц A и B является матрица $C = (c_{ij})$ размерности $n \times m$, элементы которой вычисляются по формуле

$$c_{ij} = a_{ij} + b_{ij}. \quad (8.1)$$

Операция сложения определена только для матриц одинаковой размерности. Свойства операции сложения соответствуют свойствам коммутативности и ассоциативности: $A + B = B + A$ и $(A + B) + C = A + (B + C)$.

2. *Вычитание матриц* так же определяется покомпонентно. Результатом вычитания $A - B$ является матрица $D = (d_{ij})$, элементы которой вычисляются как

$$d_{ij} = a_{ij} - b_{ij}. \quad (8.2)$$

Как и в случае сложения, операция вычитания определена только для матриц одинаковой размерности.

3. *Умножение матрицы на число* (скаляр) представляет собой операцию, при которой каждый элемент матрицы $A = (a_{ij})$ умножается на заданное число λ , в результате чего получается матрица $B = (b_{ij})$, где $b_{ij} = \lambda a_{ij}$. Эта операция

сохраняет размерность исходной матрицы и обладает свойствами дистрибутивности и ассоциативности относительно умножения на число: $\lambda(A+B) = \lambda A + \lambda B$, $(\lambda + \mu)A = \lambda A + \mu A$, $(\lambda\mu)A = \lambda(\mu A)$.

4. Умножение матриц размерности $n \times m$ и B размерности $m \times p$ определяется как матрица $C = AB$ размерности $n \times p$, элементы которой вычисляются по формуле

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}. \quad (8.3)$$

Операция умножения не коммутативна, то есть $AB \neq BA$ в общем случае. Однако она ассоциативна и дистрибутивна относительно сложения: $(AB)C = A(BC)$, $A(B+C) = AB + AC$.

5. Транспонирование матрицы $A = (a_{ij})$ представляет собой операцию, в результате которой строки и столбцы меняются местами. Обозначается как A^T и определяется следующим образом: $(A^T)_{ij} = a_{ji}$. Транспонирование удовлетворяет равенствам $(A^T)^T = A$, $(A+B)^T = A^T + B^T$ и $(AB)^T = B^T A^T$.

6. След матрицы $A = (a_{ij})$ размерности $n \times n$ определяется как сумма элементов, расположенных на главной диагонали:

$$\text{Tr}(A) = \sum_{i=1}^n a_{ii}. \quad (8.4)$$

След обладает линейностью: $\text{Tr}(A+B) = \text{Tr}(A) + \text{Tr}(B)$ и $\text{Tr}(\lambda A) = \lambda \text{Tr}(A)$. Кроме того, $\text{Tr}(AB) = \text{Tr}(BA)$ при условии согласованной размерности.

7. Определитель матрицы — скалярная величина, которая вычисляется для квадратной матрицы $A = (a_{ij})$, $i, j = \overline{1, n}$. Определитель обозначается как $\det A$ или $|A|$.

Для матрицы порядка 1 определитель равен единственному элементу матрицы: если $A = (a_{11})$, то $\det A = a_{11}$.

Для матрицы порядка 2 имеем

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix},$$

определитель вычисляется по формуле

$$\det A = a_{11}a_{22} - a_{12}a_{21}.$$

Для матрицы порядка 3

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$$

определитель равен

$$\det A = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{13}a_{22}a_{31} - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33}.$$

В общем случае для вычисления определителя применяется *метод Гаусса*, основанный на приведении матрицы к *верхнетреугольному виду* (все элементы ниже главной диагонали равны 0) с помощью *элементарных преобразований*:

- перестановка двух строк: при этом меняется знак определителя);
- умножение строки на ненулевое число: при этом определитель умножается на это число;
- прибавление к одной строке другой строки, умноженной на число: при этом определитель не изменяется.

В результате этих преобразований исходная матрица A приводится к верхнетреугольному виду U , для которого определитель равен произведению элементов главной диагонали:

$$\det A = (-1)^s \cdot \frac{1}{\prod k_i} \cdot \prod_{i=1}^n u_{ii}, \quad (8.5)$$

где s — число перестановок строк, k_i — множители, на которые были умножены строки, u_{ii} — диагональные элементы треугольной матрицы U .

Если в процессе приведения к треугольному виду один из диагональных элементов обращается в нуль, значит $\det A = 0$, и матрица A является вырожденной.

8. *Ранг матрицы* определяется как количество линейно независимых строк или столбцов. Ранг матрицы — это число ненулевых строк после приведения матрицы к верхнетреугольному виду.

9. *Обратная матрица* A^{-1} к квадратной невырожденной матрице A — это такая матрица, для которой $AA^{-1} = A^{-1}A = I$, где I — единичная матрица. Обратная матрица существует только в случае, если $\det A \neq 0$.

Метод Гаусса-Жордана нахождения обратной матрицы основан на расширении A до блочной матрицы $[A|I]$ путем приписывания справа единичной матрицы. Последовательно применяются элементарные преобразования строк (перестановка строк, умножение строки на ненулевой скаляр, прибавление к одной строке другой строки, умноженной на скаляр), направленные на приведение левой части к единичному виду. После завершения преобразований результирующая правая часть расширенной матрицы содержит обратную матрицу A^{-1} .

10. *Возведение матрицы в степень* применяется к квадратным матрицам. Возведение в степень A^k определяется как последовательное умножение матрицы A саму на себя k раз, при этом A^0 полагается равной единичной матрице I той же размерности.

Пример 8.4

Реализовать подпрограммы формирования нулевой и единичной матрицы заданных размерностей.

Решение

Программный код решения приведен в листинге 8.4.

Листинг 8.4

```
# Подпрограмма вывода двумерного списка
def MOut(a):
    for i in a:
        for j in i:
            print(f'{j:4d}', end='')
        print()

# Подпрограмма формирования нулевой матрицы
def MZeros(n, m):
    return [[0 for j in range(m)] for i in range(n)]

# Подпрограмма формирования единичной матрицы
def MOnes(n):
    return [[1 if i == j else 0 for j in range(n)] for i in
            range(n)]

n, m = 4, 6 # Размерности матриц
print('Нулевая матрицв')
MOut(MZeros(n, m))
#  0  0  0  0  0  0
#  0  0  0  0  0  0
#  0  0  0  0  0  0
#  0  0  0  0  0  0
print()

print('Единичная матрицв')
MOut(MOnes(m))
#  1  0  0  0  0  0
#  0  1  0  0  0  0
#  0  0  1  0  0  0
#  0  0  0  1  0  0
#  0  0  0  0  1  0
#  0  0  0  0  0  1
```

Пример 8.5

Реализовать подпрограммы сложения, вычитания матриц, умножения матрицы на число k . Предусмотреть проверку размерностей матриц.

Решение

Программный код решения приведен в листинге 8.5.

Листинг 8.5

```
from random import *

# Подпрограмма вывода двумерного списка
def MOut(a):
```

```

    for i in a:
        for j in i:
            print(f'{j:5d}', end='')
        print()

# Подпрограмма суммирования матриц
def MSum(a, b):
    if len(a) != len(b) or len(a[0]) != len(b[0]):
        return False
    else:
        return [[a[i][j] + b[i][j] for j in range(len(a[0]))]
                for i in range(len(a))]

# Подпрограмма вычитания матриц
def MSubst(a, b):
    if len(a) != len(b) or len(a[0]) != len(b[0]):
        return False
    else:
        return [[a[i][j] - b[i][j] for j in range(len(a[0]))]
                for i in range(len(a))]

# Подпрограмма умножения матрицы на число
def MMult(a, k):
    return [[a[i][j] * k for j in range(len(a[0]))] for i in
            range(len(a))]

# Размерности матриц и коэффициент умножения
n, m, k = 4, 6, 3
# Заполнение матриц случайными значениями
A = [[randint(-10, 10) for j in range(m)] for i in range(n)]
B = [[randint(-10, 10) for j in range(m)] for i in range(n)]

# Вывод матриц
MOut(A); print()
MOut(B); print()

# Вывод результата подпрограммы суммирования
print('Сумма матриц')
MOut(MSum(A, B)); print()

# Вывод результата подпрограммы вычитания
print('Разность матриц')
MOut(MSubst(A, B)); print()

# Вывод результата подпрограммы умножения на скаляр
print(f'Умножение первой матрицы на число k={k}')
MOut(MMult(A, k))

```

Результаты работы программы приведены в листинге 8.6.

Листинг 8.6

8	-10	10	3	-8	5
---	-----	----	---	----	---

9	-9	-9	-2	9	1
-7	-3	6	-1	-2	2
8	-1	-5	-7	3	10
9	-10	-9	-5	7	-6
10	-2	-9	-5	-2	-2
-3	1	-1	-6	-4	-2
-2	7	-4	-2	-4	8
Сумма матриц					
17	-20	1	-2	-1	-1
19	-11	-18	-7	7	-1
-10	-2	5	-7	-6	0
6	6	-9	-9	-1	18
Разность матриц					
-1	0	19	8	-15	11
-1	-7	0	3	11	3
-4	-4	7	5	2	4
10	-8	-1	-5	7	2
Умножение первой матрицы на число k=3					
24	-30	30	9	-24	15
27	-27	-27	-6	27	3
-21	-9	18	-3	-6	6
24	-3	-15	-21	9	30

Пример 8.6

Реализовать подпрограмму умножения матриц $A_{n \times m}$ и $B_{n \times p}$. Предусмотреть проверку размерностей матриц.

Решение

Программный код решения приведен в листинге 8.7.

Листинг 8.7

```

from random import *

# Подпрограмма вывода двумерного списка
def MOut(a):
    for i in a:
        for j in i:
            print(f'{j:5d}', end='')
        print()

# Подпрограмма произведения матриц
def MProd(a, b):
    # Проверка размерностей
    if len(a[0]) != len(b):
        return False

```

```

# Создаем нулевую матрицу нужной размерности.
# В ней столько же строк, сколько в матрице a
# и столько же столбцов, сколько в матрице b
c = [[0 for j in range(len(b[0]))] for i in range(len(a))]

# Вычисление сумм произведений строки матрицы a на столбец
матрицы b
for i in range(len(a)):
    for j in range(len(b[0])):
        for r in range(len(a[0])):
            c[i][j] += a[i][r] * b[r][j]

return c

# Размерности матриц
n, m, p, k = 3, 4, 5
# Заполнение матриц случайными значениями
A = [[randint(-10, 10) for j in range(m)] for i in range(n)]
B = [[randint(-10, 10) for j in range(p)] for i in range(m)]

# Вывод матриц
MOut(A); print()
MOut(B); print()

# Вывод результата произведения
print('Произведение матриц')
MOut(MProd(A, B))

```

Результаты работы программы приведены в листинге 8.8.

Листинг 8.8

```

3      -4      1      2
4      -6     -2      7
-10     4     10     -9

-1     -3      6      7     10
5      -2      6      3      6
-10     2    -10     -6      4
-9      8     -5     10     -2

Произведение матриц
-51     17    -26     23      6
-77     52    -27     92    -18
11     -30    -91   -208    -18

```

Задание 8.2

Заданы матрицы $A_{n \times m}$, $B_{m \times p}$, $C_{p \times q}$, приведенные в табл. 8.2. Вычислить следующие матрицы и вывести их на экран с соответствующими пояснениями (см. примеры выше):

1. $A^T, (B + C)^T$.
 2. $(2A + 4B^T) - 5C$.
 3. $(A^2 + C^3)^T$.
 4. След матрицы $(A + B + C)$.
 5. Матрицу, полученную присоединением единичной матрицы к C .
 6. Матрицу из матрицы A , полученную перестановкой строк с индексами i и j ($i \neq j$). Значения i и j задать самостоятельно.
 7. Матрицу из матрицы A , полученную перестановкой столбцов с индексами i и j ($i \neq j$). Значения i и j задать самостоятельно.
 8. Транспонированную матрицу B .
 9. Список из минимальных элементов столбцов матрицы C .
 10. Список из максимальных элементов столбцов матрицы B .
 11. Матрицу из матрицы A , полученную умножением строки i на число k и сложенную со строкой j ($i \neq j$). Значения i, j, k задать литерально.
 12. Матрицу из матрицы B , полученную умножением столбца i на число k и сложенную со столбцом j ($i \neq j$). Значения i, j, k задать литерально.
- Содержание отчета:
- полная формулировка задания;
 - программный код с обязательными пояснениями в виде комментариев.
 - результаты работы программы.

Таблица 8.2

№	Матрица A	Матрица B	Матрица C
1	$\begin{pmatrix} -1 & 8 & 1 & 0 & 0 \\ -8 & 60 & 9 & 6 & -3 \\ 5 & -60 & -7 & 25 & -16 \\ -10 & 108 & 17 & -32 & 16 \\ -3 & 60 & -69 & -99 & 76 \end{pmatrix}$	$\begin{pmatrix} 3 & 4 & 6 & -9 & 6 \\ -3 & -14 & 1 & 5 & 3 \\ -21 & -18 & -44 & 70 & -50 \\ 9 & 92 & -73 & -26 & -59 \\ 12 & -74 & 37 & -172 & 113 \end{pmatrix}$	$\begin{pmatrix} -1 & 1 & -4 & -10 & -6 \\ -3 & 10 & -15 & -26 & -18 \\ -6 & -36 & -11 & -92 & -46 \\ 6 & 8 & 43 & 109 & 77 \\ 7 & -70 & 75 & 60 & 146 \end{pmatrix}$
2	$\begin{pmatrix} 10 & -4 & -8 & 10 & -10 \\ 70 & -34 & -51 & 70 & -62 \\ 40 & -10 & -27 & 33 & -38 \\ 70 & -52 & 14 & 36 & 15 \\ -90 & 84 & 52 & -99 & 61 \end{pmatrix}$	$\begin{pmatrix} 4 & 8 & 8 & 9 & 10 \\ 16 & 40 & 34 & 29 & 46 \\ 16 & 40 & 35 & 37 & 49 \\ -8 & 0 & -7 & 5 & 3 \\ 16 & -40 & 8 & 48 & -34 \end{pmatrix}$	$\begin{pmatrix} 5 & -7 & 8 & 6 & -1 \\ -10 & 21 & -25 & -4 & -8 \\ 15 & 7 & -20 & 42 & -37 \\ 15 & -63 & 142 & 43 & 18 \\ -45 & 42 & -5 & 34 & 89 \end{pmatrix}$
3	$\begin{pmatrix} 4 & 2 & 1 & 1 & -3 \\ -36 & -21 & -4 & -6 & 34 \\ 12 & 18 & -16 & -9 & -33 \\ -40 & -20 & -13 & -14 & 15 \\ 0 & 21 & -35 & 19 & -9 \end{pmatrix}$	$\begin{pmatrix} -3 & 8 & 9 & 8 & -8 \\ 12 & -32 & -38 & -28 & 41 \\ 9 & -24 & -31 & -10 & 50 \\ -6 & 16 & 14 & 14 & -4 \\ -3 & 8 & -5 & -74 & -24 \end{pmatrix}$	$\begin{pmatrix} 2 & 0 & -2 & 0 & 8 \\ 0 & 6 & 2 & -4 & 2 \\ 2 & 60 & 28 & -47 & 18 \\ -18 & 30 & 38 & -30 & -74 \\ -16 & 0 & 116 & -58 & -160 \end{pmatrix}$

№	Матрица A	Матрица B	Матрица C
4	$\begin{pmatrix} -2 & 7 & 4 & 5 & 3 \\ -2 & 15 & 8 & 14 & -4 \\ 14 & -97 & -53 & -85 & 12 \\ 0 & 0 & 0 & 1 & 9 \\ 18 & 1 & -3 & 20 & -101 \end{pmatrix}$	$\begin{pmatrix} -1 & -7 & 3 & -5 & -6 \\ 3 & 23 & -2 & 8 & 15 \\ 10 & 84 & 27 & -2 & 29 \\ -6 & -42 & 2 & -18 & -26 \\ 5 & 35 & -7 & 10 & 44 \end{pmatrix}$	$\begin{pmatrix} 1 & 5 & 0 & 10 & -10 \\ 6 & 39 & -4 & 64 & -68 \\ -3 & -33 & 11 & -35 & 46 \\ -4 & 7 & 0 & -13 & 21 \\ 1 & 50 & 4 & 57 & -40 \end{pmatrix}$
5	$\begin{pmatrix} 8 & 5 & 9 & -6 & -7 \\ 72 & 40 & 84 & -59 & -56 \\ -80 & -40 & -101 & 74 & 61 \\ 40 & 0 & 65 & -60 & -14 \\ 48 & -5 & 45 & -52 & -3 \end{pmatrix}$	$\begin{pmatrix} 7 & -7 & 1 & -7 & 8 \\ 70 & -70 & 0 & -76 & 71 \\ 56 & -56 & -7 & -66 & 48 \\ -63 & 63 & 16 & 73 & -76 \\ 70 & -70 & 100 & -12 & 162 \end{pmatrix}$	$\begin{pmatrix} 10 & -4 & -8 & 10 & -10 \\ 70 & -34 & -51 & 70 & -62 \\ 40 & -10 & -27 & 33 & -38 \\ 70 & -52 & 14 & 36 & 15 \\ -90 & 84 & 52 & -99 & 61 \end{pmatrix}$
6	$\begin{pmatrix} 9 & -8 & 10 & -9 & 10 \\ -63 & 47 & -63 & 73 & -67 \\ 18 & -70 & 53 & 37 & 28 \\ -45 & 121 & -104 & -43 & -66 \\ 9 & 64 & -1 & -85 & 41 \end{pmatrix}$	$\begin{pmatrix} -8 & 9 & -4 & 3 & 10 \\ 80 & -96 & 38 & -34 & -109 \\ 72 & -135 & 20 & -62 & -161 \\ 32 & -36 & 2 & -26 & -102 \\ 40 & -87 & 2 & -73 & -107 \end{pmatrix}$	$\begin{pmatrix} 4 & -6 & 2 & 5 & 2 \\ 12 & -18 & 11 & 15 & 2 \\ 32 & -48 & 4 & 36 & 25 \\ -20 & 30 & -11 & 11 & -5 \\ -36 & 54 & 12 & 31 & -42 \end{pmatrix}$
7	$\begin{pmatrix} 4 & -6 & 2 & 5 & 2 \\ 12 & -18 & 11 & 15 & 2 \\ 32 & -48 & 4 & 36 & 25 \\ -20 & 30 & -11 & 11 & -5 \\ -36 & 54 & 12 & 31 & -42 \end{pmatrix}$	$\begin{pmatrix} 9 & 9 & 7 & 10 & -5 \\ -90 & -92 & -61 & -107 & 51 \\ 27 & 13 & 92 & -21 & -3 \\ 45 & 29 & 91 & -12 & -17 \\ 36 & 32 & 70 & 100 & -79 \end{pmatrix}$	$\begin{pmatrix} -7 & -8 & -9 & 8 & -3 \\ 49 & 48 & 57 & -55 & 31 \\ 35 & 72 & 64 & -43 & -17 \\ -14 & -16 & 32 & 14 & -96 \\ -28 & -56 & -44 & 65 & -44 \end{pmatrix}$
8	$\begin{pmatrix} 9 & -1 & 2 & 3 & -10 \\ -63 & 4 & -7 & -15 & 75 \\ -36 & -8 & 17 & 9 & 53 \\ 90 & 11 & -44 & -28 & -174 \\ -63 & 34 & -101 & -96 & -15 \end{pmatrix}$	$\begin{pmatrix} 3 & 2 & 5 & -3 & 5 \\ 24 & 10 & 31 & -29 & 39 \\ 9 & 36 & 56 & 22 & 10 \\ 24 & -2 & -3 & -10 & -5 \\ -12 & -8 & -52 & 35 & -97 \end{pmatrix}$	$\begin{pmatrix} 2 & -1 & -8 & 6 & 4 \\ -12 & -3 & 52 & -30 & -30 \\ -8 & 13 & 25 & -24 & -14 \\ 14 & 20 & -53 & 2 & 57 \\ 10 & -95 & 3 & 92 & -50 \end{pmatrix}$
9	$\begin{pmatrix} 4 & -6 & 2 & 5 & 2 \\ 12 & -18 & 11 & 15 & 2 \\ 32 & -48 & 4 & 36 & 25 \\ -20 & 30 & -11 & 11 & -5 \\ -36 & 54 & 12 & 31 & -42 \end{pmatrix}$	$\begin{pmatrix} -5 & 0 & 10 & -4 & 0 \\ 20 & 9 & -30 & 18 & 9 \\ -20 & -90 & -69 & -30 & -81 \\ 40 & -72 & -178 & 25 & -64 \\ -5 & 0 & -71 & 47 & 67 \end{pmatrix}$	$\begin{pmatrix} 10 & -6 & -2 & 3 & -4 \\ -10 & 15 & 5 & -10 & 10 \\ 0 & 27 & 1 & -24 & 22 \\ 60 & 54 & -14 & -65 & 55 \\ 70 & 39 & 5 & -42 & 31 \end{pmatrix}$
10	$\begin{pmatrix} 6 & 6 & 6 & -5 & -7 \\ -42 & -37 & -48 & 37 & 51 \\ 30 & -10 & 69 & -48 & -49 \\ 24 & -1 & 36 & -42 & -27 \\ -42 & -32 & -36 & 37 & -4 \end{pmatrix}$	$\begin{pmatrix} 9 & 9 & -6 & -7 & 3 \\ -36 & -45 & 15 & 19 & -14 \\ 27 & 108 & 63 & 70 & 26 \\ 36 & 81 & 21 & 31 & 25 \\ -63 & -9 & 96 & -17 & -32 \end{pmatrix}$	$\begin{pmatrix} -7 & -5 & 0 & 1 & 5 \\ 35 & 22 & 2 & -9 & -23 \\ -28 & -5 & -5 & 28 & 16 \\ -21 & 15 & -40 & 28 & -22 \\ -56 & -31 & -6 & 21 & 35 \end{pmatrix}$
11	$\begin{pmatrix} 3 & -8 & 4 & -6 & -4 \\ 18 & -46 & 15 & -36 & -23 \\ -3 & -4 & 48 & 14 & 3 \\ -15 & 42 & -23 & 7 & 16 \\ 24 & -74 & 81 & -59 & 4 \end{pmatrix}$	$\begin{pmatrix} -1 & 10 & 3 & 0 & 4 \\ -3 & 28 & 18 & 8 & 21 \\ 9 & -86 & -45 & -11 & -59 \\ 9 & -94 & -9 & 16 & 0 \\ 0 & 12 & -54 & -43 & -77 \end{pmatrix}$	$\begin{pmatrix} 5 & 1 & 2 & 9 & -7 \\ 5 & 3 & -5 & 3 & -16 \\ -15 & -1 & -19 & -29 & 22 \\ -5 & 1 & 51 & -46 & -107 \\ -45 & -1 & -40 & -154 & 52 \end{pmatrix}$
12	$\begin{pmatrix} -5 & 5 & -5 & -10 & 6 \\ 25 & -21 & 22 & 60 & -39 \\ -40 & 28 & -37 & -115 & 75 \\ -45 & 65 & -90 & -71 & 1 \\ -35 & 19 & 25 & -106 & 33 \end{pmatrix}$	$\begin{pmatrix} 2 & 9 & 4 & 1 & -5 \\ -4 & -17 & -5 & 0 & 18 \\ -10 & -54 & -38 & -21 & -48 \\ -2 & -19 & 47 & -8 & -92 \\ 6 & 28 & 51 & 58 & 59 \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 9 & -5 & -1 \\ 7 & 3 & 69 & -27 & -16 \\ -9 & -21 & -70 & 74 & -10 \\ -10 & -50 & -72 & 168 & -25 \\ -8 & -8 & -30 & 66 & 4 \end{pmatrix}$

№	Матрица A	Матрица B	Матрица C
13	$\begin{pmatrix} 5 & 8 & 9 & 7 & 9 \\ -10 & -22 & -12 & -20 & -16 \\ 50 & 74 & 96 & 68 & 92 \\ 20 & 56 & 12 & 56 & 36 \\ -35 & -26 & -93 & 61 & -155 \end{pmatrix}$	$\begin{pmatrix} 9 & -9 & -10 & -7 & -5 \\ 36 & -26 & -46 & -18 & -11 \\ 36 & 24 & -81 & 38 & 42 \\ -18 & 8 & 71 & -56 & -74 \\ -9 & -1 & 16 & -51 & -36 \end{pmatrix}$	$\begin{pmatrix} -1 & 0 & 3 & -6 & 7 \\ -7 & -7 & 13 & -37 & 51 \\ -8 & -7 & 6 & -34 & 65 \\ 8 & -21 & -48 & 66 & -44 \\ 2 & -7 & -4 & -10 & -60 \end{pmatrix}$
14	$\begin{pmatrix} 2 & 2 & -7 & 0 & 10 \\ -4 & 4 & 13 & -5 & -19 \\ 8 & -40 & -21 & 28 & 42 \\ -10 & -66 & 47 & 27 & -9 \\ -4 & 28 & 17 & -18 & 99 \end{pmatrix}$	$\begin{pmatrix} -3 & 6 & 7 & 2 & -10 \\ 27 & -59 & -71 & -23 & 80 \\ 3 & -56 & -79 & -46 & -87 \\ 30 & -105 & -134 & -65 & 14 \\ 15 & -20 & -83 & -36 & 37 \end{pmatrix}$	$\begin{pmatrix} 8 & 7 & -4 & -3 & 2 \\ -80 & -63 & 31 & 40 & -18 \\ -56 & -91 & 78 & -46 & -25 \\ -8 & -35 & 64 & 2 & -20 \\ 64 & -7 & 89 & -47 & -21 \end{pmatrix}$
15	$\begin{pmatrix} -7 & 4 & -1 & -10 & 7 \\ -56 & 24 & -3 & -81 & 47 \\ 28 & 8 & -8 & 42 & 3 \\ -49 & 84 & -12 & -75 & 162 \\ 28 & -80 & 20 & 26 & -62 \end{pmatrix}$	$\begin{pmatrix} 1 & 5 & 6 & -4 & 0 \\ 3 & 13 & 14 & -2 & -5 \\ -3 & -25 & -44 & 71 & -17 \\ -9 & -57 & -24 & 21 & -92 \\ -5 & -37 & -54 & 26 & -125 \end{pmatrix}$	$\begin{pmatrix} 3 & -1 & -2 & 7 & 5 \\ -15 & -1 & 13 & -35 & -35 \\ 9 & 27 & -21 & 29 & 65 \\ 9 & 39 & -27 & -43 & 77 \\ 30 & 8 & -29 & 62 & 44 \end{pmatrix}$
16	$\begin{pmatrix} -9 & -1 & -6 & -3 & 10 \\ 63 & 13 & 40 & 27 & -66 \\ -18 & -38 & -9 & -39 & -7 \\ 72 & 62 & -24 & 88 & -69 \\ 0 & 24 & 64 & 80 & 107 \end{pmatrix}$	$\begin{pmatrix} 6 & 5 & 7 & 9 & -3 \\ 6 & 15 & 5 & 19 & -7 \\ 60 & 60 & 64 & 97 & -32 \\ 48 & 130 & 54 & 172 & -67 \\ -12 & -70 & -6 & -79 & 37 \end{pmatrix}$	$\begin{pmatrix} -2 & 10 & 7 & -10 & 0 \\ -8 & 46 & 29 & -39 & -8 \\ 12 & -48 & -47 & 53 & -25 \\ 10 & -8 & 42 & 144 & 35 \\ 20 & -52 & -111 & 39 & -117 \end{pmatrix}$
17	$\begin{pmatrix} -1 & 0 & 3 & -6 & 7 \\ -7 & -7 & 13 & -37 & 51 \\ -8 & -7 & 6 & -34 & 65 \\ 8 & -21 & -48 & 66 & -44 \\ 2 & -7 & -4 & -10 & -60 \end{pmatrix}$	$\begin{pmatrix} -10 & -7 & -5 & -4 & 9 \\ 80 & 64 & 40 & 25 & -68 \\ 20 & 14 & 6 & 12 & -24 \\ 30 & -19 & -1 & 60 & -80 \\ 90 & 71 & 5 & 78 & -112 \end{pmatrix}$	$\begin{pmatrix} 4 & -9 & 3 & -2 & 1 \\ -4 & 13 & 0 & -1 & -8 \\ 24 & -34 & 41 & -28 & -23 \\ -36 & 65 & -63 & 39 & 6 \\ -8 & 42 & -52 & -42 & -118 \end{pmatrix}$
18	$\begin{pmatrix} -3 & -9 & -5 & 10 & -6 \\ -9 & -17 & -11 & 21 & -21 \\ -21 & -163 & -68 & 157 & -9 \\ -9 & 43 & -1 & -20 & -50 \\ 0 & -50 & -27 & 13 & 34 \end{pmatrix}$	$\begin{pmatrix} 9 & -8 & 8 & 5 & 2 \\ -18 & 23 & -10 & 0 & -8 \\ 90 & -80 & 79 & 46 & 15 \\ 45 & 16 & 81 & 79 & -57 \\ -54 & 83 & -22 & 10 & -45 \end{pmatrix}$	$\begin{pmatrix} 7 & 6 & 6 & -10 & -9 \\ 0 & -2 & -3 & -9 & -2 \\ 7 & -8 & -20 & -67 & -13 \\ 35 & 46 & 104 & -28 & -129 \\ -28 & -22 & -46 & 139 & 95 \end{pmatrix}$
19	$\begin{pmatrix} 4 & 6 & -5 & 4 & 10 \\ -12 & -24 & 21 & -5 & -25 \\ -16 & -66 & 60 & 32 & -3 \\ 32 & 72 & -74 & -10 & 60 \\ 32 & 30 & -30 & 76 & 131 \end{pmatrix}$	$\begin{pmatrix} 6 & -5 & -8 & -1 & 6 \\ 60 & -54 & -80 & -20 & 66 \\ -18 & -17 & 34 & -78 & 21 \\ -18 & 31 & -6 & 46 & -22 \\ -24 & -4 & 42 & -57 & 56 \end{pmatrix}$	$\begin{pmatrix} -5 & -9 & 0 & -9 & 4 \\ 15 & 28 & -2 & 33 & -4 \\ 20 & 39 & -7 & 56 & 8 \\ -25 & -45 & 6 & -67 & 29 \\ -25 & -52 & 17 & -23 & -92 \end{pmatrix}$
20	$\begin{pmatrix} 0 & 7 & 6 & -6 & -5 \\ 0 & 3 & 16 & 4 & -9 \\ 0 & 35 & -58 & -86 & 18 \\ 0 & -48 & 68 & -3 & -43 \\ 0 & 54 & -48 & 66 & 65 \end{pmatrix}$	$\begin{pmatrix} 8 & 0 & -4 & -10 & 7 \\ 0 & -6 & 6 & -3 & 4 \\ 8 & -30 & 31 & -33 & 37 \\ -48 & -60 & 89 & 31 & 16 \\ -64 & -6 & -7 & 95 & -181 \end{pmatrix}$	$\begin{pmatrix} 7 & -5 & 4 & 1 & 5 \\ 63 & -38 & 27 & 10 & 54 \\ 56 & -12 & -1 & 15 & 77 \\ -49 & 21 & -31 & -22 & -57 \\ 21 & 41 & -72 & 7 & 92 \end{pmatrix}$
21	$\begin{pmatrix} -1 & 1 & 7 & 4 & 4 \\ -10 & 12 & 77 & 30 & 32 \\ -1 & -5 & -17 & 24 & 38 \\ -7 & -13 & -48 & 43 & 191 \\ -3 & 11 & 67 & 47 & -102 \end{pmatrix}$	$\begin{pmatrix} 4 & -1 & 5 & -2 & 5 \\ -40 & 3 & -46 & 15 & -59 \\ -28 & 14 & -40 & 29 & -24 \\ 4 & 48 & -32 & 127 & 95 \\ -40 & -11 & -34 & -7 & -16 \end{pmatrix}$	$\begin{pmatrix} 2 & 0 & -5 & -4 & 0 \\ 10 & 1 & -30 & -26 & 3 \\ -2 & 0 & 5 & 12 & -7 \\ 4 & 2 & -20 & 4 & -19 \\ 4 & -6 & 20 & 84 & -22 \end{pmatrix}$

№	Матрица A	Матрица B	Матрица C
22	$\begin{pmatrix} -5 & 3 & -1 & 1 & -9 \\ 20 & -20 & 2 & -14 & 33 \\ 50 & -14 & 13 & 6 & 89 \\ 10 & 18 & 7 & 19 & 20 \\ 20 & 28 & 6 & 24 & -1 \end{pmatrix}$	$\begin{pmatrix} 9 & -5 & 4 & -5 & 0 \\ 45 & -35 & 10 & -27 & 4 \\ 45 & 25 & 75 & -14 & -14 \\ 36 & 30 & 96 & -10 & 21 \\ -9 & -5 & -29 & 48 & -48 \end{pmatrix}$	$\begin{pmatrix} -2 & -3 & 0 & 7 & -9 \\ 6 & 19 & 10 & -24 & 19 \\ 2 & 73 & 73 & -37 & -54 \\ -10 & -45 & -6 & -38 & -70 \\ 2 & -17 & -41 & 152 & 20 \end{pmatrix}$
23	$\begin{pmatrix} 3 & 6 & -9 & 3 & -1 \\ -24 & -56 & 75 & -16 & 8 \\ 9 & -62 & 12 & 84 & 3 \\ 30 & 12 & -108 & 97 & -29 \\ -21 & -98 & -6 & 94 & -102 \end{pmatrix}$	$\begin{pmatrix} -10 & 10 & 8 & -3 & 2 \\ 50 & -45 & -44 & 9 & -15 \\ 0 & 35 & -22 & -43 & -33 \\ -40 & 80 & 6 & -65 & -25 \\ -30 & 30 & -12 & 9 & -11 \end{pmatrix}$	$\begin{pmatrix} 9 & 9 & 10 & 9 & -4 \\ -27 & -29 & -36 & -30 & 13 \\ -45 & -31 & -3 & -24 & 21 \\ 0 & -10 & -10 & -7 & 40 \\ -63 & -51 & 11 & -13 & 99 \end{pmatrix}$
24	$\begin{pmatrix} 1 & -10 & 2 & -4 & -5 \\ 8 & -73 & 16 & -36 & -38 \\ -5 & 106 & -1 & -11 & 37 \\ 5 & 6 & 28 & -50 & -17 \\ 7 & -42 & -49 & -51 & -7 \end{pmatrix}$	$\begin{pmatrix} -2 & -3 & -5 & 9 & 4 \\ -2 & -1 & -15 & 6 & 11 \\ -10 & -5 & -71 & 35 & 46 \\ 20 & 26 & 62 & -103 & -37 \\ 14 & 21 & 71 & -81 & -125 \end{pmatrix}$	$\begin{pmatrix} -5 & -4 & 1 & 2 & -10 \\ 15 & 15 & -4 & -5 & 38 \\ -10 & 10 & -2 & 16 & 33 \\ 50 & 58 & -10 & 4 & 161 \\ 20 & 34 & -8 & 4 & 104 \end{pmatrix}$
25	$\begin{pmatrix} 9 & -2 & -7 & 6 & 8 \\ 9 & 2 & -8 & -3 & 15 \\ 90 & -52 & -58 & 126 & 16 \\ -18 & 4 & 42 & -54 & -80 \\ 54 & -4 & -36 & 6 & 45 \end{pmatrix}$	$\begin{pmatrix} -6 & -9 & -4 & 3 & 9 \\ -12 & -28 & -16 & 10 & 22 \\ 48 & 102 & 59 & -39 & -81 \\ -54 & -101 & -82 & 70 & 62 \\ -42 & 27 & 23 & 6 & 4 \end{pmatrix}$	$\begin{pmatrix} 9 & 7 & -4 & 6 & 7 \\ -27 & -18 & 17 & -13 & -19 \\ 27 & -3 & -45 & -24 & 12 \\ 45 & 65 & 65 & 65 & 97 \\ 27 & 18 & -45 & 61 & -64 \end{pmatrix}$
26	$\begin{pmatrix} -5 & -9 & 0 & -9 & 4 \\ 15 & 28 & -2 & 33 & -4 \\ 20 & 39 & -7 & 56 & 8 \\ -25 & -45 & 6 & -67 & 29 \\ -25 & -52 & 17 & -23 & -92 \end{pmatrix}$	$\begin{pmatrix} 1 & 8 & 3 & 8 & -2 \\ -3 & -27 & -9 & -20 & 11 \\ -4 & -35 & -5 & -20 & 18 \\ 0 & 24 & -63 & -95 & -90 \\ -1 & -5 & 53 & 133 & -15 \end{pmatrix}$	$\begin{pmatrix} 4 & 8 & -4 & 8 & 0 \\ 24 & 38 & -21 & 55 & 3 \\ 32 & 24 & -14 & 101 & 6 \\ 0 & -10 & 33 & 47 & -28 \\ 12 & -56 & 18 & 44 & 2 \end{pmatrix}$
27	$\begin{pmatrix} -9 & 5 & -1 & 9 & -3 \\ 27 & -21 & 4 & -26 & 6 \\ 0 & 30 & -1 & -7 & 19 \\ -27 & 21 & -32 & 40 & -37 \\ -18 & 4 & -17 & 27 & -15 \end{pmatrix}$	$\begin{pmatrix} -9 & 4 & 8 & 2 & 10 \\ 36 & -19 & -23 & 2 & -39 \\ 0 & 6 & -12 & -21 & -12 \\ 27 & 3 & -99 & -44 & 21 \\ 9 & -19 & 67 & -27 & -114 \end{pmatrix}$	$\begin{pmatrix} -5 & -5 & 1 & 2 & 2 \\ 35 & 36 & -3 & -16 & -22 \\ 35 & 33 & -25 & -19 & -4 \\ -10 & -3 & 60 & 26 & -35 \\ -35 & -39 & 81 & 49 & 96 \end{pmatrix}$
28	$\begin{pmatrix} -6 & -9 & -4 & 3 & 9 \\ -12 & -28 & -16 & 10 & 22 \\ 48 & 102 & 59 & -39 & -81 \\ -54 & -101 & -82 & 70 & 62 \\ -42 & 27 & 23 & 6 & 4 \end{pmatrix}$	$\begin{pmatrix} -6 & -5 & 10 & -10 & -3 \\ 18 & 13 & -40 & 30 & 16 \\ -30 & -25 & 59 & -47 & -23 \\ 42 & 47 & -10 & 72 & -13 \\ 18 & 11 & -14 & 22 & -87 \end{pmatrix}$	$\begin{pmatrix} -10 & -2 & 2 & -3 & 0 \\ 30 & -3 & -4 & 4 & -9 \\ 100 & 11 & -8 & 31 & -7 \\ 90 & -9 & -62 & -9 & -29 \\ -70 & -14 & 54 & -60 & -50 \end{pmatrix}$
29	$\begin{pmatrix} -5 & -9 & 4 & 0 & 9 \\ -35 & -55 & 22 & 3 & 69 \\ 10 & 42 & -21 & 2 & -6 \\ 40 & 120 & -28 & -42 & -92 \\ -50 & -10 & -20 & 62 & 211 \end{pmatrix}$	$\begin{pmatrix} -5 & 8 & -6 & 8 & -3 \\ 30 & -57 & 42 & -50 & 21 \\ -50 & 80 & -60 & 80 & -21 \\ 5 & 28 & -18 & 1 & 72 \\ -40 & -8 & 0 & 47 & 18 \end{pmatrix}$	$\begin{pmatrix} 1 & -3 & 9 & -10 & -4 \\ -5 & 13 & -38 & 43 & 11 \\ -2 & 2 & -9 & 14 & -15 \\ -3 & 13 & -1 & -20 & 63 \\ -8 & 16 & -89 & 124 & -39 \end{pmatrix}$
30	$\begin{pmatrix} -2 & 9 & -8 & -10 & 5 \\ 12 & -57 & 46 & 55 & -38 \\ 10 & -39 & 43 & 51 & -1 \\ 18 & -75 & 77 & 112 & -27 \\ 18 & -102 & 66 & 148 & -87 \end{pmatrix}$	$\begin{pmatrix} 7 & -7 & 8 & -2 & 0 \\ -35 & 36 & -41 & 8 & 2 \\ -70 & 72 & -74 & 20 & -1 \\ -70 & 74 & -36 & 42 & -23 \\ -35 & 45 & -98 & 2 & 34 \end{pmatrix}$	$\begin{pmatrix} 10 & 6 & -8 & -4 & -1 \\ -90 & -53 & 67 & 34 & 15 \\ -20 & -7 & -13 & 6 & 23 \\ 10 & 14 & -36 & -45 & 71 \\ -20 & -5 & -31 & 24 & 38 \end{pmatrix}$

Пример 8.7

Пусть дана матрица

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} 2 & 1 & 3 \\ 3 & 0 & 2 \\ -5 & 3 & 1 \end{pmatrix}.$$

Требуется привести матрицу A к верхнетреугольному виду с целочисленными значениями методом Гаусса и вычислить ее определитель по формуле (8.5) без программной реализации.

Решение

Элемент главной диагонали a_{kk} , ниже которого требуется получить нулевые значения, в математике называется *ведущим элементом*. Если он нулевой, то ищется такая строка, в которой элемент, расположенный ниже ведущего, ненулевой. Если такая строка существует, то эта строка и строка с ведущим элементом меняются местами (при этом знак определителя меняется на противоположный, значение s в формуле (8.5) увеличивается на 1). Если же ведущий элемент a_{kk} не равен 0, то для получения нулевого элемента ниже него в строке i требуется умножить все элементы строки k на некоторое число (скаляр) mul_k и сложить со строкой i . Результат сложения и будет строка i в матрице A .

Чтобы матрица после элементарных преобразований оставалась целочисленной, требуется умножить все элементы строки i на такое число mul_i , чтобы при сложении со строкой k , умноженной на mul_k , элемент a_{ik} стал нулевым. Значения mul_k и mul_i вычисляются следующим образом:

1. Находится наименьшее общее кратное (НОК) значений a_{kk} и a_{ik} :

$$l = \text{НОК}(|a_{kk}|, |a_{ik}|).$$

2. Абсолютные значения коэффициентов умножения mul_k и mul_i будут равны

$$mul_k = l / |a_{kk}|, \quad mul_i = l / |a_{ik}|.$$

3. Знак $sign$ коэффициента mul_i должен быть таким, чтобы при сложении строк элемент a_{ik} стал нулевым, т. е.

$$sign = \begin{cases} -1, & \text{если } a_{kk} \cdot a_{ik} > 0; \\ 0, & \text{иначе.} \end{cases}$$

В итоге элементы строки i вычисляются как

$$a'_{ik} = a_{kk} \cdot mul_k + sign \cdot a_{ik} \cdot mul_i.$$

Чтобы в рассматриваемом примере получить нулевые элементы ниже ведущего элемента $a_{11} = 2$, следует умножить первую строку на $mul_k = 3$ и сложить со второй строкой, умноженной на $mul_i = -2$, а так же умножить первую

строку на $mul_k = 5$ и сложить со второй строкой, умноженной на $mul_i = 2$. В результате получим

$$A_1 = \begin{pmatrix} 2 & 1 & 3 \\ 0 & 3 & 5 \\ 0 & 11 & 17 \end{pmatrix}.$$

Теперь все элементы ниже ведущего элемента $a_{11} = 2$ нулевые.

Следующий ведущий элемент $a_{22} = 3$. Чтобы получить нулевые элементы ниже него, следует умножить вторую строку на $mul_k = 11$ и сложить со второй строкой, умноженной на $mul_i = -3$. В результате получим верхнетреугольную матрицу

$$U = \begin{pmatrix} 2 & 1 & 3 \\ 0 & 3 & 5 \\ 0 & 0 & 4 \end{pmatrix}.$$

Для вычисления определителя воспользуемся формулой (8.5). Перестановок строк не производилось, поэтому $s = 0$. Коэффициенты u_{ii} в формуле — это элементы главной диагонали матрицы U . Коэффициенты k_i в формуле — это множители mul_i , на которые были умножены строки i при выполнении элементарных преобразований. Для рассматриваемого примера это коэффициенты -2, 2, -3. Таким образом

$$\det A = (-1)^0 \frac{1}{-2 \cdot 2 \cdot (-3)} (2 \cdot 3 \cdot 4) = 2.$$

Пример 8.8

Реализовать подпрограмму приведения квадратной матрицы размерности n к верхнетреугольному виду (элементы ниже главной диагонали равны 0) с целочисленными коэффициентами. Вывести на исходную и верхнетреугольную матрицу. С использованием этой подпрограммы вычислить определитель матрицы. Выполнить проверку вычислений с использованием модуля numpy.

Решение

Программный код решения приведен в листинге 8.9.

Листинг 8.9

```
import numpy as np
from math import *
from copy import deepcopy

# Подпрограмма вывода двумерного списка
def MOut(a):
    for i in a:
        for j in i:
```

```

        print(f"{j:8d}", end="")
    print()
print()

# Подпрограмма приведения матрицы к верхнетреугольному виду
def MUpperT(matrix):
    a = deepcopy(matrix)
    n = len(a)
    koef = []

    # Блок А
    for k in range(n):
        # Поиск ведущего элемента v
        if a[k][k] == 0:
            for i in range(k + 1, n):
                if a[i][k] != 0:
                    a[k], a[i] = a[i], a[k]
                    koef.append(-1)
                    break

        # Блок В
        v = a[k][k]
        if v == 0:
            continue # пропускаем нулевой столбец

        # Блок С
        for i in range(k + 1, n):
            if a[i][k] == 0:
                continue

            # Блок D
            # Приводим к общему знаменателю
            l = lcm(abs(a[i][k]), abs(v))
            mul_i = l // abs(a[i][k])
            mul_k = l // abs(v)
            sign = -1 if (a[i][k] * v > 0) else 1
            koef.append(mul_i)

            for j in range(k, n):
                a[i][j] = a[i][j] * mul_i + sign * a[k][j] *
mul_k
    return a, koef

# Основная программа
# Пример матрицы
A = [
    [0, -3, 6, 1, -8],
    [-2, 9, -8, 4, 0],
    [1, -7, 6, 0, -3],
    [3, 9, 4, 5, -1],
    [2, -6, 6, 8, -2],
]

print("Изначальная матрица:")

```

```

MOut (A)

U, p = MUupperT(A)
print("Верхнетреугольная форма (целые значения):")
MOut (U)

# Произведение элементов главной диагонали матрицы U
pU = prod([U[i][i] for i in range(len(U))])
# Произведение коэффициентов mul_i
pKcoef = prod([i for i in p])
det = pU // pKcoef
print("Определитель матрицы:", det)

# Проверка через модуль numpy
# Вычисляем определитель
B = np.array(A)
det1 = np.linalg.det(B)
print("Определитель матрицы (numpy):", det1)

```

Пояснения к программному коду:

1. При приведении матрицы к верхнетреугольному виду используются элементарные преобразования: перестановка строк и умножение строки на число с последующим сложением с другой строкой. При этом определитель матрицы не изменяется.

2. Если элемент a_{kk} главной диагонали матрицы A не равен 0, то требуется, чтобы все элементы ниже него были равны 0. Это достигается умножением строки k на некоторый коэффициент и сложением с некоторой строкой ниже. В блоке A программного кода последовательно перебираются все элементы главной диагонали ($a[k][k]$). Если этот элемент равен 0, то ищется такая строка i , в которой под элементом $a[k][k]$ стоит ненулевой элемент. Если такая строка найдена, то строки k и i меняются местами.

3. По элементу $a[k][k]$ (ведущий элемент, в программе это переменная v), будут определяться коэффициенты, на которые следует умножить строку k . Если ведущий элемент равен 0, это значит, что все элементы ниже $a[k][k]$ нулевые и никаких преобразований не требуется (блок B программного кода).

4. В блоке C происходит перебор строк, ниже строки k . Если $a[i][k]$ уже нулевой, то никаких преобразований не требуется и строка i пропускается. Иначе (блок D программного кода), чтобы числа в матрице оставались целыми, определяются два коэффициента mul_i и mul_k , такие, что при сложении строки k , умноженной на mul_k со строкой i , умноженной на mul_i , элемент $a[i][k]$ стал нулевым. Для этого вычисляется наименьшее общее кратное (переменная l) для $a[i][k]$ и v при помощи функции `lcm()` модуля `math`. Соответствующие коэффициенты

$$mul_i = l / a[i][k], \quad mul_k = l / a[k][k],$$

После этого строка i заменяется на строку, полученную сложением строки i , умноженной на mil_i со строкой k , умноженной на mul_k . Чтобы элемент $a[i][k]$ стал равным 0, определяется знак множителя (переменная $sign$): если v и $a[i][k]$ имеют одинаковые знаки, то строки нужно вычитать ($sign = -1$), а если разные, то строки нужно складывать ($sign = 1$).

6. Для проверки корректности вычислений используем модуль `numpy`, в котором имеется функция `linalg.det()`, позволяющая получить определитель.

Замечание. В модуле `numpy` при вычислении определителя не используется целочисленная арифметика, т. к. в общем случае значения элементов матрицы могут быть не целыми, поэтому результат имеет вещественный тип и небольшую погрешность.

Результат работы программы листинга 8.9 приведен в листинге 8.10.

Листинг 8.10

Изначальная матрица:

0	-3	6	1	-8
-2	9	-8	4	0
1	-7	6	0	-3
3	9	4	5	-1
2	-6	6	8	-2

Верхнетреугольная форма (целые значения):

-2	9	-8	4	0
0	-3	6	1	-8
0	0	-18	7	22
0	0	0	592	-284
0	0	0	0	9972

Определитель матрицы: 1108

Определитель матрицы (`numpy`): 1108.0000000000023

Задание 8.3

Привести матрицы A , B и C из табл. 8.2 к верхнетреугольному виду, вычислить их определители. Вычисление верхнетреугольной матрицы оформить в виде подпрограммы. Проверить корректность вычислений при помощи функции `linalg.det()` модуля `numpy`.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев.
- исходные матрицы;

- верхнетреугольные матрицы;
- определители матриц, вычисленный в программе, определители матриц, вычисленных через модуль numpy.

Задание 8.4

Определить ранг матриц A , B и C из табл. 8.3. Ранг матрицы — это количество ненулевых строк в матрице, приведенной к верхнетреугольному виду.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев.
- исходные матрицы;
- верхнетреугольные матрицы;
- ранги матриц.

Таблица 8.3

№	Матрица A	Матрица B	Матрица C
1	$\begin{pmatrix} 51 & 51 & 16 & 96 & 27 \\ 6 & 6 & -2 & 6 & -9 \\ 5 & 5 & 2 & 10 & 4 \\ -6 & 0 & 5 & 1 & -8 \\ -4 & -7 & 3 & -1 & 1 \end{pmatrix}$	$\begin{pmatrix} -19 & -1 & -25 & 1 & 8 \\ 0 & 10 & 4 & -8 & 7 \\ -5 & -3 & -5 & 7 & -10 \\ 7 & -1 & 10 & 3 & -9 \\ 58 & -12 & 85 & 34 & -91 \end{pmatrix}$	$\begin{pmatrix} 14 & 0 & 4 & -26 & -9 \\ 84 & -14 & 18 & -89 & -65 \\ -76 & 18 & -14 & 55 & 63 \\ -6 & 4 & 0 & -8 & 7 \\ -10 & 2 & -2 & 9 & 8 \end{pmatrix}$
2	$\begin{pmatrix} -6 & -1 & 3 & -4 & -4 \\ -4 & 5 & -6 & -8 & -3 \\ -34 & 34 & -39 & -60 & -25 \\ 2 & -3 & 8 & -5 & 9 \\ -6 & -1 & 3 & -4 & -4 \end{pmatrix}$	$\begin{pmatrix} -5 & -1 & 10 & -6 & -6 \\ 6 & 6 & -20 & 14 & 4 \\ -54 & -6 & 100 & -58 & -68 \\ -4 & 4 & 0 & 2 & -8 \\ 11 & 7 & -30 & 20 & 10 \end{pmatrix}$	$\begin{pmatrix} -6 & -4 & 0 & -6 & 4 \\ 0 & -3 & 7 & 0 & 9 \\ 9 & -10 & -6 & 9 & 1 \\ 27 & 2 & 30 & -3 & 61 \\ -3 & -2 & -6 & 2 & -10 \end{pmatrix}$
3	$\begin{pmatrix} 3 & -7 & -6 & -8 & 1 \\ 9 & -10 & -2 & -4 & -7 \\ 9 & -9 & -10 & 1 & -9 \\ -3 & 7 & -9 & -10 & 8 \\ 18 & -42 & 39 & 42 & -39 \end{pmatrix}$	$\begin{pmatrix} 15 & 39 & 44 & -41 & -51 \\ -10 & 3 & 9 & -4 & -3 \\ 0 & -1 & 9 & -6 & -6 \\ 3 & 8 & 7 & -7 & -9 \\ 6 & 15 & 23 & -20 & -24 \end{pmatrix}$	$\begin{pmatrix} 1 & -4 & 9 & -2 & -1 \\ 6 & -8 & 28 & -5 & 6 \\ 4 & 0 & 10 & -1 & 8 \\ 0 & 16 & -26 & 7 & 12 \\ 10 & -24 & 64 & -13 & 2 \end{pmatrix}$
4	$\begin{pmatrix} -99 & 15 & -78 & 15 & 66 \\ -72 & 12 & -54 & 12 & 48 \\ 9 & -1 & 8 & -1 & -6 \\ 10 & 6 & 0 & 3 & 0 \\ -9 & 5 & 2 & 5 & 6 \end{pmatrix}$	$\begin{pmatrix} -34 & -28 & -20 & 14 & 2 \\ -74 & -60 & -48 & 46 & 18 \\ -10 & -8 & -7 & 8 & 4 \\ -4 & -4 & 1 & -10 & -10 \\ -24 & -20 & -13 & 6 & -2 \end{pmatrix}$	$\begin{pmatrix} 1 & -5 & 0 & -5 & -8 \\ 7 & 4 & 9 & -5 & -10 \\ -1 & 10 & -2 & -9 & -3 \\ 18 & -33 & 9 & -50 & -62 \\ 10 & 7 & 9 & -10 & 2 \end{pmatrix}$
5	$\begin{pmatrix} 4 & -1 & -6 & -6 & 10 \\ 13 & 6 & -7 & -6 & 14 \\ -9 & -7 & 1 & 0 & -4 \\ 67 & 48 & -13 & -6 & 38 \\ 4 & -1 & -6 & -6 & 10 \end{pmatrix}$	$\begin{pmatrix} -4 & 1 & -9 & -4 & 4 \\ -6 & 4 & 10 & -3 & 5 \\ 8 & 7 & 4 & 3 & -1 \\ -6 & 1 & 9 & 1 & 6 \\ 12 & 1 & -17 & -6 & -13 \end{pmatrix}$	$\begin{pmatrix} -2 & -1 & 5 & -9 & 3 \\ -16 & -10 & 31 & -58 & 9 \\ -4 & -4 & 1 & -4 & -9 \\ 0 & -2 & -9 & 14 & -15 \\ 6 & 7 & 1 & -7 & 10 \end{pmatrix}$

6	$\begin{pmatrix} 19 & 20 & -31 & -33 & 10 \\ -33 & -110 & 86 & 71 & -3 \\ 15 & 10 & -22 & -25 & 9 \\ 4 & 10 & -9 & -8 & 1 \\ 7 & -10 & -4 & -9 & 7 \end{pmatrix}$	$\begin{pmatrix} 3 & -2 & -5 & -5 & -4 \\ -7 & -1 & -6 & 4 & 6 \\ -1 & 9 & -3 & -7 & 5 \\ -28 & 27 & 42 & 38 & 41 \\ -1 & 9 & -3 & -7 & 5 \end{pmatrix}$	$\begin{pmatrix} -2 & -8 & -7 & 0 & 4 \\ 2 & -2 & -5 & 9 & 6 \\ -2 & 0 & -3 & -10 & -10 \\ 32 & 10 & -59 & 3 & 24 \\ -5 & -2 & 9 & 1 & -3 \end{pmatrix}$
7	$\begin{pmatrix} 25 & 88 & 33 & -31 & -71 \\ 5 & 38 & 18 & -16 & -26 \\ -7 & 8 & 9 & -7 & 1 \\ -4 & -10 & -3 & 3 & 9 \\ 7 & 4 & 9 & 7 & 9 \end{pmatrix}$	$\begin{pmatrix} -11 & 34 & -1 & -5 & 31 \\ 3 & -1 & -8 & 9 & 10 \\ 2 & -5 & -1 & 2 & -3 \\ -9 & 29 & -2 & -3 & 28 \\ 9 & -16 & -11 & 15 & 1 \end{pmatrix}$	$\begin{pmatrix} 6 & -9 & 2 & -10 & 6 \\ -8 & -7 & 8 & 0 & -1 \\ -9 & 1 & 8 & -5 & 5 \\ 8 & 9 & 1 & 1 & 8 \\ -65 & -62 & 1 & -12 & -51 \end{pmatrix}$
8	$\begin{pmatrix} 73 & -42 & 19 & 35 & 82 \\ -7 & -2 & 9 & 5 & -8 \\ -71 & 30 & 1 & -19 & -80 \\ -8 & 4 & -1 & -3 & -9 \\ -23 & 6 & 7 & -1 & -26 \end{pmatrix}$	$\begin{pmatrix} 8 & 7 & -2 & -5 & -10 \\ -4 & 5 & 3 & 8 & -4 \\ -18 & 23 & -17 & 18 & -24 \\ -7 & 9 & -10 & 5 & -10 \\ -53 & 68 & -67 & 43 & -74 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 7 & 8 & -2 \\ 2 & 2 & -4 & 8 & 6 \\ 7 & -4 & 3 & -5 & 6 \\ 5 & 10 & 8 & 7 & -9 \\ 2 & 2 & -4 & 8 & 6 \end{pmatrix}$
9	$\begin{pmatrix} -7 & -10 & -4 & 4 & -9 \\ 93 & -90 & -34 & 104 & 71 \\ 10 & -8 & -3 & 10 & 8 \\ 0 & 2 & 2 & -5 & 2 \\ -10 & 9 & -5 & 5 & -2 \end{pmatrix}$	$\begin{pmatrix} 5 & -4 & 7 & -9 & -3 \\ -8 & -4 & 9 & 5 & -8 \\ -7 & 3 & 2 & 5 & 5 \\ 19 & -10 & 3 & -19 & -13 \\ 12 & -7 & 5 & -14 & -8 \end{pmatrix}$	$\begin{pmatrix} -57 & -27 & 36 & -12 & -17 \\ -3 & -3 & 6 & 0 & 1 \\ -9 & -4 & 5 & -2 & -3 \\ -93 & -43 & 56 & -20 & -29 \\ 87 & 37 & -44 & 20 & 31 \end{pmatrix}$
10	$\begin{pmatrix} 1 & 16 & 4 & 12 & -13 \\ 3 & 7 & 1 & 3 & -6 \\ 19 & -65 & -23 & -69 & 50 \\ 3 & 7 & 1 & 3 & -6 \\ 2 & -9 & -3 & -9 & 7 \end{pmatrix}$	$\begin{pmatrix} -1 & 5 & -8 & 3 & 7 \\ 3 & 9 & -5 & 10 & -8 \\ 0 & -5 & -10 & 4 & 1 \\ -24 & -77 & 30 & -76 & 65 \\ -10 & -1 & 0 & -1 & 7 \end{pmatrix}$	$\begin{pmatrix} -19 & -1 & -25 & 1 & 8 \\ 0 & 10 & 4 & -8 & 7 \\ -5 & -3 & -5 & 7 & -10 \\ 7 & -1 & 10 & 3 & -9 \\ 58 & -12 & 85 & 34 & -91 \end{pmatrix}$
11	$\begin{pmatrix} 0 & -4 & -7 & -5 & -9 \\ 1 & -7 & -5 & -6 & 6 \\ 1 & -39 & -61 & -46 & -66 \\ 1 & -43 & -68 & -51 & -75 \\ 6 & -10 & 7 & 3 & 2 \end{pmatrix}$	$\begin{pmatrix} 38 & 13 & 28 & -55 & 49 \\ -2 & 3 & 8 & -5 & 9 \\ -18 & -1 & 0 & 15 & -7 \\ -4 & -1 & -2 & 5 & -4 \\ -2 & 3 & 8 & -5 & 9 \end{pmatrix}$	$\begin{pmatrix} 5 & 1 & -4 & -10 & -1 \\ -7 & 5 & 5 & 2 & 3 \\ 2 & 6 & 4 & -2 & 6 \\ 3 & -3 & 10 & 8 & 6 \\ 2 & 6 & 4 & -2 & 6 \end{pmatrix}$
12	$\begin{pmatrix} -2 & -6 & -3 & -8 & -2 \\ -8 & 18 & -21 & -56 & 4 \\ -10 & 26 & -27 & -72 & 6 \\ 3 & -26 & 12 & 32 & -7 \\ -1 & 4 & -3 & -8 & 1 \end{pmatrix}$	$\begin{pmatrix} 8 & 1 & 1 & 1 & -7 \\ 71 & -38 & 3 & -8 & 46 \\ 8 & -10 & -4 & -1 & -10 \\ -6 & 8 & -9 & 9 & 9 \\ -9 & 4 & -1 & 1 & -8 \end{pmatrix}$	$\begin{pmatrix} -22 & 49 & -12 & 11 & -34 \\ -4 & 10 & -5 & 4 & -7 \\ -6 & 9 & 8 & -5 & -6 \\ -1 & -6 & -10 & 8 & -6 \\ 18 & -51 & 38 & -29 & 36 \end{pmatrix}$
13	$\begin{pmatrix} -6 & -9 & -2 & -10 & -1 \\ 10 & 8 & -7 & -10 & -5 \\ 70 & 98 & 13 & 90 & 5 \\ 28 & 35 & -1 & 20 & -2 \\ 8 & -4 & 5 & 2 & -10 \end{pmatrix}$	$\begin{pmatrix} -34 & 22 & -42 & -33 & 17 \\ -38 & 27 & -48 & -40 & 19 \\ -10 & -8 & -6 & 9 & 5 \\ 4 & -5 & 6 & 7 & -2 \\ 2 & -23 & 12 & 30 & -1 \end{pmatrix}$	$\begin{pmatrix} 29 & -6 & -3 & -1 & 51 \\ 3 & 8 & 7 & 0 & 2 \\ 6 & -2 & -2 & 0 & 9 \\ -1 & -9 & 1 & -2 & 3 \\ -7 & 6 & 9 & -1 & -3 \end{pmatrix}$
14	$\begin{pmatrix} -14 & 17 & -31 & 12 & 8 \\ 5 & -5 & 8 & -4 & -2 \\ -39 & 42 & -71 & 32 & 18 \\ -39 & 42 & -71 & 32 & 18 \\ 6 & -3 & 1 & -4 & 0 \end{pmatrix}$	$\begin{pmatrix} 5 & 0 & 0 & -9 & 3 \\ 1 & 2 & -10 & 10 & 0 \\ 6 & 0 & -6 & -1 & -5 \\ -1 & 7 & -6 & 6 & 8 \\ 6 & 0 & -6 & -1 & -5 \end{pmatrix}$	$\begin{pmatrix} -8 & -7 & 3 & 0 & -2 \\ -1 & -6 & -10 & 1 & 2 \\ 4 & -6 & -1 & -6 & -1 \\ -44 & -48 & 17 & -6 & -13 \\ 4 & -6 & -1 & -6 & -1 \end{pmatrix}$

15	$\begin{pmatrix} -1 & 1 & -3 & 7 & -2 \\ 5 & 8 & -9 & -8 & -10 \\ 0 & -4 & 0 & -4 & -10 \\ -8 & -9 & -8 & -9 & -3 \\ -1 & -19 & -3 & -13 & -52 \end{pmatrix}$	$\begin{pmatrix} -1 & 1 & 4 & -6 & 8 \\ -1 & 1 & 4 & -6 & 8 \\ 5 & 9 & -9 & 5 & 9 \\ -1 & 26 & -26 & -1 & 13 \\ 0 & 5 & -6 & 1 & 1 \end{pmatrix}$	$\begin{pmatrix} -4 & 7 & 1 & 5 & 2 \\ -5 & -2 & -2 & -4 & 6 \\ 19 & -44 & -8 & -34 & -6 \\ 35 & -72 & -12 & -54 & -14 \\ -13 & 12 & 0 & 6 & 10 \end{pmatrix}$
16	$\begin{pmatrix} -28 & 60 & -37 & -31 & -81 \\ 8 & 6 & -1 & 5 & 9 \\ -4 & 6 & -4 & -4 & -10 \\ 4 & 4 & -3 & 1 & 8 \\ 8 & 6 & -1 & 5 & 9 \end{pmatrix}$	$\begin{pmatrix} -10 & 17 & -22 & -6 & 18 \\ -10 & 17 & -22 & -6 & 18 \\ 4 & -3 & -6 & -6 & 10 \\ 7 & -10 & 8 & 0 & -4 \\ 25 & -33 & 18 & -6 & -2 \end{pmatrix}$	$\begin{pmatrix} -2 & 9 & -3 & 10 & 9 \\ -6 & -10 & 7 & 0 & 9 \\ -3 & 14 & -6 & 18 & 14 \\ -1 & 5 & -3 & 8 & 5 \\ 8 & -8 & -6 & -10 & 7 \end{pmatrix}$
17	$\begin{pmatrix} 9 & -10 & -6 & -7 & 8 \\ -1 & -79 & -37 & -44 & 32 \\ -1 & 9 & 5 & 5 & -4 \\ -19 & 83 & 53 & 46 & -40 \\ -10 & 2 & 8 & 1 & -4 \end{pmatrix}$	$\begin{pmatrix} 10 & 10 & -4 & 9 & 10 \\ 22 & 7 & 2 & 12 & -2 \\ 30 & 5 & 6 & 14 & -10 \\ 4 & -1 & 2 & 1 & -4 \\ 30 & 5 & 6 & 14 & -10 \end{pmatrix}$	$\begin{pmatrix} 1 & 8 & -5 & -6 & 9 \\ 90 & 9 & 41 & -80 & -63 \\ 9 & 1 & 5 & -9 & -6 \\ 0 & -1 & -9 & 10 & -3 \\ 3 & 7 & 3 & -9 & 7 \end{pmatrix}$
18	$\begin{pmatrix} -6 & -10 & -3 & -2 & -1 \\ -22 & -22 & 1 & -1 & 5 \\ -10 & -2 & 7 & 3 & 7 \\ -4 & 8 & 10 & 5 & 8 \\ -64 & -92 & -20 & -15 & -2 \end{pmatrix}$	$\begin{pmatrix} 3 & -3 & -10 & -9 & -6 \\ 93 & -41 & -56 & 15 & 8 \\ 10 & -5 & -7 & 1 & 1 \\ 8 & 4 & 2 & 7 & -8 \\ 3 & 4 & 7 & 6 & -1 \end{pmatrix}$	$\begin{pmatrix} 5 & 10 & 6 & -10 & 5 \\ -9 & 3 & 6 & 0 & -5 \\ -34 & -47 & -24 & 50 & -30 \\ 2 & 3 & 8 & 9 & -9 \\ -44 & -67 & -36 & 70 & -40 \end{pmatrix}$
19	$\begin{pmatrix} -21 & 16 & 18 & -13 & 3 \\ -41 & 31 & 38 & -43 & -7 \\ -9 & 7 & 6 & 5 & 9 \\ -2 & 5 & -6 & 7 & -2 \\ -4 & 3 & 4 & -6 & -2 \end{pmatrix}$	$\begin{pmatrix} 6 & 10 & 10 & -10 & -2 \\ -66 & -44 & -35 & -46 & 70 \\ -34 & -20 & -15 & -30 & 38 \\ -66 & -44 & -35 & -46 & 70 \\ -8 & -6 & -5 & -4 & 8 \end{pmatrix}$	$\begin{pmatrix} 2 & 4 & 2 & 10 & -6 \\ 22 & -31 & -33 & 0 & -1 \\ 4 & -7 & -7 & -2 & 1 \\ 1 & -5 & -5 & -1 & 7 \\ -6 & -4 & -5 & 6 & -9 \end{pmatrix}$
20	$\begin{pmatrix} 9 & -6 & -3 & 0 & 8 \\ -8 & 3 & -9 & 10 & -1 \\ 17 & -9 & 6 & -10 & 9 \\ 9 & -6 & -3 & 0 & 8 \\ -39 & 12 & -57 & 60 & 2 \end{pmatrix}$	$\begin{pmatrix} 5 & -10 & 4 & -4 & -1 \\ -3 & 3 & 10 & 8 & 6 \\ 0 & 0 & -4 & 7 & 5 \\ -6 & -6 & 5 & -10 & 4 \\ 5 & -10 & 36 & -60 & -41 \end{pmatrix}$	$\begin{pmatrix} 2 & -8 & 7 & -7 & -4 \\ 1 & -8 & 3 & -4 & -7 \\ 7 & 3 & 1 & -10 & -4 \\ 23 & 1 & 10 & -37 & -16 \\ -68 & -38 & -3 & 93 & 36 \end{pmatrix}$
21	$\begin{pmatrix} -49 & 71 & 16 & 46 & 53 \\ 0 & 8 & 2 & -10 & 4 \\ 0 & 5 & -2 & -9 & -2 \\ 21 & -19 & -4 & -34 & -17 \\ 7 & -9 & -2 & -8 & -7 \end{pmatrix}$	$\begin{pmatrix} 27 & 21 & 21 & -17 & 25 \\ 47 & 39 & 33 & -23 & 37 \\ -33 & -33 & -15 & 1 & -11 \\ -3 & -6 & 3 & -8 & 7 \\ 10 & 9 & 6 & -3 & 6 \end{pmatrix}$	$\begin{pmatrix} 8 & 6 & -9 & -2 & -3 \\ 7 & 7 & 2 & 6 & 0 \\ -2 & -2 & 9 & -6 & 8 \\ 1 & 1 & 4 & -4 & 0 \\ -2 & -2 & 9 & -6 & 8 \end{pmatrix}$
22	$\begin{pmatrix} 14 & 16 & -1 & 3 & 2 \\ -4 & -4 & 3 & -1 & -2 \\ 6 & 8 & 5 & 1 & -2 \\ 42 & 44 & -22 & 10 & 16 \\ 14 & 16 & -1 & 3 & 2 \end{pmatrix}$	$\begin{pmatrix} -2 & 8 & 5 & -5 & -1 \\ 8 & 3 & -9 & 8 & -4 \\ 0 & -10 & -2 & -1 & -8 \\ -1 & -6 & 7 & -6 & -1 \\ -12 & 38 & 28 & -31 & -14 \end{pmatrix}$	$\begin{pmatrix} -85 & -96 & -23 & -26 & -42 \\ 10 & 10 & 3 & 3 & 5 \\ 5 & -6 & 4 & 1 & 3 \\ 0 & -6 & -6 & -5 & 0 \\ -85 & -96 & -23 & -26 & -42 \end{pmatrix}$
23	$\begin{pmatrix} 2 & 5 & -8 & -4 & -3 \\ 7 & -3 & 8 & -8 & 5 \\ 7 & -6 & 0 & 3 & -3 \\ -6 & -5 & -10 & -8 & 5 \\ 57 & -59 & -10 & 19 & -22 \end{pmatrix}$	$\begin{pmatrix} 6 & 1 & 3 & -3 & 9 \\ -10 & -6 & -7 & -2 & 4 \\ 9 & -6 & 6 & -9 & -6 \\ 42 & -23 & 27 & -39 & -15 \\ -12 & 13 & -9 & 15 & 21 \end{pmatrix}$	$\begin{pmatrix} 23 & -45 & -40 & 23 & -14 \\ -52 & 105 & 110 & -22 & 16 \\ 5 & -10 & -10 & 3 & -2 \\ -12 & 25 & 30 & 2 & 0 \\ -2 & 5 & 10 & 8 & -4 \end{pmatrix}$

24	$\begin{pmatrix} -22 & 11 & -29 & -13 & -32 \\ -10 & 0 & -8 & 2 & 5 \\ -2 & -4 & 6 & 7 & 8 \\ -4 & 3 & -7 & -4 & -8 \\ -22 & 11 & -29 & -13 & -32 \end{pmatrix}$	$\begin{pmatrix} 1 & -8 & -6 & 8 & 1 \\ -8 & -4 & 7 & -2 & 8 \\ -9 & 4 & 13 & -10 & 7 \\ -13 & 36 & 37 & -42 & 3 \\ -13 & 36 & 37 & -42 & 3 \end{pmatrix}$	$\begin{pmatrix} -7 & -2 & -7 & 3 & -4 \\ -6 & -3 & -2 & 8 & -9 \\ -14 & -3 & -8 & -6 & -7 \\ -7 & -1 & -1 & -9 & -3 \\ 1 & -9 & -3 & 9 & 9 \end{pmatrix}$
25	$\begin{pmatrix} 8 & 5 & 2 & 8 & 7 \\ 5 & -5 & -5 & -7 & -6 \\ 6 & -3 & 0 & 10 & 1 \\ -1 & 10 & -6 & -3 & 5 \\ 51 & -48 & -45 & -53 & -53 \end{pmatrix}$	$\begin{pmatrix} -55 & 41 & 57 & 78 & -59 \\ -7 & 6 & 7 & 10 & -8 \\ -55 & 41 & 57 & 78 & -59 \\ 1 & -7 & 1 & -2 & 5 \\ 0 & -8 & 4 & 3 & -2 \end{pmatrix}$	$\begin{pmatrix} -32 & 37 & 0 & 26 & 3 \\ -72 & 87 & 5 & 66 & 23 \\ 8 & -10 & -1 & -8 & -4 \\ 48 & -63 & -10 & -54 & -37 \\ -8 & 7 & -3 & 2 & -9 \end{pmatrix}$
26	$\begin{pmatrix} -8 & 9 & 1 & 4 & -4 \\ -28 & 39 & 46 & 29 & 11 \\ -28 & 39 & 46 & 29 & 11 \\ 4 & -6 & -9 & -5 & -3 \\ -12 & 15 & 10 & 9 & -1 \end{pmatrix}$	$\begin{pmatrix} 1 & -2 & 1 & 4 & -7 \\ -3 & -7 & 10 & 0 & -8 \\ 7 & -6 & 9 & 10 & -9 \\ -9 & 0 & 9 & -7 & -10 \\ -19 & 20 & -1 & -47 & 60 \end{pmatrix}$	$\begin{pmatrix} 19 & -45 & -28 & -94 & -74 \\ -8 & -4 & 5 & 7 & -3 \\ -3 & 4 & 3 & 10 & 9 \\ 13 & -37 & -22 & -74 & -56 \\ -8 & -9 & -1 & -4 & 7 \end{pmatrix}$
27	$\begin{pmatrix} 61 & -65 & -100 & -47 & 66 \\ 7 & -11 & -10 & -2 & 12 \\ 3 & 7 & 10 & -7 & -1 \\ 6 & -6 & -10 & -5 & 6 \\ 1 & -5 & 0 & 3 & 6 \end{pmatrix}$	$\begin{pmatrix} 7 & 8 & 7 & 1 & 8 \\ -73 & 16 & 55 & -63 & -16 \\ 10 & -1 & -6 & 8 & 3 \\ -93 & 18 & 67 & -79 & -22 \\ -83 & 17 & 61 & -71 & -19 \end{pmatrix}$	$\begin{pmatrix} 0 & 6 & 1 & 1 & -1 \\ -1 & -9 & 4 & 1 & 0 \\ -10 & -52 & -3 & -4 & 18 \\ -10 & 8 & 7 & 6 & 8 \\ 5 & -6 & 2 & 2 & 5 \end{pmatrix}$
28	$\begin{pmatrix} -8 & -5 & 10 & 6 & -1 \\ -10 & -4 & -5 & -6 & 5 \\ 9 & 8 & 3 & -1 & -9 \\ 1 & -2 & -2 & -1 & -7 \\ 79 & 36 & 38 & 41 & -44 \end{pmatrix}$	$\begin{pmatrix} 54 & -70 & -22 & 8 & 85 \\ 6 & 2 & 10 & -8 & 5 \\ 0 & 11 & 14 & -10 & -5 \\ -5 & 3 & 6 & 1 & -9 \\ 6 & -9 & -4 & 2 & 10 \end{pmatrix}$	$\begin{pmatrix} 30 & 42 & 7 & -66 & -82 \\ 20 & 30 & 7 & -48 & -62 \\ -5 & -6 & 0 & 9 & 10 \\ 40 & 54 & 7 & -84 & -102 \\ -10 & -6 & 7 & 6 & -2 \end{pmatrix}$
29	$\begin{pmatrix} -4 & 7 & 6 & -9 & 2 \\ -7 & 34 & 18 & 12 & 29 \\ -4 & 7 & 6 & -9 & 2 \\ -14 & 97 & 46 & 61 & 92 \\ 1 & -9 & -4 & -7 & -9 \end{pmatrix}$	$\begin{pmatrix} -2 & 7 & 4 & -8 & -5 \\ -2 & 5 & -10 & 6 & 3 \\ 10 & -3 & 7 & 7 & -3 \\ 1 & 1 & 1 & -6 & 5 \\ 101 & -29 & 71 & 64 & -25 \end{pmatrix}$	$\begin{pmatrix} 7 & -9 & 9 & -3 & 5 \\ -15 & 36 & -22 & 6 & -12 \\ 2 & 8 & -1 & -3 & 0 \\ 6 & 9 & 5 & -3 & 3 \\ 41 & -36 & 50 & -18 & 28 \end{pmatrix}$
30	$\begin{pmatrix} -6 & 5 & 22 & -18 & 17 \\ -2 & -3 & 4 & 0 & 9 \\ -6 & 5 & 22 & -18 & 17 \\ 2 & -4 & -9 & 9 & -4 \\ -8 & 9 & 8 & 5 & 8 \end{pmatrix}$	$\begin{pmatrix} 14 & 11 & 21 & 27 & 18 \\ -18 & -1 & -7 & -9 & 2 \\ -2 & 5 & 7 & 9 & 10 \\ -8 & -3 & -7 & -9 & -4 \\ -10 & 2 & 0 & 0 & 6 \end{pmatrix}$	$\begin{pmatrix} 4 & -9 & 4 & -3 & -3 \\ 10 & -6 & -10 & -2 & 6 \\ -59 & 40 & 60 & 11 & -66 \\ 9 & -7 & -8 & -2 & 9 \\ -9 & 8 & -1 & 0 & 2 \end{pmatrix}$

Задание 8.5

Для матриц, приведенных в табл. 8.2, вычислить обратные матрицы методом Гаусса-Жордана: исходная матрица расширяется до блочной вида $[A|I]$ путем приписывания справа единичной матрицы. Путем элементарных преобразований блочной матрицы та часть, которая соответствовала A , приводится к единичной. После преобразований та часть, которая соответствовала I и будет обратной матрицей. Проверить полученный результат при помощи функции `linalg.inv(a)` модуля `numpy`.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев.
- исходные матрицы, блочные матрицы;
- блочные матрицы, полученные после приведения левой части к единичному виду;
- обратные матрицы, рассчитанные в программе и обратные матрицы, вычисленные с использованием модуля numpy.

Equation Section (Next)

9. Кортежи

Кортеж — упорядоченная, неизменяемая и итерируемая последовательность произвольных объектов. Другими словами, кортеж можно условно понимать как неизменяемый список.

Литерально кортеж записывается в круглых скобках `()`, внутри которых через запятую перечисляются элементы. Функция преобразования последовательности в кортеж — `tuple()`.

Замечание. В ряде случаев при литеральной записи кортежей круглые скобки можно опустить (см. ниже).

Кортежи используются в следующих случаях:

- когда данные не должны изменяться в процессе их обработки;
- когда данные имеют определенную структуру: набор координат точек (x_i, y_i) , значения палитры цветов (r_i, g_i, b_i) , определяемые пользователем структуры, например, $(\text{Фамилия}, \text{Имя}, \text{Отчество})$;
- когда требуется максимальная производительность: чтение и копирование кортежей выполняется быстрее, чем аналогичные операции со списками;
- когда при последующей обработке требуется получить уникальный набор значений: функция `set()` неприменима к списку из списков, но работает со списком из кортежей (см. п. 10).

Пустой кортеж и кортеж из одного элемента можно задать, как показано в листинге 9.1. Обратите внимание, что запятая в случае одноэлементного кортежа обязательна.

Листинг 9.1

```
# Пустой кортеж
p = ()
print(p) # ()

# Кортеж из одного элемента
q = (2,)
print(q) # (2,)
```

Кортежи применяются при множественном присваивании (называемом так же *распаковкой кортежей*) в частности, при обмене значениями объектов (листинг 9.2).

Листинг 9.2

```
# Множественное присваивание (скобки можно не использовать)
a, b, c = 3, 7, 5
print(a, b, c) # 3 7 5
```

```
# Множественное присваивание (аналогично предыдущей записи)
a, b, c = (3, 5, 7)
print(a, b, c) # 3 7 5

# Обмен значениями
a, b = b, a
print(a, b) # 7, 3
a, b, c = c, a, b
print(a, b, c) # 5 7 3
```

При множественном присваивании количество объектов в левой части должно совпадать с количеством значений в правой, иначе возникнет исключение. Однако, если в правой части больше значений, чем объектов в левой, то, если при указании перед именем последнего объекта *, оставшиеся значения будут распакованы в последний объект в виде списка (листинг 9.3).

Листинг 9.3

```
a, b, c, d, e = tuple(range(5))
print(a, b, c, d, e) # 0 1 2 3 4

a, b, *c = tuple(range(5))
print(a, b, c) # 0 1 [2, 3, 4]
```

Так же кортежи применяются при возвращении множественных значений результатов работы функции (листинг 9.4).

Листинг 9.4

```
def stats(data):
    return min(data), max(data), sum(data) / len(data)

a = [5, 14, 8, 0, -3, 3]
R = stats(a)
print(type(R)) # <class 'tuple'>
# Результат функции - кортеж

print(R[1], R[2]) # 14 4.5
```

Кортежи неизменяемы, поэтому операции по добавлению, удалению и изменению кортежей вызовут исключение. Любые действия, предполагающие модификацию, требуют создания нового кортежа. Основные операции над кортежами приведены в табл. 9.1.

Таблица 9.1

Операция	Описание	Пример	Результат выполнения
Индексация	Получение элемента по индексу	<code>(1, 2, 3)[0]</code>	1
Срез	Извлечение подпоследовательности	<code>(1, 2, 3, 4)[1:3]</code>	<code>(2, 3)</code>
Конкатенация	Объединение двух кортежей	<code>(1, 2) + (3, 4)</code>	<code>(1, 2, 3, 4)</code>
Повторение	Повторение кортежа заданное число раз	<code>(1, 2) * 2</code>	<code>(1, 2, 1, 2)</code>
Принадлежность	Проверка наличия элемента в кортеже	<code>2 in (1, 2, 3)</code>	True
Длина	Подсчет количества элементов	<code>len((1, 2, 3))</code>	3
Минимум	Нахождение наименьшего элемента	<code>min((4, 2, 7))</code>	2
Максимум	Нахождение наибольшего элемента	<code>max((4, 2, 7))</code>	7
Сумма	Суммирование элементов (при числовых типах)	<code>sum((1, 2, 3))</code>	6
Количество вхождений	Подсчет числа вхождений элемента	<code>(1, 2, 1, 3).count(1)</code>	2
Индекс первого вхождения	Получение индекса первого появления указанного значения	<code>(1, 2, 3, 2).index(2)</code>	1
Распаковка	Присваивание элементов кортежа переменным	<code>a, b = (1, 2)</code>	<code>a = 1, b = 2</code>
Вложенные кортежи	Кортежи могут содержать другие кортежи	<code>((1, 2), (3, 4))</code>	<code>((1, 2), (3, 4))</code>
Итерирование	Обход элементов в цикле	<code>for x in (1, 2, 3): print(x)</code>	1, 2, 3 (по строкам)
Сравнение	Лексикографическое сравнение	<code>(1, 2) < (1, 3)</code>	True
Преобразование из списка	Создание кортежа из списка	<code>tuple([1, 2, 3])</code>	<code>(1, 2, 3)</code>
Преобразование в список	Преобразование кортежа в список	<code>list((1, 2, 3))</code>	<code>[1, 2, 3]</code>

Операция	Описание	Пример	Результат выполнения
Доступ при вложенных кортежах	Обращение к вложенному элементу	<code>((1, 2), (3, 4))[1][0]</code>	3

Функция **zip()** используется для агрегации элементов из нескольких итерируемых объектов (списков, кортежей, строк) в кортежи, каждый из которых содержит элементы с одинаковыми индексами из переданных итерируемых объектов. Синтаксис использования:

```
zip(iterable1, iterable2, ..., iterableN)
```

где `iterable1, iterable2, ..., iterableN` — два или более итерируемых объектов.

Пример использования функции **zip()** приведен в листинге 9.5.

Листинг 9.5

```
a = [1, 2, 3]
b = ['x', 'y', 'z']

z = zip(a, b)
print(list(z))
# [(1, 'x'), (2, 'y'), (3, 'z')]
```

При объединении последовательностей разной длины ни одна из них не дополняется автоматически. Лишние элементы отбрасываются (листинг 9.6).

Листинг 9.6

```
a = [1, 2]
b = ['a', 'b', 'c']

print(list(zip(a, b)))
# [(1, 'a'), (2, 'b')]
```

Возможна и обратная операция — из списка кортежей получить кортежи с исходными элементами (листинг 9.7).

Листинг 9.7

```
p = [(1, 'a'), (2, 'b'), (3, 'c')]

a, b = zip(*p)
# a = (1, 2, 3), b = ('a', 'b', 'c')
```

Задание 9.1

Выполнить задания в соответствии с табл. 9.2. Кортежи в программе задать литерально. При обработке данных допускается использование списков.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев.
- результаты работы программы.

Таблица 9.2

№	Задание
1	Дан кортеж из n элементов, каждый из которых является кортежем из двух элементов: первый элемент — строка, обозначающая название товара, второй элемент — вещественное число, обозначающее цену. Требуется сформировать новый кортеж, содержащий только те товары, цена которых не превышает среднее арифметическое всех цен, с сохранением исходного порядка
2	Дан кортеж, содержащий n элементов, каждый из которых является кортежем из трех целых чисел (a, b, c) , представляющих длины сторон треугольника. Требуется построить новый кортеж, содержащий те вложенные кортежи, которые представляют корректные треугольники (удовлетворяющие неравенству треугольника), отсортированные по убыванию их периметра
3	Дан кортеж, содержащий n элементов, каждый из которых является кортежем из двух элементов: первый элемент — строка, содержащая фамилию и инициалы, второй элемент — целое число, обозначающее балл за тест. Требуется построить новый кортеж, содержащий фамилии только тех студентов, чей балл превышает медианное значение по выборке
4	Дан кортеж, содержащий n элементов, каждый из которых является кортежем из трех целых чисел (d, m, y) , представляющих день, месяц и год рождения человека. Требуется построить новый кортеж, содержащий те же даты, отсортированные в хронологическом порядке, но только для людей, родившихся после 1 января 2000 года (включительно)
5	Дан кортеж, содержащий n элементов, каждый из которых является кортежем из двух элементов: первый элемент — строка, представляющая идентификатор, второй элемент — целое число. Требуется построить новый кортеж, содержащий идентификаторы тех элементов, для которых значение второго поля является простым числом
6	Дан кортеж из n элементов, каждый из которых является кортежем из двух строк (s_1, s_2) , представляющих пары слов. Требуется построить новый кортеж, содержащий только те пары, в которых вторая строка является анаграммой первой
7	Дан кортеж из n элементов, каждый из которых является кортежем из трех элементов (Имя, Фамилия, Год рождения). Требуется построить новый кортеж, содержащий полные имена в формате Фамилия Имя, упорядоченные по возрастанию года рождения

8	Дан кортеж из n элементов, каждый из которых является кортежем из трех элементов (x, y, r) , где x и y — вещественные координаты центра окружности, r — её радиус. Требуется построить новый кортеж, содержащий только те окружности, площадь которых превышает 100
9	Дан кортеж из n элементов, каждый из которых является кортежем из четырех целых чисел (a, b, c, d) , представляющих коэффициенты биквадратного уравнения $ax^4 + bx^3 + cx^2 + dx = 0$. Требуется построить кортеж из всех таких коэффициентных кортежей, для которых уравнение имеет хотя бы один нетривиальный (ненулевой) целый корень на отрезке от -100 до 100
10	Дан кортеж из n элементов, каждый из которых является кортежем из двух элементов: первый элемент — строка в формате времени HH:MM:SS, второй элемент — целое число, представляющее длительность в секундах. Требуется построить новый кортеж, содержащий итоговое время, полученное путем прибавления указанной длительности ко всем моментам времени, выраженное в формате HH:MM:SS
11	Дан кортеж из n элементов, каждый из которых является кортежем из двух целых чисел (a, b) . Требуется построить новый кортеж, в котором каждый элемент представляет собой кортеж $(a, b, \text{НОД}(a, b))$, где НОД — наибольший общий делитель a и b
12	Дан кортеж из n элементов, каждый из которых является кортежем из двух элементов: первый — строка, обозначающая код региона, второй — строка с номером автомобиля. Требуется построить кортеж, содержащий номера только тех автомобилей, которые зарегистрированы в регионе с кодом «77»
13	Дан кортеж из n элементов, каждый из которых является кортежем из двух строк: первая — фамилия, вторая — строка, представляющая дату рождения в формате ДД.ММ.ГГГГ. Требуется построить кортеж фамилий всех лиц, родившихся весной (в марте, апреле или мае)
14	Дан кортеж из n элементов, каждый из которых является кортежем из двух элементов: первый — строка, представляющая наименование товара, второй — кортеж из двух чисел $(\text{Цена}, \text{Количество})$. Требуется построить кортеж из строк наименований всех товаров, суммарная стоимость которых (произведение цены и количества) превышает 1000
15	Дан кортеж из n элементов, каждый из которых является кортежем из четырех чисел, представляющих цвет в формате RGBA, где первые три значения — целые числа от 0 до 255, последнее — вещественное число от 0.0 до 1.0. Требуется построить новый кортеж, содержащий только те цвета, у которых уровень прозрачности меньше 0,5 и хотя бы один из компонентов R, G или B превышает 200
16	Дан кортеж из n элементов, каждый из которых является кортежем (a, b) , где a — строка, представляющая фамилию, а b — строка, представляющая дату в формате ДД.ММ.ГГГГ. Требуется построить новый кортеж, содержащий фамилии a тех людей, которые родились в марте (месяц равен 03), с сохранением исходного порядка
17	Дан кортеж из n элементов, каждый из которых является кортежем (a, b) , где a — строка, представляющая имя сотрудника, b — целое число, обозначающее количество отработанных часов. Требуется построить новый кортеж, содержащий те кортежи, для которых значение b превышает 160, отсортированные по убыванию b

18	Дан кортеж из n элементов, каждый из которых является кортежем (a, b) , где a — строка, обозначающая логин пользователя, b — строка, представляющая IP-адрес в формате «X.X.X.X». Требуется построить новый кортеж логинов a , IP-адрес которых начинается с «192.»
19	Дан кортеж из n элементов, каждый из которых является кортежем (a, b, c) , где a — строка с названием дисциплины, b — строка с фамилией преподавателя, c — целое число, обозначающее число часов. Построить кортеж названий дисциплин a , для которых $c \leq 36$ и фамилия преподавателя b начинается с «К»
20	Дан кортеж из n элементов, каждый из которых представляет собой кортеж (a, b, c) , где a — строка с названием проекта, b — строка, обозначающая статус («open», «closed», «in progress»), c — целое число, обозначающий приоритет. Сформировать новый кортеж названий проектов a , у которых $b = \text{"open"}$ и $c \leq 3$
21	Дан кортеж из n элементов, каждый из которых является кортежем (a, b, c) , где a — строка, представляющая фамилию, b — строка с названием кафедры, c — вещественное число, обозначающее средний рейтинг. Требуется построить кортеж фамилий a , кафедра которых $b = \text{"Информатика"}$, и рейтинг c превышает 4.5
22	Дан кортеж из n элементов, каждый из которых является кортежем (a, b, c) , где a — строка, обозначающая категорию, b — строка с названием товара, c — вещественное число, обозначающее цену. Построить кортеж названий b , для которых $a = \text{"Снеки"}$ и цена c является кратной 10
23	Дан кортеж из n элементов, каждый из которых является кортежем (a, b, c) , где a — строка с именем, b — строка с фамилией, c — целое число, представляющее длину имени и фамилии вместе. Построить кортеж строк вида «b a», для которых c превышает 20
24	Дан кортеж из n элементов, каждый из которых является кортежем (a, b) , где a — строка, представляющая код региона, b — целое число, обозначающее число зарегистрированных пользователей. Требуется построить кортеж значений a , у которых число пользователей b строго меньше 1000
25	Дан кортеж из n элементов, каждый из которых представляет собой кортеж (a, b) , где a — строка, представляющая электронный адрес, b — строка, представляющая дату регистрации в формате «YYYY-MM-DD». Построить кортеж адресов a , дата регистрации которых пришлась на 2024 год
26	Дан кортеж из n элементов, каждый из которых является кортежем (a, b, c) , где a — целое число, представляющее год, b — строка с названием месяца, c — целое число, представляющее количество осадков в мм. Построить кортеж тех кортежей, в которых $b = \text{"Май"}$ и $c \geq 50$, отсортированных по возрастанию c
27	Дан кортеж из n элементов, каждый из которых является кортежем (a, b) , где a — строка, обозначающая вид услуги, b — целое число, обозначающее длительность в минутах. Построить кортеж значений a , у которых длительность b делится на 30 и не превышает 180

28	Дан кортеж из n элементов, каждый из которых представляет собой кортеж (a, b, c) , где a — строка, обозначающая категорию книги, b — строка, представляющая фамилию автора, c — вещественное число, обозначающее рейтинг. Сформировать кортеж фамилий b тех авторов, которые опубликовали книгу в категории «non-fiction» с рейтингом $c > 4.0$
29	Дан кортеж из n элементов, каждый из которых является кортежем (a, b) , где a — строка, обозначающая идентификатор пользователя, b — целое число, обозначающее количество действий за сессию. Построить кортеж идентификаторов a , количество действий b у которых является четным числом и превышает 20
30	Дан кортеж из n элементов, каждый из которых представляет собой кортеж (a, b, c) , где a — строка, представляющая фамилию, b — целое число, обозначающий номер группы, c — строка с датой последней аттестации. Сформировать кортеж фамилий a студентов из групп с номером $b > 300$, у которых дата c позже «01.06.2024»

Equation Section (Next)

10. Множества

Множество (тип **set**) представляет собой неупорядоченную коллекцию уникальных объектов произвольных неизменяемых типов. Сам тип **set** является изменяемым.

Особенности типа множество:

- элементы не индексируются;
- элементы в общем случае не упорядочены;
- элементами множества могут быть только неизменяемые типы (числа, логические значения, строки, кортежи)
- множество содержит только уникальные значения;
- можно добавлять или удалять элементы;
- над множествами можно выполнять операции теории множеств.

Литерально множество задается в фигурных скобках {}, элементы перечисляются через запятую (,). Функция преобразования последовательности во множество — **set()**. При ее применении обычно изменяется порядок следования элементов и удаляются дубликаты значений (листинг 10.1).

Листинг 10.1

```
a = [4, 'abc', -3, 2, 4, 'abc', False, 'den', True, True, 7,
     False]
print(set(a))
# {False, True, 2, 4, 7, 'den', 'abc', -3}
```

Пустое множество (множество не содержащее элементов) задается только конструкцией **set()**. При попытке задать как {} получается пустой словарь (листинг 10.2).

Листинг 10.2

```
a = set()
print(type(a)) # <class 'set'>
# Множество

b = {}
print(type(b)) # <class 'dict'>
# Словарь
```

Основные операции над множествами приведены в табл. 10.1.

Таблица 10.1

Функция / оператор	Описание	Пример	Результат
<code>add(elem)</code>	Добавление элемента в множество	<code>s.add(5)</code>	Множество обновляется

Функция / оператор	Описание	Пример	Результат
<code>remove(elem)</code>	Удаление элемента, вызывает ошибку при отсутствии	<code>s.remove(2)</code>	<code>KeyError</code> , если элемента нет
<code>discard(elem)</code>	Удаление элемента, ничего не делает при отсутствии	<code>s.discard(2)</code>	Без ошибки
<code>pop()</code>	Удаление и возврат случайного элемента	<code>x = s.pop()</code>	<code>x</code> — удаленный элемент
<code>clear()</code>	Удаление всех элементов	<code>s.clear()</code>	<code>s</code> становится пустым
<code>len(s)</code>	Количество элементов	<code>len(s)</code>	Целое число
<code>in / not in</code>	Проверка наличия / отсутствия элемента	<code>2 in s</code>	<code>True</code> или <code>False</code>

Рассмотрим теоретико-множественные операции.

Пусть зафиксирована некоторая область рассмотрения, называемая универсумом или универсальной областью, обозначаемой U . *Универсальным множеством* называется множество U , содержащее в себе все элементы, которые могут входить в любые множества, рассматриваемые в данном контексте. Таким образом, для любого множества A , определенного в данной области, справедливо включение $A \subseteq U$.

Мощностью множества называется количество элементов в этом множестве.

Пустое множество $\emptyset$ — множество, не содержащее элементов.

Пусть A и B — произвольные множества, $A \subseteq U$, $B \subseteq U$.

Объединением множеств A и B называется множество

$$A \cup B = \{x \mid x \in A \vee x \in B\}.$$

Графическая иллюстрация приведена на рис. 4.

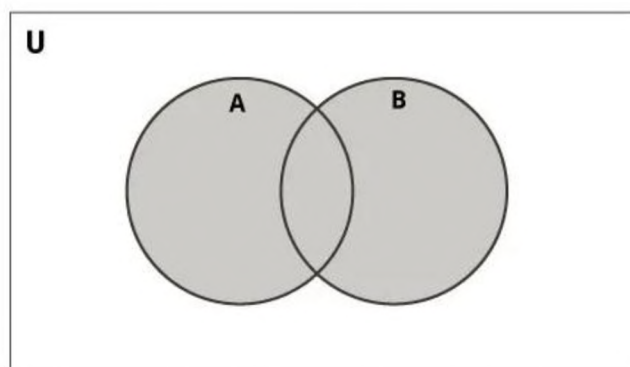


Рисунок 4. Объединение множеств

Пересечением множеств A и B называется множество

$$A \cap B = \{x \mid x \in A \wedge x \in B\}.$$

Графическая иллюстрация приведена на рис. 5.

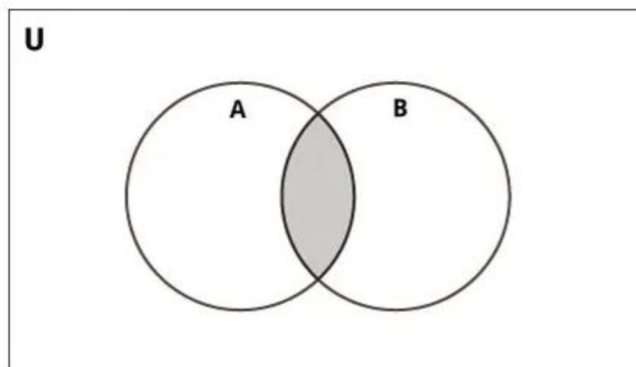


Рисунок 5. Пересечение множеств

Разностью множеств A и B называется множество
 $A \setminus B = \{x \mid x \in A \wedge x \notin B\}$.

Графическая иллюстрация приведена на рис. 6.

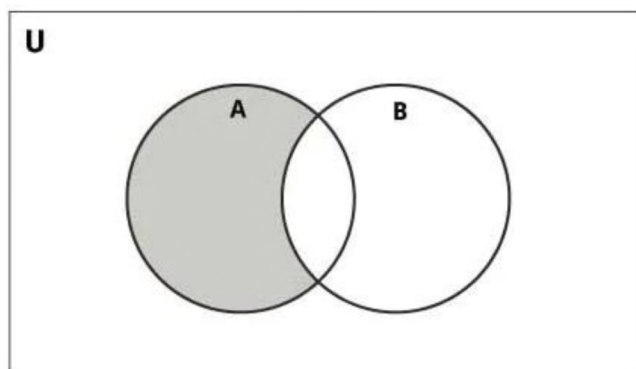


Рисунок 6. Разность множеств

Симметрической разностью множеств A и B называется множество
 $A \Delta B = \{x \mid (x \in A \wedge x \notin B) \vee (x \in B \wedge x \notin A)\}$

Графическая иллюстрация приведена на рис. 7.

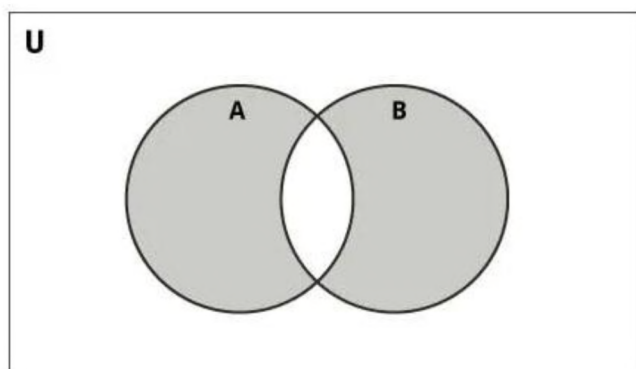


Рисунок 7. Симметрическая разность множеств

Реализация теоретико-множественных операций, а так же дополнительные способы проверки включения и охвата множеств приведены в табл. 10.2.

Таблица 10.2

Описание операции	Оператор	Метод	Результат
Объединение множеств	$A \mid B$	<code>A.union(B)</code>	Множество, содержащее все элементы, которые есть хотя бы в одном из двух множеств
Пересечение множеств	$A \& B$	<code>A.intersection(B)</code>	Множество, содержащее только те элементы, которые есть в обоих множествах
Разность множеств	$A - B$	<code>A.difference(B)</code>	Множество, содержащее элементы, входящие в A , но не входящие в B
Симметрическая разность множеств	$A \wedge B$	<code>A.symmetric_difference(B)</code>	Множество, содержащее элементы, входящие в A или в B , но не в оба
Проверка включения (подмножество)	$A \leq B$	<code>A.issubset(B)</code>	Логическое значение True , если все элементы A принадлежат B
Проверка строгого включения	$A < B$	—	Логическое значение True , если A является подмножеством B и $A \neq B$
Проверка охвата (надмножество)	$A \geq B$	<code>A.issuperset(B)</code>	Логическое значение True , если все элементы B принадлежат A
Проверка строгого охвата	$A > B$	—	Логическое значение True , если A является надмножеством B и $A \neq B$
Проверка на непересекаемость множеств	—	<code>A.isdisjoint(B)</code>	Логическое значение True , если множества не имеют общих элементов

Задание 10.1

Выполнить задания в соответствии с табл. 10.3. Данные для обработки (как числовые, так и строковые) задать в программе случайным образом.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев.
- исходные данные и результаты работы программы.

Таблица 10.3

№	Задание
1	Даны два списка одинаковой длины, содержащие только целые числа. Построить множество всех таких пар (a_i, b_i) , для которых множества цифр в десятичной записи a_i и b_i не пересекаются. Количество таких пар необходимо вывести
2	Задан список кортежей, каждый из которых содержит два множества. Для каждого кортежа проверить, являются ли множества дизъюнктными (множества, у которых нет общих элементов, то есть их пересечение является пустым множеством). Подсчитать количество кортежей, для которых объединение множеств содержит ровно k элементов, где k задается пользователем
3	Дан список строк. Для каждой строки построить множество символов, входящих в нее. Найти максимальное количество строк, в которых множества символов попарно не пересекаются. Использовать переборный алгоритм с множествами
4	Дан вложенный список S , содержащий n списков, каждый из которых содержит целые числа. Определить количество таких пар списков (S_i, S_j) , $i < j$, что множество элементов S_i совпадает с множеством элементов S_j , при этом порядок элементов может различаться и повторения игнорируются
5	Дан список L , содержащий n кортежей из двух целых чисел (a_i, b_i) , $a_i < b_i$. Для каждого кортежа построить множество всех чисел от a_i до b_i включительно. Затем определить количество таких пар множеств, пересечение которых содержит более k элементов, где k задается пользователем
6	Дан список L из строк одинаковой длины, содержащих только строчные буквы латинского алфавита. Для каждой строки построить множество пар (i, c) , где i — позиция символа, а c — сам символ. Найти две строки, множества которых имеют наибольшее пересечение. Вывести их индексы
7	Задано множество S натуральных чисел. Построить все возможные разбиения S на два подмножества A и B такие, что $A \cap B = \emptyset$, $A \cup B = S$, и суммы элементов A и B совпадают. Вернуть количество таких разбиений (учитываются только различные по элементам разбиения, не по порядку)
8	Даны три списка A , B и C , содержащие натуральные числа. Для каждого элемента x из A определить, существует ли такой $y \in B$ и $z \in C$, что множества цифр x , y и z попарно не пересекаются. Подсчитать количество таких элементов x
9	Сформировать список L из n множеств, каждое из которых содержит m уникальных целых чисел. Определить, существует ли такая тройка множеств A , B , C из L , что $(A \cup B \cup C) = (A \cap B \cap C)$. Если да, вернуть индексы тройки, если нет — вывести None
10	Дан список строк. Построить множество всех символов, встречающихся хотя бы в k строках, где k задается пользователем. Вернуть отсортированный по алфавиту кортеж этих символов
11	Даны два списка A и B , каждый из которых содержит только целые числа. Сформировать множество всех таких чисел, которые входят в A и B , но с разной кратностью появления: в одном из списков ровно один раз, а в другом — более одного. Вернуть упорядоченный по возрастанию список таких чисел

№	Задание
12	Сгенерировать список L из n вложенных списков, содержащих целые числа. Построить множество всех пар индексов (i, j) , $i \neq j$, таких что множества элементов $L[i]$ и $L[j]$ различаются ровно на один элемент. Вернуть количество таких пар
13	Дано множество S , содержащее n целых чисел. Построить список всех подмножеств S (не включая пустое и само S), для которых сумма элементов кратна заданному числу k . Подсчитать количество таких подмножеств
14	Даны два списка A и B , содержащие по n целых чисел. Построить список из n множеств, где i -й элемент — это пересечение множеств всех делителей $A[i]$ и $B[i]$. Вернуть количество множеств, содержащих ровно d элементов, где d задано пользователем
15	Задан список L из n целых чисел. Для каждой позиции i определить множество S_i , состоящее из всех таких $j \neq i$, что $L[j]$ и $L[i]$ имеют хотя бы один общий делитель, отличный от 1. Вернуть максимальное значение мощности множества S_i
16	Сформировать список T из m кортежей длины 3, содержащих вещественные числа. Каждое число округлить до двух знаков после запятой и представить в виде строки. Для каждого кортежа построить множество всех возможных упорядоченных подстрок длины 2, возникающих при конкатенации трех элементов кортежа. Найти количество уникальных множеств
17	Дано множество S , состоящее из целых чисел. Для каждой тройки $(a, b, c) \in S^3$, где $a \neq b \neq c$, определить, существует ли множество цифр, входящее одновременно в записи a , b и c . Подсчитать количество таких троек
18	Задан список строк одинаковой длины. Построить множество всех позиций, на которых встречается хотя бы k различных символов, где k задается пользователем. Вернуть отсортированный по возрастанию кортеж этих позиций
19	Даны два вложенных списка A и B , каждый из которых содержит по n подсписков длины m . Для каждого индекса i определить множество $M_i = \text{set}(A[i]) \Delta \text{set}(B[i])$. Найти сумму мощностей всех M_i , таких что M_i не пусто и содержит хотя бы одно четное число
20	Дано множество S натуральных чисел. Построить все упорядоченные пары $(a, b) \in S \times S$, для которых множества цифр a и b полностью совпадают. Вернуть общее количество таких пар с $a \neq b$
21	Сформировать список L из n целых чисел. Для каждого i построить множество D_i всех делителей $L[i]$, кроме 1 и самого $L[i]$. Вернуть количество таких i , для которых D_i не пересекается ни с одним D_j при $j \neq i$
22	Даны три списка A , B , C , содержащие строки. Для каждой тройки строк (a, b, c) построить соответствующие множества символов. Определить количество троек, для которых выполняется условие $\text{set}(a) \cup \text{set}(b) = \text{set}(c)$. Вернуть количество таких троек
23	Задан список L , содержащий n строк. Для каждой строки определить множество символов, встречающихся в ней, и построить список множеств. Определить наибольшее значение k , для которого существует подпоследовательность длины k , все множества которой попарно непересекающиеся

№	Задание
24	Задано два списка A и B длины n , содержащие строки одинаковой длины. Определить количество позиций i , для которых множества символов $A[i]$ и $B[i]$ имеют одинаковую мощность и непустое пересечение
25	Сформировать список T из n кортежей длины 2, содержащих целые числа. Для каждой пары (a, b) построить множество всех общих цифр, встречающихся в десятичной записи a и b . Подсчитать количество пар, у которых это множество содержит хотя бы три элемента
26	Даны три множества A , B и C , содержащие целые числа. Построить множество всех таких троек $(a, b, c) \in A \times B \times C$, для которых верно $a < b < c$ и множества делителей чисел a , b и c попарно различны
27	Задано множество S , содержащее n различных строк. Каждую строку представить как множество её символов. Построить список всех таких пар строк (s_1, s_2) , что пересечение соответствующих множеств состоит ровно из одного символа
28	Сформировать список L из n случайных целых чисел в диапазоне от 1000 до 9999. Для каждой цифры от 0 до 9 определить множество всех индексов в списке L , на которых эта цифра встречается в записи хотя бы одного элемента. Найти количество таких цифр, для которых соответствующее множество состоит более чем из $n/2$ индексов
29	Задано множество S , состоящее из положительных рациональных чисел, представленных в виде кортежей (p, q) , где p и q — числитель и знаменатель. Преобразовать каждую дробь в строку, содержащую только цифры, и определить количество различных множеств цифр, полученных для всех дробей
30	Дан список L , содержащий строки. Построить множество всех пар строк (s_1, s_2) , таких что множества символов в s_1 и s_2 совпадают. Удалить из списка L одну строку из каждой такой пары и вывести итоговый список

Equation Section (Next)

11. Словари

Словарь (тип **dict**) — неупорядоченная коллекция пар вида «ключ-значение», где каждому уникальному ключу соответствует некоторое значение.

Словари создаются с использованием фигурных скобок { }, внутри которых указываются пары вида `ключ: значение`, например,

```
{ "a": 1, -11:7 }.
```

Функция преобразования к словарному типу — **dict()**.

Ключами могут быть только объекты хешируемого типа (например, строки, числа, кортежи), а значения могут быть объектами произвольного типа.

Обращение к значению словаря осуществляется по ключу, используя синтаксис

```
словарь [ключ]
```

Если указанный ключ отсутствует, возбуждается исключение `KeyError`.

Добавление новых элементов производится присваиванием с синтаксисом

```
словарь [новый_ключ] = значение
```

Если ключ уже существует, его значение будет обновлено.

Пустой словарь, не содержащий ни одной пары `ключ: значение`, создается следующим образом: `a = {}`.

Основные методы для работы со словарями приведены в табл. 11.1. В последнем столбце указаны результаты применения методов к словарю

```
{4:12, -1:7, 2: 13, 'a': 9, 'b':321, 19:1000}.
```

Таблица 11.1

Метод	Описание действия	Пример использования	Результат выполнения
<code>d.get(k)</code>	Возвращает значение по ключу <code>k</code> , если ключ есть в словаре, иначе <code>None</code>	<code>d.get('a')</code>	9
<code>d.get(k, v)</code>	Возвращает значение по ключу <code>k</code> , если нет — возвращает значение <code>v</code>	<code>d.get('z', 0)</code>	0

Метод	Описание действия	Пример использования	Результат выполнения
<code>d.keys()</code>	Возвращает представление всех ключей словаря	<code>list(d.keys())</code>	<code>[4, -1, 2, 'a', 'b', 19]</code>
<code>d.values()</code>	Возвращает представление всех значений словаря	<code>list(d.values())</code>	<code>[12, 7, 13, 9, 321, 1000]</code>
<code>d.items()</code>	Возвращает представление всех пар «ключ–значение»	<code>list(d.items())</code>	<code>[(4, 12), (-1, 7), (2, 13), ('a', 9), ('b', 321), (19, 1000)]</code>
<code>d.pop(k)</code>	Удаляет ключ <code>k</code> и возвращает связанное с ним значение	<code>d.pop(2)</code>	13, словарь изменяется
<code>d.pop(k, v)</code>	То же, но возвращает <code>v</code> , если ключа нет	<code>d.pop('z', -1)</code>	-1
<code>d.popitem()</code>	Удаляет и возвращает последнюю добавленную пару	<code>d.popitem()</code>	(19, 1000), словарь изменяется
<code>d.update(m)</code>	Обновляет словарь значениями из другого словаря или итерируемого объекта	<code>d.update({1: 111, 'b': 0})</code>	словарь обновлен: 'b' стало 0, 1 добавлен
<code>d.setdefault(k)</code>	Возвращает значение по ключу <code>k</code> , если он есть, иначе вставляет <code>k:None</code>	<code>d.setdefault('x')</code>	None, ключ 'x' добавлен
<code>d.setdefault(k, v)</code>	То же, но вставляет <code>k:v</code> , если ключ отсутствует	<code>d.setdefault('y', 42)</code>	42, ключ 'y' добавлен
<code>d.clear()</code>	Удаляет все элементы словаря	<code>d.clear()</code>	словарь становится пустым
<code>dict.fromkeys(S)</code>	Создает новый словарь с ключами из <code>S</code> , все значения None	<code>dict.fromkeys(['x', 'y'])</code>	<code>{'x': None, 'y': None}</code>
<code>dict.fromkeys(S, v)</code>	То же, но значения устанавливаются равными <code>v</code>	<code>dict.fromkeys([1, 2, 3], 0)</code>	<code>{1: 0, 2: 0, 3: 0}</code>
<code>d.copy()</code>	Создает поверхностную копию словаря	<code>d.copy()</code>	копия словаря

Формирование словаря возможно из имеющихся списков ключей и значений с использованием комбинации функций **dict()** и **zip()**, пример приведен в листинге 11.1.

Листинг 11.1

```
a = [4, 7, -2, 5, 1]
b = ['a', 'b', 'c', 'd', 'e']

print(dict(zip(a, b)))
# {4: 'a', 7: 'b', -2: 'c', 5: 'd', 1: 'e'}
```

Так же возможно создание словаря через генераторное выражение — *генератор словаря* (*dictionary comprehension*). Его синтаксис похож на генераторы списков. Отличие состоит в том, что в качестве возвращаемого выражения используется пара **ключ:значение** (**key_expr: value_expr**). Синтаксис генератора словаря:

```
{key_expr: value_expr for item in iterable}
```

или синтаксис с условием

```
{key_expr: value_expr for item in iterable if condition}
```

Пример использования генераторного выражения для сортировки словаря по ключам приведен в листинге 11.2.

Листинг 11.2

```
a = [4, 7, -2, 5, 1]
b = ['a', 'b', 'c', 'd', 'e']

d = dict(zip(a, b))
print(d) # {4: 'a', 7: 'b', -2: 'c', 5: 'd', 1: 'e'}

d = {i:d[i] for i in sorted(d.keys())}
print(d) # {-2: 'c', 1: 'e', 4: 'a', 5: 'd', 7: 'b'}
```

Пример 11.1

Дан список строк вида 'W321; отдел: Снабжение; закупки: [{дата: 2025-01-10, сумма: 52000}, {дата: 2025-01-15, сумма: 47000}, {дата: 2025-02-01, сумма: 62000}]'. Построить словарь, в котором ключами будут ID отделов, а значениями — вложенные словари, где ключи — месяцы, а значения — суммарные закупки в каждом месяце. Определить отделы, у которых суммарные закупки в январе превышают 90000.

Решение

Программный код с комментариями приведен в листинге 11.3.

Листинг 11.3

```
# Исходные данные
s = 'W321;Снабжение;2025-01-10:52000;2025-01-15:48000;2025-02-15:64000;2025-03-12:71000' + '\n' + \
    'W322;Ремонт;2025-01-18:50000;2025-02-02:36000;2025-02-19:74000;2025-03-18:55000' + '\n' + \
    'W321;Снабжение;2025-01-16:32000;2025-01-25:40000' + '\n' + \
    'W322;Ремонт;2025-01-14:60000;2025-05-30:33000;2025-01-30:46700;2025-02-12:55000' + '\n' + \
    'W323;Обслуживание;2025-01-17:41000;2025-03-20:51000' + \
    '\n' + \
    'W324;Производство;2025-01-19:123000;2025-01-19:17000;2025-02-21:63700;2025-03-23:49000' + '\n' + \
    'W321;Снабжение;2025-01-10:50000;2025-03-15:27000;2025-04-11:60000;2025-04-12:35000' + '\n' + \
    'W324;Производство;2025-01-14:12000;2025-01-15:17000;2025-02-15:14000;2025-03-12:11500'

# Подпрограмма, формирующая словарь
def newDict(s):
    # Новый пустой словарь
    new_dict = {}
    # Проход по отдельным строкам
    for i in s.split('\n'):
        # Определение ID
        ind = i[:i.index(';')]
        # Если ID еще нет в словаре,...
        if ind not in new_dict:
            # ...то добавляем. Значение по ключу ind - пустой словарь
            new_dict[ind] = {}
        # Проход по всем парам дата:стоимость, разделенных ;
        for j in i.split(';')[2:]:
            # Определение месяца
            t = j.split('-')[1]
            # Если месяца нет в словаре...
            if t not in new_dict[ind]:
                # ...то добавляем ключ во вложенный словарь со значением стоимости
                new_dict[ind][t] = int(j.split(':')[1])
            else:
                # ...иначе прибавляем к значению потраченную сумму
                new_dict[ind][t] += int(j.split(':')[1])
    # Возвращаем словарь
    return new_dict

# Применяем к строке s функцию и выводим словарь
```

```

d = newDict(s)
print(d); print()

# Отбираем нужные значения и выводим результат на экран
res = [i for i in d if d[i]['01'] > 90000]
print('Отделы с суммарными закупками в январе больше 90000:',
      *res)

```

Результаты работы программы представлены в листинге 11.4.

Листинг 11.4

```

{'W321': {'01': 222000, '02': 64000, '03': 98000, '04': 95000},
 'W322': {'01': 156700, '02': 165000, '03': 55000, '05': 33000},
 'W323': {'01': 41000, '03': 51000}, 'W324': {'01': 169000,
 '02': 77700, '03': 60500}}

```

```

Отделы с суммарными закупками в январе больше 90000: W321 W322
W324

```

Задание 11.1 (практико-ориентированное задание)

Имеется список строк, отражающий состояние некоторой предметной области (задать литерально в тексте программы, не менее 15 строк). Поясняющего текста ('клиент', 'товар' и пр.) в строках быть не должно — см. пример 11.1. Требуется выполнить задания в соответствии с табл. 11.2. Формирование словаря оформить в виде подпрограммы.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев;
- сформированный словарь;
- результаты работы программы.

Таблица 11.2

№	Задание
1	Имеется список строк, каждая из которых имеет формат 'ID; ФИО; предмет1:оценка1, предмет2:оценка2, ..., предметN:оценкаN', например: 'S102; Кузнецов А.В.; математика:85, физика:78, информатика:91'. Построить словарь, ключами которого являются ID студентов, а значениями — словари с полями 'ФИО' и 'оценки'. Затем вычислить среднюю оценку каждого студента и вывести ID и ФИО тех студентов, у которых не менее двух оценок ниже 80 и средний балл выше 70

№	Задание
2	Имеется список заказов, каждый из которых представлен строкой формата 'Z231; 2024-07-10; клиент K103; товар: телевизор; цена: 42000'. Построить словарь, где ключами будут ID клиентов, а значениями — списки словарей с информацией о заказанных товарах и их ценах. Для каждого клиента подсчитать общее количество заказов, суммарную стоимость всех заказов и определить клиентов, у которых средняя стоимость одного заказа превышает 30000
3	Дан список журналов посещаемости сотрудников, где каждая запись имеет формат 'ID; 2024-07-12; Сидоров И.И.; офис А; пришел в 08:45; ушел в 17:30'. Построить словарь, где ключами будут ID сотрудников, а значениями — вложенные словари с датами и количеством отработанных часов в соответствующий день. Затем определить сотрудников, которые хотя бы дважды за последние 10 рабочих дней работали менее 6 часов
4	Дан список строк формата 'P007; книга: Мастер и Маргарита; автор: Булгаков; жанр: роман; год: 1940'. Построить словарь, где ключами будут жанры, а значениями — списки словарей с информацией о книгах, упорядоченные по году издания по убыванию. Затем найти жанры, в которых есть хотя бы одна книга, изданная до 1950 года, и вывести ID всех таких книг
5	Дан список строк, описывающих транзакции пользователей, в формате 'T984; пользователь: U123; дата: 2024-06-20; сумма: 5800; категория: техника'. Построить словарь, ключами которого являются ID пользователей, а значениями — словари, в которых ключами будут категории, а значениями — суммарные расходы в данной категории. Определить пользователей, у которых более двух категорий расходов, и при этом расходы в любой одной категории превышают 50% от всех его расходов
6	Дан список строк формата 'E021; Иванов С.П.; отдел: аналитика; проекты: [PR101, PR205, PR330]'. Построить словарь, где ключами будут ID сотрудников, а значениями — словари с полями 'ФИО', 'отдел', 'проекты'. Затем составить список ID сотрудников, участвующих более чем в двух проектах, названия которых начинаются с 'PR2'
7	Дан список строк формата 'S731; товар: ноутбук; категория: техника; продажи: [43000, 47000, 39000]'. Построить словарь, в котором ключами будут ID товаров, а значениями — словари с полями 'категория' и 'средняя продажа'. Отсортировать словарь по убыванию средней продажи и вывести ID трех товаров с максимальной средней продажей в категории 'техника'
8	Дан список строк, каждая из которых имеет формат 'U101; пользователь: Смирнов Д.К.; действия: [вход, просмотр, покупка, выход, вход, просмотр]'. Построить словарь, где ключами будут ID пользователей, а значениями — словари с количеством каждого типа действия. Затем вывести ID пользователей, у которых количество действий 'покупка' превышает 20% от общего числа действий
9	Имеется журнал звонков в формате 'C909; оператор: Лебедев А.Е.; абонент: 89281234567; длительность: 03:21'. Построить словарь, где ключами будут ID операторов, а значениями — суммарное количество звонков и общая длительность всех разговоров (в секундах). Определить операторов, средняя длительность звонков которых превышает 180 секунд

№	Задание
10	Дан список строк формата 'R214; район: Центральный; показания: {2024-01: 1240, 2024-02: 1305, 2024-03: 1412}', где показания — это значения потребления электроэнергии. Построить словарь, где ключами будут ID районов, а значениями — список помесечных приращений потребления. Вывести ID районов, в которых прирост за любой месяц превышал 100
11	Дан список строк формата 'T718; маршрут: М34; водитель: Козлов Н.А.; остановки: [ул. Ленина, пл. Свободы, ул. Гагарина, депо]'. Построить словарь, где ключами будут ID маршрутов, а значениями — списки остановок. Определить маршруты, в которых общее количество остановок не менее четырех и которые начинаются с улицы
12	Дан список строк формата 'A001; агент: Петров В.П.; сделки: {2024-06: 6, 2024-07: 9, 2024-08: 3}'. Построить словарь, где ключами будут ID агентов, а значениями — количество сделок за каждый месяц. Определить агентов, у которых число сделок за июль не менее чем в два раза превышает число сделок за июнь
13	Имеется список данных о заявках клиентов в формате 'Z800; клиент: K432; услуги: [видеонаблюдение, контроль доступа, облачное хранилище]'. Построить словарь, где ключами будут ID клиентов, а значениями — списки заказанных услуг. Определить клиентов, заказавших не менее трех различных услуг, содержащих в названии слово 'доступ' или 'облако'
14	Дан список строк вида 'F556; файл: data.csv; размер: 1341 КБ; тип: таблица'. Построить словарь, в котором ключами будут ID файлов, а значениями — словари с полями 'тип' и 'размер'. Найти суммарный размер всех файлов типа 'таблица', а также определить ID файлов этого типа, размер которых превышает 1,5 МБ
15	Имеется список строк формата 'H102; пациент: Громов С.А.; диагнозы: [гипертония, диабет]; препараты: [препарат А, препарат В]'. Построить словарь, где ключами будут ID пациентов, а значениями — словари с диагнозами и назначенными препаратами. Вывести ID пациентов, у которых одновременно более одного диагноза и не менее двух назначенных препаратов
16	Дан список строк вида 'B320; банк: Альфа; клиенты: {K001: [депозит, кредит], K002: [ипотека], K003: [депозит, инвестиции]}'. Необходимо построить словарь, где ключами будут ID банков, а значениями — словари, в которых ключами являются ID клиентов, а значениями — списки услуг. Определить банки, у которых не менее двух клиентов оформили услугу «депозит»
17	Имеется список строк формата 'D207; дата: 2025-05-03; товар: сок; чек: {позиции: 4, сумма: 213.50}'. Сформировать словарь, где ключами будут ID чеков, а значениями — словари с количеством позиций и суммой. Отобрать ID всех чеков, где средняя цена одной позиции превышает 50
18	Дан список записей формата 'Q611; устройство: датчик температуры; измерения: {08:00: 22.4, 12:00: 25.6, 16:00: 28.1}'. Построить словарь, где ключами будут ID устройств, а значениями — списки всех измерений. Найти устройства, у которых разность между максимальным и минимальным значением температуры превышает 5

№	Задание
19	Дан список строк формата 'L874; библиотека: №12; книги: {K001: 5, K002: 0, K003: 2}', где значения обозначают количество экземпляров. Построить словарь, в котором ключами будут ID библиотек, а значениями — количество книг, отсутствующих в наличии (количество экземпляров равно нулю). Определить ID библиотек, где отсутствует не менее двух различных книг
20	Имеется список строк вида 'W119; сотрудник: Климов И.С.; задания: [{тип: отчет, время: 3.2}, {тип: презентация, время: 2.1}]'. Построить словарь, где ключами будут ID сотрудников, а значениями — общее количество времени на каждое задание. Найти сотрудников, общее время работы которых на задания типа «отчет» превышает 5
21	Дан список строк формата 'K708; клиент: 924315; отзывы: {2024-05: 4.1, 2024-06: 3.8, 2024-07: 2.9}'. Построить словарь, где ключами будут ID клиентов, а значениями — оценки по месяцам. Найти клиентов, у которых средняя оценка за три месяца снизилась более чем на 1.0 между первой и последней датой
22	Дан список строк формата 'E912; заказ: Z341; позиции: [{товар: карандаш, количество: 10}, {товар: бумага, количество: 5}]'. Построить словарь, где ключами будут ID заказов, а значениями — суммарное количество товаров. Найти заказы, в которых количество позиций превышает 12 и присутствует товар с названием, содержащим «бумага»
23	Имеется список записей формата 'X101; проект: Alpha; фазы: {планирование: 4, реализация: 8, тестирование: 3}', где значения — продолжительность в неделях. Сформировать словарь, где ключами будут ID проектов, а значениями — общая продолжительность всех фаз. Определить проекты, у которых реализация составляет не менее 50% от общей продолжительности
24	Дан список строк формата 'M005; пользователь: user21; сессии: [{дата: 2025-04-01, длительность: 23}, {дата: 2025-04-02, длительность: 45}]'. Построить словарь, где ключами будут ID пользователей, а значениями — средняя длительность сессии. Определить пользователей, средняя сессия которых превышает 30 минут
25	Имеется список строк вида 'N400; склад: SKL12; остатки: {продукт А: 150, продукт В: 0, продукт С: 80}'. Построить словарь, в котором ключами будут ID складов, а значениями — словари остатков по товарам. Найти склады, на которых количество позиций с нулевым остатком превышает одну
26	Дан список строк вида 'R381; врач: Михайлова В.В.; приемы: {2025-06-01: 5, 2025-06-02: 8, 2025-06-03: 3}', где значения обозначают количество пациентов. Необходимо построить словарь, где ключами будут ID врачей, а значениями — общая загруженность по дням. Определить врачей, у которых общее число приемов за все дни превышает 12
27	Имеется список записей формата 'F210; магазин: Магнит; продажи: [{товар: яблоки, сумма: 234.50}, {товар: хлеб, сумма: 89.10}]'. Построить словарь, где ключами будут ID магазинов, а значениями — суммарная выручка по каждому товару. Найти магазины, в которых продажи по товару «яблоки» составляют более 200

№	Задание
28	В текстовом файле содержатся строки формата 'Z930; контракт: C781; этапы: {начало: 2024-11-01, окончание: 2025-03-30, продление: 2025-04-15}'. Сформировать словарь, где ключами будут ID контрактов, а значениями — количество дней между каждой парой последовательных этапов. Вывести контракты, у которых интервал между окончанием и продлением превышает 10 дней
29	Дан список строк вида 'A104; ученик: Иванов А.С.; оценки: {математика: [5, 4, 3], физика: [4, 4, 5]}'. Построить словарь, где ключами будут ID учеников, а значениями — средние оценки по каждому предмету. Найти учеников, средний балл которых по математике выше, чем по физике
30	Имеется список записей вида 'T667; предприятие: Энергия; потребление: {январь: 1200, февраль: 1350, март: 980}', где значения — объемы потребленной энергии в кВт·ч. Построить словарь, где ключами будут ID предприятий, а значениями — среднее месячное потребление. Найти предприятия, у которых месячное потребление возросло по сравнению с предыдущим месяцем более чем на 200 кВт·ч

Equation Section (Next)

12. Создание интерфейса консольной программы

Создание *консольных интерфейсов* с вложенными меню в языке Python основывается на организации пользовательского взаимодействия по иерархической структуре, в которой каждое меню представляет собой логический блок, связанный с определенным набором функций. Основная идея заключается в циклическом отображении возможных действий, управляемых пользователем через числовой или символьный выбор.

Принципы построения консольных интерфейсов:

1. Консольный интерфейс представляет собой дерево, в котором каждый узел — это меню, а переход между уровнями реализуется с помощью управляющих конструкций (обычно условных операторов и циклов). Главное меню является корневым элементом, а все остальные — дочерними подменю.

2. Каждое меню организовано в виде цикла (чаще всего **while**), внутри которого осуществляется:

- вывод вариантов действий;
- ввод пользовательского выбора;
- проверка корректности ввода;
- выполнение соответствующих действий;
- предоставление возможности возврата к предыдущему уровню или выхода из программы.

Проверка ввода выполняется с помощью условий и возможной обработки исключений для защиты от недопустимых значений.

3. Для возврата из подменю в родительское меню используется организация циклов таким образом, чтобы условие завершения зависело от пользовательского выбора (например, при выборе опции «Назад» происходит прерывание текущего цикла и возврат на уровень выше).

4. Для повышения модульности и читаемости программы каждый уровень меню и его логика должны быть реализованы в виде отдельных функций.

5. Если требуется передавать параметры между меню (например, список, введенный в одном меню, используется в другом), используются аргументы и возвращаемые значения функций или глобальные переменные, однако предпочтительнее использовать локальные переменные и передачу через параметры.

6. Подменю часто вызываются многократно при возвращении из других пунктов. Поэтому функции подменю должны быть построены таким образом, чтобы могли корректно запускаться повторно и сохранять свою логику независимо от предшествующих вызовов.

7. Выход из интерфейса осуществляется по специальной команде пользователя, которая приводит к завершению всех активных циклов. Обычно это реализуется через ввод специального значения (например, 0) на любом уровне.

8. Интерфейс не должен производить необратимых действий без подтверждения (например, удаление данных), что реализуется через дополнительный ввод подтверждения (y/n) и соответствующую проверку.

Пример 12.1

Создать интерфейс консольной программы, реализующий взаимодействие с пользователем. Программа должна складывать два числа, введенные пользователем, проверять введенное число на четность, посчитывать количество символов во введенной строке, инвертировать строку. Интерфейс должен содержать главное меню и соответствующие подменю.

Решение

Программный код решения приведен в листинге 12.1.

Листинг 12.1

```
def mainM():
    while True:
        print("\n--- ГЛАВНОЕ МЕНЮ ---")
        print("1. Операции с числами")
        print("2. Работа со строками")
        print("3. Выход из программы")

        choice = input("Введите номер пункта: ")

        if choice == '1':
            numbersM()
        elif choice == '2':
            stringsM()
        elif choice == '3':
            print("Завершение программы.")
            break
        else:
            print("Ошибка: введите корректный номер.")

def numbersM():
    while True:
        print("\n--- МЕНЮ: ОПЕРАЦИИ С ЧИСЛАМИ ---")
        print("1. Сумма двух чисел")
        print("2. Проверка на четность")
        print("3. Возврат в главное меню")

        choice = input("Введите номер пункта: ")

        if choice == '1':
            a = float(input("Введите первое число: "))
            b = float(input("Введите второе число: "))
            print("Сумма:", a + b)
        elif choice == '2':
            n = int(input("Введите целое число: "))
            if n % 2 == 0:
                print("Число четное.")
            else:
                print("Число нечетное.")
        elif choice == '3':
            return
```

```

        else:
            print("Ошибка: введите корректный номер.")

def stringsM():
    while True:
        print("\n--- МЕНЮ: РАБОТА СО СТРОКАМИ ---")
        print("1. Подсчет количества символов")
        print("2. Инвертирование строки")
        print("3. Возврат в главное меню")

        choice = input("Введите номер пункта: ")

        if choice == '1':
            s = input("Введите строку: ")
            print("Длина строки:", len(s))
        elif choice == '2':
            s = input("Введите строку: ")
            print("Инвертированная строка:", s[::-1])
        elif choice == '3':
            return
        else:
            print("Ошибка: введите корректный номер.")

# Запуск интерфейса
mainM()

```

Результат работы программы приведен в листинге 12.2.

Листинг 12.2

```

--- ГЛАВНОЕ МЕНЮ ---
1. Операции с числами
2. Работа со строками
3. Выход из программы
Введите номер пункта: 2

--- МЕНЮ: РАБОТА СО СТРОКАМИ ---
1. Подсчет количества символов
2. Инвертирование строки
3. Возврат в главное меню
Введите номер пункта: 1
Введите строку: abcdef
Длина строки: 6

--- МЕНЮ: РАБОТА СО СТРОКАМИ ---
1. Подсчет количества символов
2. Инвертирование строки
3. Возврат в главное меню
Введите номер пункта: 3

--- ГЛАВНОЕ МЕНЮ ---
1. Операции с числами
2. Работа со строками

```

3. Выход из программы
Введите номер пункта: 3
Завершение программы.

Задание 12.1 (практико-ориентированное задание)

Создать интерфейс консольной программы в соответствии с табл. 12.1. Если в задании требуется добавить или удалить элемент, задать в программе литерально требуемые данные (не менее 5).

Таблица 12.1

№	Задание
1	Реализовать консольный интерфейс справочной системы по основам Python. Главное меню должно содержать пункты: «Типы данных», «Условные операторы», «Циклы». Каждый пункт раскрывается в подменю с краткими текстовыми пояснениями. Предусмотреть возврат в предыдущее меню
2	Создать консольное меню, в котором пользователь выбирает категорию (например, «Длина», «Масса», «Температура»), а затем конкретный тип преобразования (например, «метры в километры» или «Цельсий в Фаренгейт»). Предусмотреть ввод значений и вывод результата
3	Разработать консольный интерфейс калькулятора. Главное меню содержит пункты: «Сложение и вычитание», «Умножение и деление», «Возведение в степень и корень». Подменю позволяют ввести два числа и выбрать конкретную операцию. Обеспечить возврат в главное меню
4	Создать консольное приложение, позволяющее пользователю генерировать числовые последовательности. Главное меню: «Арифметическая», «Геометрическая», «Фибоначчи». Каждый раздел — подменю, где запрашиваются параметры генерации и выводятся элементы последовательности
5	Реализовать программу с главным меню: «Статистика текста», «Поиск и замена», «Инвертирование слов». В подменю реализовать ввод строки и соответствующую обработку. Предусмотреть повторный ввод и возврат в основное меню
6	Создать консольное приложение для управления списком задач. Главное меню: «Добавить задачу», «Просмотреть задачи», «Удалить задачу». Каждый пункт открывает подменю с соответствующими запросами. Список задач должен сохраняться в оперативной памяти
7	Разработать интерфейс, моделирующий систему голосования. Главное меню: «Создать опрос», «Проголосовать», «Просмотреть результаты». В подменю реализовать ввод вопроса, вариантов ответа, подсчет голосов и вывод результатов голосования
8	Создать консольное приложение, моделирующее систему самопроверки. Главное меню: «Математика», «Информатика», «Физика». В каждом разделе — подменю с выбором тестов. Каждый тест содержит вопросы с вариантами ответа. Обеспечить подсчет правильных ответов
9	Реализовать консольное приложение, в котором пользователь выбирает раздел «Геометрические вычисления», «Алгебраические преобразования» или «Анализ функций». В каждом разделе предусмотреть подменю с типовыми задачами, например, вычисление площади треугольника или нахождение корней квадратного уравнения

№	Задание
10	Разработать интерфейс консольного меню для управления банковским счетом. Главное меню должно содержать пункты: «Пополнить счет», «Снять со счета», «Показать баланс». Каждая операция реализуется во вложенном подменю с запросом значений и подтверждением
11	Создать меню программы, моделирующей поведение бытовых приборов. Главное меню: «Холодильник», «Микроволновая печь», «Стиральная машина». Подменю каждого устройства позволяет включить или выключить устройство и установить параметры работы
12	Реализовать консольную систему учета посещения студентов. Главное меню должно содержать пункты: «Добавить запись», «Показать журнал посещений», «Удалить запись». Подменю обрабатывает конкретные действия с вводом имени студента и даты
13	Создать программу, моделирующую простую библиотечную систему. Главное меню содержит пункты: «Добавить книгу», «Удалить книгу», «Показать список книг». В подменю реализовать работу с названиями, авторами и уникальными идентификаторами
14	Реализовать консольную игру с меню. Главное меню должно содержать пункты: «Начать игру», «Настройки», «Выход». Подменю «Настройки» позволяет выбрать уровень сложности и стиль интерфейса
15	Разработать интерфейс для работы с валютами. Главное меню: «Конвертация валют», «Курс валют», «Настройки отображения». Подменю «Конвертация» позволяет выбрать направление обмена, ввести сумму и вывести результат
16	Создать консольное приложение для учета расходов. Главное меню содержит пункты: «Добавить расход», «Показать все расходы», «Фильтрация по категории». В подменю предусмотреть ввод суммы, категории и даты
17	Реализовать консольное приложение для работы с пользовательскими учетными записями. Главное меню должно содержать пункты: «Создать пользователя», «Изменить данные пользователя», «Удалить пользователя». Каждое подменю запрашивает соответствующие данные: имя, пароль и идентификатор
18	Разработать консольную систему для работы с электронным журналом успеваемости. Главное меню должно содержать пункты: «Добавить оценку», «Просмотреть оценки ученика», «Рассчитать средний балл». Подменю должны обеспечивать корректный ввод имени и числовых значений
19	Создать интерфейс программы, которая позволяет выполнять численные преобразования. Главное меню: «Округление чисел», «Изменение знака», «Перевод в экспоненциальную форму». В каждом подменю выполняется ввод числа и соответствующее преобразование
20	Реализовать консольное меню для запуска простых математических тестов. Главное меню содержит пункты: «Проверка знаний таблицы умножения», «Проверка сложения», «Проверка деления». Каждый пункт запускает серию вопросов с подсчетом правильных ответов
21	Разработать программу, моделирующую систему управления обучающими модулями. Главное меню: «Выбрать модуль», «Пройти тест», «Просмотреть результаты». Подменю модуля позволяет выбрать тему и приступить к выполнению заданий
22	Создать интерфейс для приложения, рассчитывающего стоимость поездки. Главное меню: «Такси», «Поезд», «Самолет». Каждое подменю запрашивает расстояние, категорию тарифа и рассчитывает итоговую стоимость

№	Задание
23	Разработать консольное меню по анализу списков чисел. Главное меню: «Ввод списка», «Минимум и максимум», «Сумма и среднее значение». Подменю реализует операции с введенным ранее списком, без использования внешних структур хранения
24	Реализовать интерфейс взаимодействия с географическими объектами. Главное меню: «Континенты», «Страны», «Города». В каждом подменю пользователь выбирает объект, после чего отображается краткая информация или статистика, предварительно заданная в коде
25	Реализовать консольное приложение для моделирования работы справочной телефонной службы. Главное меню должно содержать пункты: «Добавить абонента», «Найти по имени», «Удалить запись». В подменю пользователь вводит данные абонента, включая имя и номер телефона
26	Создать программу, предоставляющую доступ к разделам новостного дайджеста. Главное меню: «Мировые новости», «Технологии», «Спорт». Каждый пункт открывает подменю, отображающее список кратких заголовков с возможностью выбора и просмотра расширенной информации
27	Разработать интерфейс для симуляции навигации по каталогу фильмов. Главное меню должно содержать пункты: «Жанры», «Рейтинг», «Поиск по названию». Подменю «Жанры» позволяет выбрать категорию, затем просмотреть список фильмов и краткое описание
28	Создать систему учета книг в домашней библиотеке. Главное меню должно содержать пункты: «Добавить книгу», «Просмотреть список книг», «Поиск по автору». Каждое подменю запрашивает необходимые сведения и выводит результаты в текстовом виде
29	Разработать консольный интерфейс настройки параметров пользовательского профиля. Главное меню: «Имя пользователя», «Электронная почта», «Пароль». В подменю реализовать ввод новых значений и отображение текущих
30	Разработать приложение для анализа целочисленного ряда. Главное меню должно содержать пункты: «Ввод последовательности», «Анализ на монотонность», «Вывод отсортированного ряда». Подменю реализует соответствующие операции над последовательностью, введенной пользователем

Equation Section (Next)

13. Исследование функций с использованием языка Python

13.1. Построение графиков функции, ее первой и второй производной. Анализ функции по графикам

Цель работы: визуальное исследование поведения функции одной переменной на заданном конечном интервале посредством построения графиков функции и ее производных первого и второго порядка с использованием аппроксимации конечными разностями.

При построении графиков функции, ее первой и второй производных можно придерживаться следующего общего алгоритма.

Пусть задана функция $f(x)$, определенная на интервале $[a, b]$. Необходимо построить равномерную сетку из N узлов

$$x_i = a + i \cdot h, \quad (13.1)$$

где h — шаг, вычисляемый как

$$h = \frac{b - a}{N - 1}, \quad (13.2)$$

и $i = 0, 1, \dots, (N - 1)$. В каждой точке x_i вычисляется значение $f_i = f(x_i)$.

Чем больше значение N , тем точнее получается результат, но больше и затрачиваемое процессорное время.

Замечание. При делении отрезка $[a, b]$ на N точек, включающих, в том числе, концы отрезка, он будет разбит на $(N - 1)$ интервал. Поэтому, чтобы получить «ровные» точки разбиения, следует указать $(N + 1)$ точку деления. Например, при делении отрезка $[2, 4]$ на 10 равных интервалов, следует указать $N = 11$ и тогда отрезок будет иметь сетку $[2.0, 2.2, 2.4, 2.6, 2.8, 3.0, 3.2, 3.4, 3.6, 3.8, 4.0]$.

Приближенные значения первой производной $f'(x)$ на внутренних узлах определяются по формуле

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{2h} \quad (13.3)$$

при $1 \leq i \leq N - 2$, а на граничных узлах — по формулам односторонних разностей:

$$f'(x_0) \approx \frac{f(x_1) - f(x_0)}{h} \quad (13.4)$$

и

$$f'(x_{N-1}) \approx \frac{f(x_{N-1}) - f(x_{N-2})}{h}, \quad (13.5)$$

где h определяется по формуле (13.2).

Приближенные значения второй производной $f''(x)$ на внутренних узлах определяются по формуле

$$f''(x_i) \approx \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{h^2} \quad (13.6)$$

при $1 \leq i \leq N-2$, а на концах интервала применяются аналогичные односторонние формулы:

$$f''(x_0) \approx \frac{f(x_2) - 2f(x_1) + f(x_0))}{h^2} \quad (13.7)$$

и

$$f''(x_{N-1}) \approx \frac{f(x_{N-1}) - 2f(x_{N-2}) + f(x_{N-3}))}{h^2}. \quad (13.8)$$

После вычисления списков значений $f(x_i)$, $f'(x_i)$ и $f''(x_i)$ производится их визуализация средствами библиотеки Matplotlib.

По графику функции одной переменной $f(x)$ на заданном интервале можно установить следующие свойства:

1. Факт непрерывности функции на рассматриваемом интервале. При наличии разрывов они визуально идентифицируются как скачки, разрывы или асимптотические разлеты значений $f(x)$.

1. Примерные значения корней функции: в них $f(x) = 0$.

2. Интервалы монотонности, то есть участки возрастания и убывания функции. На участках возрастания график направлен вверх при движении слева направо, на участках убывания — вниз.

3. Точки экстремума — максимумы и минимумы функции, соответствующие вершинам или впадинам графика. Точка экстремума характеризуется сменой монотонности функции в окрестности этой точки.

4. Интервалы выпуклости и вогнутости функции. Выпуклость определяется направлением кривизны графика: если дуга обращена вверх, функция выпукла; если дуга обращена вниз, функция вогнута.

5. Точки перегиба, в которых происходит смена выпуклости функции. На графике это соответствует точкам, где кривизна графика меняет знак.

6. Границы области определения функции. При наличии вертикальных асимптот или недопустимых значений x график может иметь разрывы в этих точках.

7. Периодичность функции, если график обладает регулярной повторяющейся структурой на равных промежутках аргумента.

8. Поведение функции на бесконечности (если график строится на достаточно широком интервале): наличие горизонтальных или наклонных асимптот.

По графику первой производной функции $f'(x)$ можно установить следующие свойства функции $f(x)$:

1. Интервалы монотонности функции $f(x)$. Если $f'(x) > 0$ на некотором интервале, то функция $f(x)$ возрастает на этом интервале. Если $f'(x) < 0$, то функция убывает. Следовательно, знак производной указывает характер монотонности функции $f(x)$.

2. Точки экстремума функции $f(x)$. Если производная $f'(x)$ обращается в нуль в точке x_0 , то эта точка называется стационарной — это потенциальная точка экстремума функции. При этом, если $f'(x)$ меняет знак с положительного на отрицательный в точке x_0 , то x_0 — точка локального максимума функции $f(x)$. Если производная меняет знак с отрицательного на положительный, то x_0 — точка локального минимума.

3. Положение горизонтальных касательных к графику функции $f(x)$. Точки, в которых $f'(x) = 0$, соответствуют точкам, где касательная к графику $f(x)$ горизонтальна.

4. Резкие изменения значений $f'(x)$ (скачки) могут указывать на точки разрыва производной и, соответственно, на особенности формы графика $f(x)$, в том числе, изломы и изменения угла наклона.

По графику второй производной функции $f''(x)$ можно установить следующие свойства функции $f(x)$:

1. Интервалы выпуклости и вогнутости функции $f(x)$. Если $f''(x) < 0$ на некотором интервале, то функция $f(x)$ выпукла, что означает, что ее график расположен ниже касательных. Если $f''(x) > 0$, то функция $f(x)$ вогнута, то есть ее график расположен выше касательных.

2. Точки перегиба функции $f(x)$. Если $f''(x_0) = 0$ и в окрестности точки x_0 вторая производная меняет знак, то x_0 является точкой перегиба функции $f(x)$, то есть точкой, в которой функция меняет характер выпуклости. При этом необходимо, чтобы производная $f''(x)$ действительно меняла знак в проколотой окрестности x_0 , поскольку простое обращение второй производной в нуль не гарантирует наличие перегиба.

3. Характер экстремума функции $f(x)$ в стационарной точке x_0 , если одновременно известно, что $f'(x_0) = 0$. Если $f''(x_0) > 0$, то x_0 является точкой локального минимума функции $f(x)$. Если $f''(x_0) < 0$, то x_0 является точкой локального максимума.

Пример 13.1

Построить средствами Python графики функции $f(x) = x^4 - 10x^3 + 35x^2$, ее первой и второй производной на отрезке $[0,10]$. По графикам определить:

1. Факт непрерывности функции.
2. Приближенные корни уравнения $f(x) = 0$
3. Приблизительные интервалы монотонности.
4. Точки экстремума.
5. Примерные интервалы выпуклости и вогнутости.
6. Имеются или нет точки перегиба и приблизительное значение аргумента в точках перегиба.
7. Примерное значение аргумента, в которых функция имеет касательные, параллельные оси абсцисс.

Решение

Создадим функции $f(x)$, $fFirst(x0, x1)$ и $fSecond(x0, x1, x2)$ для вычисления значений функции, значений первой и второй производной соответственно (листинг 13.1). Возвращаемое значение в функциях $fFirst$ и $fSecond$ вычисляются по формулам (13.3) и (13.6) соответственно.

Листинг 13.1

```
# Подпрограмма вычисления f(x)
def f(x):
    return x**4 - 10 * x**3 + 35 * x

# Подпрограмма вычисления f'(x)
def fFirst(x0, x1):
    return (f(x1) - f(x0)) / (2 * h)

# Подпрограмма вычисления f''(x)
def fSecond(x0, x1, x2):
    return (f(x2) - 2 * f(x1) + f(x0)) / (h**2)
```

Далее введем в программу значения концов отрезка $[a,b]$, количество точек разбиения выберем 1001. Рассчитаем шаг h по формуле (13.2).

Используя генератор списков, составим список X , содержащий значения узлов сетки, вычисляемые по формуле (13.1).

Используя генератор списков, составим список F , содержащий значения функции $f(x)$ в узлах сетки.

Реализация указанных действий приведена а листинге 13.2.

Листинг 13.2

```
# Концы отрезка
a, b = 0, 10
# Количество точек разбиения
N = 1000 + 1
```

```
# Шаг
h = (b - a) / (N - 1)
# Точки сетки
X = [a + i * h for i in range(N)]
# Список значений функции f(x) в точках сетки
F = [f(x) for x in X]
```

Составим список `FFirst`, содержащий значения первой производной функции в узлах сетки. Первоначально составим список для внутренних узлов (начиная с индекса 1 и заканчивая индексом $(N-2)$ включительно — в программе это реализовано как проход по значениям функции `range(1, len(X) - 1)` (не забываем, что значение второго аргумента не достигается). Для граничных узлов сетки расчеты выполняются по формуле (13.4) для $x_0 = a$ и по формуле (13.5) для $x_{N-1} = b$. Первое значение добавляется в список `FFirst` слева, второе — справа. Добавление значений выполняется через конкатенацию списков (листинг 13.3). Последний элемент списка `X` имеет индекс `len(X) - 1`, а предпоследний — индекс `len(X) - 2`.

Листинг 13.3

```
FFirst = [fFirst(X[i-1], X[i+1]) for i in range(1, len(X) - 1)]
FFirst = [(f(X[1]) - f(X[0])) / h] + FFirst + \
          [(f(X[len(X) - 1]) - f(X[len(X) - 2])) / h]
```

Аналогично составим список `FSecond`, содержащий значения второй производной функции в узлах сетки. Для граничных узлов сетки расчеты выполняются по формуле (13.7) для $x_0 = a$ и по формуле (13.8) для $x_{N-1} = b$. Первое значение добавляется в список `FSecond` слева, второе — справа. Добавление значений выполняется через конкатенацию списков (листинг 13.4). Последний элемент списка `X` имеет индекс `len(X) - 1`, предпоследний — индекс `len(X) - 2`, третий элемент с конца — индекс `len(X) - 3`.

Листинг 13.4

```
FSecond = [fSecond(X[i - 1], X[i], X[i + 1])\
            for i in range(1, len(X) - 1)]
FSecond = [(f(X[2]) - 2 * f(X[1]) + f(X[0])) / (h**2)] + \
          FSecond + [(f(X[len(X) - 1]) - 2 * f(X[len(X) - 2])\
            + f(X[len(X) - 3])) / (h**2)]
```

Таким образом, получены четыре списка:

- `X` — значения узлов сетки,
- `F` — значения функции в узлах сетки,
- `FFirst` — значения первой производной в узлах сетки,
- `FSecond` — значения второй производной в узлах сетки.

Далее выполним визуализацию (построим графики) самой функции и ее производных. Для этого будем использовать модуль Matplotlib. Если модуль не установлен, то необходимо выполнить следующие действия:

1. Нажать сочетание клавиш *Win+R*, в появившемся окне набрать команду `cmd` (рис. 8) и нажать на клавишу *Enter*.

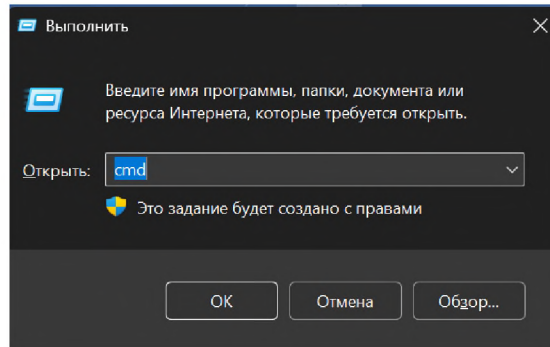


Рисунок 8

2. В появившемся консольном окне (рис. 9) набрать команду `pip install matplotlib` и нажать на клавишу *Enter*.

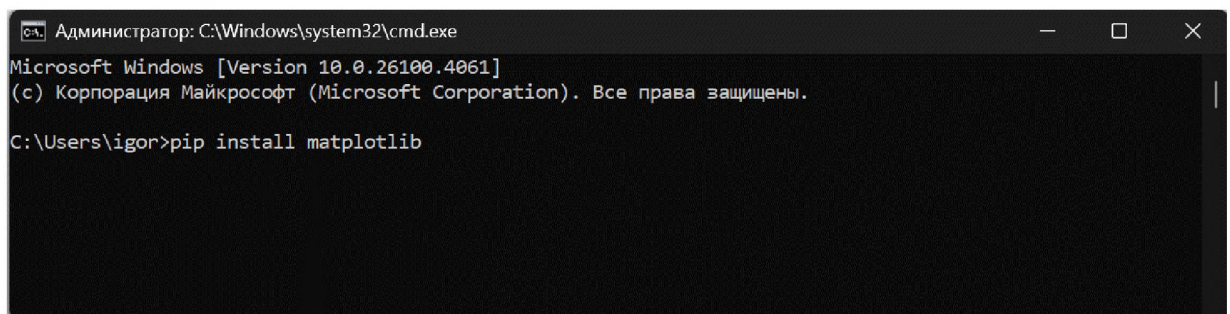


Рисунок 9

3. После этого запустится процесс установки модуля (предполагается наличие соединения с сетью Интернет для скачивания файлов).
4. После установки можно использовать пакет Matplotlib в пайтон-скриптах.

Подключение модуля осуществляется в начале скрипта. Подключаем модуль стандартным способом (листинг 13.5) — создаем псевдоним `plt`, чтобы каждый раз не обращаться к полному имени функций этого модуля.

Листинг 13.5

```
import matplotlib.pyplot as plt
```

Программный код создания графика с пояснениями приведен в листинге 13.6.

Листинг 13.6

```
plt.figure(figsize=(12, 6)) # Размер изображения в дюймах
plt.title('Графики функции  $f(x) = x^4 - 10 * x^3 + 35 * x$  и  
ее производных') # Заголовок
plt.xlabel('X') # Подпись оси X
plt.ylabel('f, f', f''') # Подпись оси Y
plt.grid(True) # Сетка

plt.plot(X, F, label='f(x) =  $x^4 - 10 * x^3 + 35 * x$ ',
color='blue', linewidth=2) # Линия f(x)
plt.plot(X, FFirst, label='Первая производная', color='green',
linewidth=2) # Линия f'(x)
plt.plot(X, FSecond, label='Вторая производная', color='red',
linewidth=2) # Линия f''(x)
plt.legend() # Легенда (подпись функции)

plt.show() # Отображение графика
```

В результате получим все три линии на графике (рис. 10).

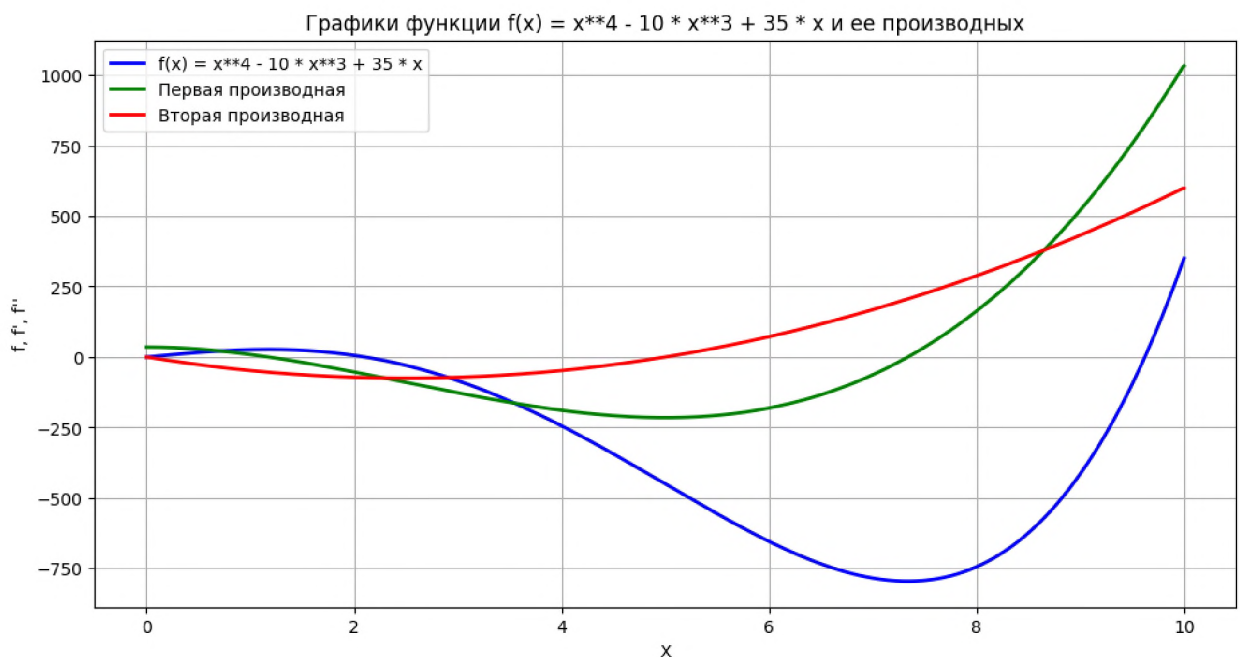


Рисунок 10

Полный код вычислений и построения графика приведен в листинге 13.7.

Листинг 13.7

```
import matplotlib.pyplot as plt

# Подпрограмма вычисления f(x)
def f(x):
    return x**4 - 10 * x**3 + 35 * x
```

```

# Подпрограмма вычисления  $f'(x)$ 
def fFirst(x0, x1):
    return (f(x1) - f(x0)) / (2 * h)

# Подпрограмма вычисления  $f''(x)$ 
def fSecond(x0, x1, x2):
    return (f(x2) - 2 * f(x1) + f(x0)) / (h**2)

# Концы отрезка
a, b = 0, 10

# Количество точек разбиения
N = 1000 + 1

# Шаг
h = (b - a) / (N - 1)

# Точки сетки
X = [a + i * h for i in range(N)]

# Список значений функции  $f(x)$  в точках сетки
F = [f(x) for x in X]

# Список значений первой производной
FFirst = [fFirst(X[i - 1], X[i + 1]) for i in range(1, len(X) - 1)]
FFirst = [(f(X[1]) - f(X[0])) / h] + FFirst + \
    [(f(X[len(X) - 1]) - f(X[len(X) - 2])) / h]

# Список значений второй производной
FSecond = [fSecond(X[i - 1], X[i], X[i + 1]) for i in range(1, len(X) - 1)]
FSecond = [(f(X[2]) - 2 * f(X[1]) + f(X[0])) / (h**2)] + \
    [(f(X[len(X) - 1]) - 2 * f(X[len(X) - 2]) + \
    f(X[len(X) - 3])) / (h**2)]

plt.figure(figsize=(12, 6)) # Размер изображения в дюймах
plt.title('Графики функции  $f(x) = x^4 - 10 * x^3 + 35 * x$  и ее производных') # Заголовок
plt.xlabel('X') # Подпись оси X
plt.ylabel('f, f\'', f\'\'') # Подпись оси Y
plt.grid(True) # Сетка

plt.plot(X, F, label='f(x) = x**4 - 10 * x**3 + 35 * x',
color='blue', linewidth=2) # Линия f(x)
plt.plot(X, FFirst, label='Первая производная', color='green',
linewidth=2) # Линия f'(x)
plt.plot(X, FSecond, label='Вторая производная', color='red',
linewidth=2) # Линия f''(x)
plt.legend() # Легенда (подпись функции)

```

```
plt.show() # Отображение графика
```

Если графики сливаются при изображении на одной координатной сетке, можно изобразить точки $f(x)$, $f'(x)$, $f''(x)$ на разных графиках. Для этого достаточно закомментировать строки кода, отвечающие за построение ненужных графиков (эти строки приведены в листинге 13.8).

Листинг 13.8

```
plt.plot(X, F, label='f(x) = x**4 - 10 * x**3 + 35 * x',  
color='blue', linewidth=2) # Линия f(x)  
plt.plot(X, FFirst, label='Первая производная', color='green',  
linewidth=2) # Линия f'(x)  
plt.plot(X, FSecond, label='Вторая производная', color='red',  
linewidth=2) # Линия f''(x)
```

Если оставить только первую строку, то получим график, приведенный на рис. 11.

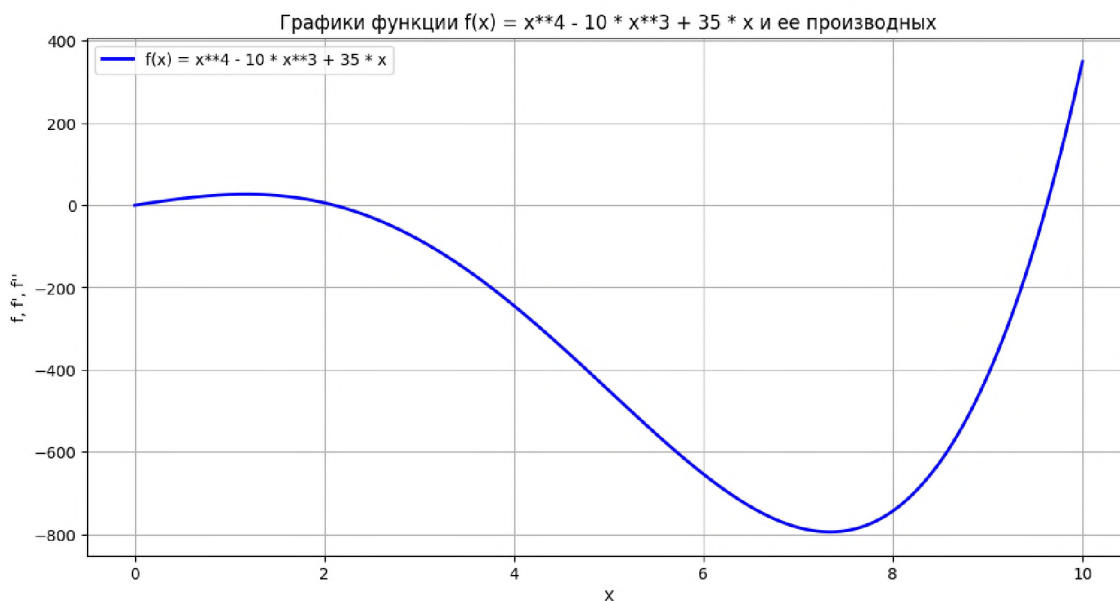


Рисунок 11

Если оставить только вторую строку (построение первой производной), то получим график, приведенный на рис. 12.



Рисунок 12

Если оставить только третью строку (построение второй производной), то получим график, приведенный на рис. 13.



Рисунок 13

Далее выполним исследование функции по визуальному представлению (синяя линия):

1. Функция $f(x)$ является многочленом четвертой степени и, следовательно, непрерывна на всем интервале $[0, 10]$. На графике отсутствуют разрывы, скачки, асимптоты, разлеты значений.
2. Примерные корни значений функции: $x = 0$, $x \approx 2$, $x \approx 9.5$.
3. Функция возрастает на интервалах $[0, \approx 0.7]$ и $(\approx 7.2, 10]$; убывает на интервале $(1.2, \approx 7.2)$.

4. На графике функции наблюдаются две стационарные точки, в которых достигаются локальные экстремумы: локальный максимум при $x \approx 1.2$ и локальный минимум при $x \approx 7.2$.
5. Вопрос о выпуклости и вогнутости функции по графику $f(x)$ выяснить только приблизительно. В интервале $(3,6)$ происходит смена выпуклости на вогнутость. Данный вопрос будет исследован далее по визуальному представлению второй производной функции.
6. Исходя из предыдущего пункта, точка перегиба находится в интервале $(3,6)$.
7. Функция определена на всем интервале $[0,10]$ без ограничений. На графике отсутствуют вертикальные асимптоты и недопустимые значения аргумента.
8. Функция не является периодической. На графике отсутствует регулярная повторяющаяся структура на равных промежутках аргумента.
9. Анализ на бесконечности невозможен, поскольку график рассматривается только на ограниченном интервале $[0,10]$. Однако из характера роста функции на правом краю ($x = 10$) видно, что функция стремится к $+\infty$, то есть возрастает без наличия горизонтальных или наклонных асимптот.

Выполним исследование функции по визуальному представлению графика первой производной (зеленая линия):

1. График $f'(x) > 0$ на интервалах $[0, \approx 1.2)$ и $(\approx 7.2, 10]$, следовательно, на этих интервалах функция возрастает. График $f'(x) < 0$ на интервале $(\approx 1.2, \approx 7.2)$, следовательно, на этом интервале функция убывает.
2. Точка $x \approx 1.2$ — точка локального максимума, т.к. в ней производная меняет знак с положительного на отрицательный. Точка $x \approx 7.2$ — точка локального минимума, т.к. в ней производная меняет знак с отрицательного на положительный.
3. Горизонтальные касательные имеются в точках $x \approx 1.2$ и $x \approx 7.2$.
4. Резких изменений графика $f'(x)$ не наблюдается.

Выполним исследование функции по визуальному представлению графика второй производной (красная линия):

1. При $x \in (0, \approx 5)$ значения $f''(x)$ отрицательны, следовательно, на этом интервале функция выпукла вверх; при $x \in (\approx 5, 10]$ значения $f''(x)$ положительны, следовательно, на этом интервале функция выпукла вниз.
2. Исходя из предыдущего пункта, точка $x \approx 5$ является точкой перегиба, т.к. в этой точке $f''(x)$ меняет знак и в этой точке $f''(x) = 0$.
3. В точке локального экстремума $x \approx 1.2$ значение $f''(x) < 0$, следовательно $x \approx 1.2$ является точкой локального максимума. В точке

локального экстремума $x \approx 7.2$ значение $f''(x) > 0$, следовательно $x \approx 7.2$ является точкой локального минимума.
Сведем результаты в табл. 13.1.

Таблица 13.1

Свойство функции	Наличие / Приближенное значение
Область определения	$[0, 10]$
Непрерывность	Да
Периодичность	Нет
Корни	$x = 0$, $x \approx 2$, $x \approx 9.5$
Интервал(ы) возрастания	$[0, \approx 0.7)$, $(\approx 7.2, 10]$
Интервал(ы) убывания	$(1.2, \approx 7.2)$
Точка(и) локального максимума	$x \approx 1.2$
Точка(и) локального минимума	$x \approx 7.2$
Горизонтальные касательные	$x \approx 1.2$, $x \approx 7.2$
Интервал(ы) выпуклости вверх	$(0, \approx 5)$
Интервал(ы) выпуклости вниз	$(\approx 5, 10]$
Точка(и) перегиба	$x \approx 5$

Задание 13.1

Построить график функции на интервале $[a, b]$, ее первой и второй производной, выполнить анализ функции по графикам в соответствии с вариантом (табл. 13.2).

Таблица 13.2

№	Функция	Интервал
1	$f(x) = \frac{(x^3 - 4x)e^{-x/2}}{5}$	$[-3, 5]$
2	$f(x) = \frac{1}{10}(x^4 - 13x^2 + 36)\ln(x + 1)$	$[-4, 4]$
3	$f(x) = \frac{x(x^2 - 9)(x^2 - 1)}{20}$	$[-4, 4]$
4	$f(x) = \frac{\sqrt{ x + 1}(x^3 - 7x + 6)}{5}$	$[-3, 4]$
5	$f(x) = \frac{(x^2 - 4)(x^3 - x)}{10(x^2 + 1)}$	$[-3, 3]$
6	$f(x) = \frac{(x^4 - 10x^2 + 9)}{15}e^{-x^2/10}$	$[-4, 4]$
7	$f(x) = \frac{(x^2 - 1)(x^2 - 4)}{10(1 + e^{-x})}$	$[-3, 3]$

№	Функция	Интервал
8	$f(x) = \frac{x(x-2)(x+2)(x-3)(x+3)}{50}$	$[-4, 4]$.
9	$f(x) = \frac{(x^3 - 3x)}{10} e^{-x^2/4}$	$[-3, 3]$
10	$f(x) = \frac{\ln(x^2 + 1)(x^3 - 2x)}{5}$	$[-2, 3]$
11	$f(x) = \frac{\sqrt{x^2 + 1}(x^2 - 9)}{20}$	$[-4, 4]$
12	$f(x) = \frac{ x (x^2 - 1)}{5} e^{- x /2}$	$[-3, 3]$
13	$f(x) = \frac{(x^5 - 5x^3 + 4x)}{30} e^{-x^2/5}$	$[-4, 3]$
14	$f(x) = \frac{x(x^2 - 1)(x^2 - 4)(x - 2)}{50}$	$[-3, 3]$
15	$f(x) = \frac{\ln(x^2 + 2)(x^4 - 4x^2 + 3)}{20}$	$[-3, 3]$
16	$f(x) = \frac{\sqrt{x^2 + 1}(x^3 - 6x)}{25}$	$[-4, 4]$
17	$f(x) = \frac{(x^6 - 9x^4 + 15x^2 - 5)}{100} e^{-x^2/6}$	$[-4, 4]$
18	$f(x) = \frac{x(x^2 - 1)(x^2 - 9)(x^2 - 4)}{120}$	$[-3, 4]$
19	$f(x) = \frac{\ln(x^2 + 3)(x^5 - 7x^3 + 6x)}{60}$	$[-3, 4]$
20	$f(x) = \frac{\sqrt{x^2 + 2}(x^4 - 5x^2 + 4)}{40}$	$[-4, 4]$
21	$f(x) = \frac{(x^7 - 14x^5 + 49x^3 - 36x)}{200} e^{-x^2/7}$	$[-4, 4]$
22	$f(x) = \frac{x(x^2 - 1)(x^2 - 16)(x^2 - 9)}{200}$	$[-4, 4]$.
23	$f(x) = \frac{\ln(x^2 + 0.5)(x^3 - x)}{6}$	$[-2, 3]$
24	$f(x) = \frac{\sqrt{x^2 + 0.8}(x^2 - 4)}{18}$	$[-4, 4]$
25	$f(x) = \frac{(x^4 - 5x^2 + 4)}{12} e^{-x^2/6}$	$[-3, 3]$
26	$f(x) = \frac{x(x^3 - x^2 - 4x + 4)}{15}$	$[-2, 3]$
27	$f(x) = \frac{\ln(x + 0.7)(x^4 - 3x^2)}{8}$	$[-2, 3]$
28	$f(x) = \frac{(x^2 + 0.5)^{1/3}(x^2 - 9)}{20}$	$[-4, 4]$
29	$f(x) = \frac{ x ^{1.5}(x^3 - x)}{10} e^{- x /3}$	$[-3, 3]$

№	Функция	Интервал
30	$f(x) = \frac{\ln(x^2 + 0.6)(x^5 - 5x^3 + 4x)}{25}$	$[-2, 3]$

Отчет по выполнению задания должен содержать:

1. Титульный лист.
2. Программный код решения с обязательными комментариями.
3. Рисунок, на котором представлены все три графика (функции, ее первой и второй производных).
4. Отдельные графики, отображающие функцию, ее первую и вторую производные.
5. Текстовую часть исследования функции, ее первой и второй производных.
6. Итоговую таблицу исследования функции (см. табл. 13.1).

13.2. Нахождение корней функции численными методами

Цель работы: научиться находить решение уравнения $f(x) = 0$ на заданном интервале численными методами.

Алгоритм нахождения корня уравнения методом деления пополам (бисекции) основан на применении теоремы Больцано-Коши, согласно которой если функция $f(x)$ непрерывна на отрезке $[a, b]$ и значения $f(a)$ и $f(b)$ имеют противоположные знаки, то на этом отрезке существует хотя бы один корень уравнения $f(x) = 0$. Метод предполагает последовательное деление отрезка пополам и выбор подотрезка, на котором функция сохраняет смену знака.

Пусть задана непрерывная функция $f(x)$ и начальный отрезок $[a, b]$, на концах которого выполняется условие $f(a) \cdot f(b) < 0$. Также пусть задана требуемая точность вычисления $\varepsilon > 0$. Алгоритм нахождения корня уравнения $f(x) = 0$:

1. Проверяется, что $f(a) \cdot f(b) < 0$. В противном случае метод неприменим на данном отрезке.

2. Вычисляется середина отрезка $c = \frac{a+b}{2}$.

3. Вычисляется значение $f(c)$.

4. Если $|f(c)| < \varepsilon$ или длина отрезка $|b - a| < \varepsilon$, то c принимается за приближенный корень.

В противном случае:

если $f(a) \cdot f(c) < 0$, то корень заключен в отрезке $[a, c]$ и присваивается $b = c$; иначе корень заключен в отрезке $[c, b]$ и присваивается $a = c$.

5. Переход к шагу 2. Процесс повторяется до достижения заданной точности.

6. Результатом (корнем) будет значение $x = \frac{a+b}{2}$.

Пример 13.2

Рассматривается функция $f(x) = x^3 - 2$ на отрезке $[0, 2]$. Вычислить корень уравнения $f(x) = 0$ методом бисекции с точностью не менее 5 знаков после десятичного разделителя.

Решение

Создадим функцию для вычисления $f(x)$ (листинг 13.9).

Листинг 13.9

```
def f(x):  
    return x**2 - 3
```

Далее создадим функцию, реализующую рассмотренный выше алгоритм (листинг 13.10). Функция имеет три аргумента — концы отрезка и точность, с которой нужно определить корень.

Листинг 13.10

```
def solve(a, b, eps):  
    if f(a) * f(b) > 0:  
        return False  
    while (b - a) / 2 > eps:  
        c = (a + b) / 2  
        fc = f(c)  
        if abs(fc) < eps:  
            return c  
        if f(a) * fc < 0:  
            b = c  
        else:  
            a = c  
    return (a + b) / 2
```

Зададим значения отрезка $[a, b]$ и точность ε . Так как предполагается точность не менее 5 знаков после десятичного разделителя, то зададим точность ε на один разряд больше: 10^{-6} . Все, что остается сделать — применить функцию `solve()` к заданным значениям (листинг 13.11).

Листинг 13.11

```
a, b = 0, 2  
eps = 10**-6  
print(solve(a, b, eps))
```

Программный код целиком с пояснениями приведен в листинге 13.12.

Листинг 13.12

```
# Подпрограмма вычисления функции
def f(x):
    return x**2 - 3

# Подпрограмма решения уравнения методом бисекции
def solve(a, b, eps):
    # Если функция на концах отрезка имеем одинаковые знаки, то
    алгоритм неприменим
    if f(a) * f(b) > 0:
        return False
    # Пока точность превышает eps.
    while (b - a) / 2 > eps:
        # .реализуем алгоритм
        c = (a + b) / 2
        fc = f(c)
        # Если значение функции в центре отрезка меньше
        требуемой точности.
        if abs(fc) < eps:
            # Решение найдено!
            return c
        # Пересчитываем концы отрезка [a,b]
        if f(a) * fc < 0:
            b = c
        else:
            a = c
    # Возвращаем решение, как среднее значение отрезка [a,b]
    return (a + b) / 2

# Задаем границы отрезка
a, b = 0, 2

# Задаем точность
eps = 10**-6

# Выводим на экран значение, вычисляемое при заданных a, b и
eps
print(solve(a, b, eps))
```

В результате получим вещественное значение 1.7320507764816284. Так как предполагает выполнение точность не менее 5 знаков, то получим ответ $x=1.73205$.

Замечание. В ряде случаев при переопределении концов отрезка $[a,b]$ возникает ситуация, когда процесс вычислений заикливается. В этом случае вводят счетчик количества итераций цикла **while**. При достижении некоторого порогового значения счетчика (обычно ≥ 1000) происходит выход из цикла. В данном примере заикливания не происходит, но при возникновении

подобной ситуации реализация алгоритма выглядит, как показано в листинге 13.13. В счетчике `ci` хранится количество итераций. Как только перестанет выполняться одно из условий $(b - a) / 2 > \text{eps}$ или $ci < 10000$, происходит выход из цикла. Рекомендуется всегда включать подобную проверку количества итераций.

Листинг 13.13

```
def solve(a, b, eps):
    ci = 0
    if f(a) * f(b) > 0:
        return False
    while (b - a) / 2 > eps and ci < 10000:
        c = (a + b) / 2
        fc = f(c)
        if abs(fc) < eps:
            return c
        if f(a) * fc < 0:
            b = c
        else:
            a = c
        ci += 1
    return (a + b) / 2
```

Задание 13.2

Для функции из предыдущего задания вычислить все корни на отрезке определения с точностью до 7 знаков после десятичного разделителя. Количество корней и отрезки, на которых они могут быть вычислены, определить самостоятельно.

Отчет по выполнению задания должен содержать:

1. Титульный лист.
2. График функции $f(x)$.
3. Программный код нахождения корней уравнения методом бисекции.
4. Значения вещественных корней, полученных в результате выполнения программы и значения корней с заданной точностью.

13.3. Нахождение точек экстремума функции численными методами

Цель работы: научиться находить точки экстремума (максимума или минимума) функции $f(x)$ на заданном интервале численными методами с определенной точностью.

Алгоритм нахождения локального экстремума (максимума или минимума) на отрезке для унимодальной функции методом дихотомии применяется к непрерывной и унимодальной функции $f(x)$, определенной на отрезке $[a, b]$. Под *унимодальностью* понимается наличие единственной точки экстремума на данном отрезке. Метод основан на последовательном сужении интервала, содержащего точку экстремума, с использованием двух контрольных точек, расположенных симметрично относительно середины отрезка.

Пусть функция $f(x)$ определена и непрерывна на отрезке $[a, b]$, и на этом отрезке существует единственный минимум. Задан параметр точности $\varepsilon > 0$, а также малая положительная константа $\delta > 0$, такая что $0 < \delta \ll \varepsilon$.

1. Инициализируется отрезок $[a, b]$ и параметры ε, δ .
2. Вычисляются точки:

$$x_1 = \frac{a+b}{2} - \delta, \quad x_2 = \frac{a+b}{2} + \delta.$$
3. Вычисляются значения функции $f(x_1)$ и $f(x_2)$.
4. Если $f(x_1) < f(x_2)$, то минимум находится в интервале $[a, x_2]$, и устанавливается $b = x_2$. Если $f(x_1) > f(x_2)$, то минимум находится в интервале $[x_1, b]$, и устанавливается $a = x_1$. Если $f(x_1) = f(x_2)$, то минимум находится в интервале $[x_1, x_2]$, и устанавливается $a = x_1, b = x_2$.
5. Переход к шагу 2, пока длина интервала $b - a > \varepsilon$.
6. После завершения итераций точка минимума приближенно вычисляется как $x = \frac{a+b}{2}$.

Для поиска максимума используется аналогичная процедура с противоположными сравнениями на шаге 4: $f(x_1) > f(x_2)$ и $f(x_1) < f(x_2)$.

Пример 13.3

Для функции $f(x) = -x^3 + 12x^2 - 47x + 60$, заданной на отрезке $[3, 6]$, определить точки экстремума с точностью до 5 знаков после десятичного делителя.

Решение

Построим график функции $f(x) = -x^3 + 12x^2 - 47x + 60$ на отрезке $[3, 6]$ (листинг 13.14).

Листинг 13.14

```
import matplotlib.pyplot as plt

def f(x):
    return -x**3 + 12*x**2 - 47*x + 60
```

```

a1, b1 = 3, 6
N = 1001
h = (b1 - a1) / N
X = [a1 + i * h for i in range(N)]
F = [f(x) for x in X]

plt.figure(figsize=(12, 6))
plt.grid(True)
plt.plot(X, F, label='f(x)', color='blue', linewidth=2)
plt.show()

```

В результате получим график, представленный на рис. 14.

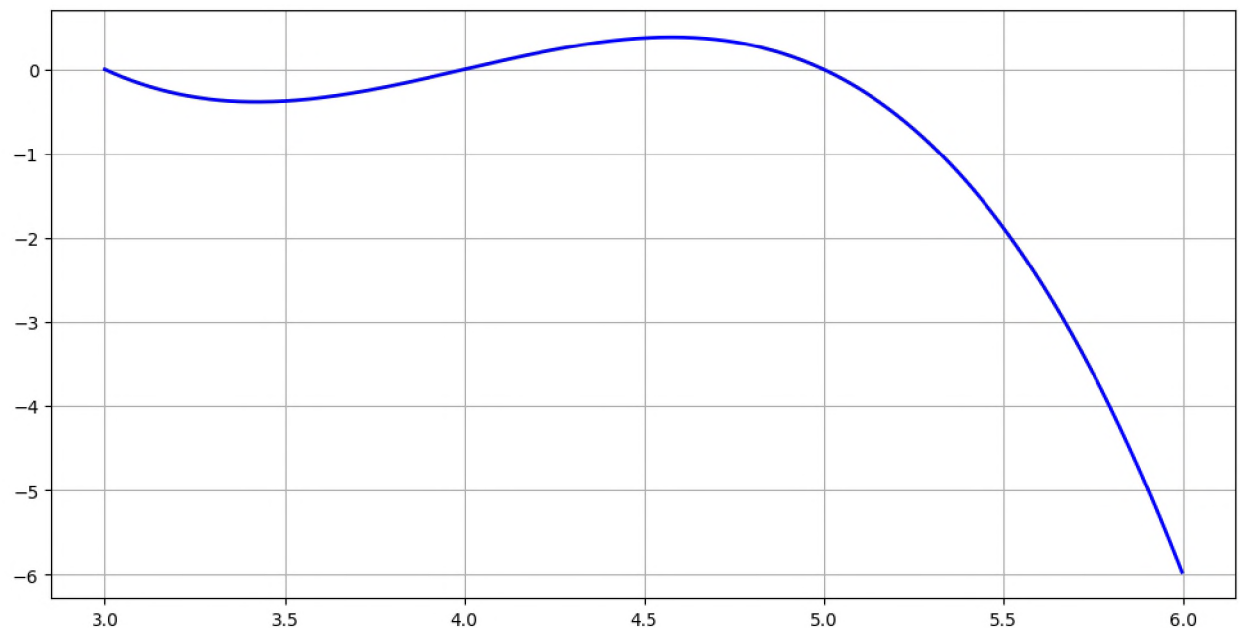


Рисунок 14

Визуально определяем, что функция имеет две экстремальные точки: минимум на отрезке $[3, 4]$ и максимум на отрезке $[4, 5]$.

Реализуем алгоритм, представленный выше в виде функций `fmax()` и `fmin()` с аргументами a, b (концы отрезка), ε , $\delta = \varepsilon / 2$ (листинг 13.15).

Листинг 13.15

```

eps, delta = 10**-6, 10**-6 / 2

def fmax(a, b, eps, delta):
    ci = 0
    while b - a > eps and ci <= 10000:
        x1 = (a + b) / 2 - delta
        x2 = (a + b) / 2 + delta
        f1, f2 = f(x1), f(x2)
        if f1 > f2:
            b = x2
        if f1 < f2:
            a = x1

```

```

        if f1 == f2:
            a, b = x1, x2
            ci += 1
        return (a + b) / 2

def fmin(a, b, eps, delta):
    ci = 0
    while b - a > eps and ci <= 10000:
        x1 = (a + b) / 2 - delta
        x2 = (a + b) / 2 + delta
        f1, f2 = f(x1), f(x2)
        if f1 < f2:
            b = x2
        if f1 > f2:
            a = x1
        if f1 == f2:
            a, b = x1, x2
        ci += 1
    return (a + b) / 2

a, b = 3, 4
print(fmin(a, b, eps, delta))

a, b = 4, 5
print(fmax(a, b, eps, delta))

```

В результате выполнения этого программного кода получены результаты для минимума $x_{f_{\min}} = 3.42264$, для максимума $x_{f_{\max}} = 4.57735$.

Задание 13.3

Для функции из первого задания вычислить все точки экстремума с точностью 7 знаков после десятичного разделителя на области задания функции. Количество экстремумов, их тип и отрезки, на которых они могут быть вычислены, определить самостоятельно.

Отчет по выполнению задания должен содержать:

1. Титульный лист.
2. График функции $f(x)$.
3. Программный код нахождения точек экстремума.
4. Значения аргумента функции в точках экстремума с требуемой точностью.
5. Значения функции в точках экстремума с требуемой точностью.

13.4. Нахождение точек перегиба функции численными методами

Цель работы: численное исследование поведения дважды дифференцируемой функции одной переменной на заданном интервале с использованием метода конечных разностей для приближенного вычисления второй производной и определения координат точек перегиба.

Проверка, что некоторая точка x_0 является точкой перегиба функции $f(x)$, основывается на исследовании поведения второй производной $f''(x)$. Точка x_0 называется точкой перегиба, если в ней происходит смена знака второй производной, то есть если $f''(x)$ меняет знак при переходе через x_0 . Это означает, что график функции меняет характер кривизны: из выпуклого вверх становится выпуклым вниз или наоборот.

Пусть $f(x)$ — дважды дифференцируемая функция на некоторой окрестности точки x_0 . Тогда x_0 является точкой перегиба функции $f(x)$, если одновременно выполняются следующие условия:

1. Существует вторая производная $f''(x)$ в окрестности точки x_0 .
2. $f''(x_0) = 0$.
3. Существует окрестность точки x_0 , такая что $f''(x)$ меняет знак при переходе через x_0 . То есть, существует $\delta > 0$, такое, что $f''(x_0 - \delta) \cdot f''(x_0 + \delta) < 0$.

Если эти условия выполнены, то x_0 является точкой перегиба.

В случае численного анализа используется центральная разностная аппроксимация второй производной в точке x , которая вычисляется по формуле (13.7), где $\delta = h$.

Знак $f''(x)$ определяется в точках $x_0 - \delta$ и $x_0 + \delta$. Если значения $f''(x_0 - \delta)$ и $f''(x_0 + \delta)$ имеют противоположные знаки, то x_0 — точка перегиба.

Описание алгоритма:

1. Задать исследуемую функцию $f(x)$, интервал $[a, b]$ и шаг дискретизации $h > 0$. Чем меньше шаг h , тем точнее результат. Например, если предполагается точность не менее 4 знаков после десятичного разделителя, следует указать $h = 10^{-5}$. При заданных a , b и h количество точек разбиения определяется как

$$N = \left\lfloor \frac{b-a}{h} \right\rfloor, \quad (13.9)$$

где $\lfloor \cdot \rfloor$ — обозначение операции округления в меньшую сторону.

2. Отрезок $[a, b]$ разбить на сетку значений с шагом h .

3. Для каждого значения x_i сетки из интервала (a, b) вычислить приближенное значение второй производной $f''(x_i)$ по формуле (13.7).
4. Для всех последовательных пар значений $f''(x_i)$ и $f''(x_{i+1})$ проверить условие смены знака

$$f''(x_i) \cdot f''(x_{i+1}) < 0. \quad (13.10)$$

Если условие выполнено, то между точками x_i и x_{i+1} имеется точка перегиба. В качестве результата выбирается значение

$$\frac{x_i + x_{i+1}}{2}. \quad (13.11)$$

Замечание. Из-за особенностей вещественной арифметики в Python и ошибок округления при численной реализации указанного алгоритма возможно появление пар значений, удовлетворяющих условию (13.10). Для нахождения наиболее точного значения в пределах заданной точности применяют специальные численные методы, рассмотрение которых выходит за рамки данного курса.

Пример 13.4

Для функции, рассмотренной в п. 13.1, численно определить точку перегиба с точностью не менее 3 знаков после десятичного разделителя.

Решение

В п. 13.1 было установлено, что точкой перегиба для функции является значение $x \approx 5$. Найдем значение x с заданной точностью.

Определим пользовательские функции `f()` и `fSecond()` для вычисления значений функции и ее второй производной соответственно (листинг 13.16).

Листинг 13.16

```
def f(x):
    return x**4 - 10 * x**3 + 35 * x

def fSecond(x0, x1, x2):
    return (f(x2) - 2 * f(x1) + f(x0)) / (h**2)
```

Зададим шаг $h = 10^{-5}$, значения концов отрезка, в которой находится предполагаемая точка перегиба, как [4,6]. Для вычисления количества точек разбиения по формуле (13.9) воспользуемся функцией `floor()` модуля `math`. Рассчитаем значения функции (список `X`) и второй производной (список `FSecond`) в точках сетки (листинг 13.17).

Листинг 13.17

```
from math import *

h = 10**-5
a, b = 4, 6
N = floor((b - a) / h)

X = [a + i * h for i in range(N + 1)]

FSecond = [fSecond(X[i - 1], X[i], X[i + 1]) for i in range(1,
len(X) - 1)]
```

Отберем из пар значений списка X только такие, которые удовлетворяют условию (13.10) (листинг 13.18).

Листинг 13.18

```
R = [(X[i] + X[i + 1]) / 2 for i in range(len(FSecond) - 1) if
FSecond[i] * FSecond[i + 1] < 0]

print(R)
```

Полностью программный код приведен в листинге 13.19.

Листинг 13.19

```
from math import *

def f(x):
    return x**4 - 10 * x**3 + 35 * x

def fSecond(x0, x1, x2):
    return (f(x2) - 2 * f(x1) + f(x0)) / (h**2)

h = 10**-5
a, b = 4, 6
N = floor((b - a) / h)
X = [a + i * h for i in range(N + 1)]
FSecond = [fSecond(X[i - 1], X[i], X[i + 1]) for i in range(1,
len(X) - 1)]

R = [(X[i] + X[i + 1]) / 2 for i in range(len(FSecond) - 1) if
FSecond[i] * FSecond[i + 1] < 0]

print(R)
```

В результате получим список значений R:

```
[4.999925, 4.9999350000000001, 4.999975, 4.999995,
5.000005, 5.0000149999999999, 5.000025,
5.00003500000000005, 5.000045].
```

Делаем вывод: при точности $h = 10^{-5}$ получаем несколько потенциальных значений точки перегиба, т. е. возникла ситуация, рассмотренная в замечании перед этим примером.

Попробуем уменьшить точность. При задании шага $h = 10^{-4}$ имеем единственное значение 4.99995, что при данном шаге означает точность 3 знака после десятичного разделителя. Таким образом, численный ответ для значения точки перегиба функции будет 4,999.

Задание 13.4

Для функции из первого задания численно вычислить все точки перегиба на отрезке определения функции. Количество точек перегиба и отрезки, на которых они могут быть вычислены, определить самостоятельно. Подобрать такое значение шага h , при котором программа выдаст единственный результат.

Отчет по выполнению задания должен содержать:

1. Титульный лист.
2. График функции $f(x)$.
3. Программный код нахождения точек перегиба.
4. Значения точек перегиба с найденной точностью.

Equation Section (Next)

14. Обработка файлов

14.1. Базовые операции

Обработка файлов представляет собой совокупность операций по созданию, открытию, чтению, записи, изменению и закрытию файлов, расположенных в файловой системе.

Обработка файлов в Python включает следующие этапы:

1. Открытие файла с помощью встроенной функции **open** ().
2. Выполнение операций чтения, записи или изменения содержимого файла.
3. Закрытие файла с помощью метода **close** () либо автоматическое закрытие при использовании конструкции **with**.

Функция

```
open(file, mode='r', encoding=None)
```

возвращает файловый объект. Аргумент `file` задает путь к файлу, **mode** определяет режим открытия, **encoding** — кодировку для текстового файла (обычно `'utf-8'`). Ключи **mode** и **encoding** являются необязательными.

Режимы открытия файла приведены в табл. 14.1.

Таблица 14.1

Режим	Назначение
'r'	Чтение (файл должен существовать)
'w'	Запись (создание нового файла или обнуление)
'a'	Добавление в конец (создание при отсутствии)
'x'	Создание нового файла, ошибка при наличии файла
'b'	Бинарный режим (в комбинации с 'r', 'w', 'a')
't'	Текстовый режим (по умолчанию)
'+'	Обновление: чтение и запись

Выполнять действия с данными, которые хранятся в файлах, рекомендуется с использованием конструкции `with`, называемой так же *контекстным менеджером*. При использовании `with` после выполнения операций над содержимым файла происходит автоматическое закрытие сессии работы с файлом. Синтаксис контекстного менеджера:

```
with open(...) as f:  
    инструкции
```

Здесь `f` — имя переменной (файлового объекта), связываемой с открываемым файлом

Основные методы, применяемые к файловому объекту `f` представлены в табл. 14.2.

Таблица 14.2

Метод	Назначение	Пример использования
<code>read(size=k)</code>	Считывает весь файл или указанное количество байт/символов	<code>f.read(), f.read(100)</code>
<code>readline()</code>	Считывает одну строку из файла	<code>line = f.readline()</code>
<code>readlines()</code>	Считывает все строки и возвращает их в виде списка	<code>lines = f.readlines()</code>
<code>write(s)</code>	Записывает строку <code>s</code> в файл	<code>f.write("пример\n")</code>
<code>writelines(lines)</code>	Записывает итерируемую последовательность строк в файл	<code>f.writelines(["одна\n", "две\n"])</code>
<code>seek(offset, whence=0)</code>	Перемещает курсор чтения/записи в указанную позицию	<code>f.seek(0), f.seek(10, 1)</code>
<code>tell()</code>	Возвращает текущую позицию курсора в файле	<code>pos = f.tell()</code>
<code>close()</code>	Закрывает файл	<code>f.close()</code>

Все считываемые данные имеют строковый тип.

Примеры использования контекстного менеджера и базовых операций с содержимым файла приведены в листинге 14.1.

Листинг 14.1

```
# Чтение всего файла
with open('data.txt', 'r') as f:
    content = f.read()

# Построчное чтение
with open('data.txt', 'r') as f:
    for line in f:
        print(line.strip())

# Запись в файл
with open('output.txt', 'w') as f:
    f.write('Пример строки\n')
    f.write('Еще одна строка\n')

# Добавление в файл
with open('output.txt', 'a') as f:
    f.write('Дополнительная запись\n')
```

Задание 14.1

Написать программу в соответствии с заданием в табл. 14.3.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев.
- исходный и итоговый файлы.

Таблица 14.3

№	Задание
1	В файле file1.txt содержатся строки с произвольным текстом. Пользователь задает число n . Отобразить все строки, длина которых строго больше n символов, и сохранить их в файл file2.txt
2	В файле file1.txt содержатся строки с произвольным текстом. Пользователь задает слово. Сформировать файл file2.txt, содержащий только те строки исходного файла, которые содержат заданное слово без учета регистра
3	В файле file1.txt содержатся строки, среди которых есть пустые и состоящие из пробелов. Удалить такие строки и сохранить результат в файл file2.txt
4	В файле file1.txt содержатся строки с произвольным текстом. Определить количество строк, состоящих исключительно из цифр (без пробелов и других символов), и записать это число в файл file2.txt
5	В файле file1.txt могут встречаться повторяющиеся строки. Удалить повторяющиеся строки, сохранив только первые вхождения, и записать результат в файл file2.txt
6	В файле file1.txt содержатся строки в определенном порядке. Записать все строки в обратном порядке в файл file2.txt
7	В файле file1.txt содержатся строки с произвольным текстом. Подсчитать общее количество слов во всех строках и записать результат в файл file2.txt
8	В файле file1.txt содержатся строки произвольной длины. Создать файл file2.txt, в котором каждая строка исходного файла будет предварена ее порядковым номером и символом табуляции
9	В файле file1.txt содержатся строки с произвольным текстом. Отобразить и записать в файл file2.txt только те строки, которые состоят исключительно из латинских букв без пробелов и других символов
10	В файле file1.txt содержатся строки с произвольным текстом. Пользователь задает слово. Заменить все вхождения этого слова (без учета регистра), являющегося отдельным словом, на строку <REDACTED>. Результат сохранить в файл file2.txt
11	В файле file1.txt содержатся строки с произвольным текстом. Для каждой строки определить количество слов и записать в файл file2.txt пары: номер строки и количество слов, разделенные символом табуляции
12	В файле file1.txt содержатся строки с произвольным содержимым. Удалить из каждой строки все цифры и сохранить результат в файл file2.txt
13	В файле file1.txt содержатся строки с произвольным текстом. Заменить в каждой строке все пробелы на символ подчеркивания и записать результат в файл file2.txt
14	В файле file1.txt содержатся строки с текстом. Пользователь задает число k . Сформировать файл file2.txt, содержащий только те строки, в которых количество символов пунктуации (точек, запятых, точек с запятой, двоеточий, восклицательных и вопросительных знаков) не менее k

№	Задание
15	В файле file1.txt содержатся строки, содержащие слова, разделенные пробелами. В каждой строке отсортировать слова по длине в порядке неубывания и сохранить результат в файл file2.txt
16	В файле file1.txt содержатся строки с текстом. Заменить каждое вхождение самой длинной подстроки, состоящей из повторяющихся подряд одинаковых символов, на строку [REPEAT], и сохранить результат в файл file2.txt
17	В файле file1.txt содержатся строки с произвольным текстом. Определить, какие строки содержат одинаковое количество букв и цифр, и записать их в файл file2.txt
18	В файле file1.txt содержатся строки с произвольным содержимым. Удалить из файла все строки, содержащие хотя бы одну заглавную латинскую букву, и сохранить оставшиеся строки в файл file2.txt
19	В файле file1.txt содержатся строки с текстом. Удалить из каждой строки все повторяющиеся слова, сохранив только первое вхождение каждого, и записать результат в файл file2.txt
20	В файле file1.txt содержатся строки с текстом. Пользователь вводит строку-шаблон, содержащую символ *, обозначающий любое количество произвольных символов. Сформировать файл file2.txt, содержащий строки, соответствующие заданному шаблону
21	В файле file1.txt содержатся строки с произвольным текстом. Для каждой строки определить количество заглавных букв латинского и кириллического алфавитов. Записать в файл file2.txt пары: исходная строка и соответствующее количество заглавных букв, разделенные символом табуляции
22	В файле file1.txt содержатся строки, каждая из которых может содержать как текст, так и числа. В каждой строке заменить все числовые последовательности на строку [NUMBER] и записать результат в файл file2.txt
23	В файле file1.txt содержатся строки с произвольным текстом. Определить строку с наибольшим количеством пробелов и записать только ее в файл file2.txt. Если таких строк несколько, выбрать первую из них
24	В файле file1.txt содержатся строки. Отсортировать все строки по числу вхождений символа а (латинская и кириллическая буквы считаются отдельно) и сохранить результат в файл file2.txt
25	В файле file1.txt содержатся строки с текстом. Удалить из каждой строки все гласные буквы (как латинские, так и кириллические) и сохранить результат в файл file2.txt
26	В файле file1.txt содержатся строки с текстом. Пользователь вводит два слова. В каждой строке заменить все вхождения первого слова на второе и сохранить результат в файл file2.txt
27	В файле file1.txt содержатся строки с текстом. Сформировать файл file2.txt, в который записать только те строки, в которых слова идут в лексикографически отсортированном порядке
28	В файле file1.txt содержатся строки с текстом. Удалить все строки, в которых количество слов не является простым числом, и сохранить результат в файл file2.txt
29	В файле file1.txt содержатся строки с текстом. Определить количество строк, в которых содержатся подряд идущие одинаковые слова, и записать это число в файл file2.txt
30	В файле file1.txt содержатся строки с произвольным текстом. Удалить из каждой строки подстроку между первым и последним вхождением символа # (включительно) и сохранить результат в файл file2.txt

14.2. Обработка csv-файлов

CSV (Comma-Separated Values, разделенные запятыми значения) — это текстовый формат хранения табличных данных, в котором каждая строка соответствует одной записи, а поля внутри строки разделяются символом `' , '` (запятая) или другими символами-разделителями (точкой с запятой, табуляцией).

Общая структура формата CSV:

1. Файл состоит из последовательности строк. Каждая строка представляет собой запись с одним или несколькими полями.
2. Поля разделяются символом-разделителем.
3. Часто первая строка содержит имена полей (заголовки), так же через разделитель.

4. Если поле содержит символ-разделитель, символ кавычек или перевод строки, оно должно быть заключено в двойные кавычки `" ... "`. Внутри таких полей все двойные кавычки экранируются удвоением — `""`.

5. Все значения в CSV-файле представлены текстом. Преобразование типов (числа, даты и т. п.) выполняется отдельно при обработке данных.

Модуль `csv`, входящий в стандартную библиотеку Python, предназначен для чтения и записи файлов в формате CSV. Он предоставляет функциональные средства для корректной обработки полей, содержащих специальные символы (запятые, кавычки, переводы строки), а также для использования различных символов-разделителей.

Диалект — это именованная совокупность параметров форматирования CSV-файла, определяющая способ обработки строк. Параметры диалекта указываются при открытии файла для чтения/записи. Основные параметры диалектов приведены в табл. 14.4.

Таблица 14.4

Параметр	Тип значения	Значение по умолчанию	Назначение
<code>delimiter</code>	<code>str</code> (1 символ)	<code>' , '</code>	Символ, используемый для разделения полей
<code>quotechar</code>	<code>str</code> (1 символ)	<code>' " '</code>	Символ, которым заключаются текстовые поля
<code>escapechar</code>	<code>str</code> (1 символ) или <code>None</code>	<code>None</code>	Символ для экранирования специальных символов, если <code>quoting=csv.QUOTE_NONE</code>
<code>doublequote</code>	<code>bool</code>	<code>True</code>	Если <code>True</code> , символ кавычек внутри поля дублируется
<code>skipinitialspace</code>	<code>bool</code>	<code>False</code>	Если <code>True</code> , пробелы сразу после разделителя игнорируются
<code>lineterminator</code>	<code>str</code>	<code>'\r\n'</code>	Строка, используемая для завершения строки при записи

Параметр	Тип значения	Значение по умолчанию	Назначение
quoting	int (константа csv)	csv.QUOTE_MINIMAL	Управляет тем, какие поля следует заключать в кавычки
comment	str (1 символ) или None	None	Указывает символ, с которого начинается строка-комментарий при чтении
strict	bool	False	Если True, возбуждается исключение при нарушении формата CSV

Основные функции, методы и константы модуля csv приведены в табл. 14.5.

Таблица 14.5

Название	Назначение	Пример использования
reader()	Создает объект для последовательного чтения строк из CSV-файла	reader = csv.reader(file)
writer()	Создает объект для записи строк в CSV-файл	writer = csv.writer(file)
DictReader()	Читает строки CSV-файла как словари, используя заголовки или переданные имена полей	reader = csv.DictReader(file)
DictWriter()	Записывает строки в CSV-файл в виде словарей, используя список полей	writer = csv.DictWriter(file, fieldnames=['id', 'name'])
register_dialect()	Регистрирует новый диалект CSV с пользовательскими параметрами	csv.register_dialect('my', delimiter=';')
unregister_dialect()	Удаляет ранее зарегистрированный диалект по имени	csv.unregister_dialect('my')
get_dialect()	Возвращает объект диалекта по его имени	dialect = csv.get_dialect('my')
list_dialects()	Возвращает список всех зарегистрированных диалектов	csv.list_dialects()
field_size_limit()	Получает или задает максимально допустимый размер поля в байтах	csv.field_size_limit(1000000)

Название	Назначение	Пример использования
QUOTE_MINIMAL	Константа, указывающая, что кавычки используются только при необходимости	<code>csv.writer(file, quoting=csv.QUOTE_MINIMAL)</code>
QUOTE_ALL	Константа, указывающая, что все поля должны быть заключены в кавычки	<code>csv.writer(file, quoting=csv.QUOTE_ALL)</code>
QUOTE_NONNUMERIC	Константа, указывающая, что все нечисловые поля должны быть в кавычках и интерпретируются как строки	<code>csv.writer(file, quoting=csv.QUOTE_NONNUMERIC)</code>
QUOTE_NONE	Константа, запрещающая использование кавычек в любых полях	<code>csv.writer(file, quoting=csv.QUOTE_NONE, escapechar='\\')</code>

Задание 14.2

Написать программу в соответствии с заданием в табл. 14.6.

Содержание отчета:

- полная формулировка задания;
- программный код с обязательными пояснениями в виде комментариев.
- исходный и итоговый файлы.

Таблица 14.6

№	Задание
1	Пользователь вводит с клавиатуры данные о сотрудниках в формате ФИО;отдел;оклад. Признаком окончания ввода является пустая строка. После завершения ввода необходимо запросить имя выходного файла. Требуется сгруппировать сотрудников по отделам, вычислить средний оклад по каждому отделу и сохранить результат в CSV-файл, где каждая строка содержит название отдела и соответствующее среднее значение оклада

№	Задание
2	Пользователь вводит сведения о заказах в формате номер_заказа; дата; сумма; статус. После ввода всех данных (признак — пустая строка) программа должна запросить имя файла для сохранения. Требуется сформировать CSV-файл, в котором для каждого уникального статуса указывается количество заказов с этим статусом и суммарная стоимость таких заказов. Первая строка файла должна содержать заголовки статус, количество, сумма
3	Пользователь вводит сведения о результатах тестирования студентов в формате ФИО; группа; балл. Ввод завершается пустой строкой. После запроса имени файла программа должна сохранить в файл таблицу в формате CSV, где каждая строка содержит название группы, количество студентов в этой группе и средний балл по группе. Результаты предварительно отсортировать по убыванию среднего балла
4	Пользователь вводит данные о произведенных операциях в формате ID; операция; время_в_минутах. Признаком окончания ввода является пустая строка. После завершения ввода необходимо запросить имя выходного CSV-файла. Требуется сгруппировать данные фпо типу операции и для каждой операции рассчитать общее время выполнения и среднее время на одну операцию. Полученные значения округлить до двух знаков после запятой и сохранить в формате операция, количество, общее время, среднее время
5	Пользователь вводит сведения о продажах в формате менеджер; месяц; сумма_продаж. Признаком окончания ввода является пустая строка. После завершения ввода (пустая строка) требуется запросить имя файла для сохранения. Для каждого менеджера необходимо рассчитать общий объем продаж и количество месяцев, в которых он участвовал в продажах. Результаты сохранить в CSV-файл, содержащий строки вида менеджер, количество_месяцев, суммарные_продажи, отсортированные по убыванию суммарных продаж
6	Пользователь вводит данные о показаниях температуры в формате город; дата; температура. Ввод завершается пустой строкой. После запроса имени выходного файла требуется рассчитать для каждого города среднее значение температуры и дату максимального значения температуры. Результаты сохранить в CSV-файл, каждая строка которого содержит город, средняя_температура, дата_макс_температуры
7	Пользователь вводит информацию о перемещениях товаров на складе в формате код_товара; тип_операции; количество, где тип_операции — одно из значений поступление или отгрузка. Признаком окончания ввода является пустая строка. После окончания ввода необходимо запросить имя выходного CSV-файла. Требуется рассчитать итоговый остаток по каждому товару и сохранить в файл строки вида код_товара, остаток, отсортированные по коду товара
8	Пользователь вводит данные о расходах по проектам в формате проект; категория; сумма. Признаком окончания ввода является пустая строка. После завершения ввода необходимо запросить имя файла. Требуется рассчитать общие расходы по каждому проекту и по каждой категории. Полученные результаты сохранить в файл в виде двух блоков, разделенных пустой строкой: первый блок — проект, общая_сумма, второй — категория, общая_сумма

№	Задание
9	Пользователь вводит данные о результатах сдачи лабораторных работ в формате ФИО; лабораторная_работа; оценка. Признаком окончания ввода является пустая строка. После завершения ввода и запроса имени выходного файла требуется построить таблицу, в которой по строкам перечислены студенты, по столбцам — номера лабораторных работ, а на пересечении — соответствующие оценки. Отсутствующие значения оставить пустыми. Таблицу сохранить в формате CSV
10	Пользователь вводит сведения о посещаемости студентов в формате ФИО; дата; присутствие, где присутствие — значение да или нет. Признаком окончания ввода является пустая строка. После завершения ввода требуется запросить имя CSV-файла. Программа должна вычислить для каждого студента процент посещений (отношение количества да к общему числу записей) и сохранить результат в виде таблицы ФИО, процент_посещений, отсортированной по убыванию значения процента
11	Пользователь поочередно вводит информацию о заказах клиентов, совершенных в интернет-магазине. Каждая строка имеет формат номер_заказа; наименование_товара; количество_единиц; стоимость_за_единицу. Признаком окончания ввода является пустая строка. После завершения ввода требуется сгруппировать данные по номеру заказа и для каждого заказа вычислить общую стоимость (произведение количества на стоимость, суммированное по всем позициям). Программа должна запросить имя выходного CSV-файла и сохранить в него таблицу с полями номер_заказа, общая_стоимость
12	Пользователь вводит результаты замеров температуры воздуха, зафиксированные в течение нескольких дней, в формате дата; время; температура. Признаком окончания ввода является пустая строка. После окончания ввода необходимо рассчитать для каждой уникальной даты среднюю температуру на основе всех измерений, относящихся к этой дате. Полученные значения необходимо сохранить в CSV-файл, имя которого пользователь задает самостоятельно, в формате дата, средняя_температура
13	Пользователь вводит данные по операциям на расчетном счете предприятия. Формат ввода каждой записи: дата; тип_операции; сумма, где тип_операции принимает значения пополнение или снятие. Признаком окончания ввода является пустая строка. Необходимо для каждой уникальной даты определить итоговый баланс счета, начиная с нулевого значения. Изменения баланса должны учитываться в хронологическом порядке. После завершения обработки программа сохраняет в указанный пользователем CSV-файл таблицу с полями дата, баланс
14	Пользователь вводит протоколы тестирования производственных изделий в формате идентификатор_изделия; дата; результат, где результат — символ П (прошел) или Н (не прошел). Признаком окончания ввода является пустая строка. Требуется для каждого уникального идентификатора изделия вычислить долю успешных тестов по отношению к общему числу проведенных проверок. Итоговая таблица, содержащая поля идентификатор_изделия, доля_успешных_тестов, сохраняется в файл с именем, указанным пользователем

№	Задание
15	Пользователь вводит информацию об участии сотрудников в образовательных курсах. Каждая запись содержит ФИО; название_курса; оценка. Признаком окончания ввода является пустая строка. Требуется определить для каждого сотрудника курс, по которому он получил наивысшую итоговую оценку. В случае одинаковых оценок выбрать курс, введенный первым. Сформированную таблицу с полями ФИО, лучший_курс, оценка необходимо сохранить в CSV-файл, имя которого пользователь указывает при запуске программы
16	Пользователь вводит данные о нагрузке серверов в вычислительном центре. Каждая строка имеет вид дата; имя_сервера; загрузка_в_процентах. Признаком окончания ввода является пустая строка. Задача состоит в том, чтобы для каждого сервера определить максимально зафиксированное значение загрузки среди всех дат и сохранить в указанный пользователем CSV-файл таблицу вида имя_сервера, максимальная_загрузка
17	Пользователь вводит информацию о посещениях пользователями интернет-страниц. Каждая запись имеет формат имя_пользователя; адрес_страницы; время_просмотра_в_секундах. Признаком окончания ввода является пустая строка. После завершения ввода необходимо подсчитать общее время, проведенное каждым пользователем на сайте, и сохранить в CSV-файл таблицу с двумя полями: имя_пользователя, общее_время_просмотра
18	Пользователь вводит данные о доставках грузов из логистического центра. Каждая строка имеет вид номер_доставки; пункт_назначения; расстояние_в_км. Признаком окончания ввода является пустая строка. Требуется суммировать общее расстояние доставок для каждого уникального пункта назначения. Результирующая таблица, содержащая поля пункт_назначения, общее_расстояние, должна быть сохранена в CSV-файл с именем, заданным пользователем
19	Пользователь вводит протоколы соревнований, содержащие информацию в формате ФИО_участника; название_дисциплины; полученные_баллы. Признаком окончания ввода является пустая строка. Для каждого участника необходимо вычислить суммарный балл, набранный по всем дисциплинам, и сохранить в указанный пользователем CSV-файл таблицу с полями ФИО_участника, общий_балл
20	Пользователь вводит данные о количестве продукции, произведенной в различных цехах предприятия. Каждая запись имеет формат цех; дата; количество_единиц. Признаком окончания ввода является пустая строка. Необходимо определить для каждого цеха дату, на которую пришелся наибольший объем выпуска. Если объемы равны, выбирается более ранняя дата. Полученные результаты сохраняются в CSV-файл с полями цех, дата_максимума, максимальный_выпуск
21	Пользователь вводит с клавиатуры данные о сотрудниках в формате: ФИО; отдел; оклад (например, Иванов И.И.; Бухгалтерия; 42000). Ввод продолжается до появления пустой строки. После завершения ввода программа должна запросить имя выходного файла. Необходимо сгруппировать сотрудников по отделам, вычислить средний оклад по каждому отделу и сохранить результат в CSV-файл. Формат вывода: отдел, средний_оклад

№	Задание
22	Пользователь вводит информацию о заказах в формате: номер_заказа; дата; сумма; статус (например, Z0193; 2024-12-01; 15800; выполнен). Ввод продолжается до пустой строки. После завершения необходимо запросить имя CSV-файла. Требуется сформировать таблицу, в которой для каждого статуса указано количество заказов и суммарная сумма. Формат вывода: статус, количество, сумма
23	Пользователь вручную вводит данные о посещении фитнес-клуба в формате: ФИО; дата; тренировка (например, Сидоров П.С.; 2025-01-05; кардио). Признаком окончания ввода является пустая строка. После завершения ввода (по пустой строке) требуется запрос имени файла. Необходимо подсчитать общее количество тренировок каждого типа и сохранить результат в формате: тренировка, количество
24	Пользователь вводит информацию о начислениях по договорам в формате: номер_договора; месяц; тип_начисления; сумма (например, D1102; 2025-03; электроэнергия; 920.5). Признаком окончания ввода является пустая строка. После завершения ввода необходимо запросить имя файла. Нужно сгруппировать начисления по типу и по месяцу, а затем рассчитать суммарные значения. Результат сохранить в формате: месяц, тип_начисления, сумма
25	Пользователь вводит данные о медицинских обследованиях в формате: ФИО; отделение; дата; результат (например, Орлова М.И.; терапия; 2025-06-20; положительный). Признаком окончания ввода является пустая строка. После завершения ввода необходимо запросить имя CSV-файла. Требуется подсчитать количество пациентов с каждым результатом по каждому отделению. Формат вывода: отделение, результат, количество
26	Пользователь вводит данные об учебных баллах студентов в формате: ФИО; дисциплина; балл (например, Николаев Р.А.; математика; 74). Признаком окончания ввода является пустая строка. После завершения ввода (по пустой строке) программа должна запросить имя CSV-файла. Требуется по каждой дисциплине вычислить максимальный и минимальный баллы. Сохранить результат в формате: дисциплина, макс_балл, мин_балл
27	Пользователь вводит информацию о продажах в формате: менеджер; категория; количество; стоимость_единицы (например, Петрова Л.И.; мебель; 2; 15500). Признаком окончания ввода является пустая строка. После завершения ввода необходимо запросить имя файла. Требуется вычислить общую сумму продаж по каждой категории товаров и сохранить данные в формате: категория, сумма
28	Пользователь вводит данные о времени выполнения задач в формате: ФИО; проект; время_в_часах (например, Фролов А.Д.; Alpha; 3.5). Признаком окончания ввода является пустая строка. После завершения ввода необходимо запросить имя выходного файла. Требуется подсчитать общее количество часов по каждому проекту и сохранить результат в формате: проект, суммарное_время
29	Пользователь вводит информацию о результатах проверок оборудования в формате: номер_устройства; тип; дата; состояние (например, UTX-0321; сервер; 2025-07-10; исправен). Признаком окончания ввода является пустая строка. После завершения ввода необходимо запросить имя CSV-файла. Требуется по каждому типу оборудования определить количество устройств в каждом состоянии. Формат вывода: тип, состояние, количество

№	Задание
30	Пользователь вручную вводит сведения о бронировании номеров в гостинице в формате: ФИО; дата_заезда; кол-во_ночей; тип_номера (например, Козлов Е.Н. ; 2025-08-01; 3; люкс). Признаком окончания ввода является пустая строка. После завершения ввода (пустая строка) программа должна запросить имя файла. Необходимо подсчитать количество забронированных ночей по каждому типу номера и сохранить результат в формате: тип_номера, количество_ночей

Equation Section (Next)

15. Регрессионный анализ

15.1. Линейная парная регрессия

15.2. Линейная множественная регрессия

Предметный указатель

CSV формат	266	Конкатенация	88
escape-последовательность	86	Константы модуля math	27
F-строка	100	Конъюнкция	24
Алгоритм	6	Копирование	
Алгоритмизация	7	глубокое	158
Алфавит исполнителя	7	поверхностное	158
Аргумент	15	Кортеж	208
ключевой	75	пустой	208
Атрибут	17	Лексема	15
Бесконечность		Литерал	16
отрицательная	23	Лямбда-функция	176
положительная	23	Матрица	
Ветвление	См. Инструкция, условная	верхнетреугольная	189
Возведение матрицы в степень ..	189	единичная	187
Вычитание матриц	187	нулевая	187
Генератор		Менеджер контекстный	262
словаря	225	Метод	17
списка	150	Гаусса	189
Генерация последовательности		Гаусса-Жордана	189
целых чисел	55	Методы файлового объекта	263
Данные входные допустимые	7	Множество	216
Диалект модуля csv	266	пустое	216, 217
Дизъюнкция	24	универсальное	217
Задание литеральное	16	Мощность множества	217
Знаки пунктуации	16	Ноль	
Значение объекта	18	замыкающий	22
Идентификатор	16	лидирующий	22
объекта	18	Обработка файлов	262
Идентичность объектов	18	Обратная матрица	189
Импликация	24	Объединение множеств	217
Индекс		Объект	17
итерации	43	изменяемый	18
Инструкция	15	итерируемый	19
условная	34	неизменяемый	18
цикла с параметром	61	хешируемый	19
цикла с предусловием	44	Операнд	15
Интерфейс консольный	232	Оператор	15
Исполнитель алгоритма	7	тернарный условный	35
Итерация	43	Операторы	
Класс	17	сравнения	22
Код программный	14	Операции	
Команда исполнителя	7	арифметические	21
Комментарий	17	логические	24
		над кортежами	209
		над множествами	216
		с присваиванием	21

теоретико-множественные	217	двумерных списков	176
Операция	15	на месте	174
бинарная	15	Состояние	
тернарная	15	конечное	7
унарная	15	начальное	7
Определитель матрицы	188	Состояние исполнителя	7
Отрицание	24	Список	
Параметр		двумерный	155
именованный	75	одномерный	116
позиционный	75	пустой	116
со значением по умолчанию	75	Сравнение вещественных чисел ..	23
Пересечение множеств	217	Сравнение списков	158
Подпрограмма	74	Среда исполнения алгоритма	7
Предусловие	43	Среда программирования	14
Преобразования элементарные ..	189	Срез	89
Прерывание цикла	43	Строка	
Приоритет операций	25	пустая	85
Программа	14	Тело цикла	43
Пропуск итерации	44	Тип данных	18
Развилка См. Инструкция, условная		вещественный	22
Разность множеств	218	логический	23
симметрическая	218	строка	85
Ранг матрицы	189	целочисленный	22
Распаковка кортежей	208	Транспонирование матрицы	188
Режим открытия файла	262	Умножение	
Результат выполнения алгоритма ..	7	матриц	188
Ряд		матрицы на число	187
Маклорена	81	Форма записи числа	
Тейлора	81	экспоненциальная	22
Свойства		Формы описания алгоритмов	8
алгоритма	6	Функции модуля math	26
Свойство класса	17	тригонометрические	27
Семантика	14	Функция	
Символ		высшего порядка	176
экранированный	86	пользовательская См. Подпрограмма	
Синтаксис	14	Хеш-значение	19
Система команд исполнителя	7	Цепочки сравнений	26
Система программирования	14	Цикл	43
След матрицы	188	бесконечный	44
Словарь	223	с параметром	43
пустой	223	с предусловием	43
Слово ключевое	16	Эквиваленция	24
Сложение матриц	187	Экземпляр класса	17
Сортировка	174	Этапы алгоритмизации	7

